KU LEUVEN

FACULTY OF SCIENCE

Connectivity Bounds in Graphs and Beyond

Erdős Problem 915, from Proof to Search

Louis Vandenbruwaene

Supervisor: Prof. Stijn Cambie
KU Leuven

Thesis presented in
fulfillment of the requirements
for the degree of Master of Science
in Mathematics

Academic year 2026–2027

Open Access Disclaimer

This master's thesis is a student work submitted as part of an academic programme at KU Leuven. The work is made available for examination and archiving in accordance with the applicable KU Leuven regulations and open-access policy for master's theses.

Foreword

This thesis studies how many edges or arcs a network can contain when no pair of vertices is allowed to have too many independent routes between them. The starting point is Erdős Problem 915, an extremal question about edge-disjoint paths in simple graphs. The work is told as a journey. It begins with the variants whose answers can be proved cleanly by hand, watches those proofs grow longer and more delicate as the model changes, and at the point where a single argument no longer suffices it turns to a unified program. That program holds all twelve variants behind one common interface, so a single piece of code serves them all. It measures connectivity exactly with a maximum-flow routine, the standard way to count the independent routes between two points. It runs a prover that sweeps every graph of a small fixed size and, when none beats a target, reports that sweep as a genuine upper-bound proof. And it runs a search that, like hot metal cooling slowly into shape, settles random graphs toward the densest ones the rules allow.

I am grateful to my supervisor, Prof. Stijn Cambie, and my daily advisor for their guidance, corrections, and mathematical feedback throughout the project. One review in particular changed the shape of this thesis: Prof. Cambie put a late draft through a language model different from the one I had been working with, and it came back with a misapplied inequality I had read past many times, together with the counting idea that grew into Proposition A.22 and Theorem A.24. I thank my promotor, Prof. Jan Goedgebeur, whose suggestion of the `nauty` generation pipeline sharply accelerated the directed enumeration. I also thank the people who discussed early versions of the arguments, checked examples, or helped clarify the presentation. Above all I thank my girlfriend Sara, whose patience and encouragement carried me through the long stretches of this work. Any remaining mistakes are my own.

Contribution Statement

This thesis is my own work. I carried out the literature study, recast the twelve variants of the problem in one common language, and designed and wrote the single program that runs through all of them. I built every example, by hand or by computer search, checked each one with that program, wrote the proofs that appear in the thesis, and typeset the whole document. Where a result comes from the literature I say so and cite it. The theorems I lean on but do not claim as mine are those of Mader, Leonard, Sørensen and Thomassen, Menger, Gomory and Hu, and Baranyai, the hypergraph connectivity results of Bérczi, Chandrasekaran, Király and Kulkarni, and the standard probabilistic tools used for the random graphs.

The program has three jobs, and it keeps them separate. It *measures*: given any graph, it computes exactly, through maximum flow, how many independent routes the busiest pair of vertices has. It *proves*: for a fixed number of vertices it sweeps a finite space of graphs, and when nothing in that space beats a target, that sweep is a genuine upper-bound proof rather than a guess. And it *discovers*: a search that cools like a heated metal settling into shape wanders the space of graphs and finds dense ones, which give lower bounds and suggest conjectures but never prove an upper bound by themselves. Every general statement in the thesis rests on a proof by hand or a cited theorem. The computer searches are evidence that points the way, never a replacement for the argument. I used AI language models, including Claude Code, ChatGPT, and Gemini, as tools for literature search, LaTeX editing, proofreading, and trying out proof drafts. I checked every mathematical statement and proof in the thesis myself and take full responsibility for the content.

One thing about that use is worth recording, because it is a methodological point and not just a disclosure. A model that has helped write an argument is a poor reviewer of it: it reads the text the way its author does, follows the same intended reading of each sentence, and slides over the same gaps. Handing the finished draft to a *different* model, cold, behaved much more like handing it to a different person. It read each sentence as a standalone claim rather than as a step in an argument it already believed, and that is exactly how it caught a use of Whitney's inequality that runs the wrong way, in a sentence that had survived several careful readings by me and by the model that helped write it. The same pass also proposed a counting argument I had not tried. It was wrong about a good deal else, and every item it raised had to be checked against the source before anything was changed, several of them turning out to be objections to conventions the thesis states rather than to errors. But the pattern is the familiar one from working with people: the value came from the disagreement, not from the agreement, and from the fact that the second reader had no stake in the first reader's phrasing.

The genuinely new results are the following. For hyperedge networks I settle the vertex-disjoint problem completely at the two strictest route limits, $m = 2$ and $m = 3$, for every hyperedge size r and every number of vertices n (Theorems A.45 and A.48): the vertex-disjoint and edge-disjoint answers turn out to be equal, both $\lfloor (m-1)(n-1)/(r-1) \rfloor$. The $m = 3$ case rests on a bound for incidence graphs that I prove using Tutte's decomposition of a graph into its three-connected pieces (Lemma A.47). For one-way multigraphs at $m = 3$ I prove an unconditional attachment lemma (Lemma A.34) and an odd-step theorem (Theorem A.36, Remark A.39) that together shrink the whole problem, for every n, down to two finite statements about multigraphs on just seven vertices that a computer can check, one of which the accompanying program has already verified outright, and they do so without assuming the unproved minimum-degree conjecture the earlier route needed. For one-way simple graphs I prove an unconditional quadratic bound (Proposition A.22 and Theorem A.24): under any route limit m, such a graph has at most $\lfloor n^2/4 \rfloor + 4(m-1)(n-1)$ arcs, and deleting a comparable number of arcs always leaves a one-way bipartite graph. This fixes both the leading constant and the order of the second-order term of the standing conjecture (Conjecture 1.14) without any assumption about the shape of the extremal graph, which is what every previous route to that bound needed, and leaves only a constant factor of about eight in one coefficient. The counting idea behind it came out of the external review recorded in the foreword, and I verified it and worked out the sharpening myself. For multigraphs under the alternative counting convention I settle the vertex problem at $m = 2$ and disprove the natural guess, suggested by the data through $m = 4$, that a thickened tree stays optimal for all larger n: chaining thickened cliques through single bridge vertices (Theorem A.56) beats it by a margin growing linearly in n, at a rate that is quadratic rather than linear in m (Section A.8).

Short Summary

Erdős Problem 915, posed by Bollobás and Erdős [1], asks for the maximum number of edges in a graph on n vertices such that no two vertices are connected by m edge-disjoint paths. In modern notation this asks for

$$\ell_m(n) = \max\{|E(G)| : |V(G)| = n,\ \lambda_G^{\max} \leq m-1\}.$$

Mader proved that, for simple graphs and $n \geq m \geq 2$,

$$\ell_m(n) = \left\lfloor \frac{m(n-1)}{2} \right\rfloor.$$

This thesis revisits this theorem and studies the corresponding extremal problem under three independent changes: the graph model (simple graphs, multigraphs, and hypergraphs), the path-separation notion (edge-disjoint or internally vertex-disjoint), and the presence or absence of directions. Sampled random graphs $G(n,p)$ run alongside throughout as a lens on the typical case rather than as a separate model.

The thesis is organised as a journey from hand proofs to computation. The early variants are settled rigorously: the multigraph value $L_m(n) = (m-1)(n-1)$, the random-graph threshold $p^* = m/n$, the equality of the edge and vertex problems for $m \leq 4$, and the first divergence at $m = 5$, where Sørensen and Thomassen prove $k_5(n) = \lfloor 8n/3 \rfloor - 3$. The directed case forces the turn: the value $\ell_2^{\mathrm{dir}}(n) = \max(2(n-1), \lfloor n^2/4 \rfloor)$ is proved by a long induction, and for $m \geq 3$ the hand proof gives way to a conjecture, after an explicit ten-vertex graph rules out the naive guess. The centrepiece is then a unified program. One multiplicity-matrix representation models almost all twelve variants, an exact maximum-flow checker measures local connectivity, a cut-counting optimisation certifies upper bounds for small sizes, and a search guided by edge sensitivity, run as both simulated annealing and tabu search, discovers extremal graphs. The program rediscovers, each in its own variant, the double star (undirected multigraphs at $m = 2$), the directed hub, the one-directional bipartite digraph, and the augmented-bipartite construction (the directed cases), certifies the directed multigraph values $L_m^{\mathrm{dir}}(n) = 2(n-1)(m-1)$ for $n \leq 6$, and frames the remaining open problems on the directed frontier.

The interplay then pays back in the other direction. The hypergraph vertex problem is settled completely at $m = 2$ and $m = 3$: $k_m^{(r)}(n) = \lfloor (m-1)(n-1)/(r-1) \rfloor$ for every uniformity r, the $m = 3$ case through a new rank bound on incidence graphs proved by way of Tutte's triconnected decomposition. On the directed side, an unconditional attachment lemma and a conditional odd-step theorem reduce the entire directed multigraph problem at $m = 3$, value and extremal characterisation for all n, to two finite, machine-checkable statements about multigraphs on seven vertices, one of which the program has since verified outright, and directed hypergraphs

receive their first bounds, quadratic in n.

Abbreviations and Symbols

GH	Gomory-Hu
MILP	mixed-integer linear program
SA	simulated annealing
SPQR	the triconnected decomposition tree (series, parallel, rigid pieces)

$G = (V, E)$	graph with vertex set V and edge set E
$D = (V, A)$	directed graph with vertex set V and arc set A
n	number of vertices
m	forbidden-connectivity parameter, with all local connectivities at most $m-1$
r	hyperedge size (every hyperedge spans r vertices)
$\lambda_G(u, v)$	maximum number of edge-disjoint u-v paths
$\lambda_G^{\max}$	maximum local edge-connectivity over all vertex pairs
$\kappa_G(u, v)$	maximum number of internally vertex-disjoint u-v paths
$\kappa_G^{\max}$	maximum local vertex-connectivity over all vertex pairs
$\ell_m(n)$	simple undirected edge-disjoint extremal function
$L_m(n)$	undirected multigraph edge-disjoint extremal function
$k_m(n)$	simple undirected vertex-disjoint extremal function
$\ell_m^{\mathrm{dir}}(n)$	simple directed arc-disjoint extremal function
$L_m^{\mathrm{dir}}(n)$	directed multigraph arc-disjoint extremal function
$k_m^{\mathrm{dir}}(n)$	simple directed vertex-disjoint extremal function
$\ell_m^{(r)}(n)$, $k_m^{(r)}(n)$	r-uniform hypergraph edge- and vertex-disjoint extremal functions
$M^*(n)$	the m-free directed multigraph optimum, $L_m^{\mathrm{dir}}(n) = (m-1)\, M^*(n)$
$B_{p,q}$	one-directional complete bipartite digraph, all arcs from a p-part to a q-part
$\mu(u, v)$	multiplicity of an edge or arc between u and v
$\sigma(e)$	edge sensitivity $\lambda^{\max}(G) - \lambda^{\max}(G - e)$
$G(n, p)$	binomial random graph

Contents

Chapter 1

The Problem and Its Base Cases

A network is useful not only because it connects, but because it connects in more than one way. A single road between two towns is a liability, and a second independent road is insurance. The mathematical content of that intuition is the number of routes between two points that share no segment, and the question this thesis circles is how many connections a network can hold before some pair of points is forced to have too many such routes. This chapter builds the question from nothing, settles the variants whose proofs are short enough to read in an afternoon, and then follows the same question along three axes until the proofs stretch past breaking. By the end the case for a machine has made itself.

1.1 Graphs and routes

To state the question precisely we need only a few ideas. A *graph* G is a finite set of points, called *vertices*, together with a set of links, called *edges*, each joining two of the vertices. We write $V(G)$ for the set of vertices and $E(G)$ for the set of edges, so that $|E(G)|$ is the number of edges, the very quantity this thesis tries to make as large as possible. One pictures the vertices as dots and the edges as lines between them.

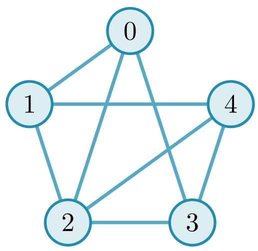


Figure 1.1. A simple graph on five vertices. It is K_5, the *complete* graph in which every one of the $\binom{5}{2} = 10$ pairs is joined by an edge (the binomial coefficient $\binom{5}{2}$ counts the ways to choose two of the five vertices), with the two edges $\{0, 4\}$ and $\{1, 3\}$ removed, so its edge set $E(G)$ holds eight edges. The five dots are the vertex set $V(G)$ and the lines are the edges. Because at most one line joins any given pair and no edge loops back to its own endpoint, this is a simple graph. The same graph returns in Figure 1.2, once we can count the independent routes between a pair.

A graph is the bare skeleton of a network: the vertices are the towns or the devices, and the edges are the roads or the links between them. We write n for the number of vertices, and the *degree* of a vertex is the number of edges meeting it. Unless we say otherwise our graphs are *simple*, meaning that at most one edge joins any given pair of vertices and its endpoints are distinct.

A *path* from a vertex u to a vertex v is a sequence of distinct vertices that starts at u, ends at v, and joins each consecutive pair by an edge. It is the formal version of a route through the network. Two paths between the same endpoints are *edge-disjoint* if no edge belongs to both, so that the loss of a single edge can break at most one of them. The largest number of pairwise edge-disjoint paths between u and v is their *local edge-connectivity* $\lambda_G(u, v)$, the count of genuinely independent routes the network offers to that pair. The pair with the most independent routes sets the ceiling for the whole graph:

$$\lambda_G^{\max} = \max_{u \neq v} \lambda_G(u, v),$$

the largest local edge-connectivity anywhere in G. A theorem of Menger [2], recalled in Appendix A, gives this quantity a second face: $\lambda_G(u, v)$ also equals the least number of edges one must remove to cut every route from u to v (Figure 1.2). Many independent routes between a pair is exactly the same thing as a costly cut between them.

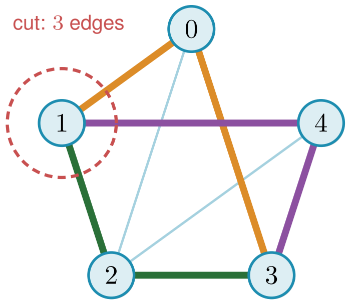


Figure 1.2. Menger's theorem on the graph of Figure 1.1. The pair $1, 3$ carries no edge of its own, yet three routes still join them with no shared edge, 1-0-3 (orange), 1-2-3 (green), and 1-4-3 (purple), each through a different middle vertex, so $\lambda(1, 3) = 3$. The three edges meeting vertex 1 (inside the red dashed ring) are a smallest set whose removal cuts every 1-3 route, and no set of just two edges can cut them all, because the three routes share no edge, so each removed edge breaks at most one route and two removals leave one route intact. The cheapest cut therefore also has size three. The most independent routes a pair can muster always equals the cheapest cut between them, the fact every bound in this thesis leans on.

1.2 Erdős Problem 915

The constraint at the heart of this thesis is a ceiling on connectivity. Requiring $\lambda_G^{\max} \leq m - 1$ says that no pair of vertices is joined by as many as m edge-disjoint paths, or equivalently that every pair can be severed by removing fewer than m edges. The question is how many edges a graph can hold while staying under that ceiling.

Question 1.1 (Erdős Problem 915 [3]). How many edges can a graph on n vertices have if no two of its vertices are joined by m edge-disjoint paths?

Writing the answer as an extremal function,

$$\ell_m(n) = \max\{\, |E(G)| : |V(G)| = n,\ \lambda_G^{\max} \leq m - 1 \,\},$$

the problem asks for $\ell_m(n)$, the most edges possible under the ceiling. Mader settled the simple-graph case completely, proving both halves at once: no graph under the ceiling has more than $\lfloor m(n-1)/2 \rfloor$ edges (the upper bound), and some graph reaches that count (the lower bound). The brackets $\lfloor \cdot \rfloor$ round their content down to the nearest whole number, and their ceiling counterpart $\lceil \cdot \rceil$ rounds up. Both recur throughout, because an edge count is always an integer while the formulas that bound it need not be.

Theorem 1.2 (Mader [4])**.** *For all* $n \geq m \geq 2$*,*

$$\ell_m(n) = \left\lfloor \frac{m(n-1)}{2} \right\rfloor .$$

Mader's own proof is a direct structural induction. The proof given in full in Appendix A takes a different road, by way of the Gomory–Hu tree, found independently of Mader, together with a characterisation of the graphs that attain equality (Theorem A.11). This is the road the thesis adopts throughout, because the very same tree returns for the multigraph and hypergraph variants (Theorem 1.3, Proposition 1.15), so a single argument serves all of them rather than each needing its own. Every graph has such a tree (Theorem A.2), and we single out the one identity that makes it work, because the same idea reappears when the program of Chapter 2 stores connectivities as a tree (a graph without cycles): a Gomory–Hu tree of G is a weighted tree on the same vertex set in which $\lambda_G(u, v)$ equals the lightest edge on the tree path from u to v. All $\binom{n}{2}$ local connectivities are therefore encoded by $n-1$ numbers, and $\lambda_G^{\max}$ is simply the heaviest edge of the tree.

The whole argument (Appendix A) is one act of double counting on that tree. The word *charged* below is just accounting: to charge an edge to a tree edge is to assign it there for the count, and the proof works by charging every edge of G to tree edges and then adding the charges up. Each of the $n-1$ tree edges stands for a minimum cut of G, and an original edge of G is charged once for every tree edge lying between its endpoints, so summing the tree-edge weights counts every edge of G at least once and, crucially, counts most of them at least twice. The only edges charged a single time are those joining tree-adjacent vertices, and a *simple* graph has at most $n-1$ of those, one per tree edge. Every other edge is charged twice. Since each tree weight is a local connectivity and so at most $m-1$, the two counts meet at $2|E(G)| - (n-1) \leq (m-1)(n-1)$, which rearranges to the floor. The factor of two, and with it the gap between Mader's $\lfloor m(n-1)/2 \rfloor$ and the larger value $(m-1)(n-1)$ that parallel edges will buy in Theorem 1.3, is exactly the dividend of simplicity, the one place the argument uses that no pair carries a parallel edge.

One example fixes the scale. The complete graph K_n joins every pair, so every pair has $n-1$ routes that share no edge, $\lambda(u, v) = n-1$, while it carries all $\binom{n}{2}$ edges. It therefore meets the ceiling $\lambda^{\max} \leq m-1$ exactly when $m \geq n$, and from that point on the question is settled trivially: the complete graph is feasible, no simple graph on n vertices is denser, so $\ell_m(n) = \binom{n}{2}$ for every $m \geq n$ and raising m further changes nothing. For smaller m the extremal graph must be strictly sparser than complete, and pinning down exactly how much sparser is the whole problem. That is why the theorem, and the thesis, assume $m \leq n$.

1.3 Three axes of variation

Mader's theorem is the floor we build on, and everything afterwards is a variation on it. There are three independent ways to change the rules, and they generate the landscape this thesis traverses.

The first axis is the *model*, drawn in Figure 1.3. Two independent choices generate it. The first is *arity*: an edge may join exactly two vertices, or a *hyperedge* may join any $r \geq 2$ of them, a route then passing through it between any two of its members. At $r = 2$ a hyperedge is an ordinary edge, so graphs are the $r = 2$ special case and the genuinely new setting begins at $r \geq 3$. The second is *multiplicity*: a *simple* object carries at most one copy of each edge, while a *multi* object allows parallel copies, recorded by a multiplicity $\mu(u, v)$, so a single pair can already carry several independent routes. Because the two choices are independent they form a small lattice. At the top sits the most general object, the *multi-hypergraph*, and lowering one toggle at a time restricts it downward: forbidding repeats gives the simple hypergraph, dropping the arity to two gives the multigraph, and doing both gives the simple graph at the bottom. Every theorem in this thesis is a statement about one node of this lattice, and the three rows of every comparison figure are its simple-graph, multigraph, and hypergraph levels.

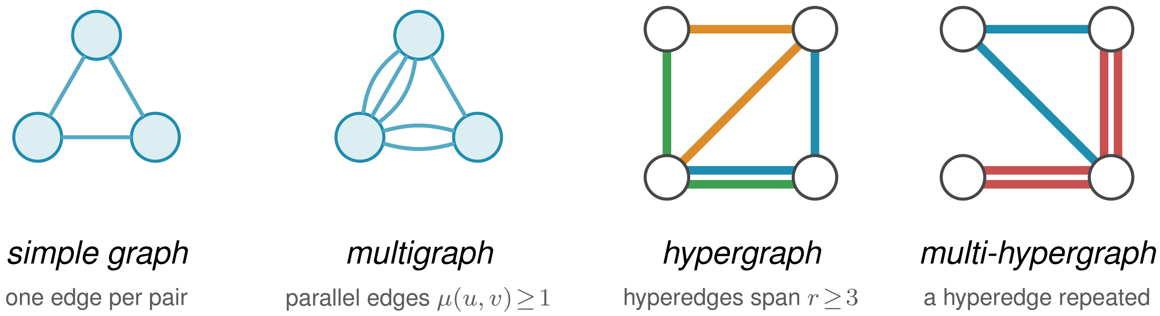


Figure 1.3. The first axis of variation, the *model*, shown for the four object classes this thesis uses. In a *simple graph* each pair of vertices is joined by at most one edge. A *multigraph* permits parallel edges, here a triple and a double, so the multiplicity $\mu(u, v)$ counts how many copies join a pair. A *hypergraph* replaces pairwise edges by hyperedges, each a set of $r \geq 2$ vertices ($r = 2$ gives back ordinary edges, and the depicted examples use $r = 3$), drawn the way a metro map draws a line, threading all of its stations, one colour per hyperedge. Where two hyperedges share a stretch between stations the two lines run side by side, one colour each, exactly as a metro map separates lines along a shared stretch of track. A *multi-hypergraph* lets a hyperedge be repeated, drawn as two parallel lines through the same stations, the hypergraph counterpart of parallel edges. When all hyperedges have the same size r the hypergraph is r-uniform. The same connectivity question, how many independent routes a pair can be forced to share, is asked of all four.

The second axis is the *direction*, a third toggle independent of the other two. Setting it at the top of the model lattice gives the single most general object in the thesis, the *directed multi-hypergraph*, of which every other class is a restriction. Figure 1.4 redraws the four models of Figure 1.3 with arcs in place of edges. An undirected object is the symmetric special case of a directed one, each edge standing for a matched pair of opposite arcs $u \to v$ and $v \to u$, so direction strictly enlarges the family of objects. What it does *not* do is let the directed and undirected problems merge into one. The two opposite arcs offer a round trip the single edge uv does not, the extremal digraphs (directed graphs) are deliberately *lopsided*, with out-degree and in-degree pulled apart in a way no symmetric object can imitate, and the Gomory-Hu tree that settles every undirected case has no directed analogue (Appendix A), so the one-way variants stay distinct in both difficulty and method.

A digraph carries three degree counts at each vertex. The *out-degree* $d^+(v)$ counts the arcs leaving v, and the vertices they reach are its *out-neighbours*. The *in-degree* $d^-(v)$ counts the arcs entering v, and the vertices they come from are its *in-neighbours*. The *total degree* $d(v) = d^+(v) + d^-(v)$ adds the two. A vertex with high out-degree fans arcs outward, one with high in-degree gathers them, and keeping the two balanced is the structural idea behind several of the directed constructions to come. The directed measure itself is easiest to meet on one concrete digraph. Figure 1.5 shows a network with three arc-disjoint routes between the two chosen vertices, which equals the smallest arc-cut that separates them, and the degree bound that limits both, which is exactly how Menger reads in a one-way network.

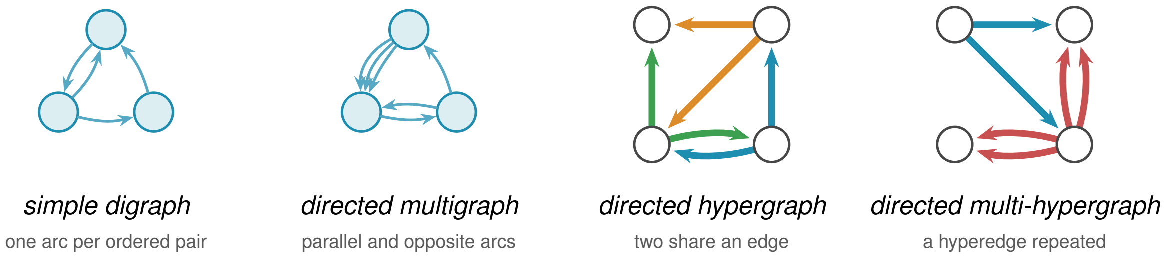


Figure 1.4. The four models of Figure 1.3 redrawn *directed*, the second axis applied to the first. Every edge becomes an arc, and a pair may now carry arcs both ways, the round trip an undirected edge cannot offer. The *simple digraph* still allows at most one arc per ordered pair, so the two-cycle on the top pair is simple. The *directed multigraph* adds parallel arcs, here a triple one way and a bidirected pair. The *directed hypergraph* draws the same three hyperedges as Figure 1.3, in the same colours, each now a metro-styled star, the tail sending one thick coloured arc to each of its heads, the star convention of Figure A.8. Taking each star's tail to be the middle station of the undirected line keeps every arc on the same edge as before. The blue and green hyperedges still share the bottom edge, now one arc running each way, the directed counterpart of two metro lines sharing a stretch of track. The *directed multi-hypergraph* repeats the red hyperedge, so each of its arcs becomes a parallel pair. This last panel is the directed multi-hypergraph at the top of the model lattice, the most general object in the thesis.

The third axis is the *separation*: routes may be required to avoid sharing an edge, as above, or the stricter *internally vertex-disjoint*, sharing no intermediate vertex, which gives the local vertex-connectivity $\kappa_G(u, v)$ and its maximum $\kappa_G^{\max}$. Whitney's inequality $\kappa_G(u, v) \le \lambda_G(u, v)$ [5] records that avoiding shared vertices is at least as demanding as avoiding shared edges, and Figure 1.6 shows the gap on a graph where two routes are edge-disjoint yet forced through one common vertex.

Three models, two directions, two separations: twelve versions of one question. The thesis is the attempt to answer as many of them as honest mathematics allows, and to build a machine

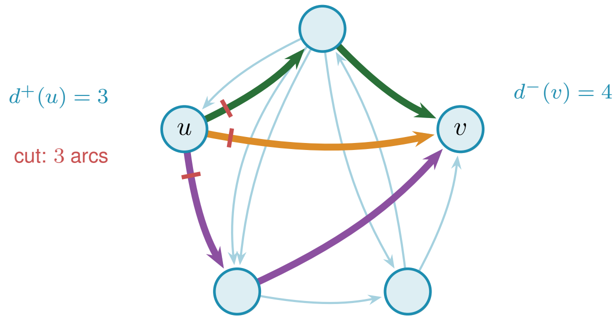


Figure 1.5. A directed version of the K_5 example with multiple arcs. Three arc-disjoint directed routes connect u to v (orange, green, purple). The three red marks cut the arcs leaving u, the first arc of each route, so $\lambda(u, v) = 3$ by Menger's theorem. A directed cut counts only the arcs leaving u's side, so the lone arc entering u from the top is not part of it. The out-degree $d^+(u) = 3$ and in-degree $d^-(v) = 4$ bound the value from above, and $\lambda(u, v) = \min(3, 4) = 3$ meets the smaller one, at u.

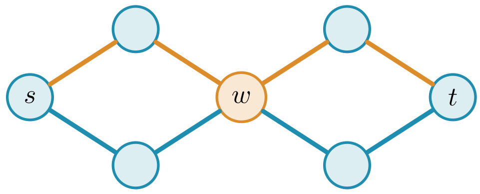


Figure 1.6. The third axis, the *separation*. The two s–t routes drawn here, one orange and one blue, share no edge, so they count as two edge-disjoint paths and $\lambda(s, t) = 2$. Both pass through the single vertex w, however, so they are *not* internally vertex-disjoint, and since every s–t route must cross w we have $\kappa(s, t) = 1$. This is Whitney's inequality $\kappa \leq \lambda$ made concrete: the gap between the two separations is exactly what the vertex-disjoint problem measures, and the rest of the chapter shows it stays shut until $m = 5$.

for the rest. Some answers are short and we prove them here. Some answers are long, and their length is the reason Chapter 2 exists. The rest of this chapter collects the variants whose proofs are short enough to belong in the body, so that the reader meets the easy slope before the climb. Figure 1.7 draws the whole space as one tree, so that the reader has a single map to return to as the chapters visit its branches one at a time.

1.4 Cases with short proofs

Allowing parallel edges removes the parity subtlety, the dependence on whether $m(n-1)$ is even or odd, that produces the floor in Mader's theorem, and the answer becomes exactly linear. Write $L_m(n)$ for the multigraph analogue of $\ell_m(n)$.

Theorem 1.3. *For undirected multigraphs on n vertices, $L_m(n) = (m-1)(n-1)$.*

Proof. For the lower bound, take a spanning tree of K_n, that is, a connected cycle-free subgraph that touches all n vertices and so has exactly $n-1$ edges, and give every tree edge multiplicity $m-1$, meaning $m-1$ parallel copies. The result has $(m-1)(n-1)$ edges. Any two vertices are joined by a unique path in the tree, of some length $k \geq 1$. To separate them one must cut some edge of that path, and the cheapest choice is the lightest tree edge on it, which still carries $m-1$

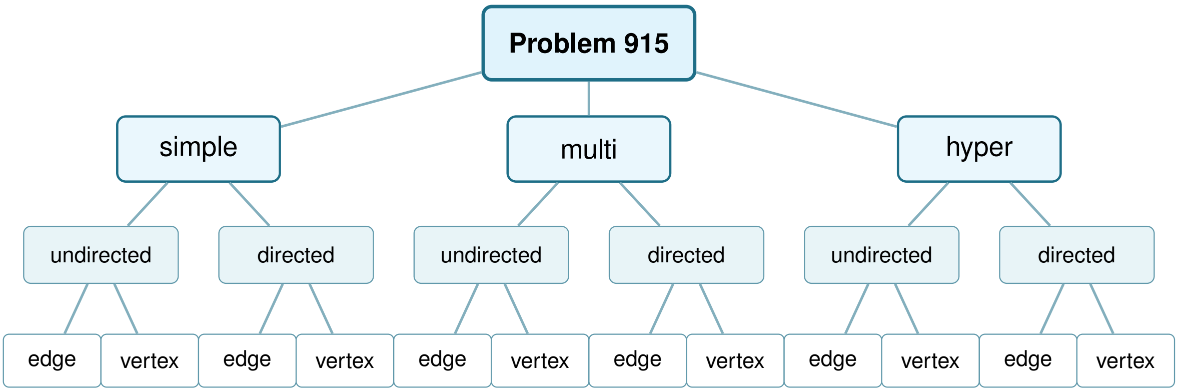


Figure 1.7. The whole space this thesis studies, drawn as one tree. Three model choices (simple graph, multigraph, r-uniform hypergraph), each splitting into undirected or directed, each in turn splitting into edge-disjoint or internally vertex-disjoint routes: twelve leaves, twelve variants. Figure 4.7 redraws this exact tree once every leaf's status is known.

copies, so $\lambda(u, v) = m - 1$ for every pair and $\lambda^{\max} = m - 1$. The graph sits right at the ceiling.

For the upper bound, suppose $\lambda_G^{\max} \leq m - 1$ and build a Gomory–Hu tree T of G (Theorem A.2), the same $(n - 1)$-edge summary of all connectivities used for Mader's theorem. Each tree edge e stands for a minimum cut of G, of capacity $c(e) = \lambda_G(u, v) \leq m - 1$ for the pair (u, v) it represents. Now charge each edge of G to a tree edge: an edge $\{x, y\}$ crosses the cut of the lightest tree edge on the tree path from x to y, so it is counted inside that tree edge's capacity. Summing the $n - 1$ capacities therefore counts every edge of G at least once, which already gives $|E(G)| \leq \sum_{e \in T} c(e) \leq (m - 1)(n - 1)$. Unlike Mader's simple-graph count there is no second charge to halve, because parallel edges are now allowed and nothing forces an edge to be counted twice. The tree-of-multiplicity-$(m - 1)$ construction above meets this bound exactly, which is why the answer is the clean product $(m - 1)(n - 1)$ and not a floor. □

The vertex-disjoint question is just as quick at the first non-trivial value. Forbidding two internally vertex-disjoint paths is a strong constraint: it forbids cycles.

Proposition 1.4. *For simple graphs, $k_2(n) = n - 1$, attained exactly by trees.*

Proof. If G contains a cycle, a closed loop of distinct vertices, then any two vertices on that cycle are joined by the two paths of the cycle, which are internally vertex-disjoint, so $\kappa_G^{\max} \geq 2$. Hence $\kappa_G^{\max} \leq 1$ forces G to be acyclic, free of cycles, and an acyclic graph on n vertices has at most $n - 1$ edges, with equality for trees. A tree indeed has $\kappa^{\max} = 1$, since removing the single intermediate vertex on any path disconnects its endpoints. □

Here $k_m(n)$ denotes the vertex-disjoint extremal function, the counterpart of $\ell_m(n)$ for $\kappa^{\max}$. We have just seen $k_2(n) = n - 1 = \ell_2(n)$. The agreement of the edge and vertex problems is no accident at small m, and the moment it breaks is the first place the proofs begin to strain.

1.5 The first divergence: vertex-disjoint paths

Replacing edge-disjoint by internally vertex-disjoint routes changes nothing at first. Leonard proved that the edge and vertex problems give the same answer while m stays small.

Theorem 1.5 (Leonard [6]). *For simple graphs, all $n \geq m$ and $m \leq 4$,*

$$k_m(n) = \ell_m(n) = \left\lfloor \frac{m(n-1)}{2} \right\rfloor .$$

The hypothesis $n \geq m$ is the same one Mader's theorem carries, and it is not decoration. Below it the complete graph K_n is itself feasible, since $\kappa_{K_n}^{\max} = \lambda_{K_n}^{\max} = n-1 \leq m-1$, so the answer is the trivial $\binom{n}{2}$ and the formula overshoots it. At $n = 3$, $m = 4$ the formula gives 4 where a graph on three vertices has only 3 edges. Every closed form in this thesis carries the same caveat, stated or inherited from its constructions, and the figures cap each curve at the trivial maximum for exactly this reason.

For $m \leq 4$ the extremal graphs for the two problems coincide: Whitney's inequality $\kappa \leq \lambda$ is tight where it matters and the harder vertex constraint costs nothing (Figure 1.9, the three overlapping blue curves). One is tempted to expect this forever. It fails at the very next value, and the failure is sharp.

Theorem 1.6 (Sørensen-Thomassen [7]). *For simple graphs and all $n \geq 13$,*

$$k_5(n) = \left\lfloor \frac{8n}{3} \right\rfloor - 3 \ > \ \left\lfloor \frac{5(n-1)}{2} \right\rfloor = \ell_5(n).$$

The restriction $n \geq 13$ is the range over which Sørensen and Thomassen certify that $k_5(n)$ equals the formula, as the Erdős Problems database records it [3]. Below it the true $k_5(n)$ needs the case-by-case structural analysis [7] carries out rather than the closed form, which is why the stated domain begins there.

Remark 1.7 (A counting convention that shifts every quoted constant by one). The additive constant above deserves a warning, because two conventions are in circulation and they differ by exactly one. This thesis defines $\ell_m(n)$ and $k_m(n)$ as the *largest* number of edges a graph can carry while *no* pair is joined by m independent routes. The Erdős Problems database [3] states the same results the other way round, as the *smallest* number of edges that *forces* such a pair, which is one more.

The shift is visible in every entry that both sources state. The database gives $k_2(n) = n$ where a tree has $n-1$ edges, $k_3(2n) = 3n-1$ against $\lfloor 3(2n-1)/2 \rfloor = 3n-2$ here, $k_4(n) = 2n-1$ against $2n-2$, $\ell_5(2n) = 5n-2$ against $5n-3$, $\ell_6(n) = 3n-2$ against $3n-3$, and it writes the Bollobás-Erdős conjecture as $1 + \binom{m}{2}n$ where the convention here gives $\binom{m}{2}n$. Every one of those is exactly one apart, and the values in this thesis are the ones an exhaustive search returns: $k_3(4) = 4$, $k_3(5) = 6$ and $k_4(5) = 8$ were each confirmed by walking every graph of that size.

Applied to Theorem 1.6 the same shift would read $\lfloor 8n/3 \rfloor - 4$ rather than the $\lfloor 8n/3 \rfloor - 3$ printed above, which is the database's figure carried over unadjusted. Both candidates respect $k_5 \geq \ell_5$, which Whitney's inequality forces, so the thesis's own arithmetic cannot decide between them, and the original paper is the only thing that can. Nothing here depends on which is right: k_5 enters only as the cited first divergence between the edge and vertex problems, and that divergence holds under either constant. A reader who needs the exact value should take it from [7] directly and should note which convention that paper uses before comparing it with anything above.

At $m = 5$ the vertex problem genuinely diverges from the edge problem, visible in Figure 1.9 as the shaded gap that opens between the dashed blue and solid orange curves. Leonard had already found the first crack, an explicit 57-vertex, 141-edge counterexample to the general conjecture at $m = 5$ [8], before Sørensen and Thomassen pinned the exact value down. The reason the crack opens at all is structural. Leonard's earlier method, the one that closes $m \leq 4$, leans on the extremal graphs being assembled from cliques, vertex sets joined in all pairs, glued along shared cliques. A k-*tree* is built up one vertex at a time: start from a complete graph on $k + 1$ vertices, then again and again add a single new vertex and join it to all k vertices of some clique of size k that is already there. Figure 1.8 grows a 2-tree, hanging each new vertex on an edge already present. These clique scaffolds are exactly the building blocks the method uses. Through $m = 4$ those scaffolds are forced, and counting their edges gives the bound.

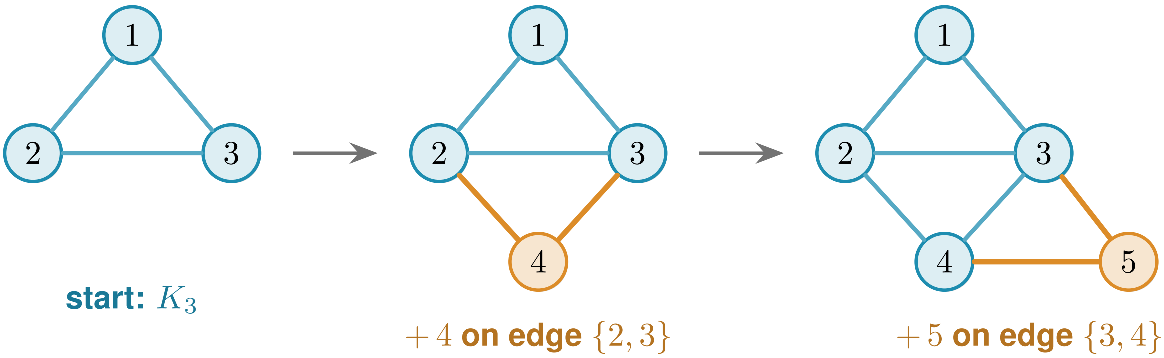


Figure 1.8. A 2-tree grown one vertex at a time, read left to right. The blue triangle on $\{1, 2, 3\}$ is the starting K_3. Vertex 4 is then joined to the two ends of the edge $\{2, 3\}$, and vertex 5 to the two ends of the edge $\{3, 4\}$ (the new vertex and its two orange edges at each step), each new vertex hung on an edge already present. A general k-tree follows the same recipe with K_{k+1} in place of the triangle and a k-clique in place of the edge. These are the clique scaffolds Leonard's count rests on.

At $m = 5$ they are no longer the unique extremal shape. Additional configurations appear, and the clean count breaks. Sørensen and Thomassen recover the exact value by a delicate structural induction, which we cite rather than reproduce. For $m \geq 6$ even that much is unknown: the vertex-disjoint problem has stood open since 1974, and the extremal structure is genuinely not understood. It is the one direction this thesis does not attempt to force, and Chapter 4 explains why.

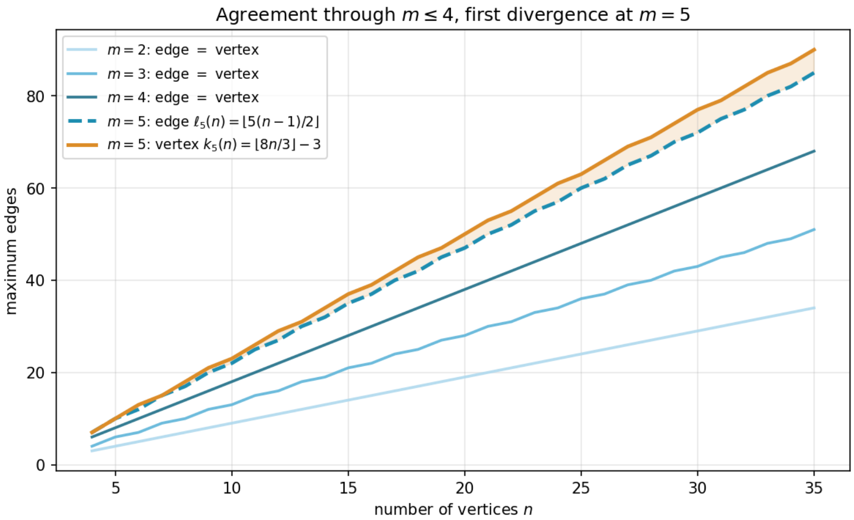


Figure 1.9. The edge and vertex extremal functions for $m = 2, 3, 4$ (blue shades, single curve per m since they agree) and $m = 5$ (dashed blue for edge, solid orange for vertex). Through $m = 4$ the two problems give identical answers, and at $m = 5$ the vertex formula $k_5(n) = \lfloor 8n/3 \rfloor - 3$ separates above the edge formula $\ell_5(n) = \lfloor 5(n-1)/2 \rfloor$, with the gap shaded.

1.6 The directed case and the quadratic phenomenon

Direction breaks a symmetry that undirected graphs never had. In an undirected graph the number of edges is tied to the connectivity through every vertex's degree. In a digraph a vertex can have many arcs in and none out, and a whole graph can lean entirely one way. Send every arc from one half of the vertices to the other and nothing comes back: between any two vertices there is at most one route, yet the number of arcs is quadratic.

Construction 1.8 (One-directional bipartite digraph)**.** Split V into parts A and B with $|A| = \lfloor n/2 \rfloor$ and $|B| = \lceil n/2 \rceil$, and include every arc from A to B. All arcs run from A to B, and none within A, within B, or from B back to A. Since B has no outgoing arcs, a vertex in A cannot reach any other vertex of A, and two vertices of B cannot communicate at all. For any $a \in A$ and $b \in B$ the only directed path from a to b is the single arc $a \to b$, so $\lambda(a, b) = 1$ and $\lambda^{\max} = 1$. The arc count is $|A| \cdot |B| = \lfloor n^2/4 \rfloor$.

Figure 1.10 draws the construction. Every arrowhead lands in B and none leaves it, so the picture is a wall of one-way traffic: from any a on the left there is exactly one way to reach any b on the right, the single arc between them, and there is no way at all to travel between two vertices on the same side. The arc count $|A| \cdot |B|$ is quadratic, yet no pair carries more than one route. This is the lopsidedness direction buys, and it has no undirected analogue. It competes with a second construction built around a single centre.

That second construction is the directed hub. It keeps each vertex's in-degree and out-degree balanced, the directed measure already met in Figure 1.5, and spends its arcs on a central vertex rather than a one-way wall.

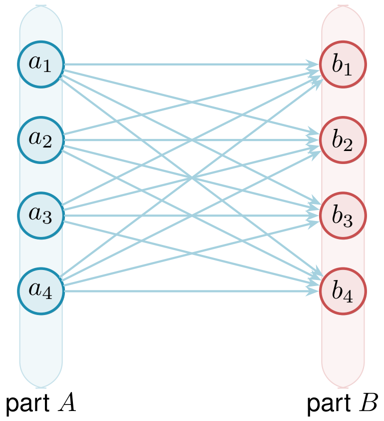


Figure 1.10. The one-directional complete bipartite digraph $B_{k,k}$ of Construction 1.8, drawn here at $n = 8$ with $|A| = |B| = 4$. Every one of the $|A| \cdot |B| = \lfloor n^2/4 \rfloor$ arcs runs left to right, from A to B. There are no arcs inside A, inside B, or back from B to A. Because B has no outgoing arcs and A no incoming ones, the only route from any a_i to any b_j is the single arc $a_i \to b_j$, so every pair has local edge-connectivity 1 while the arc count grows quadratically.

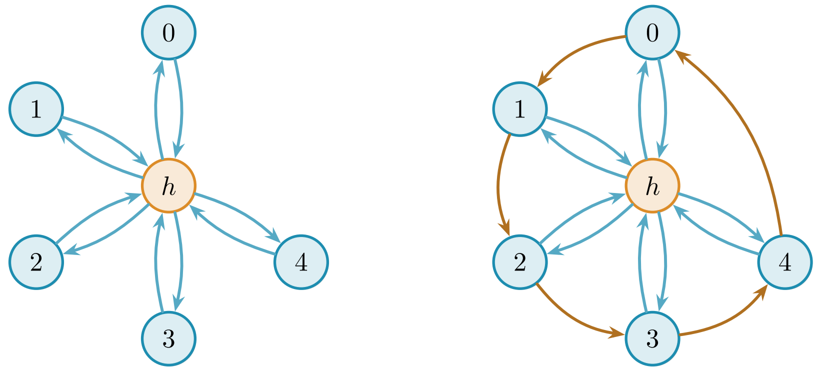
Construction 1.9 (Directed hub)**.** For $n \geq m \geq 2$, take a hub vertex h and label the remaining $N = n - 1$ vertices $0, 1, \ldots, N - 1$. Add the $2N$ hub arcs $h \to i$ and $i \to h$ for every i, and on the non-hub vertices add the circulant arcs $i \to i + j \pmod N$ for $j = 1, \ldots, m - 2$, the target label wrapping around past N back to 0 (that ring pattern is what *circulant* means, and "$\mathrm{mod}\ N$" is the wrap-around). Counting these gives $2N$ hub arcs (each of the N spokes joined to h in both directions) plus $(m - 2)N$ circulant arcs ($m - 2$ layers, one arc per spoke), so the total is $2N+(m-2)N = mN = m(n-1)$ arcs. Every non-hub vertex has $d^+ = d^- = 1+(m-2) = m-1$, and any ordered pair of vertices contains a non-hub endpoint, so the directed degree bound (Lemma A.3, which says $\lambda(u, v) \leq \min(d^+(u), d^-(v))$, a route out of u must use one of u's out-arcs and one of v's in-arcs) gives $\lambda^{\max} \leq m - 1$.

At $m = 2$ this construction is easiest to picture: the circulant layer is empty and what remains is the *bidirected star* of Figure 1.11, a hub joined to each spoke by a matched pair of opposite arcs. Every spoke can reach every other only by going in to the hub and back out, two hops, so no pair is ever joined by two independent routes, and the arc total is exactly $2(n - 1)$. For $m \geq 3$ not all $m(n-1)$ arcs can touch the hub, since a simple digraph has at most $2(n-1)$ arcs incident to one vertex: the surplus lives in the circulant layers among the spokes, tuned so that no degree exceeds the budget. The two constructions compete on different scales, linear against quadratic, and which one wins depends on n. At $m = 2$ the competition resolves exactly.

Theorem 1.10 (Directed arc value at $m = 2$)**.** *For simple digraphs,*

$$\ell_2^{\mathrm{dir}}(n) = \max\left(2(n-1),\ \left\lfloor \frac{n^2}{4} \right\rfloor\right).$$

The hub construction leads for $n < 7$, the one-directional bipartite construction leads for $n > 7$, and the two tie at $n = 7$, where both reach 12 arcs.



$m = 2$, $n = 6$: $2(n-1) = 10$ arcs $\qquad$ $m = 3$, $n = 6$: $m(n-1) = 15$ arcs

Figure 1.11. The directed hub at $n = 6$ for two values of m. *Left ($m = 2$):* the bidirected star. The hub h is joined to each spoke by one arc in each direction, giving $2(n-1) = 10$ arcs. No circulant layer is added, so all spoke communication routes through h, so $\lambda^{\max} = 1$. *Right ($m = 3$):* a single circulant layer is added among the five spokes. The blue arcs are the same hub-spoke pairs as on the left. The orange arcs are that one circulant layer, the cycle $0 \to 1 \to 2 \to 3 \to 4 \to 0$, so each spoke has $d^+ = d^- = m - 1 = 2$. The total is $m(n-1) = 15$ arcs and $\lambda^{\max} = 2$. For $m \geq 3$ the spokes are no longer isolated from one another, and tracing all the routes becomes the argument that fills the rest of the chapter.

The lower bound is the better of the two constructions. The upper bound is where the trouble starts.

Idea of the upper bound at $m = 2$. The value $M(n) = \max(2(n-1), \lfloor n^2/4 \rfloor)$ is a contest between a linear term and a quadratic one, and which leads is pure arithmetic: $2(n-1) \geq \lfloor n^2/4 \rfloor$ precisely for $n \leq 7$, with equality at $n = 7$, where both give 12. So M follows the bidirected star up to the crossover and the bipartite digraph past it, and $n = 7$ is the single value where the two constructions tie. The proof that no digraph beats $M(n)$ is an induction on n (Appendix A). Delete a vertex of *minimum* total degree: since the degrees sum to twice the arc count, an averaging argument bounds that degree by $2a/n$, where a is the number of arcs, and the remaining digraph on $n - 1$ vertices still respects the ceiling, so the inductive bound applies to it. Feeding the two facts together gives $a \leq \frac{n}{n-2} \lfloor (n-1)^2/4 \rfloor$, and a short parity check shows the right-hand side never exceeds $\lfloor n^2/4 \rfloor$ once $n \geq 8$: for even n it lands exactly on the bipartite value, and for odd n a sub-unit slack is swallowed by the fact that a is an integer. The averaging closes the step with no separate hub case, which is why the difficulty is not depth but bulk. Nothing in the argument is deep. It is a careful walk through two parity regimes, with the base cases $n \leq 7$ settled by exhaustive computer search rather than by hand. That experience of a correct but long and almost entirely mechanical case-check is exactly what motivates the chapters that follow. One genuinely wants to hand the case-checking to a machine.

The vertex-disjoint version of the same question has the same answer at $m = 2$, but it is worth being careful about why, because Whitney's inequality is easy to read backwards here. Since $\kappa \leq \lambda$ pairwise, a digraph that respects the arc ceiling automatically respects the vertex one, so the vertex-feasible family *contains* the arc-feasible one and is in general strictly larger. The free consequence therefore runs one way only: the two constructions above keep $\kappa^{\max} = 1$

as readily as $\lambda^{\max} = 1$, which makes them lower bounds for the vertex problem too. No upper bound comes with them. What settles the vertex value is that the same induction goes through unchanged, because deleting a vertex lowers $\kappa^{\max}$ for exactly the reason it lowers $\lambda^{\max}$, and because the base cases $n \le 7$ were re-run under the stricter separation and returned the same six numbers. Appendix A gives both halves, and Remark A.16 there spells out the direction trap in full.

Theorem 1.11 (Directed vertex value at $m = 2$)**.** *For simple digraphs,*

$$k_2^{\mathrm{dir}}(n) = \max\left(2(n-1),\ \left\lfloor \frac{n^2}{4} \right\rfloor\right) = \ell_2^{\mathrm{dir}}(n).$$

1.7 The failure of direct proof for $m \ge 3$

For $m \ge 3$ the induction no longer closes, and the natural guess is wrong. The natural initial conjecture was that the two constructions above continued to be the whole story, giving $\max(m(n-1), \lfloor n^2/4 \rfloor)$. A single graph on ten vertices refutes it.

Remark 1.12 (A thirty-arc counterexample). Take $A = \{a_1, \dots, a_5\}$ and $B = \{b_1, \dots, b_5\}$ with all 25 arcs from A to B, and add a directed five-cycle $b_1 \to b_2 \to b_3 \to b_4 \to b_5 \to b_1$ inside B. This digraph has 30 arcs. Every vertex of B has exactly one extra in-arc from inside B, so two arc-disjoint routes from a_i to b_j exist: the direct arc $a_i \to b_j$, and the detour $a_i \to b_{j-1} \to b_j$ using the 5-cycle predecessor of b_j (indices mod 5). For the matching upper bound, the only in-arcs of b_j reachable on a path from a_i are the direct arc $a_i \to b_j$ and the cycle arc $b_{j-1} \to b_j$. Removing both disconnects a_i from b_j, while neither alone does, so the minimum cut has size 2. Hence $\lambda(a_i, b_j) = 2$. Routes between two vertices of B never leave B and give $\lambda(b_i, b_j) = 1$. Hence $\lambda^{\max} = 2$, so the graph is feasible at $m = 3$, yet $30 > \max(3 \cdot 9,\ 25) = 27$.

Figure 1.12 draws this thirty-arc digraph and traces the mechanism. The twenty-five faint arcs are the complete one-directional bipartite layer from A to B, the construction we already understand. On top of them the orange five-cycle gives each vertex of B a single extra in-arc from inside B. That one extra arc is exactly what doubles the connectivity. For the marked pair a_1, b_2 the two heavy routes show how: the blue arc is the direct hop $a_1 \to b_2$, and the green path $a_1 \to b_1 \to b_2$ borrows the cycle arc $b_1 \to b_2$ for its second step. The two share no arc, so $\lambda(a_1, b_2) = 2$, and the same pair of routes exists for every a_i, b_j. Five extra arcs lift the whole digraph from one independent route per pair to two, and the thirty arcs overshoot the old prediction of twenty-seven.

The counterexample generalises: the five-cycle gave every vertex of B exactly one extra in-arc from inside B, and the budget for such arcs is $m - 2$ per vertex, not one. Figure 1.12 is the instance $m = 3$, where that budget is one and a single cycle suffices. For larger m each b_j collects $m - 2$ cyclic predecessors and the same picture acquires more concentric chords inside B.

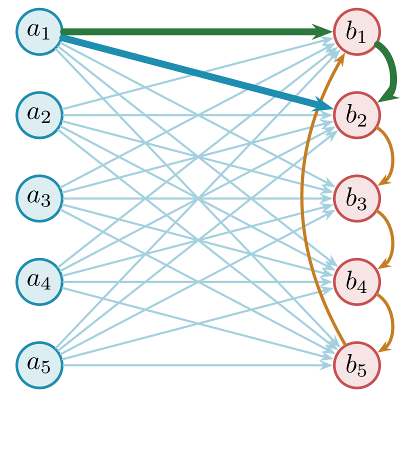


Figure 1.12. The thirty-arc counterexample of Remark 1.12: the augmented bipartite digraph (Construction 1.13) at $m = 3$, $n = 10$, $|A| = |B| = 5$. The 25 faint arcs are the complete one-directional layer from A to B, and the 5 orange arcs form the directed cycle $b_1 \to b_2 \to b_3 \to b_4 \to b_5 \to b_1$ inside B, one extra in-arc per vertex of B. The heavy blue arc and the heavy green detour show the pair a_1, b_2 carrying 2 arc-disjoint routes: the direct arc $a_1 \to b_2$ (blue) and the two-step detour $a_1 \to b_1 \to b_2$ (green) through the cycle predecessor b_1. Hence $\lambda^{\max} = 2$, feasible at $m = 3$, while $30 > \max(3 \cdot 9,\, 25) = 27$.

Construction 1.13 (Augmented bipartite digraph)**.** For $m \geq 2$ and $n \geq m$, set $|B| = \lceil (n + m - 2)/2 \rceil$ and $|A| = n - |B| = \lfloor (n - m + 2)/2 \rfloor$, include every arc from A to B, and inside B give each vertex b_j the $m - 2$ in-arcs from its cyclic predecessors $b_{j-1}, \ldots, b_{j-(m-2)}$ (indices mod $|B|$). The total is $\lfloor (n + m - 2)^2/4 \rfloor$ arcs.

For the connectivity, fix $a \in A$ and $b \in B$: the direct arc and the $m - 2$ two-step detours $a \to b' \to b$ through the in-neighbours b' of b inside B are $m - 1$ arc-disjoint routes, and they are all there are, because the only in-arcs of b reachable from a are exactly those $m - 1$ arcs, which therefore form a cut. Pairs inside B are limited by the in-degree $m - 2$, pairs inside A and pairs from B to A have no routes at all, so $\lambda^{\max} = m - 1$.

At $m = 2$ the inside of B is empty and the construction reduces to Construction 1.8, with the balanced partition $(\lfloor n/2 \rfloor, \lceil n/2 \rceil)$. At $m = 3$ the formula gives $|B| = \lceil (n + 1)/2 \rceil$, which coincides with $\lceil n/2 \rceil$ at odd n but gives $|B| = n/2 + 1$ at even n. Both the balanced and the shifted partition achieve the same arc count $\lfloor n^2/4 \rfloor + \lceil n/2 \rceil$ in this case, so the two are interchangeable at $m = 3$. The shift to a strictly larger B is structurally significant only at $m \geq 4$, where the balanced partition is suboptimal. At $m = 3$, $n = 10$ it is the thirty-arc counterexample of Remark 1.12, drawn in Figure 1.12 (using the equivalent balanced partition $|A| = |B| = 5$).

The two rival shapes invite a political picture. The hub is a single king through whom all traffic must pass, cheap to build but limited by the one throne. The bipartite family is a flat cabinet of ministers, each handling its own dossier with no central seat, and it scales quadratically. Which arrangement packs more arcs under the ceiling depends on n, and for $m \geq 3$ the cabinet overtakes the king once n is large. This is the corrected shape, and it leads to the standing conjecture for the directed arc problem.

Conjecture 1.14. *For simple digraphs and all $n \geq m \geq 2$,*

$$\ell_m^{\mathrm{dir}}(n) = \max\left(m(n-1),\ \left\lfloor \frac{(n+m-2)^2}{4} \right\rfloor\right).$$

The hypothesis $n \geq m$ is the one both constructions already carry, and it is needed for the same reason as in Theorem 1.5: below it the complete digraph is feasible and the answer is the trivial $n(n-1)$, which the formula exceeds. At $n = 3$, $m = 4$ the formula gives 8 where three vertices carry only 6 arcs. The exhaustive search confirms the value is 6 there.

For $m = 2$ this is Theorem 1.10. For $m = 3$ the formula $\lfloor (n+1)^2/4 \rfloor$ equals $\lfloor n^2/4 \rfloor + \lceil n/2 \rceil$, so the conjecture reduces to the previous formula for $m = 2$ and $m = 3$. For $m = 3$, $n = 10$ the second branch gives $\lfloor 121/4 \rfloor = 30$, matching Remark 1.12. For $m \geq 4$ the new formula and the old balanced-partition count $\lfloor n^2/4 \rfloor + (m-2)\lceil n/2 \rceil$ no longer agree: the quadratic branches already differ (for $m = 4$ the new one is larger by exactly one at every even n), but the hub term $m(n-1)$ dominates both at small n, so the full conjectured value first overtakes the old one at $n = 12$ for $m = 4$. The construction of Construction 1.13 with the shifted partition witnesses the difference. The lower bound holds for all m, witnessed by Construction 1.9 on the first branch and Construction 1.13 on the second. The upper bound is open, and Chapter 4 returns to it with what structural progress the machine and the hand can jointly supply.

1.8 The hypergraph variant

The third model deserves its own short treatment, not because its base case is hard but because it is the one variant whose objects are stored and manipulated differently from all the rest, and it is worth meeting that difference now rather than at the moment of computation. A hypergraph keeps no multiplicity matrix, only the list of its hyperedges, each a set of r vertices. We fix the edge size r, following the extremal literature, for a structural reason: under a connectivity ceiling a small edge is always at least as efficient per route as a large one, so a hypergraph free to mix sizes would maximise its edge count by using pairs only, and the question would collapse back to the graph case. Fixing r is what makes the hypergraph question a new question.

The route between two vertices is a *Berge path*, which alternates vertices and hyperedges so that each consecutive pair of vertices lies in the hyperedge listed between them, and two such routes are edge-disjoint when they share no hyperedge. Concretely, in a three-uniform hypergraph a route from u to w might be the single step $u, \{u, x, w\}, w$ that crosses one hyperedge, or a longer alternation $u, \{u, x, y\}, y, \{y, z, w\}, w$ that switches hyperedge at the intermediate vertex y.

With routes redefined this way the same Gomory–Hu machinery that proved the multigraph value carries over, and the base case has the same linear shape. The picture to keep in mind is a metro map: each hyperedge is a line, and a route from one vertex to another rides a line and changes to another line at a station the two share.

A single hyperedge may serve only one route, so two Berge routes that share no hyperedge are edge-disjoint, the hypergraph counterpart of two edge-disjoint paths in a graph. Reading routes off a metro map is a matter of seeing which lines share track: two hyperedges that share several vertices run side by side along the stretch between them, as in Figure 1.3, and two routes are hyperedge-disjoint exactly when the lines they ride share no track at all.

The separation axis needs its own definition here, and it is worth fixing now because the natural guess is not the one this thesis uses. Two Berge routes are called *internally disjoint* when they share neither an intermediate vertex nor a hyperedge. Both halves are required. Demanding only that the routes avoid each other's intermediate stations would let two routes ride the same line, and a line that can run only one train at a time cannot serve two routes at once, so the resulting count would not correspond to anything the metro map can carry. Requiring both is also what makes the vertex measure the exact analogue of the graph one under the translation of Appendix A, where a hypergraph becomes its incidence graph, each hyperedge becomes a node, and internally disjoint routes become internally disjoint paths in the ordinary sense. It has the pleasant side effect that Whitney's inequality survives the move to hypergraphs by definition, since an internally disjoint family is in particular hyperedge-disjoint. Write $\kappa_{\mathcal{H}}(u,v)$ for the largest such family and $\kappa_{\mathcal{H}}^{\max}$ for its maximum over pairs.

Proposition 1.15 (Hypergraph edge bound)**.** *An r-uniform hypergraph on n vertices with Berge edge-connectivity at most $m-1$ has at most $\left\lfloor \frac{(m-1)(n-1)}{r-1} \right\rfloor$ hyperedges. A simple hypergraph attains the bound whenever $m-1 \leq \binom{n-2}{r-2}$, and a multihypergraph attains it whenever $(r-1) \mid n-1$.*

The proof, given in Appendix A, runs the multigraph argument through the hypergraph Gomory–Hu theorem of Bérczi, Chandrasekaran, Király and Kulkarni [9], charging each hyperedge to the at least $r-1$ tree cuts it crosses. The star hypertree (Figure 1.13) attains the $m=2$ case, a hub joined to disjoint blocks of $r-1$ vertices, with Berge connectivity one everywhere and $\lfloor (n-1)/(r-1) \rfloor$ hyperedges, exactly as a tree attained the multigraph bound. One genuine surprise is worth recording here: for $r \geq 3$ the bound is attained by *simple* hypergraphs, with no repeated hyperedge (Theorem A.42). In the graph case simplicity is exactly what separates Mader's $\lfloor m(n-1)/2 \rfloor$ from the multigraph value $(m-1)(n-1)$. One edge size higher, it costs nothing.

The proposition counts hyperedges, but it does not say how to *measure* the Berge connectivity of a hypergraph we are handed, and that measurement is the genuinely new part. The metro picture shows why. An ordinary edge is a one-stop link between a single pair, so policing edge-disjointness means watching each pair on its own. A hyperedge is a whole line: it stops at all r of its vertices and so touches $\binom{r}{2}$ pairs at once, yet like a line that can run only one train at a time it may serve only one route. When several routes want to board the same line at different stations, the checker must wave exactly one through and turn the rest away, and it must do this for every

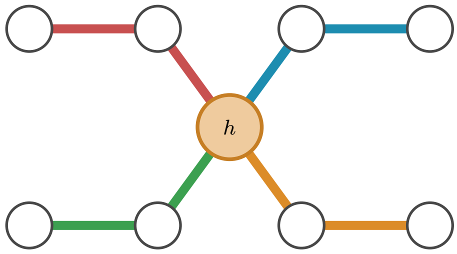


Figure 1.13. The star hypertree that attains the $m = 2$ hypergraph bound, here $r = 3$ and $n = 9$, drawn as a metro map with the hub h centred between two rows of stations. The single hub station lies on every line, and each of the $\lfloor (n-1)/(r-1) \rfloor = 4$ hyperedges is one metro line of $r = 3$ stations: it leaves the hub diagonally to an inner station and then runs horizontally to its outer station, hanging a fresh block of $r - 1 = 2$ vertices off the hub. Because every line meets every other only at h, no pair has two hyperedge-disjoint routes, so the Berge connectivity is one everywhere, exactly as a spanning tree attained the multigraph bound.

line simultaneously. That bookkeeping is the one new piece of machinery the next chapter builds, and it is why the hypergraph rejoins the family only once the checker is in place.

1.9 Random graphs as intuition

The short proofs above hinted that the edge and vertex problems travel together for a while, agreeing through $m = 4$ before the split at $m = 5$. That separation is a statement about extremal graphs, the densest ones the ceiling allows, and it is worth asking whether the same split can be seen in ordinary graphs drawn at random, with no extremal tuning at all. The random model is the natural place to build intuition, because it shows the qualitative behaviour of the connectivity notions before any construction is pushed to its limit, and we will return to it whenever a picture of the typical graph clarifies a result. It is not a detour from the program to come, either: a sampled $G(n, p)$ is an ordinary graph, measured by the very same connectivity checker the next chapter builds for every other variant, so the random model is simply the one pipeline turned to *observe* the typical case rather than to prove or to discover the extremal one.

Each of the $\binom{n}{2}$ possible edges of $G(n, p)$ is included independently with probability p, and on the resulting graph we read off both binding connectivities, the edge version $\lambda_G^{\max}$ and the vertex version $\kappa_G^{\max}$, from the very same sample. Whitney's inequality $\kappa_G^{\max} \leq \lambda_G^{\max}$ guarantees the vertex value never exceeds the edge one, but it is silent on how often the two actually come apart.

Figure 1.14 looks directly at the distributions, fixing $n = 16$ and histogramming both connectivities over many samples at three densities. Two things are visible at once. As the density rises the whole distribution marches rightward, from a binding connectivity around five at $p = \frac{1}{4}$ to around thirteen at $p = \frac{3}{4}$, so by $n = 16$ the typical graph already sits well past the $m = 4$ at which the proofs agreed. And comparing the edge row to the vertex row, the vertex distribution sits at or just to the left of the edge one, never to its right, which is Whitney's inequality seen across a whole distribution rather than a single graph.

The two pull apart on a fraction of samples that shrinks as the density grows, from about a fifth of the samples at $p = \frac{1}{4}$ to only a handful by $p = \frac{3}{4}$. The reason is structural. For $\kappa^{\max}$ to fall below $\lambda^{\max}$ some pair must carry more edge-disjoint routes than internally vertex-disjoint ones, which happens only when several of those routes are forced through one shared intermediate vertex. In a sparse graph routes are scarce and a single low-degree vertex can be exactly such a bottleneck, but as p grows every vertex gains degree and the graph offers many alternative intermediate vertices, so independent routes almost never need to reuse one. In the dense limit the graph approaches the complete graph, where $\kappa^{\max} = \lambda^{\max} = n - 1$ for every pair and the gap closes entirely. The divergence the thesis records at $m = 5$ is therefore not an artefact of the extremal construction. It is already there, in miniature, in the random model.

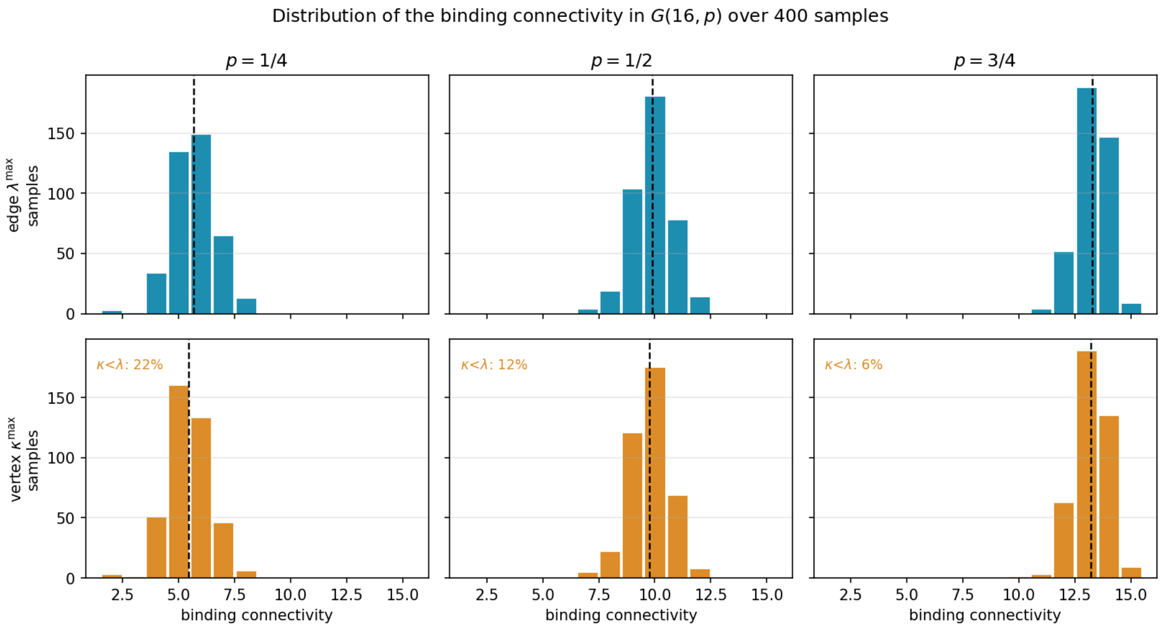


Figure 1.14. Distributions of the binding connectivity in $G(16, p)$ over 400 samples, at densities $p = \frac{1}{4}, \frac{1}{2}, \frac{3}{4}$. There are $2^{\binom{16}{2}} = 2^{120}$ labelled graphs on 16 vertices, far too many to enumerate, so these are Monte Carlo samples rather than the full population, and 400 is plenty to show the shape of each distribution. The top row histograms the edge-connectivity $\lambda^{\max}$, the bottom row the vertex-connectivity $\kappa^{\max}$, measured on the same samples, with the mean marked by a dashed line and the fraction of samples on which $\kappa^{\max} < \lambda^{\max}$ printed on each vertex panel. Reading a column top to bottom, the vertex distribution sits at or to the left of the edge one, which is Whitney's inequality, and the gap is largest at the sparsest density (22% of samples) and closes as p grows (6%). All six panels share one axis range.

The random model also supplies a landmark of a different flavour. Below a sharp density the forbidden configuration is almost surely absent, and above it almost surely present, and the location of that threshold is the same for every one of the connectivity notions above.

Theorem 1.16 (Appearance threshold)**.** *Let $m = m(n) \geq 2$ grow with n so that $m/\ln n \to \infty$ and $m = o(n)$, and let $c > 0$ be a constant. In the binomial random graph $G(n, p)$ with $p = c \cdot m/n$,*

$$\lim_{n\to\infty} \Pr\left[\lambda^{\max} \geq m\right] = \begin{cases} 1 & \textit{if } c > 1, \\ 0 & \textit{if } c < 1, \end{cases}$$

so the threshold for the appearance of a pair with local edge-connectivity at least m is $p^* = m/n$. The same threshold governs local vertex-connectivity and the directed analogues (Remark A.59). It does not govern the hypergraph models, which live on a different density scale (Remark A.60).

The proof is a concentration argument with two halves (Appendix A). Above the threshold the edge count concentrates past Mader's bound, so Theorem 1.2 itself forces a high pair into existence. Below it the maximum degree stays under m, and the degree bound caps every local connectivity. The two hypotheses earn their keep. The first, $m/\ln n \to \infty$, says m must outgrow the natural logarithm of n, and that is what lets a union bound over the n vertices close, the union bound being nothing more than adding up the n individual failure chances and needing even their sum to vanish. The second, $m = o(n)$, says m stays negligible next to n, which keeps the density $p = cm/n$ below one and in the sparse regime where the threshold lives. Two caveats on scope, and the second is the one that limits how far the result may be quoted. The upper half quoted here uses Mader's bound, which is the simple undirected edge case, so it is that variant the proof settles directly. The vertex-disjoint and directed analogues sit at the same threshold $p^* = m/n$ but reach it through their own extremal bounds rather than Mader's, possibly with a different leading constant, as Remark A.59 records. The hypergraph models are a different matter and are *not* covered. A threshold of this kind sits where a vertex's expected degree reaches m, and in a binomial r-uniform hypergraph a vertex lies in an expected $p\binom{n-1}{r-1}$ hyperedges, so the balance point is $m/\binom{n-1}{r-1}$, smaller than m/n by a factor of order n^{r-2} once $r \geq 3$. Remark A.60 works this out and Figure B.5 plots each model against its own scale. So the collapse of complexity is real but bounded: within a fixed edge size the distinctions between edges and vertices and between the two directions wash out and one density decides the matter, while changing the edge size changes the density at which everything happens.

One limitation must be stated plainly, lest the threshold be read as evidence about the problem this thesis actually studies. The growth hypothesis $m/\ln n \to \infty$ forces m to climb with n, so the theorem says nothing whatever about a fixed small m, the regime of every exact value above, where m is 2, 3, or 4 and n runs to infinity around it. There the union bound is vacuous and the appearance of a dense pair is governed by rare low-degree configurations that need finer, Poisson-type tools. The random model therefore enters as contrast and not as evidence: it shows how cleanly the question collapses in one limit, which throws into relief how stubbornly it resists in the fixed-m limit the rest of the thesis must attack by hand and by machine.

1.10 Where this work sits

It is worth pausing, before the machine, to place the question and the method against the two traditions they descend from: the extremal graph theory that posed the problem and the computational mathematics that will help settle it.

The mathematical lineage is the older of the two. The problem belongs to the family of extremal connectivity questions opened by Bollobás and Erdős, who asked how dense a graph can be while every pair of vertices stays below a fixed connectivity [1], logged today as Erdős Problem 915 [3]. Mader's theorem [4] closed the simple undirected edge case with the clean floor $\lfloor m(n-1)/2 \rfloor$ recalled in Theorem 1.2, and it is the floor every variant in this thesis is measured against. The vertex-disjoint branch has a longer and rougher history: Leonard [6] showed the edge and vertex problems coincide through $m \leq 4$, and Sørensen and Thomassen [7] found the first divergence at $m = 5$ with $k_5(n) = \lfloor 8n/3 \rfloor - 3$, after which the exact values $k_m(n)$ remain unknown. That gap between a settled edge case and a stalled vertex case is exactly the terrain Chapter 1 has been mapping, and the directed and hypergraph axes extend it further. The thesis adds to this lineage not a single new closed form but a uniform way of pushing each axis as far as exact computation allows.

The computational tradition this thesis's pipeline belongs to is the practice of settling combinatorial questions by machine in a way a human can audit, and it splits into a few schools worth contrasting with the certify-then-prove loop built next. The oldest is *exhaustive generation*: enumerate every object of a given size up to isomorphism and check each one. McKay and Piperno's `nauty`/`geng` toolkit [10] is the standard engine for this, and its flagship results include exact small Ramsey numbers such as $R(4, 5) = 25$ [11]. The enumeration of Chapter 2 is the same idea, run on the twelve variants of Problem 915. The second school is *SAT-assisted proof*, in which a question is encoded in propositional logic and a solver emits a machine-checkable certificate: Heule, Kullmann and Marek's resolution of the Boolean Pythagorean triples problem [12], whose verified proof runs to some two hundred terabytes, and Heule's determination of Schur number five [13] are the canonical examples of certified solver output at scale. The third is *flag algebras*, Razborov's framework [14] that converts asymptotic extremal bounds into semidefinite-programming certificates, the dominant tool for density problems in the large-n limit. Closest of all to the method here is the fourth, *certification by linear and integer programming* (LP/ILP), where a feasibility or optimality argument over the rationals or integers is written as a linear program, an optimisation with linear constraints, and the proof is the certificate of optimality the solved program hands back, a short list of numbers anyone can verify by direct arithmetic. This is precisely the form the cut-counting certifier of Chapter 2 takes.

Against that backdrop, what is distinctive here is consolidation rather than a new solver. A single checker measures connectivity exactly across all twelve problem variants on one data model, so search and certification share one codebase rather than living in separate tools. The certificates it emits are exact rational or integral objects, not floating-point estimates, so an upper bound it reports is a proof and not evidence. And because the same program both hunts for dense graphs and certifies the bounds they suggest, the discover step and the prove step are two modes of one loop, which is the arrangement the next chapter builds. What makes the consolidation honest is that it never blurs three kinds of knowledge: a value can be *proved* (by hand, or by exhausting every graph of a size), *certified* (by the same optimisation, reaching a vertex or two further), or only *conjectured* (witnessed by a dense graph the search exhibits, with no matching upper bound). This trichotomy, named in Section 3.6 and obeyed by the summary of Chapter 4, is the through-line of everything that follows.

Computer note

We are now stuck in three different ways: a fifty-year-old open problem at $m \geq 6$ for vertices, a long proof we would rather not check by hand at $m = 2$ for arcs, and a conjecture with no proof at all at $m \geq 3$. The base cases of the long induction were verified by exhaustive search, and the thirty-arc counterexample was found the same way. The next chapter builds a single program that models all of these cases at once, measures their connectivity exactly, and proves the small-case upper bounds outright.

Chapter 2

Certifying Bounds by Machine

The last chapter ended in three different kinds of stuckness, and all three share a shape. The objects are graphs of one flavour or another, the constraint is always a ceiling on connectivity, and the work is always either measuring a given graph or checking many small ones. That shared shape is what a single program can exploit. This chapter builds it, and then uses it for one purpose only: to *prove*. It gives every variant one representation, measures connectivity exactly through a max-flow checker, checks small cases by honest enumeration, and turns the finite checks the proofs of Chapter 1 relied on into a single auditable certifier. The hunt for dense examples, which proves nothing on its own, waits until Chapter 3.

A word on what the program is for. It is a microscope, not a source of final answers. It measures connectivity exactly, so what it reports about a given graph is a fact. It enumerates graphs of a fixed size, so for that size it can prove a theorem. It will later discover dense graphs, which are only lower bounds, but not here. Each of these has a different epistemic weight, and the program is built so that the three never get confused. Figure 2.1 draws this spine: a single *model* holds every variant, one exact *measure* reads connectivity off it, and from that measurement a *prove* step bounds a fixed size from above, with the *discover* step of the next chapter and a final *observe* step for the random model added later. Every routine introduced from here on is shown as a small card carrying its name, its inputs and output, a one-line account of how it works, and a coloured badge naming its place in the spine. The function bodies are not reproduced, because the point is the interface, not the code.

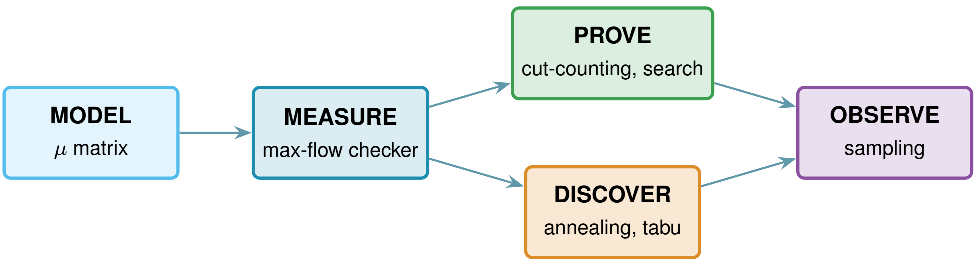


Figure 2.1. The architecture as one pipeline. A single data model, the multiplicity matrix μ, feeds an exact measurement, the max-flow checker. That measurement feeds the two bound-producing roles: a prover for upper bounds (the cut-counting optimisation and the exhaustive search), built in this chapter, and a discoverer for lower bounds (simulated annealing, or tabu search), built in Chapter 3. A final sampling role studies the random model. The badge colours on the code cards below match the boxes here.

2.1 The solver

Everything in this chapter is a single function, `solve`, and it helps to hold its most basic version in mind from the start, because every later idea is a refinement of it and nothing is ever a separate program for a separate case. Given n and m, the basic strategy is as plain as it sounds. Walk through every graph on n vertices, measure the largest local connectivity $\lambda^{\max}$ of each with a max-flow computation, and keep the densest graph that stays at or below the ceiling $m-1$. Because it has looked at every graph, the count it returns is the exact answer, not an estimate.

That one loop already settles the smallest cases, and the rest of the chapter is built on top of it without ever leaving `solve`. We first make the measurement step trustworthy and quick, via the connectivity checker and a Gomory–Hu shortcut for undirected graphs. We then make the enumeration step skip whole families of hopeless graphs, via a pruned search. Finally, for the directed multigraph case, we replace enumeration altogether with one small optimisation that proves every m at once, which is the certifier. Each of these is the same `solve` doing less work for the same guarantee, and the choice among them is made automatically from the booleans it receives.

`solve(n, m, directed, simple, separation, exhaustive, max_seconds)` **DRIVER**

in the variant booleans `directed` and `simple` (or `hypergraph` with uniformity `r`), the `separation` (edge or vertex), n, m, the `method`, and a time budget `max_seconds`

out a `SolveResult` carrying a value with an honest label: exact, lower bound, or upper bound

how picks the method the case allows: brute-force enumeration, the pruned search, or the cut-counting certifier when proving, or the temperature search or tabu search when discovering (Chapter 3). It always respects the budget, and it is the one function the rest of the thesis ever calls.

2.2 A single representation for all variants

Two of the three axes from Chapter 1, the model and the direction, are properties of the object, and a single data structure holds almost all of them. Of the three structural models, simple graphs and multigraphs fit the matrix directly, while the hypergraph keeps the separate storage promised in Chapter 1 and rejoins at the end of this chapter. The hypergraph could in principle be pressed into the same shape as a higher-order tensor, an r-dimensional array marking which r-sets are present, but that array is almost entirely zero and costs n^r entries, so the program keeps the compact list of hyperedges instead and pays nothing for the empty cells. Sampled $G(n,p)$ graphs are ordinary simple graphs and therefore use this same representation without any change. A graph or digraph on n vertices is stored as an integer matrix μ, where $\mu(u,v)$ is the multiplicity of the edge or arc from u to v. A simple graph is the case $\mu \in \{0,1\}$, an undirected graph satisfies $\mu = \mu^\top$, and a multigraph allows larger entries. The third axis, edge against vertex separation, is a property of the question rather than the object, so it lives in how we measure μ, not in μ itself. Figure 2.2 illustrates the encoding.

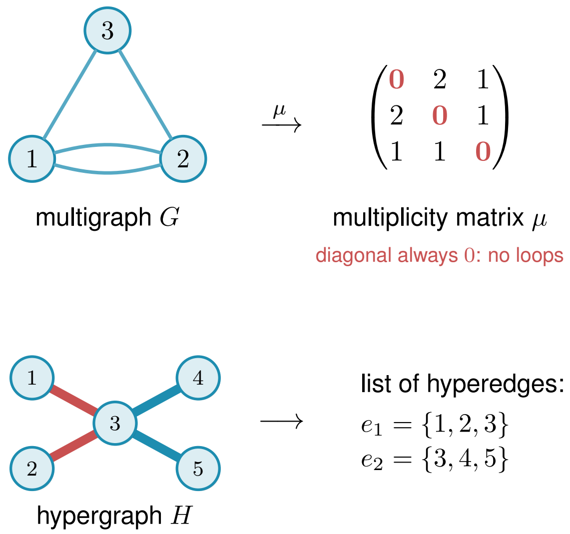


Figure 2.2. How the program stores a graph. Top: a graph or digraph on n vertices is one $n \times n$ integer matrix μ, with $\mu(u, v)$ the number of parallel edges or arcs from u to v. The double edge between 1 and 2 gives $\mu(1, 2) = 2$. The diagonal is always zero (red), since no vertex carries an edge to itself, an undirected graph keeps $\mu = \mu^\top$, a directed graph allows $\mu(u, v) \neq \mu(v, u)$, and a simple graph caps every entry at one. Bottom: the hypergraph is the one model that does not use the matrix at all, since a matrix has no room for an edge on three vertices, so it is kept as a plain list of its hyperedges, here two metro lines that share vertex 3.

`Graph(n, Variant(directed, simple))` **MODEL**

in the vertex count n and a `Variant`, the small record holding the two booleans

out a graph whose whole state is the matrix μ, stored as the `numpy` array `self.mu`

how one $n \times n$ integer `numpy` array is the object, an undirected graph keeps $\mu = \mu^\top$, and a simple graph caps every entry at one.

Keeping a single representation is what lets one measurement routine, one certifier, and one search serve every case. Nothing downstream asks which of the twelve problems it is solving. It asks only the two booleans (directed or undirected, simple or multi) and the matrix. Everything that builds or edits a graph is a thin wrapper around μ that respects those booleans, and these operations are so routine that they need no individual exposition, and Table 2.1 lists them together.

The two booleans are all the mirroring above needs, and the whole trick fits in one short routine. Every write to μ passes through it, so this is the single place the undirected symmetry is kept, not an invariant a caller has to remember to maintain.

```
def _assign(self, u, v, value):
    self.mu[u, v] = value
    if not self.variant.directed:
```

Operation	Meaning
`addEdge(G, u, v, k)`	add k parallel edges or arcs $u \to v$ (a simple graph saturates at one)
`removeEdge(G, u, v, k)`	remove k copies, never below zero
`setMultiplicity(G, u, v, c)`	set $\mu(u, v) = c$ directly
`multiplicity(G, u, v)`	read $\mu(u, v)$
`hasEdge(G, u, v)`	whether $\mu(u, v) > 0$
`degree(G, v)`	total degree of v (in- plus out-degree if directed)
`edgeCount(G)`	number of edges or arcs, counted with multiplicity
`edges(G), vertices(G)`	iterate the present edges, and the vertex set
`copy(G)`	an independent duplicate

Table 2.1. The routine operations on the graph model. Each is a one-line wrapper that reads or writes the matrix μ while keeping an undirected graph symmetric and a simple graph binary. They are bookkeeping, and the chapter's attention belongs to the routines that follow.

```
        self.mu[v, u] = value    # keep undirected graphs symmetric
```

Listing 2.1. How every write keeps an undirected graph symmetric. A directed graph simply skips the mirrored write.

2.3 The connectivity checker built on maximum flow

The whole question reduces to a flow computation. A *maximum flow* asks how much can be pushed from one point to another through a network whose links each carry a limited capacity. Menger's theorem says the most edge-disjoint u–v paths a graph offers equals its smallest u–v cut, the fewest links whose removal severs every route between the two. The max-flow / min-cut theorem says that smallest cut is exactly the value of a maximum flow, once each multiplicity $\mu(u, v)$ is read as the capacity of the link $u \to v$. So the route count we are after is just a maximum-flow value: measure the flow and you have measured the connectivity. The routine that runs this flow and reports the route count is the connectivity *checker*. It is the single most important routine in the program: it is the only place graph theory meets computation, and every later claim about connectivity passes through it. Because the search of Chapter 3 calls it by the million on tiny graphs, the program ships an optional compiled C helper that runs this same flow (named later in this chapter's reproducibility section). It is a speed-up only: it returns the identical value the pure-Python checker does, verified against it, so no number in the thesis depends on whether the helper is built.

`local_edge_connectivity(G, s, t)` $\to \mathbb{N}$ **MEASURE**

in a graph G and an ordered pair (s, t)

out $\lambda_G(s, t)$, the maximum number of edge-disjoint s–t paths

how a single `scipy.sparse.csgraph.maximum_flow` on the network whose arc capacities are the multiplicities μ, so by Menger the flow value is the path count.

`max_edge_connectivity(G)` $\to \mathbb{N}$ **MEASURE**

in a graph G

out $\lambda^{\max}(G)$, the largest local edge-connectivity over all pairs

how one max-flow per pair (ordered if directed), take the largest. This is the quantity the edge problem caps at $m-1$.

The vertex-disjoint version follows the same idea after one construction: split each vertex v into an in-copy v^{in} and an out-copy v^{out} joined by a single unit-capacity arc (Figure 2.3). Every arc $u \to v$ of the original graph becomes $u^{\text{out}} \to v^{\text{in}}$ in the split network. The construction is not special to digraphs: for an undirected graph each edge $\{u, v\}$ is first read as the two opposite arcs $u \to v$ and $v \to u$, and then the same split applies, so one routine serves the undirected and directed vertex problems alike. Routing through v now consumes the one unit of capacity on its internal arc, so two paths sharing v as an intermediate vertex compete for that unit and cannot coexist in any maximum flow. A max-flow from u^{out} to v^{in} in the split network therefore equals the number of internally vertex-disjoint u–v paths in the original graph.

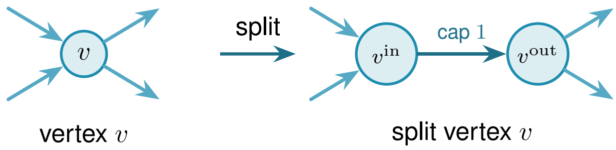


Figure 2.3. Vertex splitting for the vertex-disjoint checker. Incoming arcs attach to v^{in} and outgoing arcs leave from v^{out}, connected by a single unit-capacity arc. Any directed path that uses v as an intermediate vertex passes through this bottleneck, and two such paths would need two units of capacity and are therefore separated.

Figure 2.4 applies the split to a whole digraph. Three arc-disjoint routes are forced through one shared vertex w, so on the split network all three must cross the single unit-capacity arc inside w, which caps the number of internally vertex-disjoint routes below the edge-disjoint count.

`max_vertex_connectivity(G)` $\to \mathbb{N}$ **MEASURE**

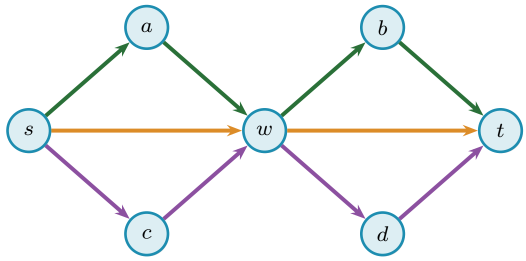
in a graph G

out $\kappa^{\max}(G)$, the largest local vertex-connectivity over all pairs

how the same sweep as the edge checker, but each flow runs on the split capacity matrix `_split_capacity_matrix(G)` of Figure 2.3, so the unit internal arcs count internally vertex-disjoint paths. Parallel edges are given capacity one here, so they never raise κ.

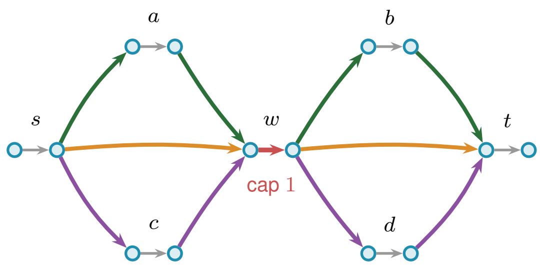
One convention in that codecard deserves to be said in prose, because it does not merely change how the checker computes, it changes what the multigraph vertex question asks. In vertex mode every adjacency carries capacity one, so a bundle of parallel edges counts as a single direct route. The rationale is that vertex-disjointness is about intermediate vertices, and parallel copies offer no new intermediate vertices to be disjoint over.

Remark 2.1 (What parallel edges mean in vertex mode)*.* That choice has a consequence which



$\lambda(s,t) = 3$: all three routes pass through w

(a) original digraph



(b) split network: $\kappa(s,t) = 1$

Figure 2.4. The vertex-split checker on a larger example. **(a)** A digraph in which three arc-disjoint s–t routes, $s \to w \to t$ (orange), $s \to a \to w \to b \to t$ (green), and $s \to c \to w \to d \to t$ (purple), all pass through the single vertex w, so $\lambda(s,t) = 3$. **(b)** The split network replaces every vertex V by an arc $V^{\text{in}} \to V^{\text{out}}$ of capacity one. All three routes must now traverse the single unit arc inside w (red), which carries only one unit, so at most one internally vertex-disjoint route gets through and $\kappa(s,t) = 1$. Splitting every vertex this way turns the vertex-disjoint count into an ordinary maximum flow.

has to be stated rather than left implicit. If parallel copies never raise κ, they never break feasibility either, so one could pile arbitrarily many copies onto any feasible multigraph and stay under the ceiling forever. Counted with multiplicity, the maximum number of edges would simply be infinite, and the extremal question would be empty. The two multigraph vertex variants are therefore posed here with the objective counting *adjacencies*, that is, pairs joined by at least one edge, rather than edges with multiplicity. Under that reading the question is exactly the simple question on the underlying graph, which is why those two rows of Table 4.1 reduce to their simple counterparts and why every figure in this thesis reports the reduced problem for them.

The other convention is perfectly available and asks something genuinely different. A single edge is a path with no interior, so q parallel copies between the same pair are q internally vertex-disjoint paths, and feasibility caps every multiplicity at $m - 1$ by itself. Nothing runs away, and the multigraph vertex problem separates from the simple one immediately: at $n = 2$ and $m = 3$ two parallel edges are feasible where the simple graph allows one. That is a new extremal problem rather than a variant of a solved one, and Chapter 4 lists it among the open problems. The appendix uses precisely this second reading whenever it argues about a general multigraph, most visibly in Lemma A.47, whose hypothesis (iv) caps a multiplicity at two for exactly this reason. The two readings never meet, because that lemma is applied only to incidence graphs, which carry no parallel edges at all, but a reader moving between the chapters and the appendix should know that the word “multigraph” is doing different work in the two places.

The two checkers are the entire interface between graph theory and computation in this thesis. As a sanity check, applied to the bidirected star of Figure 1.11 on n vertices, the edge checker reports $\lambda^{\max} = 1$, and the arc count $2(n-1)$ matches the proved values $\ell_2^{\mathrm{dir}}(n) = 2, 4, 6, 8$ for $n = 2, 3, 4, 5$.

The same checker answers two different questions. Throughout this chapter it runs in its exact-value mode, returning $\lambda^{\max}$ or $\kappa^{\max}$ on the nose, and every proved or reported number rests on that exact reading. The search of Chapter 3 needs only the cheaper yes-or-no question “is the binding connectivity above the bound?”, so there the same flow runs in a capped mode that stops as soon as it has seen one route too many. The capped mode never changes a reported value: it is a faster way to ask for a single bit, and Chapter 3 shows that the search built on it reproduces the exact search step for step.

The Gomory–Hu tree from Chapter 1 reappears here as an efficiency, and it is the honest kind. For undirected graphs all $\binom{n}{2}$ edge-connectivities are read off a single weighted tree, so $\lambda^{\max}$ is the heaviest tree edge rather than the maximum over $\binom{n}{2}$ separate flows. The speed-up is real, but it is bought with a theorem, not a heuristic.

`max_edge_connectivity_via_tree(G)` $\to \mathbb{N}$ **MEASURE**

- **in** an undirected graph G
- **out** $\lambda^{\max}(G)$, identical to the pairwise sweep
- **how** build one Gomory–Hu tree with `networkx.gomory_hu_tree`, then return its heaviest edge, which costs $n-1$ flows in place of $\binom{n}{2}$ and cross-checks the checker.

The hypergraph is the one model that never used the multiplicity matrix, since it is stored as a plain list of hyperedges, each one a set of r vertices. The checker still has to read it, and the trick is to run the same flow computation as before on a slightly larger network that we build on the spot. The difficulty is that one hyperedge ties several vertices together at once. By the definition of a Berge route in Chapter 1, only one route may use a hyperedge at a time, so we cannot just treat it as a bundle of ordinary links. Instead, for every hyperedge we add one extra helper point and connect each of that hyperedge’s member vertices to it (Figure 2.5). A route that wants to use the hyperedge now enters through one member vertex, passes through the helper point, and leaves through another.

The key is to make the helper point carry a single unit of capacity, built by the very same device as the vertex bottleneck of Figure 2.3: the helper point is split into an in-copy and an out-copy joined by one unit-capacity arc, and every route boarding the hyperedge must squeeze through that one arc. A second route would need a second unit of capacity there and so is shut out. That one unit is what captures the rule that one hyperedge serves one route.

Counting routes between two vertices is then the very same maximum flow as before, run on this enlarged network, and when a hyperedge happens to hold just two vertices the helper point collapses back to an ordinary link, so the answer agrees with the graph case. That the flow on the enlarged network counts exactly the hyperedge-disjoint Berge routes is the hypergraph version of Menger's theorem, proved in Appendix A (Theorem A.40), so the checker rests on a theorem rather than an analogy.

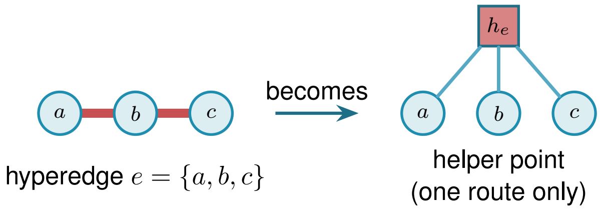
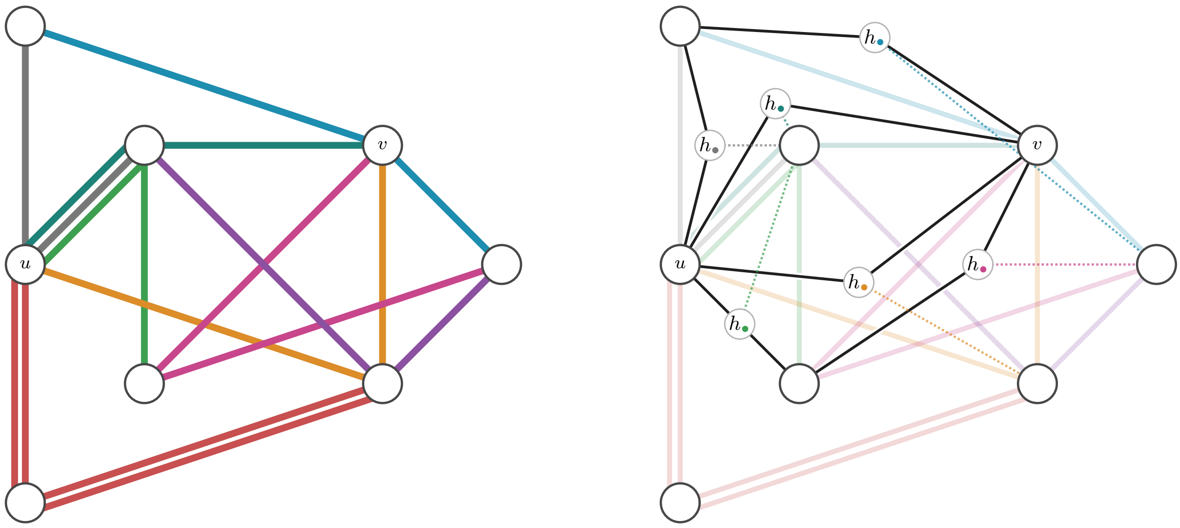


Figure 2.5. Measuring Berge connectivity by maximum flow. A hyperedge e, drawn as a metro line through its members (left), is replaced by a helper point h_e joined to each member (right). The helper point holds a single unit of capacity inside it, so at most one route may pass through h_e and the line runs one train at a time, which is what makes the hyperedge serve only one route. Counting independent routes between two vertices is then an ordinary maximum flow on this enlarged network, and a two-vertex hyperedge collapses to a single ordinary edge.

Figure 2.5 shows one hyperedge in isolation. Figure 2.6 carries the idea to a whole hypergraph in two panels. Panel (a) is the hypergraph drawn as a metro map: each of the eight hyperedges is one coloured line threading its three member stations, vertex to vertex, and a station on several hyperedges shows several coloured lines running through it. Panel (b) reads the same map as the flow network. For one pair of stations, u and v, it adds the helper point of every hyperedge a route boards, drawn as a small gate $h_\bullet$ in that line's colour, and traces the edge-disjoint Berge routes between u and v as thin black lines passing through those gates, with every hyperedge rail faded to the background. The checker reads $\lambda(u, v) = 4$: two routes run straight through the two hyperedges that contain both u and v, and two longer ones change lines at an intermediate station, four in all, while a fifth would have to reuse a gate.

One way to picture the helper point is the metro map of Chapter 1: each hyperedge is a metro line, its member vertices are the stations on it, and the helper is the line itself. Boarding at any station counts as taking that line. Because only one route may pass through the helper, only one train runs on the line at a time, so a second route must reach its destination without ever boarding it. The same flow computation that counts edge-disjoint paths through ordinary links now counts routes that share no line, which is exactly what Berge edge-disjointness means.



(a) (b)

Figure 2.6. A 3-uniform hypergraph on eight stations, with the pair u, v singled out. Panel (a) draws it as a metro map: every hyperedge is one coloured line threading its three members, vertex to vertex, the metro reading of Chapter 1. Panel (b) reads the same picture as the helper-point flow network. Each hyperedge a route boards becomes its one-route gate $h_\bullet$ of Figure 2.5, shown in that line's colour and placed on the route, and the thin black lines are the edge-disjoint Berge routes from u to v threading those gates. A dotted leg ties each gate to the hyperedge's third member, the station that route does not use, and every hyperedge rail is faded to the background so the black routes read clearly. Two routes pass straight through the two hyperedges that hold both u and v, and two longer ones transfer at an intermediate station, so the checker reads $\lambda(u, v) = 4$. A fifth route would have to reuse a gate.

`max_hyperedge_connectivity(H)` $\to \mathbb{N}$ **MEASURE**

in an r-uniform hypergraph H

out $\lambda^{\max}(H)$, the largest Berge edge-connectivity over all pairs

how build the enlarged capacity matrix `_hyper_capacity_matrix(H)` in which every hyperedge becomes a helper point that only one route may pass through, then take the same maximum flow as the graph checker. On two-vertex hyperedges it matches ordinary edge-connectivity.

On the complete graph K_n the hypergraph checker returns $n-1$, and on the complete three-uniform hypergraph $K_4^{(3)}$ it returns 3, larger than the two hyperedges through each pair because longer Berge routes also carry flow. The self-test confirms both of these values, so the construction is checked rather than merely asserted. The same helper-point network also answers the vertex-disjoint hypergraph question without alteration: splitting each ordinary vertex as in Figure 2.3 counts internally vertex-disjoint Berge routes. The one remaining direction, the *directed* hypergraph, needs the helper point to learn which way is which, and that is the subject of the next subsection.

2.3.1 The directed hypergraph checker

A *directed* hyperedge carries an orientation: it is a single *tail* vertex together with $r-1$ *head* vertices, drawn as the star of Figure A.8, and a directed Berge route may only enter it at the tail and leave it at one of its heads. The undirected helper point treated a hyperedge as a line any route could board at any station. The directed one must instead be a one-way turnstile: a route may step onto the hyperedge only at its tail and step off only at a head. The gadget that enforces this is the helper point made directional. For a hyperedge $e = (a; b, c, \dots)$ we add a gate node g_e, send one capacity-one arc from the tail into it, $a \to g_e$, and send an arc out of it to each head, $g_e \to b$, $g_e \to c$, and so on (Figure 2.7). A route reaches g_e only through the tail a, and the single unit of capacity on $a \to g_e$ lets just one route use the hyperedge, exactly as the undirected turnstile did, but now it can leave only towards a head. The gate is built once per hyperedge, directed or not, by the same loop:

```
gate_in, gate_out = base + 2 * index, base + 2 * index + 1
cap[gate_in, gate_out] = 1            # one route through this hyperedge
if hypergraph.directed:
    tails, heads = _dir_tails_heads(edge)
    for tail in tails:
        cap[leave(tail), gate_in] = _UNBOUNDED
    for head in heads:
        cap[gate_out, enter(head)] = _UNBOUNDED
```

Listing 2.2. The gate loop behind Figure 2.5 and Figure 2.7: one capacity-one gate per hyperedge, wired one-way for a directed hyperedge and both ways for an undirected one.

Figure 2.8 reads the same hypergraph directed, again in two panels. Panel (a) gives the whole

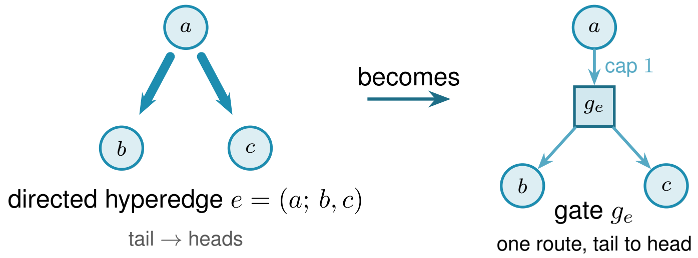


Figure 2.7. The directed helper point. A directed hyperedge $e = (a;\, b, c)$, the tail a fanning to its heads b, c (left), becomes a gate node g_e fed by one capacity-one arc from the tail and feeding an arc to each head (right). A route may reach g_e only through the tail and may leave only towards a head, so the gate is a one-way version of the turnstile of Figure 2.5, and the single unit of capacity still lets only one route use the hyperedge.

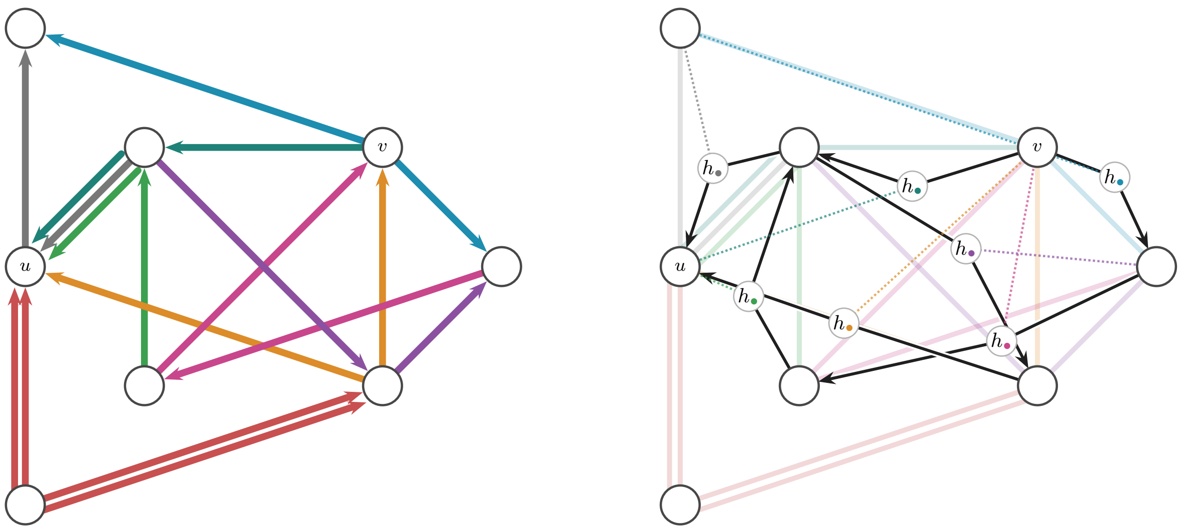
hypergraph oriented, every hyperedge a one-way line whose arrows point away from its tail, so the middle-tailed lines fan out from their centre and the end-tailed ones run one way along their length. Panel (b) traces the routes from v to u, the gate rule of Figure 2.7, fading every hyperedge rail to the background and threading a black arrow from the route's entry station through the one-route gate $h_\bullet$, set in a clear gap off the rails, to its exit, with a dotted leg to the hyperedge's third member. Orientation costs connectivity. Of the four undirected routes between u and v only two survive the arrows, $\lambda(v, u) = 2$, because v can set out through only the two hyperedges it heads, and the two survivors cross at the interchange l so they share no hyperedge. The same vertex-splitting trick of Figure 2.3 laid on top then counts internally vertex-disjoint directed routes, so this one construction measures all four hypergraph variants and `solve` reaches every one of the twelve problems through a single checker.

The gate is indifferent to how a hyperedge's vertices are split between tails and heads. It draws one capacity-one arc in from every tail and one out to every head, so the single forward tail it was built for is only the simplest case: give it a hyperedge with several tails and one head, or several of each, and it still counts the routes that enter at a tail and leave at a head. The same checker therefore measures every *orientation model* of a directed Berge edge, not just the one-tail one, and Section 4.3.1 takes up which of those models are worth distinguishing and what the program finds for them.

With the checker in place the proposed value of Proposition 1.15 can be tested on any small hypergraph the way every other variant is, which is the subject of the next section.

2.4 Checking every graph of a fixed size

The checker measures one graph. To bound *every* graph of a given size, the most direct method is the one the base cases of Chapter 1 already needed: enumerate them all. A simple graph on n vertices is a choice of $\mu(u, v) \in \{0, 1\}$ on each off-diagonal cell, a multigraph a choice in $\{0, \dots, m-1\}$, and a digraph varies every ordered pair rather than only the upper triangle. Walking that product, measuring each candidate with the checker, and keeping the densest feasible one settles the value exactly, and because it has looked at every graph it is a genuine upper



(a) (b)

Figure 2.8. The hypergraph of Figure 2.6 read directed. Panel (a) gives the whole directed hypergraph, every hyperedge a one-way line with its arrows pointing away from the tail station, so a route may join a line at its tail and ride it to a head. Panel (b) traces the routes from v to u. Every hyperedge rail is faded to the background, and a black arrow runs from the route's entry station through the hyperedge's one-route gate $h_\bullet$ to its exit, the directed turnstile of Figure 2.7, with the gate set in a clear gap off the rails. A dotted leg in the line's colour ties each gate to the hyperedge's third member, the station the route does not visit. Two edge-disjoint directed Berge routes from v to u survive. The long route $v \to j \to c \to l \to u$ rides nmj, mcj, elc and nel in turn, and the short route $v \to l \to d \to u$ rides elm, ldj and edm. They cross only at the interchange l, so they share no hyperedge, and the checker reads $\lambda(v, u) = 2$, since v can set out through only the two hyperedges it heads. Against the undirected four of Figure 2.6, the arrows have halved the count.

bound, not evidence.

This is the first thing `solve` does when it is asked to prove a small case. With its exhaustive flag set it walks that product, measures each candidate with the checker, and keeps the densest feasible one. The value it returns is exact and exhausted, a true upper bound for that n, but only while the count of graphs stays small. No new routine is involved, only `solve` choosing to enumerate.

Run on the small cases this confirms the proved values directly, with no induction and no cleverness. Table 2.2 collects a spread of variants where the enumeration finishes: in every row the machine returns exactly the value Chapter 1 proved by hand. This is the first payoff of the uniform representation. One driver, `solve`, pointed at six different problems, settles all six.

Variant	m	n	`solve`	Theory
simple undirected, edge	2	5	4	$\lfloor m(n-1)/2 \rfloor = 4$
simple undirected, edge	3	5	6	$\lfloor m(n-1)/2 \rfloor = 6$
multigraph undirected, edge	3	5	8	$(m-1)(n-1) = 8$
simple undirected, vertex	2	6	5	$k_2(n) = n-1 = 5$
simple directed, arc	2	5	8	$\max(2(n-1), \lfloor n^2/4 \rfloor) = 8$
simple directed, arc	2	6	10	$\max(2(n-1), \lfloor n^2/4 \rfloor) = 10$

Table 2.2. Variants that converge under plain enumeration. For each small case `solve`, in its exhaustive mode, walks every graph of the variant, measures it with the checker, and returns the densest feasible one. Every value matches the hand proof of Chapter 1 exactly.

The honesty of the method is also its limit. The number of graphs it must visit grows as $2^{\binom{n}{2}}$ for simple undirected graphs and $2^{n(n-1)}$ for digraphs, and capping multiplicities at $m-1$, so that each cell ranges over the m values $\{0, \dots, m-1\}$, raises the base from two to m, giving $m^{\binom{n}{2}}$ undirected multigraphs and $m^{n(n-1)}$ directed ones. The enumeration is exact wherever it finishes, and it finishes only for the smallest sizes. Chapter 3 opens with exactly how fast this wall arrives. For now the point is that the wall arrives precisely where Chapter 1 needed help, at the directed base cases $n = 6, 7$, so the rest of this chapter is about reaching a little further than blind enumeration can.

2.5 Reaching further: a pruned search for the directed base cases

The base cases $n \leq 7$ of the directed $m = 2$ theorem (Theorem 1.10) are the finite checks the induction of Appendix A depends on, and they sit just past where blind enumeration of digraphs is tractable. They can be reached by enumerating more cleverly. Feasibility is monotone: adding an arc can never lower $\lambda^{\max}$, so once a partial digraph is infeasible every completion of it is infeasible too and the whole branch is abandoned at once. A counting bound prunes further. At any partial digraph, count the arcs already placed and add the most the still-undecided cells could ever contribute. That sum is a ceiling on every completion of the branch, so if even the ceiling cannot beat the best feasible digraph found so far, the branch is dropped unexplored (Figure 2.9). What remains is a branch and bound over simple digraphs with the connectivity checker as the feasibility test, exact and complete at these sizes where the blind walk is not.

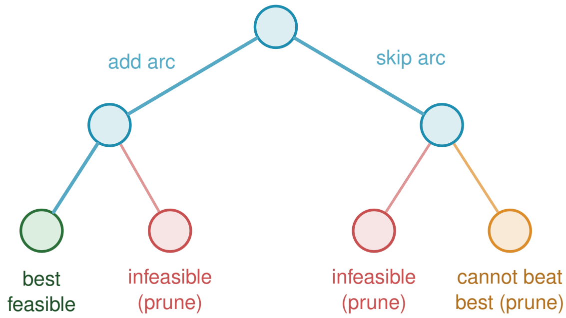


Figure 2.9. Branch and bound over digraphs. Each node fixes one more cell, arc present or absent, so the leaves are whole digraphs. Two rules cut entire subtrees before they are walked. A branch whose partial digraph already breaks the connectivity ceiling is infeasible, and so is every completion of it (red). A branch whose best possible completion still cannot beat the densest feasible digraph found so far is dropped on the counting bound (orange). Only the surviving branches are explored, which is what lets the search reach the base cases $n \leq 7$ the blind walk cannot.

Here `solve` changes its implementation without changing its interface. For a simple digraph it runs this branch and bound in place of the blind walk, still with the connectivity checker as the feasibility test, and so reaches the base cases $n \leq 7$ that blind enumeration cannot. The booleans alone decide which of the two it uses, and the caller never asks.

Run on the directed $m = 2$ problem `solve` returns the complete sequence $2, 4, 6, 8, 10, 12$ for $n = 2, \ldots, 7$, each proved complete at its size. The densest digraph it finds at each n is exactly the extremal shape Chapter 1 named, the bidirected star (the hub) below the crossover at $n = 7$ and the one-directional bipartite digraph above it, so the search confirms not just the values but the winning constructions. These are exactly the base cases the induction requires, and with them the long proof of Theorem 1.10 rests on a single auditable search rather than on anything done by hand. The exhaustive-search transcript is in Appendix A.

2.6 Generating the directed cases faster

The $m = 3$ directed multigraph result reduces, through Remark A.39, to a handful of finite statements about multigraphs on six and seven vertices, and proving them means listing every such multigraph up to a renaming of its vertices. The program does this in two independent ways, on purpose: when two different methods return the same list, that agreement is itself a check.

The first way is a search written from scratch. It is a depth-first walk over multiplicity matrices that abandons a branch the moment it turns infeasible or can no longer beat the best graph found, and it keeps just one representative of each isomorphism class by reducing every finished graph to a canonical relabelling, the smallest matrix in dictionary order over all relabellings. The canonical form keeps the memory small, but the time is spent walking labelled partial graphs, and time is what runs out: the full classification at $n = 5$ costs this in-house search about 456 seconds, and it climbs steeply from there.

The second way (Table 4.2 names it) does not search at all, it generates, and it was suggested by Prof. Jan Goedgebeur. The idea is to hand the hard part, listing one graph per isomorphism class, to a tool that already does it supremely well: the `geng` program from the `nauty` toolkit of McKay and Piperno [10], invoked through a subprocess and decoded from its `graph6` output (a compact one-line text encoding of a graph). `geng` lists the non-isomorphic *undirected* graphs on n vertices, and the program then decorates each one with the directed multiplicities the problem needs, so the isomorphism reduction is inherited for free from the support graph. The decoration itself is plain to state. For one support graph, walk its edges in a fixed order and assign each edge $\{u, v\}$ an ordered pair of multiplicities $\mu(u, v), \mu(v, u)$, each between 0 and $m-1$ but not both zero, since an edge of the support must carry at least one arc. Along the way the checker measures the partly decorated multigraph, and the moment some pair already exceeds the connectivity ceiling the whole branch of assignments is abandoned, since adding further arcs can never lower a connectivity, the same monotonicity prune as Figure 2.9. A decoration that survives to the end with the target arc count is kept, reduced to its canonical form so that each isomorphism class is reported once. Different supports share nothing, so they are processed independently across the processor cores. The companion orientation tools `directg` and `watercluster2` are not used, because each gives only one orientation per edge and so cannot express the bidirected arcs and repeated multiplicities the extremal graphs are built from. Layering the multiplicities in-house on top of `geng` is what fits the problem. To be precise about credit: Prof. Goedgebeur is not an author of `nauty`, which is the work of McKay and Piperno cited above. He suggested the generation route, and it is his suggestion that the speed-up rests on.

The two ways agree to the last graph. The generated pipeline reproduces the in-house classification exactly at every settled size, two non-isomorphic extremal families at $n = 4$ and three at $n = 5$, and it runs substantially faster, since the isomorphism work it never repeats is precisely what `geng` has already done. One difference decides which to trust further out: the in-house search leans on a conjectured counting bound once $n \geq 7$, while the generated enumeration prunes only with proved bounds, so it is a sound and complete search at every n. That soundness has now paid off. Pointed at the degree-bounded classification at $n = 7$, fact (b) of Remark A.39, the generation enumerator completed the sweep in about seven hours across ten processor cores and returned exactly the three predicted extremal families, settling fact (b) outright and leaving the value statement, fact (a), as the $m = 3$ argument's one remaining computation.

2.7 Proving every m at once

The pruned search proves one (n, m) at a time. For the directed multigraph arc problem `solve` has a sharper implementation, one that proves every m at a fixed n in a single pass, and it is worth the extra machinery because it closes the directed multigraph case outright for the sizes it reaches.

The key is that m can be scaled out of the problem completely, so that it never appears in the computation at all. Start with any directed multigraph that is feasible at level m, so $\lambda^{\max} \leq m-1$, and divide every multiplicity by $m - 1$. Two things happen at once. The multiplicities become weights $w(u, v)$ between 0 and 1, and since dividing every capacity by $m - 1$ divides every flow by the same factor, the maximum flow between each pair drops to at most one. The arc count, meanwhile, turns into the total weight $\sum_{u \neq v} w(u,v) = |A|/(m-1)$. So a dense feasible multigraph becomes a heavy feasible weight matrix, and the other way round. Write

$$M^*(n) = \max\Big\{\textstyle\sum_{u \neq v} w(u,v) : w(u,v) \in [0,1],\ \operatorname{maxflow}(s,t;w) \leq 1 \text{ for all } s,t\Big\}$$

for the largest total weight any such matrix can carry. It is one number for each n, with no m inside it, and scaling a feasible multigraph down gives

$$L_m^{\mathrm{dir}}(n) \ \leq\ (m-1)\,M^*(n) \qquad \text{for every } m \geq 2.$$

That inequality holds always, and it is the direction that does the proving. The reverse does not come for free, and it is worth being exact about why, because the two are easy to run together. Scaling *up* needs an optimal weight matrix whose entries are whole multiples of $1/(m-1)$, and a real optimum has no reason to oblige. What closes the gap is not an argument but an observation about the answer the solve returns: the heaviest matrix it finds is the *double star*, the bidirected star carrying weight one on both arcs $h \to v$ and $v \to h$ between the hub and every other vertex. Its weights are all 0 or 1, so multiplying them by $m-1$ gives an honest integer multigraph, not a fractional one, with $2(n-1)(m-1)$ arcs and $\lambda^{\max} = m-1$, for every m at once. Whenever the optimum is attained by such a $\{0,1\}$ matrix, the two bounds meet and

$$L_m^{\mathrm{dir}}(n) = (m-1)\,M^*(n) \qquad \text{for every } m \geq 2,$$

so that one solve settles infinitely many values. This is exactly what happens at every $n \leq 6$, which is what Theorem 2.2 rests on. It is a verified property of those optima rather than a theorem about all n, and where it is not verified the safe reading is the inequality: an integral question such as fact (a) of Remark A.39 is then asked of the integral program `prove_integral_arc_bound` directly, not deduced from a fractional value.

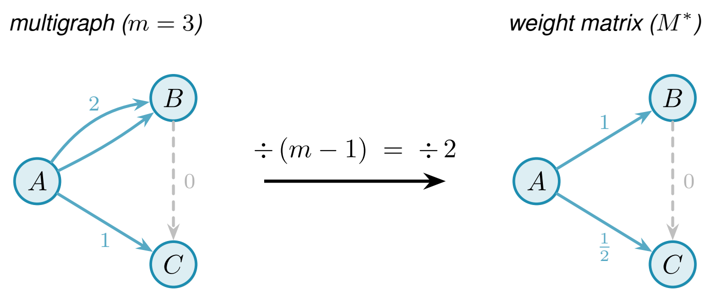


Figure 2.10. The scaling step for $m = 3$. Dividing each multiplicity by $m - 1 = 2$ maps the integer multigraph (left) to a weight matrix with all values in $[0,1]$ and max-flow at most one between every pair (right). The double arc $A \to B$ (multiplicity 2) becomes weight 1, the single arc $A \to C$ (multiplicity 1) becomes weight $\frac{1}{2}$, and the absent arc $B \to C$ (multiplicity 0, shown dashed) stays absent. Crucially, m drops out on the right, so a single optimisation over all weight matrices pins $L_m^{\mathrm{dir}}(n) = (m-1)\,M^*(n)$ for every m simultaneously.

What remains is to compute $M^*(n)$, and the difficulty is that the constraint "maximum flow at most one" is itself the answer to a flow problem rather than a plain inequality on w. The max-flow / min-cut theorem removes it. The maximum s–t flow equals the smallest capacity of any *cut* separating s from t. A cut just splits the vertices into a source side holding s and a sink side holding t, and its capacity is the total weight of the arcs that cross from the source side to the sink side. So $\mathrm{maxflow}(s,t) \leq 1$ holds exactly when some such cut has capacity at most one. That lets us drop the flow entirely: instead, for every ordered pair (s,t), the optimisation picks one cut and we require its capacity to stay at or below one.

A cut is described by a yes/no label x_u^{st} on each vertex, equal to one when u is on the source side, with s fixed on the source side and t on the sink side. An arc $u \to v$ crosses the cut exactly when u is on the source side and v is not, and a helper variable p_{uv}^{st} then picks up its weight $w(u,v)$. Forcing the crossing weights to sum to at most one for every pair forces every pair to admit a cut of capacity one, hence maximum flow at most one. This is the exact feasible region, not a relaxation of it, a relaxation being a loosened version that would also admit weight matrices no multigraph realises, and that exactness is what turns the result into a proof. The optimisation reports both the best total weight it has found and a value it has shown no matrix can exceed, and when those two meet the gap is zero and $M^*(n)$ is certified.

For the directed multigraph arc problem, then, `solve` runs this cut-counting optimisation in place of any enumeration: it chooses the minimum cut for every ordered pair, and a zero gap makes the reported $M^*(n)$ a proved upper bound, from which $L_m^{\mathrm{dir}}(n) \leq (m-1)M^*(n)$ delivers a ceiling for every m at once, met from below by the double star whenever the reported optimum is integral.

`prove_directed_multigraph(n)` $\to$ `ProofResult` **PROVE**

in the vertex count n (and tightening flags such as `use_deletion_cuts`)

out $M^*(n)$ with a proof status, giving $L_m^{\mathrm{dir}}(n) \leq (m-1)M^*(n)$ for every m, with equality when the optimum found is integral

how builds the cut-counting optimisation, the mixed-integer linear program written out below, once with `_cut_counting_model` as a `pulp` model, then `_pick_solver` runs it on CBC, the free solver bundled with `pulp`, or on the commercial Gurobi solver by a one-line switch. A zero optimality gap is a proof. The integer-weight companion `prove_integral_arc_bound(n, m, target)` answers the single feasibility question behind fact (a) of Remark A.39.

`solve` assembles this optimisation from n alone, and a reader content to take it on the strength of the cut argument just given can skip to Theorem 2.2 without loss. Written out it reads more heavily than it behaves, so we first name every symbol in plain words, then show the one inequality that looks cryptic is a simple yes-or-no switch.

The optimisation is a *Mixed-Integer Linear Program*, or MILP for short. "Linear" means every constraint and the objective are linear functions of the variables: no products, no squares, no exponentials. “Mixed-integer” means the variables split into two kinds: most are ordinary real numbers that may take any value in a continuous range, but a few are forced to be whole numbers, either 0 or 1. The reason for the split is conceptual. Arc weights w are naturally continuous: they are scaled multiplicities and can sit anywhere in $[0, 1]$. But the cut assignment for a vertex is a hard binary choice: a vertex is either on the source side of a given cut or on the sink side, with no meaningful notion of being “half on each side”. Allowing the cut labels to be fractional would describe a different, weaker condition and could report an optimistic value that no integer-weight multigraph can actually achieve. Keeping those labels in $\{0, 1\}$ is what keeps the optimal gap honest, and it is what makes the program's result a proof rather than an estimate.

Three quantities appear, and only three. The first is already familiar.

- $w(u, v) \in [0, 1]$ is the *scaled multiplicity* of the arc $u \to v$, the amount of arc weight the matrix places on $u \to v$ after the division by $m - 1$ of the previous section. It is what we are maximising the total of. This is a continuous variable: it may be any real number in $[0, 1]$.
- $x_u^{st} \in \{0, 1\}$ is the *side label* for vertex u with respect to the ordered pair (s, t). The optimisation chooses one cut for each pair, and x_u^{st} records whether u is on the source side (1) or the sink side (0). The two endpoints are pinned: $x_s^{st} = 1$ and $x_t^{st} = 0$. This is the integer variable: a vertex cannot straddle the cut.
- $p_{uv}^{st} \geq 0$ is the *crossing weight*, the part of $w(u, v)$ that the pair (s, t)'s chosen cut is forced to count. It equals $w(u, v)$ when the arc $u \to v$ crosses from source to sink, and is otherwise left at zero. This is a continuous variable.

With those three named, the program is short:

$$\begin{array}{lll}
\text{maximise} & \displaystyle\sum_{u \neq v} w(u, v) & \\
\text{subject to} & p_{uv}^{st} \geq w(u, v) + x_u^{st} - x_v^{st} - 1 & \text{for all } (s, t),\ (u, v), \\
& \displaystyle\sum_{u \neq v} p_{uv}^{st} \leq 1 & \text{for all } (s, t), \\
& w(s, t) + \displaystyle\sum_{x} \min\big(w(s, x), w(x, t)\big) \leq 1 & \text{for all } (s, t), \\
& d(v_0) \leq d(v_1) \leq \cdots \leq d(v_{n-1}), & \text{with } d(v) = \sum_{u \neq v} \big(w(u, v) + w(v, u)\big), \\
& w \in [0, 1], \quad x \in \{0, 1\}. &
\end{array}$$

The first line is the only one that looks cryptic, and it is just an indicator written as an inequality. Its right-hand side $w(u, v) + x_u^{st} - x_v^{st} - 1$ takes one of four shapes according to which side the cut puts the two endpoints u and v on, and Table 2.3 works all four out. Because $w(u, v) \leq 1$, the right-hand side is positive in one case only, the crossing case, where it equals $w(u, v)$ and so forces $p_{uv}^{st} \geq w(u, v)$. In the other three cases it is zero or negative, so the constraint reads $p_{uv}^{st} \geq$ (something ≤ 0) and leaves p_{uv}^{st} free to stay at zero, which the maximisation is happy to let it do.

x_u^{st}	x_v^{st}	meaning	$w + x_u^{st} - x_v^{st} - 1$	forces
1	0	u source side, v sink side: arc *crosses*	$w(u, v)$	$p_{uv}^{st} \geq w(u, v)$
1	1	both on the source side	$w(u, v) - 1 \leq 0$	$p_{uv}^{st} \geq 0$
0	0	both on the sink side	$w(u, v) - 1 \leq 0$	$p_{uv}^{st} \geq 0$
0	1	u sink side, v source side	$w(u, v) - 2 \leq 0$	$p_{uv}^{st} \geq 0$

Table 2.3. The first constraint, case by case. This is where the bound $w \leq 1$ earns its keep: it makes the three non-crossing rows land at or below zero, so the helper p_{uv}^{st} is pushed up to the arc weight in exactly one case, the crossing one. The constraint is therefore an exact indicator, "count $w(u, v)$ when the arc crosses this cut, and nothing otherwise", with no slack and no fudge.

Figure 2.11 shows one chosen cut and the weights it counts. The second line of the program now reads plainly. It says the total crossing weight of this pair's cut, $\sum_{u \neq v} p_{uv}^{st}$, is at most one. By max-flow / min-cut a cut of capacity at most one exists for (s, t) exactly when $\operatorname{maxflow}(s, t) \leq 1$, so the two lines together say "every ordered pair admits a cut of capacity at most one", which is precisely the feasibility we want. This is the true feasible region, not a relaxation of it, so an optimal solution reported with zero gap is a proof and not merely evidence.

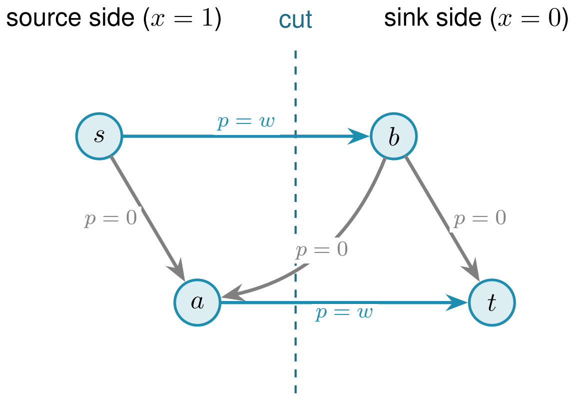


Figure 2.11. One ordered pair's chosen cut. The dashed line splits the vertices into the source side, where $x_u^{st} = 1$, and the sink side, where $x_u^{st} = 0$. Only arcs running from source side to sink side cross the cut, and the first constraint forces their helper p_{uv}^{st} up to the arc weight w (the two thick arcs). Arcs that stay on one side, and arcs that run the other way across the line, contribute nothing. The second constraint then caps the total counted weight, the thick arcs, at one.

The last two lines change no answer. They only help the solver finish in time. The third is a redundant shortcut. The direct arc $s \to t$, together with one two-step detour $s \to x \to t$ through each intermediate vertex x, already forms that many routes between s and t. So $w(s, t) + \sum_x \min(w(s, x), w(x, t)) \leq 1$ holds of any feasible matrix anyway, and stating it outright just sharpens the continuous picture the solver reasons from.

The $\min$ in that line is not a linear function, so it cannot appear in the MILP as written. To keep the program linear, each $\min(w(s, x), w(x, t))$ is replaced by a fresh helper variable z_x^{st} together with a binary selector $b_x^{st} \in \{0, 1\}$. The selector records which of the two arguments is smaller: $b_x^{st} = 1$ means $w(s, x)$ is no larger than $w(x, t)$. Four linear inequalities then pin z_x^{st} exactly to the minimum. Two upper bounds, $z_x^{st} \leq w(s, x)$ and $z_x^{st} \leq w(x, t)$, prevent z from exceeding either argument. Two lower bounds, one of the form $z_x^{st} \geq w(s,x) - (1 - b_x^{st})$ and the other $z_x^{st} \geq w(x,t) - b_x^{st}$, each become active only when the selector chooses that argument: at any integer value of b exactly one lower bound bites and forces z up to the chosen weight. Together with the upper bounds this pins $z_x^{st} = \min(w(s,x), w(x,t))$ exactly. This technique is called the McCormick linearisation of a minimum. It adds one continuous variable z and one binary variable b per intermediate vertex per ordered pair, all of them appearing only in the tightening constraint and nowhere else, so the proof structure of the program is unchanged.

The fourth line orders the vertices by weighted degree. Renaming vertices cannot change any count, so insisting on this order rules out no graph, and it spares the solver the many relabelled copies of a graph that differ only in vertex names.

Theorem 2.2 (Directed multigraph value, small n)**.** *For $n \leq 6$, $M^*(n) = 2(n-1)$, and the optimum is attained by a $\{0,1\}$ weight matrix. Hence for all $m \geq 2$, $L_m^{\mathrm{dir}}(n) = 2(n-1)(m-1)$, attained by the double star.*

It is worth being clear about what "for all m" rests on, because the theorem proves infinitely many values from finitely many solves. It is not that each m was tested. In this mode `solve` never sees m at all. It proves the single m-free number $M^*(n)$, and because that optimum turns out to be attained by a $\{0,1\}$ matrix, $L_m^{\mathrm{dir}}(n) = (m-1)M^*(n)$ then carries that one fact to every m at once. The only quantity that is checked case by case, and the only genuine limit on how far the result reaches, is n.

`solve` proves Theorem 2.2 by returning $M^*(3) = 4$, $M^*(4) = 6$, $M^*(5) = 8$, and $M^*(6) = 10$, each optimal with zero gap, so that $M^*(n) = 2(n-1)$ over this range. The first three finish quickly, $M^*(3)$ and $M^*(4)$ in well under a second and $M^*(5)$ in a few seconds, while $n = 6$ takes about twenty minutes. At $n = 7$ the same encoding does not close, and we do not pretend otherwise: it is a finite obstacle of scale, not a new idea, and it marks the current edge of certified knowledge for this variant. The solver transcript is in Appendix A.

2.8 What the solver may claim

Now that the pieces inside `solve` are built, it is worth stating plainly, because the rest of the thesis depends on it, what each is allowed to claim. The checker *measures*: its connectivity values are exact. Its three proving implementations, the exhaustive enumeration, the pruned search, and the cut-counting, *prove*: an exhausted enumeration or an optimal, zero-gap solve is an upper bound for that fixed n. The search of Chapter 3 will *discover*: it produces concrete graphs, hence lower bounds, and never an upper bound. These three claims are exactly what `solve` labels its answer with, so that an exact value, a lower bound, and an upper bound can never be quietly confused with one another.

With the microscope built and aimed at proof, the variants with small answers are settled and the directed base cases the hand proof needed are in hand. What proof by enumeration cannot do is reach the sizes where the answers are only conjectured, because the number of graphs explodes long before the interesting cases arrive. The next chapter measures that explosion exactly, and then turns the same program loose to *discover* the dense graphs that blind enumeration can never reach.

2.9 Reproducibility

Every claim in this thesis that rests on computation can be re-run from a single Python driver. All of the program's logic lives in `program/erdos915_unified.py`, one self-contained file. Its core needs only `numpy` and `scipy`, because every connectivity value it reports is a single `scipy` integer maximum flow on a capacity matrix. The model, the checker, the search, the enumeration, and the self-test all run on those two libraries alone.

A few more libraries sit on top, each for one job. The certifier uses `pulp`, a Python library for writing an optimisation once and handing it to any solver, and runs it on CBC, the free solver `pulp` bundles, or on the commercial Gurobi solver by a one-line switch. Two others are optional and only for presentation: `matplotlib` draws the figures, and `networkx` supplies the Gomory–Hu tree behind one of them. An optional compiled helper, `_erdos_fast.so` (built from `_erdos_fast.c` by `build_fast.sh`), speeds up the hot-path `_tiny_maxflow` and `_canonical_form` calls, but the pure-Python path returns the same answers.

A handful of named entry points reach the whole pipeline. `solve` drives every variant. The measures are `max_edge_connectivity` and `max_vertex_connectivity`, with the hypergraph and Gomory–Hu views beside them. The provers are `prove_directed_multigraph` and `enumerate_extremal_directed_multigraphs`, and `search_for_dense_graph` is the discoverer. Running `python erdos915_unified.py` executes the built-in self-test, the suite of invariant checks that confirms the connectivity values quoted throughout this chapter, including the sanity checks on the bidirected star and on K_n and $K_4^{(3)}$.

The figures and proofs are just as reproducible. Every figure is regenerated by the companion driver `program/make_figures.py`, which imports the same file and fixes every random seed, so the pictures are reproduced exactly rather than merely resembled, and the table in `program/README.md` records which routine produces which figure. The solver transcripts behind this chapter's certified values, the pruned-search log for Theorem 1.10 and the cut-counting certificate for Theorem 2.2, are reproduced verbatim in Appendix A, so a reader may check the proofs without running anything at all.

2.9.1 What a machine-checked claim in this thesis means

Reproducibility is not the same as certification, and it is worth stating plainly what standard the computed results here meet, since a solver reporting `OPTIMAL` is not by itself a mathematical certificate. Four practices carry the weight.

First, the exhaustive searches are self-certifying in a way the optimisations are not. A finished include-or-exclude sweep has visited a superset of the objects it needed to visit, and the two prunes are proved sound in Chapter 3, so its verdict depends on no solver and no floating-point arithmetic, only on the checker. Where a value in this thesis is called *proved*, that is usually the reason.

Second, every reported connectivity is exact and integral. The checker is a single integer maximum flow, so no value in the thesis is a rounded floating-point quantity. The one place real numbers enter is the fractional optimisation of Theorem 2.2, and there the reported optimum is a $\{0, 1\}$ matrix, which is checked exactly.

Third, the load-bearing computations are run twice, by implementations that share no code. The base cases of Theorem 1.10 and Theorem 1.11 were each confirmed by the program and by separately written checkers, agreeing on every size. The values of Section A.8 were computed by the program's routine and by an independent script, agreeing on every cell. Fact (b) of Remark A.39 was produced by the generation enumerator and re-verified graph by graph with the exact checker. Two implementations agreeing is not a proof, but it is the practical guard against the failure mode a certificate would catch, namely a correct method with a wrong program.

Fourth, the self-test at the foot of the file is a standing regression check on every invariant the text quotes, and it is run after every change to the program.

What this does not amount to is a formally checkable certificate of the kind a SAT solver can emit, such as a resolution proof a small independent verifier could replay. For the one result here that rests on a solver's infeasibility verdict rather than on an exhaustion, fact (a) of Remark A.39, that would be the right standard to aim at, and it is recorded among the open problems rather than claimed.

Chapter 3

Discovering Bounds by Search

The previous chapter proved what could be proved by looking at every graph of a fixed size. That method is honest and exact, and it stops almost at once. This chapter begins by measuring exactly how fast it stops, because the speed of that collapse is the whole reason a second kind of tool is needed. Then it builds that tool. Where the certifier walks all graphs, the search walks a few of them well, cooling like a physical system toward the densest feasible shape. What it returns is never a proof. It is a concrete graph, hence a lower bound and a conjecture, a stepping stone toward an upper bound that a later proof or certificate must supply. The chapter ends by asking the question any such tool must face: how good is it, and how far from the truth might its answers be.

3.1 The wall: how enumeration explodes

The brute force of Chapter 2 visits every graph of a given size. The trouble is how many that is. A simple undirected graph on n vertices is a yes-or-no choice on each of the $\binom{n}{2}$ possible edges, so there are $2^{\binom{n}{2}}$ of them. A digraph chooses on each of the $n(n-1)$ ordered pairs, giving $2^{n(n-1)}$, and allowing multiplicities up to a cap c raises the base from two to $c+1$. These are not numbers that merely grow. They detonate. Figure 3.1 draws the growth on a logarithmic scale, where each step up the axis multiplies the count tenfold, and the message is the same in every panel: the count passes any reachable budget while n is still in the single digits.

Three readings of the figure matter for what follows. Adding vertices is the sharpest cost: each new vertex multiplies the count by an exponential in n, so a single extra vertex can move a problem from seconds to centuries. Raising the connectivity budget is the next cost, lifting the base of the exponent rather than its height, which is why the multigraph curves sit above the simple one in every panel. Direction is the third: a digraph has twice as many cells to fill as the undirected graph on the same vertices (and a directed 3-uniform hyperedge, a tail with two heads, has three times as many), so the directed panel climbs fastest of all, and those are precisely the variants whose answers Chapter 1 could not prove. The separation, by contrast, costs nothing here, because edge and vertex disjointness search the same graphs and differ only in what the checker measures, which is why a single pair of panels suffices for all of them. The certifier of Chapter 2 reaches a few vertices further than blind enumeration, because its cut-counting optimisation is polynomial in the size of the encoding rather than in the number of graphs, but solving that encoding is itself hard and the gain is measured in single vertices, not orders of magnitude. Past that, no exhaustive method reaches the interesting sizes. Something

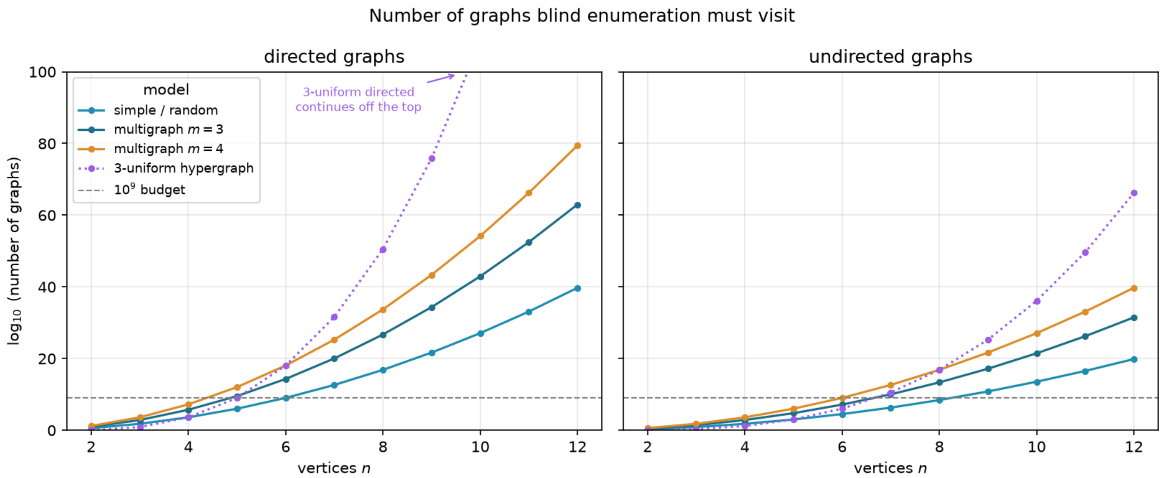


Figure 3.1. The number of graphs blind enumeration must visit, on a logarithmic vertical axis, for directed graphs (left) and undirected graphs (right). Each panel shows the simple model, two multigraph connectivity caps, and the three-uniform hypergraph (dotted), with a dashed line at a generous budget of 10^9 candidates, and every curve crosses that budget while n is still in the single digits. Writing E for the number of off-diagonal cells a model may fill, a simple graph offers 2^E candidates, a multigraph with multiplicity cap $m-1$ (the connectivity ceiling) offers m^E (one choice per cell from $\{0,\ldots,m-1\}$), and the 3-uniform hypergraph offers $2^{\binom{n}{3}}$ undirected or $2^{n\binom{n-1}{2}}$ directed, since a directed hyperedge is a tail together with two heads. The count depends only on the model and the direction, never on whether one later asks for edge- or vertex-disjoint routes, so a single pair of panels covers all of those variants at once. Direction multiplies the number of cells to fill, by two for graphs (ordered versus unordered pairs) but by three for the 3-uniform hypergraph, so the directed panel sits far above the undirected one, and the directed cases, exactly the ones Chapter 1 left open, are the first to go.

that does not look at every graph is required.

3.2 The temperature search

We want, for fixed n and m, a graph with as many edges as possible subject to $\lambda^{\max} \le m-1$ (or $\kappa^{\max} \le m-1$ for the vertex variants). The space of graphs on n vertices is enormous and its feasible region has no helpful shape: adding an edge can be harmless until, together with edges added long before, it suddenly creates a forbidden pair. A greedy walk that only ever accepts an improving move stalls at the first such collision, in a graph that is locally maximal but globally far from extremal.

The classical method for this kind of landscape is *simulated annealing* (SA), named after the metallurgical process of cooling a heated material slowly so that its atoms settle into a low-energy crystal rather than a disordered glass. Applied to graphs, annealing escapes local optima by sometimes accepting a worsening move, and by accepting fewer of them as the run proceeds. Each graph G is a state with energy

$$E(G) = -|E(G)| + \Lambda \cdot \max\big(0,\ \lambda^{\max}(G) - (m-1)\big),$$

Algorithm 1: Temperature search for a dense feasible graph

Input: vertices n, value m, start T_0, cooling α, penalty Λ, steps S
Output: the densest feasible graph encountered

$G \leftarrow$ empty graph on n vertices;
$T \leftarrow T_0$;
for S *steps* **do**
  propose G' by toggling one adjacency, biasing removals toward low-sensitivity edges;
  $\Delta \leftarrow E(G') - E(G)$;
  if $\Delta \leq 0$ ***or*** $\mathrm{rand}() < e^{-\Delta/T}$ **then**
    $G \leftarrow G'$;
  if G *is feasible* ***and*** *denser than the best so far* **then**
    record G as the best;
  $T \leftarrow \alpha T$;
return the best recorded graph;

which rewards edges and penalises any excess connectivity. The penalty is a soft constraint rather than a hard one, so the walk may pass briefly through infeasible graphs rather than being walled inside the feasible region, and we return below to why that freedom is worth having. A move toggles one adjacency. The Metropolis rule accepts an improving move always and a worsening move only with probability $e^{-\Delta E/T}$, a chance that shrinks both as the move gets worse and as the temperature drops, and the temperature T falls geometrically, multiplied by a fixed factor just below one at every step. Algorithm 1 is the whole loop, the one place in this chapter where a body is worth showing, because the loop *is* the method.

This loop is what `solve` runs in its discovery mode. With the exhaustive flag off it anneals instead of enumerating, following the Metropolis walk of Algorithm 1 under geometric cooling, removals biased toward low-sensitivity edges, and returns the densest feasible graph it encountered together with the full temperature trace. What it returns is a witness, hence a lower bound, never a proof.

`search_for_dense_graph(n, m, variant, separation)` $\rightarrow$ `SearchResult` **DISCOVER**

in the size n, the ceiling m, the `Variant`, the separation, and a step budget

out a `SearchResult`: the densest feasible graph found, with the full step-by-step trace

how one Metropolis cooling run of Algorithm 1. The driver `best_of_searches` dispatches several independent runs across processor cores and keeps the densest, so a single local optimum never decides the answer.

It is fair to ask why the walk is allowed above the ceiling at all. Reaching every feasible graph does not require it. Removing an edge can only lower $\lambda^{\max}$ and $\kappa^{\max}$, never raise them, so every subgraph of a feasible graph is again feasible, and the empty graph is feasible trivially. From any feasible target we can therefore delete edges one at a time down to the empty graph without ever crossing the ceiling, and reading that path backward builds the target up the same way. The feasible region is connected through the empty graph by single-edge moves that stay inside it, so a walk that never went infeasible could in principle visit every feasible graph there is.

We let it go infeasible anyway, because reachability and efficiency are different things. A feasible graph can be locally maximal, meaning every edge we might add creates a forbidden pair, and yet still sit far below the densest feasible graph. To improve such a graph the walk has to add an edge and then remove a different one, a swap whose first half is infeasible. A walk confined to feasible graphs could only find that better graph by retreating most of the way back to the empty graph and climbing a different slope, a detour so long that the cooling schedule usually ends first. Allowing a brief excursion above the ceiling lets the walk tunnel straight across instead, trading one load-bearing edge for a better one in two moves rather than dozens. The soft penalty also turns the search into a single smooth landscape with no hard walls for the Metropolis rule to bounce off, and the same machinery then carries over unchanged to variants whose feasible region is not so conveniently connected. Going above the bound is never the goal, then. It is the shortcut the annealer takes on its way back down to it.

3.3 Temperature and edge sensitivity

The phrase "biasing removals toward low-sensitivity edges" in Algorithm 1 is where the temperature acquires its meaning, and it is the heart of the method. For an edge e define its *sensitivity*

$$\sigma(e) = \lambda^{\max}(G) - \lambda^{\max}(G - e),$$

the amount by which deleting e relaxes the binding connectivity. A load-bearing edge sits on a tightest cut and has $\sigma(e) > 0$. Removing it would drop $\lambda^{\max}$ and so loosen the very constraint that limits how many edges the graph may hold. A free edge has $\sigma(e) = 0$: it can be removed without easing the binding pair, which usually means it can be put to better use elsewhere.

`edge_sensitivity(G, u, v)` $\to \mathbb{N}$ **MEASURE**

in a graph G and one of its edges $e = (u, v)$

out $\sigma(e) = \lambda^{\max}(G) - \lambda^{\max}(G - e)$

how delete e and let the checker remeasure, where a positive value marks a load-bearing edge. The whole-graph version `sensitivity_map(G)` reuses one shared baseline measurement. This single number is the dial the cooling turns.

When the search proposes a removal it draws the edge with probability proportional to $e^{-\sigma(e)/T}$. The dial is now explicit. At high temperature these weights flatten and the walk toggles edges regardless of how load-bearing they are, exploring widely and willing to dismantle essential structure. As the temperature falls the weights concentrate on $\sigma = 0$, so the walk almost never disturbs a load-bearing edge and instead rearranges only the free ones, until the graph crystallises onto an extremal shape. This is exactly the behaviour one wants, and it is the behaviour the formulation asks for: the temperature tracks how strongly removing an edge changes the bound. Figure 3.2 shows one such run settling onto a certified optimum. The other variants behave identically, and their traces are collected in Section B.2.

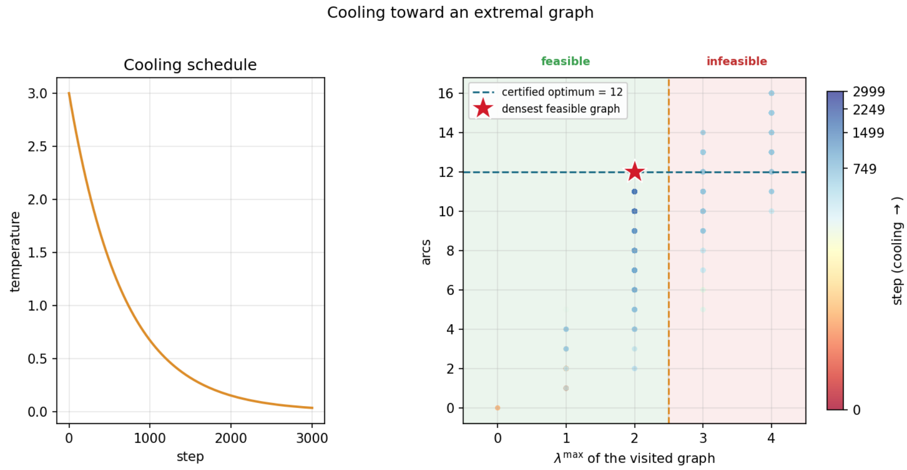


Figure 3.2. A single cooling run on the directed multigraph problem at $n = 4$, $m = 3$. *Left:* the temperature schedule, cooling geometrically from its starting value. *Right:* every visited graph plotted by its binding connectivity $\lambda^{\max}$ (horizontal) against its arc count (vertical), coloured by the step at which it was visited. Annealing uses a soft penalty rather than a hard wall, so while hot (dark points) the walk strays into the infeasible region $\lambda^{\max} > m - 1 = 2$, picking up extra arcs it is not allowed to keep. As the temperature falls (bright points) the penalty drives the walk back across the feasibility boundary and onto the densest graph that stays feasible, the certified optimum of 12 arcs at $\lambda^{\max} = 2$ (star).

The same sensitivities also explain a finished graph. Colouring a graph's arcs by σ shows at a glance which arcs are structural and which are slack. Figure 3.3 illustrates all four sensitivity levels on one directed multigraph: the darkest arcs carry the most flow and lower $\lambda^{\max}$ the most when removed, the cool arc contributes nothing. The picture is not decoration: it is the certificate of tightness made visible, and it tells the search exactly which arcs to leave alone as the temperature falls.

A single cooling schedule can still settle into a local optimum, so in discovery mode `solve` runs several independent schedules from different starting seeds and keeps the densest result across all of them. Because the restarts share no state and none depends on the outcome of another, they run in parallel. The program dispatches them across the available processor cores, so the wall-clock cost of k restarts is a small multiple of a single run rather than k times it, bounded below by the slowest restart and by the cost of starting the worker processes. A handful of parallel restarts is enough to reach the known optima at the sizes we examine. This is a reliability measure, not a change of method, and the parallelism is invisible to every number the thesis reports: the same best graph would be found by running the schedules sequentially, just more slowly.

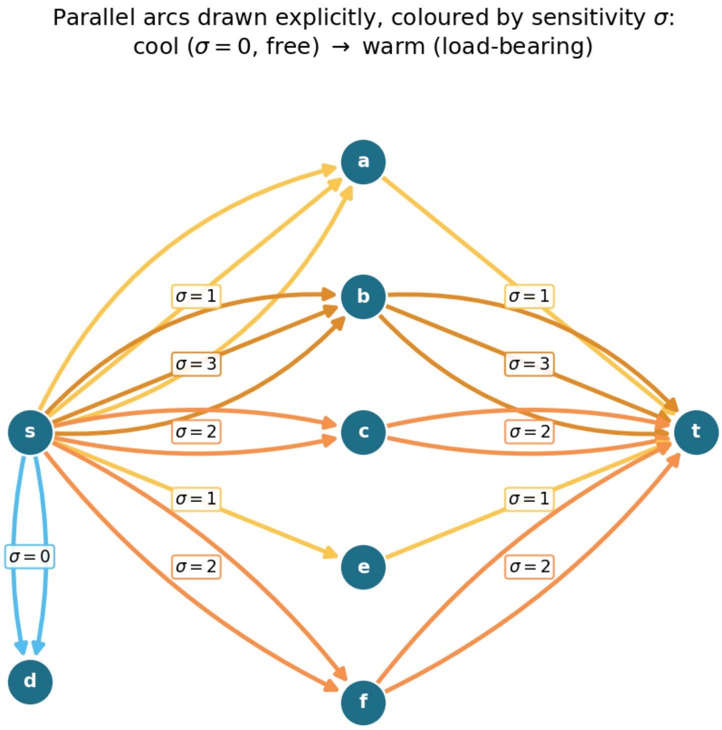


Figure 3.3. A directed multigraph on eight vertices with $\lambda^{\max} = 9$, every parallel arc drawn explicitly and coloured by its sensitivity σ, so multiplicity is read by counting arcs rather than from a label. The point is the contrast between $s \to a$ and $s \to b$: both carry three parallel arcs, yet $s \to a$ is cool $(\sigma = 1)$ because the single arc $a \to t$ downstream throttles that route to one unit, while $s \to b$ is hot $(\sigma = 3)$ because all three of its copies carry flow. The arcs $s \to d$ are coolest of all $(\sigma = 0)$: d is a dead end, so deleting them changes nothing. Removing a hot adjacency drops $\lambda^{\max}$ by its σ while removing a cool one does not, which is exactly what tells the search which arcs to leave alone as the temperature falls.

3.4 Keeping the checker cheap inside the search

The energy of Algorithm 1 reads the binding connectivity $\lambda^{\max}(G)$, and a literal reading of the loop would call the checker afresh on the whole graph at every step. That checker is the most expensive routine in the program, one maximum flow per ordered pair, and the walk runs for thousands of steps with several restarts, so such a search would spend nearly all of its time remeasuring graphs that barely changed from the step before. Two elementary facts remove almost all of that work without altering a single number the search reports.

Proposition 3.1 (Monotonicity of the binding connectivity)**.** *Let G be a graph or directed multigraph, and let G' be obtained from G by changing one adjacency by a single unit of multiplicity. Then*

(1) (monotonicity) *removing the unit cannot raise $\lambda^{\max}$, and adding it cannot lower $\lambda^{\max}$, and*

(2) (unit step) *adding the unit raises $\lambda^{\max}$ by at most one.*

Both statements hold verbatim for the vertex measure $\kappa^{\max}$.

Proof. Fix an ordered pair (s,t). By the max-flow / min-cut theorem $\lambda_G(s,t)$ is the smallest capacity of any cut separating s from t, where the capacity of a cut is the total multiplicity of the arcs that cross it from the source side to the sink side.

For part (1), reducing one adjacency by a unit lowers the capacity of every cut that the changed arc crosses and leaves all other cut capacities untouched, so no cut capacity rises. The smallest cut capacity therefore cannot rise either, which gives $\lambda_{G'}(s,t) \leq \lambda_G(s,t)$. Taking the maximum over all pairs yields $\lambda^{\max}(G') \leq \lambda^{\max}(G)$. Adding a unit is the same statement read backwards.

For part (2), write G'' for G with one unit added to a single adjacency. That extra unit of capacity crosses any fixed cut at most once, so every cut has capacity at most one greater in G'' than in G. The smallest cut capacity thus grows by at most one, giving $\lambda_{G''}(s,t) \leq \lambda_G(s,t)+1$, and together with part (1) we have $\lambda_G(s,t) \leq \lambda_{G''}(s,t) \leq \lambda_G(s,t)+1$. The maximum over pairs gives $\lambda^{\max}(G'') \leq \lambda^{\max}(G)+1$.

For $\kappa^{\max}$ the argument is identical on the vertex-split network of Figure 2.3, where changing an adjacency changes the capacity of its single arc $u^{\text{out}} \to v^{\text{in}}$ in the same way. Parallel copies leave that capacity at one and so change $\kappa^{\max}$ not at all, which only strengthens both parts. □

These two facts settle almost every step without a flow. The energy depends on $\lambda^{\max}$ only through the excess $\max(0,\ \lambda^{\max}-(m-1))$, which is zero exactly when the graph is feasible. So a removal applied to a feasible graph leaves it feasible by part (1), with excess still zero, and needs no measurement at all. An addition made while the graph sits strictly below the bound, at $\lambda^{\max} \leq m-2$, stays feasible by part (2), again with excess zero and no measurement. Only an addition made right at the boundary $\lambda^{\max} = m-1$ can tip the graph over, and only there does the search run a single flow, capped so that it stops the instant it finds one route too many. The exact connectivity value is recomputed only on the rare step whose proposal is genuinely infeasible, where the penalty term needs it. The three cases of Proposition 3.1 read directly off the code:

```
if (is_add and lam_ub <= m - 2) or (not is_add and feasible_current):
    excess = 0                    # part (2) or part (1): provably feasible
elif not exceeds_bound(proposal, m - 1, separation=separation):
    excess = 0                    # at the boundary, but the flow clears it
else:
    excess = measure_full(proposal) - (m - 1)    # infeasible: exact penalty
```

Listing 3.1. The three cases behind Proposition 3.1, in the order the proof gives them: removal from a feasible graph (part 1), addition below the bound (part 2), and only otherwise a single capped flow.

To know which case applies, the search carries a running upper bound on $\lambda^{\max}$, raised by one whenever it accepts an addition (part (2)) and left untouched when it accepts a removal (part (1)), and resynchronised to the exact value at the cadence where it already pays for flows to refresh

Construction	n	m	Search	Theory
spanning tree (simple undirected, m=2)	6	2	5	5
multiplicity-$(m-1)$ star (multi undirected, m=3)	6	3	10	10
bidirected star (simple directed, m=2)	4	2	6	6
bidirected star (simple directed, m=2)	6	2	10	10
hub–bipartite tie (simple directed, m=2)	7	2	12	12
double star (multi directed, m=3)	4	3	12	12
double star (multi directed, m=3)	5	3	16	16

Table 3.1. Validation by rediscovery. For each variant the temperature search, run from scratch with a handful of restarts, recovers exactly the extremal value proved or certified in the earlier chapters. The "Search" column is an honest lower bound found by annealing, and the "Theory" column is the proved or certified value.

the sensitivity map. The bound can only ever sit at or above the truth, so any step it clears is certainly feasible.

The key point is that the energy returned on every step is, by Proposition 3.1, exactly the value the naive implementation would have computed. Every accept-or-reject decision, every random draw, and the graph the walk finally returns are therefore unchanged, and the speed-up is invisible to every number the thesis reports. This is verified rather than asserted. A regression test runs both implementations, the fast one and a reference that measures exactly at every step, across all four graph variants and both separations, and checks that the two agree step for step in energy, in every accept-or-reject decision, and in the final graph, and that the returned graph is feasible under the exact checker. Nothing about the optimisation hides inside the bound: the witness the search reports is always certified by an exact measurement, exactly as in the slower search.

It is worth noting where this method earns its keep. For undirected graphs one could resynchronise the exact value cheaply from the Gomory–Hu tree of Chapter 2, which encodes every pairwise edge-connectivity in $n-1$ flows. The directed and vertex variants admit no such tree, so there the monotone bound and the capped predicate are the only inexpensive handle on feasibility, and they are what let one annealer serve every variant at speed.

3.5 Rediscovering the known extremisers

A search method is only worth trusting if it recovers what is already known before it is pointed at what is not. Run from the empty graph on the cases whose answers Chapter 1 and Chapter 2 settled, and told only to pack in edges without breaching the connectivity ceiling, the cooled search arrives unprompted at the named constructions. Table 3.1 reports the densest graph it found against the value the theory predicts, and in every case the two agree.

What the table hides is that the graphs themselves are the named constructions, not merely graphs of the right size. The simple undirected run at $m=2$ settles on a spanning tree, the only shape with $n-1$ edges and no cycle. The undirected multigraph run settles on a star with every edge at multiplicity $m-1$, an instance of the multi-tree extremiser behind Theorem 1.3. The directed multigraph runs settle on the double star of Chapter 2, both directions of every spoke at full multiplicity. The simple directed runs find the bidirected star at $n=4$ and, at $n=7$, land on twelve arcs, the value at which the hub and the one-directional wall tie, exactly the crossover Theorem 1.10 predicts. The cooled search does not know these names. It rediscovers the structures because, under the connectivity ceiling, there is nowhere denser to go.

Some extremisers are past the size at which a quick search lands reliably, but the exact checker of Chapter 2 settles them immediately by measuring their connectivity. The one-directional bipartite digraph on eight vertices has 16 arcs at $\lambda^{\max}=1$, and 16 already exceeds the linear hub value $2(n-1)=14$: the smallest size at which the quadratic phenomenon strictly wins, made concrete. It is also a measured illustration of the search's limit: repeated cooling runs at $n=8$, $m=2$ stall at the hub value of 14, because dismantling a finished hub and rebuilding a one-directional wall is far more than the single-arc moves of the walk can tunnel through. The known answer there comes from the construction and the checker, not from the search, which is exactly the division of labour this chapter argues for. The augmented bipartite digraph on ten vertices at $m=3$ has 30 arcs at $\lambda^{\max}=2$, the thirty-arc counterexample of Remark 1.12, confirmed pair by pair against the naive bound of 27 it refutes. The directed double star on seven vertices at $m=4$ has 36 arcs at $\lambda^{\max}=3$, matching $2(n-1)(m-1)$. None of these is a proof of optimality. Each is an exactly measured feasible graph, a lower bound the checker stamps as genuine.

3.5.1 Simulated annealing versus tabu search

The annealer is not the only way to walk this landscape. A natural alternative, suggested by Jan Goedgebeur, is *tabu search* (Table 4.2 names the implementation), which keeps the energy and the neighbourhood of Algorithm 1 but replaces the cooling schedule with a deterministic best-improving step. At each step it evaluates every single-edge move from the current graph and takes the one that lowers the energy most, then forbids the pair it just changed for a fixed number of steps, the *tabu tenure*, so the walk cannot immediately undo its last move and stall in a two-step cycle. When no improving move is left and the search stalls, it perturbs the incumbent with a few random moves and carries on. There is no temperature and no accept-worse coin: apart from the perturbation kicks the walk is deterministic. Where annealing explores by occasionally stepping uphill and trusts the cooling to settle it, tabu climbs greedily and trusts the tabu list to keep it from retracing its path.

The two engines rediscover the same values, but at very different speeds, and on the harder directed cases the difference is decisive. Table 3.2 runs them head to head at an equal budget of six seconds each, both restarted within that budget. On the simple undirected problem both reach the optimum at once. On the two directed problems past the certified range tabu reaches the optimum within the budget while the annealer plateaus below it, assembling the eighteen-arc simple-directed extremiser and the full twenty-four-arc double star at $n=7$ where the annealer stalls at sixteen and twenty. The cause is the one already met at the $n=8$, $m=2$ hub stall above. The annealer cools onto whatever dense shape it first reaches and then cannot dismantle it within the budget, whereas tabu's best-improving step follows the steepest density gradient and the tabu list stops it sliding back, so it assembles the structured extremiser the cooled walk struggles to find.

Variant	n	optimum	best in 6 s		time to optimum	
			SA	tabu	SA	tabu
simple undirected	7	9	9	9	1.3 s	0.1 s
simple directed	7	18	16	18	—	2.6 s
directed multigraph	7	24	20	24	—	2.5 s

Table 3.2. Simulated annealing (SA) against tabu search at an equal six-second budget per method, the annealer restarted from fresh seeds and tabu restarted on stalls. Only the simple undirected value is a proved optimum (Mader). The two directed values are the best known rather than proved, found by search and construction past the certified range that stops below $n = 7$ for those variants (Section 3.6), so here the "optimum" column is the target each engine is trying to reach. Both reach the simple undirected optimum. On the two directed problems tabu reaches the best known value while the annealer plateaus below it. The last two columns give the wall-clock time each method first reached that value on the reference machine, with a dash where it did not reach it in the budget.

Figure 3.4 shows the divergence directly, plotting the densest feasible graph each engine has found against wall-clock time on the two directed multigraph cases. Tabu climbs in a few decisive steps and holds at the optimum, while a single annealing schedule rises quickly and then flattens a few arcs below it. The annealer's independent restarts recover the optimum on the smaller case, as Table 3.1 records, so a single plateau is not the whole story there. But at $n = 7$ the plateau persists within the budget, and one tabu schedule already reaches what the restarted annealer does not.

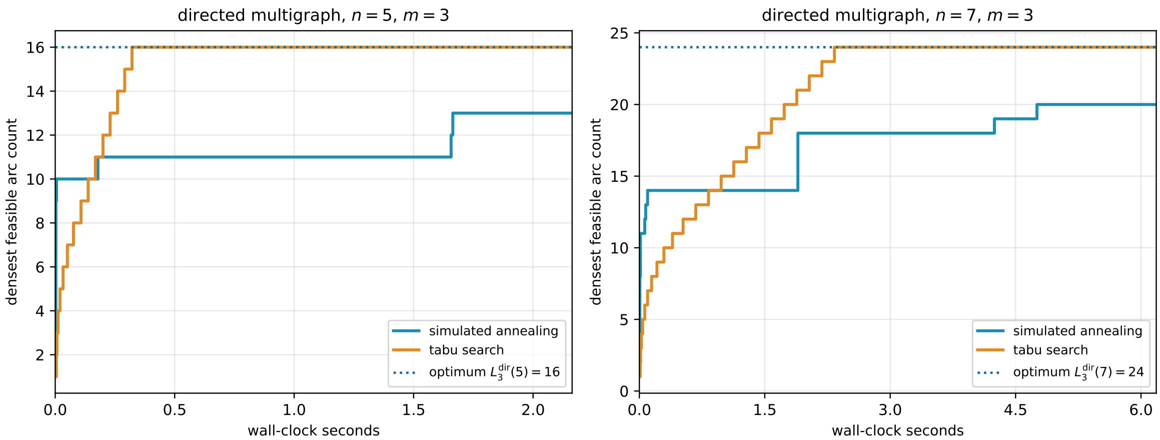


Figure 3.4. Densest feasible graph found against wall-clock time, for one simulated-annealing schedule and one tabu schedule on the directed multigraph at $n = 5$ and $n = 7$, $m = 3$. Tabu climbs in a few best-improving steps to the optimum (dotted) and holds, while a single annealing schedule rises quickly and then flattens below it. The time axis is wall-clock on the reference machine, so unlike the seed-reproducible figures elsewhere this one is a representative timed run.

This advantage comes at a cost the equal-time comparison already absorbs but is worth naming: each tabu step scans the whole neighbourhood, so it is far more expensive than the annealer's single random proposal, and tabu takes many fewer but better-aimed steps in the same wall-clock. Tabu search is therefore the program's default for solving the problems, reaching every value this thesis reports, the rediscovery of Table 3.1 and the conjectural lower bounds alike, and reaching the harder directed optima past the certified range where the annealer plateaus. Simulated annealing is kept for the self-check and verification, where the small values fall at once, the lighter per-step cost is all that is needed, and its independent restarts parallelise trivially across cores.

3.6 The trichotomy: proved, certified, conjectured

It is worth stopping to lay out, for every variant, the three kinds of knowledge the computation can hold about it, because the whole value of the pipeline is that they are kept apart and never quietly merged. About any given value the program can do one of three things. It can *prove* the value, closing a fixed size from above by exhausting every graph of that size, an upper bound with the standing of a hand proof. It can *certify* it, the same kind of upper bound but obtained by the cut-counting optimisation rather than blind exhaustion, reaching a vertex or two further. Or, where proof and certificate both run out, it can only *conjecture* the value, exhibiting a concrete dense graph that pins down a lower bound and leaving the matching upper bound to a later argument.

Proof and certificate bound the answer from above. The search bounds it only from below. Where a closed *formula* exists it can tie the two together, threading through the certified small cases and the conjectured large ones as a single curve. This is the discipline the whole thesis runs on, and it is worth stating as one: the search proposes a lower bound, a certificate or a hand proof disposes the upper bound, and only when both meet is a value called settled.

We begin with the single variant where all three kinds of knowledge are non-trivial and visibly distinct, the directed simple arc problem at a fixed $m \geq 3$, and then sweep the whole family. For that variant the knowledge splits cleanly by the size of the graph.

For small n the answer is not conjectured at all. The pruned exhaustion inside `solve` walks every simple digraph on n vertices and returns the exact maximum, complete and certain, an upper bound with the standing of a hand proof. At $m = 3$ it certifies $6, 9, 12$ for $n = 3, 4, 5$, and there the exhaustion stops within a reasonable budget, the pruned walk no longer closing $n = 6$ in tractable time. That is the certified region, finite and closed.

Beyond it the ceiling is gone and only lower bounds remain, each witnessed by a concrete feasible graph. The search finds the extremal 15 arcs at $n = 6$ on its own. Past that size it falls behind: across six restarts it reaches $17, 17, 20, 23$ for $n = 7, \ldots, 10$, while the named constructions of Chapter 1, measured exactly by the checker, supply $18, 21, 25, 30$, the last being the thirty-arc graph of Remark 1.12. The reported lower bound at each size is the better of the two, exactly as the rediscovery section combined search and checker by hand. These are conjectures with an honest floor and no matching ceiling. That is the searched region, unbounded and forever one-sided.

The two regions are tied together by a single closed form. The conjectured value

$$\ell_m^{\mathrm{dir}}(n) = \max\left(m(n-1),\ \left\lfloor \frac{(n+m-2)^2}{4} \right\rfloor \right) \qquad (n \geq m)$$

of Conjecture 1.14 is not a fresh guess bolted on past the certified range. It is one formula that already passes through every certified value at small n and then continues, without a seam, through every searched value at large n. The directed-arc panels of Figure 3.5 show this directly, the certified squares at the small sizes and the open lower-bound circles beyond both sitting on the one red conjecture curve, proof below the formula and witnessed graphs above it. The formula is the through-line that makes the small certified cases and the large conjectured ones a single statement rather than two.

Two features of those directed-arc panels are worth naming. The crossover from the linear hub term to the quadratic bipartite term happens at $n = 9$, where the second argument of the maximum first wins, and that is past the last certified size. The shape of the answer therefore changes in exactly the region where only the search can see it, which is precisely why a discovery tool was needed and a certifier alone would not do. And at $n = 10$ the formula reads 30, the count of the very counterexample that overturned the natural initial conjecture, so the closed form is not a smooth fit to easy data but one that survives the case that broke the previous guess.

Asking the same three questions of every variant, the answers sort into three regimes, and Figure 3.5 shows all twelve at once, one panel per variant, in exactly this proved, conjectured, and guessed language.

In the first regime the closed form is already a theorem for every n, proved by hand in Chapter 1 or cited, and the computation only confirms it. The undirected families live here, edge and vertex alike, together with the directed problems at $m = 2$. For these the certified small cases and the rediscovered constructions of Table 3.1 are validation, not new knowledge: the formula was never in doubt, and the program's role is to check that the pipeline reproduces it. That the two multigraph vertex rows measure exactly as their underlying simple graphs is a matter of the counting convention of Remark 2.1 rather than a theorem, so those rows need no computation of their own at all. The directed vertex rows do need their own, since Whitney's inequality gives a lower bound there but no upper one, and they get it: every exact point on those panels comes from the search and the exhaustion run under the vertex test.

In the second regime the three columns genuinely differ and the computation is load-bearing. This is the regime of the worked example above. The directed simple arc and vertex problems at $m \geq 3$ are certified only at the small n the exhaustion reaches and conjectured by the search beyond, with the formula of Conjecture 1.14 the through-line.

The directed multigraph arc problem sits here too, with one extra economy: in its cut-counting mode `solve` proves the m-free number $M^*(n) = 2(n-1)$ outright for $n \leq 6$, and the scaling identity $L_m^{\mathrm{dir}}(n) = (m-1)\, M^*(n)$ of Chapter 2 carries that one certified number to every m at once, so the closed form $2(n-1)(m-1)$ is proved across the whole m axis at those sizes. Past the crossover at $n = 7$ the conjectured value switches to the bipartite branch $(m-1)\lfloor n^2/4 \rfloor$, where the even sizes are proved conditional on the odd ones and only a single mixed-regime minimum-degree statement keeps the induction from closing (Appendix A).

In every row of this regime the formula agrees with proof where proof reaches and with the search where it does not, exactly the linkage the worked example displayed.

In the third regime there is no formula at all, and the honesty is in drawing the guess as a guess. The undirected vertex problem at $m \geq 6$ has been open since 1974 with its extremal structure genuinely unknown, and the hypergraph variants beyond the proved edge case are unexplored, the directed ones needing a one-tail, many-head reading of a Berge edge that this thesis defines itself only so that `solve` can measure it at all. Here the program still computes exact values at the few sizes that finish, and its search still exhibits feasible graphs at a handful more, but no closed form is in sight. The panels of Figure 3.5 carry, instead of a red conjecture curve, a yellow line that simply interpolates the machine points in straight segments, trusted no further than the eye. We draw it *through* the points rather than fitting a smooth curve above them: the points are honest lower bounds, the truth lies on or above them, and a line riding above the points would claim more than the search ever found.

Two cautions govern how to read the dotted curves. At small n the complete graph or hypergraph is itself feasible, its binding connectivity being only $n-1$, so the earliest points merely trace the trivial maximum and reveal nothing of the open behaviour: the undirected-vertex points run up $\binom{n}{2}$ until n passes m, and only then bend toward the linear growth the proved edge value predicts.

Past that bend the dotted line is as strong as the best lower bound behind it, which at each size is the better of the quick search and a named construction the checker verifies. The search alone stalls, exactly as it stalled on the quadratic wall in the rediscovery above: where it cannot build a denser construction its own reports climb only linearly. The clearest case is the directed hypergraph, whose bipartite family is quadratic, reaching $\sim (m-1)n^2/\big(4(r-1)\big)$ hyperarcs by Proposition A.50: there the walk slips below that construction at larger n, so it is the construction that carries the circles, with the search free to beat it wherever it can. We therefore read the dotted trends as honest lower bounds and not as fitted laws, and the one closed-form hypothesis we record, the quadratic for the directed hypergraph, comes from the construction and not from the shape of the dots.

So to the question of whether one formula links what is proved to what is conjectured, the answer is three-valued and depends on the variant, and the grid of Figure 3.5 shows all three at a glance. Where the problem is already solved the curve is blue, a theorem the computation merely agrees with. Where the problem is open but tractable the curve is red, a conjecture the certified squares confirm below and the searched circles confirm above, and that is the case the program was built for. Where the problem is genuinely hard the curve is yellow, a guess interpolated through the search points and an open question of what, if anything, the dots truly trace.

One feature stands out in the bottom row: the directed hypergraph panels, both arc and vertex, climb steeply at $m = 3$, the yellow guess curving upward rather than running straight. This is genuine and not search instability: the directed Berge edge packs a tail and $r - 1$ heads into a single object, and the bipartite construction of Proposition A.50 already reaches roughly $(m-1)n^2/(4(r-1))$ hyperarcs, so the value grows with n^2. The same quadratic shape sits one column over, in the directed arc and vertex panels, whose conjectured red curves carry the $\left\lfloor \frac{(n+m-2)^2}{4} \right\rfloor$ term and rise just as fast. Super-linear growth is the signature of *direction* across the grid, not of the hypergraph alone: every directed column bends upward while the two undirected columns stay linear. Figure 3.6 redraws the same grid at $m = 6$: the proved curves climb linearly with m exactly as their formulas predict, while the open variants' search points and their yellow interpolation climb higher without a proved curve to meet, so raising the threshold does not change which regime a variant lives in, only how far the red and yellow curves have travelled.

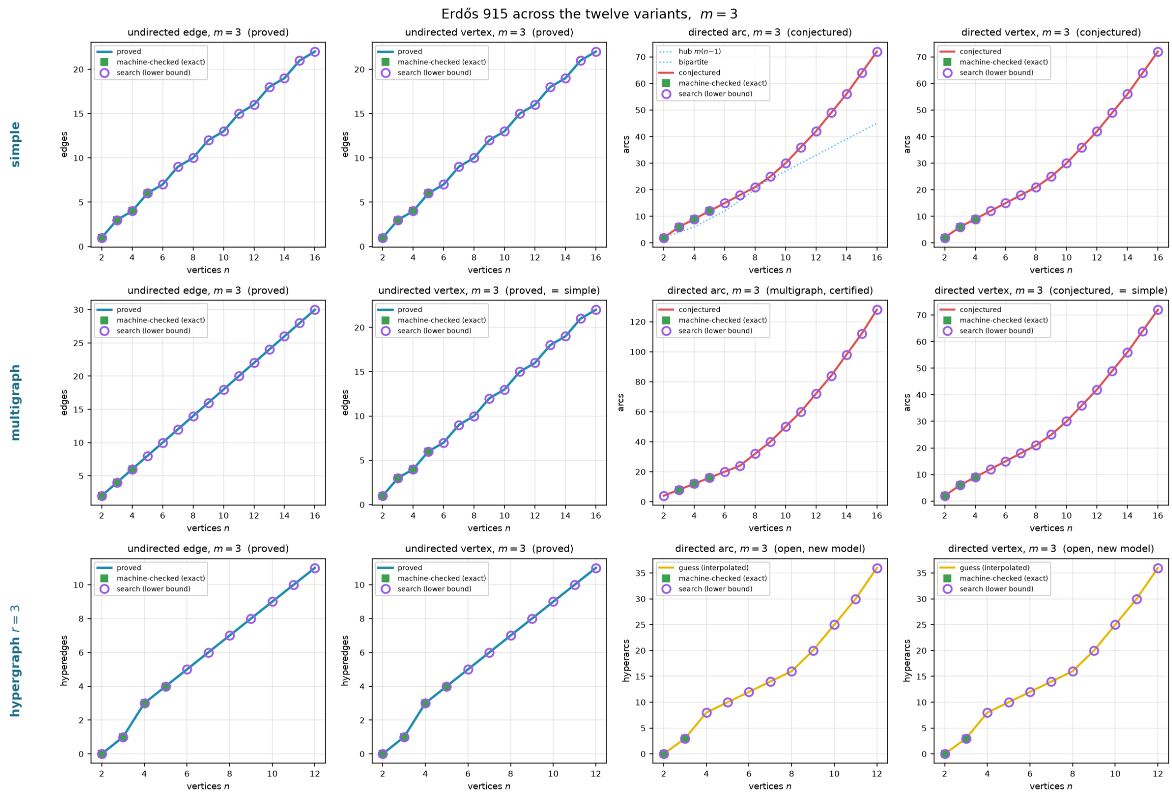


Figure 3.5. All twelve variants at $m = 3$, edges against n, laid out by model (rows: simple, multigraph, 3-uniform hypergraph) and by separation and direction (columns: undirected edge, undirected vertex, directed arc, directed vertex). A *blue* curve is a value proved for all n, a *red* curve a conjectured formula, and a *yellow* curve a bare interpolation of the machine points where no formula is known, the three told apart by colour rather than by dash pattern. Filled squares are the sizes `solve` proved or certified exactly. Open circles are lower bounds at larger n, the better at each size of the search's densest find and a named construction measured by the checker, reported as a running maximum so a feasible graph at one size is never lost at the next. On the open variants the yellow curve is the piecewise-linear interpolation *through* those circles, never above them, and the axes are scaled to the data rather than to a loose certain interval. The directed hypergraph panels in the bottom row rise steeply because their value is quadratic in n, not linear.

The same grid answers the harder question the trichotomy raises, which is what the search is

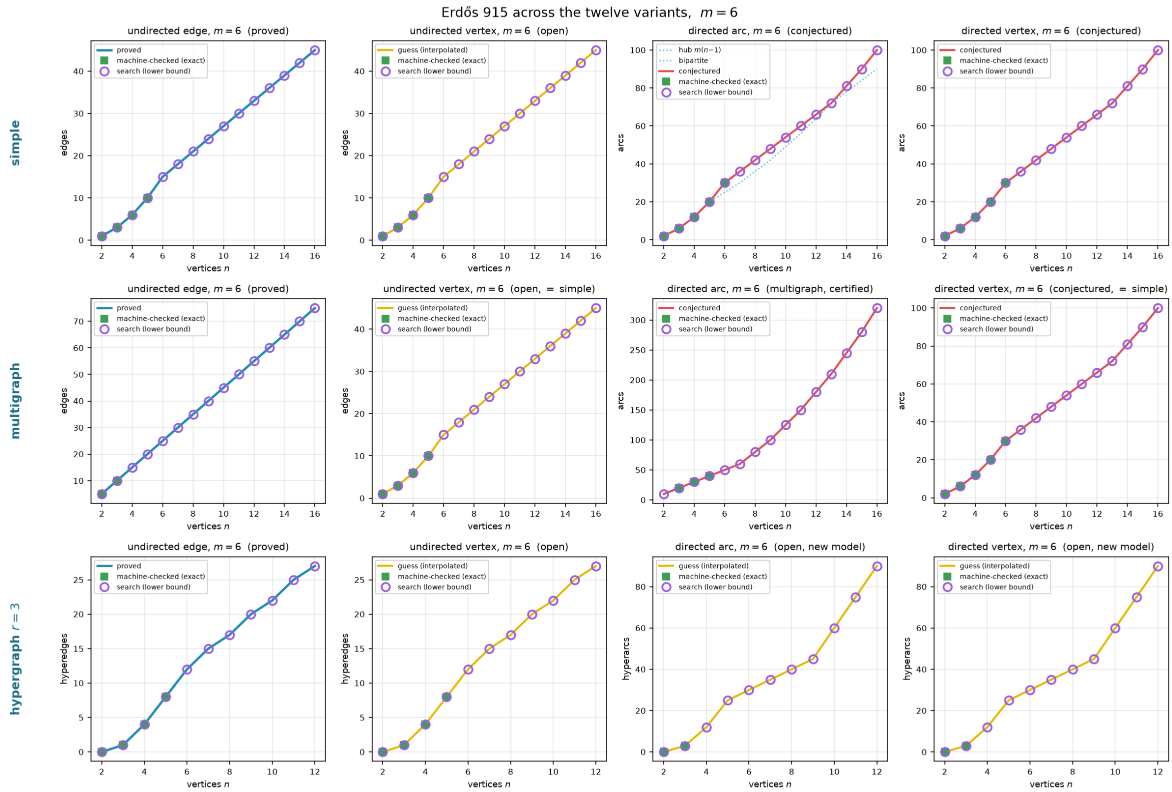


Figure 3.6. The same twelve variants at $m = 6$. The undirected edge and multigraph edge cases remain proved (Mader's theorem holds for all m), while the undirected vertex separation is the famous open problem at $m \geq 5$ and the directed cases carry the same conjectured formula scaled to $m = 6$. On the open vertex panels the circles never sit below the proved edge value at the same size, because every graph feasible for the edge problem is feasible for the vertex problem (Whitney), so the edge extremiser is an honest lower bound there, and the search is free to climb past that floor where the vertex problem is genuinely richer. Comparing this figure with the $m = 3$ version above shows that the proved cases grow linearly with m as the formulas predict, while on the open cases the circles and their yellow interpolation climb steadily without a proved curve to meet.

worth where no upper bound exists to check it against. It says two things at once. First, wherever an upper bound exists, proved or certified, the search lands exactly on it and never above it at every plotted size, which is strong evidence that the search is not leaving edges on the table, with the one instructive exception recorded above: the directed $m = 2$ run at $n = 8$, where it stalls on the hub and the bipartite optimum must be supplied by the construction instead.

Second, on the directed arc problem at $m \geq 3$, where no upper bound is known past the certified range, the plotted lower bounds land exactly on the conjectured value $\max(m(n-1), \lfloor (n+m-2)^2/4 \rfloor)$ at every reachable n, and the certified points agree with it as far as they go, but past the first open size it is the constructions, not the cooled walk, that carry the bounds there. Run from scratch the search confirms the conjecture at $n = 6$ and then plateaus a few arcs short ($17, 17, 20, 23$ against $18, 21, 25, 30$ at $n = 7, \ldots, 10$, across six restarts), for the same reason as the $m = 2$ stall recorded earlier: the extremal shapes past the crossover are bipartite walls, and a walk that has crystallised onto a hub cannot rebuild itself into a wall by single-arc moves within the cooling budget.

So to the question "do we believe the reported lower bounds are best possible," the grid answers: wherever a proof or a certificate can check them, yes, and on the open sizes they sit on the only value anyone has conjectured, with the search confirming the first open size and the constructions carrying the rest. The gap between those lower bounds and what is proved is exactly the gap between the conjecture and the missing upper bound, which Chapter 4 names precisely. The search has not closed that gap, and it cannot. What the pipeline supplies is reproducible evidence for which side of the gap the truth lies on.

Honesty

A number found by the search is reported as a lower bound, and is marked as certified or proved only when a separate cut-counting optimisation or a hand proof supplies the matching upper bound. No extremal value is ever asserted on the strength of the search alone. The figures in this chapter use fixed random seeds, so every search point is reproducible.

Chapter 4

Synthesis, Results, and Open Problems

The journey set out from one clean theorem and followed it across twelve variants, proving what proved cleanly, building a machine where the proofs ran out, certifying the small cases the machine could close, and using the search to discover the dense graphs the proofs had singled out and to point at the ones still open. This closing chapter draws the threads together. It maps the directed frontier, where proof and certificate both still fall short of the full answer, names the two genuine surprises the landscape contains, gathers the twelve answers into one picture, and says plainly what the program changed and where it stops.

4.1 The directed frontier and the single open lemma

Two of the open directions are directed, and they share a structure worth stating exactly rather than hiding. The directed arc problem is a race between two constructions on different scales. The directed hub (Construction 1.9) reaches $m(n-1)$ arcs and grows linearly. The augmented bipartite construction (Construction 1.13) packs $\lfloor (n+m-2)^2/4 \rfloor$ arcs and grows quadratically. Which leads depends on n, and the two cross at an order linear in m, near $n=9$ at the $m=3$ of Figure 4.1. Conjecture 1.14 asserts that together they are the whole story.

Conjecture 1.14 is proved as a lower bound for all m, and it is a theorem at $m=2$ by Theorem 1.10. Past $m=2$ it sits squarely in the second regime of the trichotomy of Section 3.6: certified by exhaustion at the small sizes `solve` can close for $m=3$, conjectured by the temperature search beyond, and never beaten anywhere the search reaches. What is missing is a general upper bound for $m \geq 3$, and the obstacle is structural rather than computational.

The route to the upper bound is to show that an extremal digraph, once it is past the hub regime, must look like the augmented bipartite construction: its vertices split into a part A and a part B with every arc running $A \to B$, and B thickened only mildly. Three structural facts, each proved in Appendix A, push in this direction. At $m=2$ out-neighbourhoods are mutually unreachable and second out-neighbourhoods are disjoint, and for every m a complete $A \to B$ partition forces each vertex of B to absorb at most $m-2$ arcs from inside B. With the partition free to vary, the total arc count $|B|(|A|+m-2) = |B|(n-|B|+m-2)$ is maximised at $|B| = \lceil (n+m-2)/2 \rceil$, which gives $\lfloor (n+m-2)^2/4 \rfloor$, precisely the conjectured upper bound.

The reduction is clean except at a single point, and it is worth naming the point exactly rather

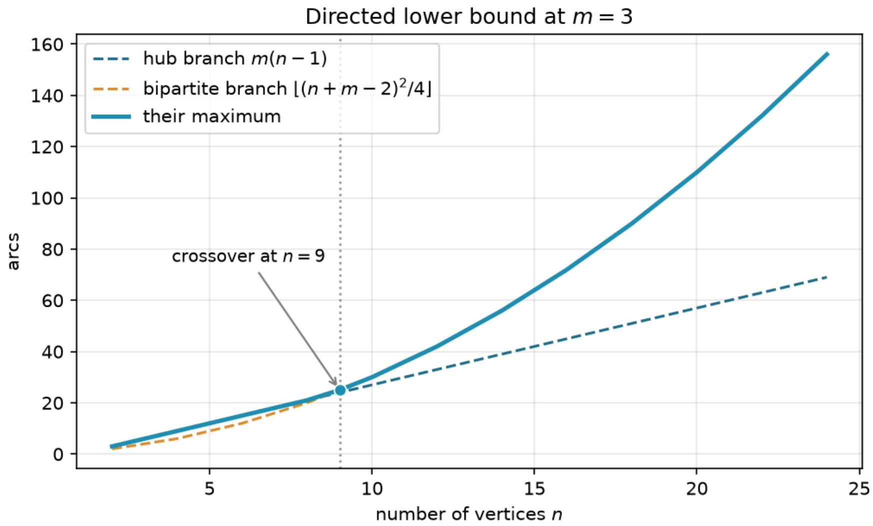
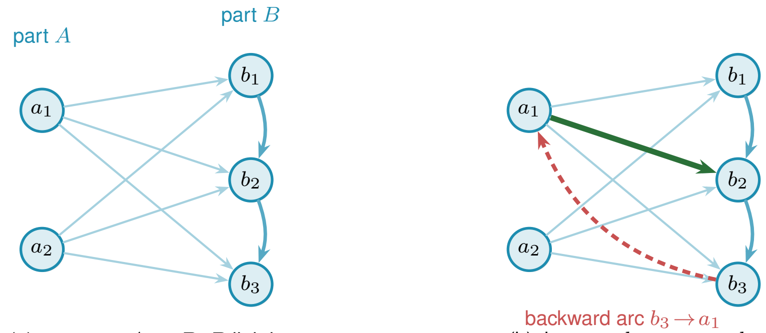


Figure 4.1. The two directed lower bounds at $m = 3$. The linear hub branch leads for small n. The quadratic augmented-bipartite branch overtakes it near $n = 9$ and leads thereafter. Conjecture 1.14 asserts their maximum is the exact value.

than hiding it. The count goes through provided the extremal digraph decomposes the way Figure 4.2(a) draws it, and that shape is four conditions rather than one: a partition $V = A \cup B$ exists, every arc from A to B is present, no arc lies inside A, and no arc runs from B back to A. The last condition is the one with a name and a picture. Under it a route between two vertices of B can never escape B, so the connectivity inside B is governed entirely by B's own arcs and the two-hop budget applies, and Figure 4.2(b) shows how a single backward arc destroys exactly that. It is also the condition where the natural delete-and-compensate exchange fails to be monotone, which is why it is the one usually named as the obstacle.

Naming it alone would nonetheless overstate the position, so the appendix states the whole hypothesis as Conjecture A.21: an extremal non-hub digraph admits such a partition, complete forwards, empty inside A, and with nothing coming back. Nothing proved here forces an arbitrary extremal digraph into that shape, and the small-case evidence for it is exactly the kind of thing the search of Chapter 3 can probe. So the honest summary is that Conjecture 1.14 for $m \geq 3$ awaits one structural decomposition theorem, of which the backward-arc clause is the hardest quarter.

What does not wait is the leading constant. Proposition A.22 proves, with no hypothesis at all beyond feasibility, that a digraph with $\lambda^{\max} \leq m-1$ has at most $\lfloor n^2/4 \rfloor + \sqrt{m}\, n^{3/2}$ arcs, and that deleting at most $\sqrt{m}\, n^{3/2}$ of them leaves a one-directional bipartite digraph. Its engine is a single observation the rest of the chapter has been circling: for any ordered pair, the direct arc and the two-step detours through distinct midpoints are automatically arc-disjoint, so feasibility caps their number, and summing that cap over every pair caps $\sum_x d^+(x)d^-(x)$. A vertex with both a large in-degree and a large out-degree is therefore expensive, every vertex is nearly a pure source or a pure sink, and throwing away each vertex's smaller side costs only $O_m(n^{3/2})$ arcs and leaves the bipartite wall standing. So $\ell_m^{\mathrm{dir}}(n) = n^2/4 + O_m(n^{3/2})$ for every fixed m, and what Conjecture 1.14 still claims beyond that is the second-order term $(m-2)n/2$, not the shape



(a) every arc $A \to B$, B lightly thickened, none returns to A

(b) the route $b_3 \to a_1 \to b_2$ now escapes B through A

Figure 4.2. The directed frontier reduction and the single obstruction. **(a)** The structure the upper bound wants. Every arc runs from A to B (faint), B carries only a light internal thickening (the $m-2$ budget, dark blue), and no arc returns to A, so a route between two vertices of B never leaves B and the count $|A||B| + (m-2)|B|$ is complete. **(b)** A single backward arc $b_3 \to a_1$ (red) breaks this. The route $b_3 \to a_1 \to b_2$ (red leg then green leg) now escapes B through A, an extra route the budget never counted. Ruling out backward arcs in an extremal digraph is the one missing lemma behind the $m \geq 3$ upper bound, and the checker refuted a proposed counterexample by finding exactly such a hidden route.

of the answer.

The machine has been useful here in the negative direction too: a proposed counterexample to the conjecture, a backward arc bolted onto the thirty-arc graph, was refuted by the checker, which found the third route the construction's author had missed.

The directed vertex-disjoint problem shares the constructions but not the proofs, and the distinction matters more than it looks. Whitney's inequality $\kappa \leq \lambda$ says a digraph obeying the arc ceiling obeys the vertex one, so the arc constructions are vertex-feasible and every lower bound carries across for free. It says nothing in the other direction, and the vertex-feasible family is the larger of the two, so no arc upper bound restricts it. At $m = 2$ the vertex value is nonetheless the same, and Theorem 1.11 proves it by running the induction a second time under the stricter separation, with its own exhaustive base cases through $n = 7$. For $m \geq 3$ the vertex value is conjectured equal to the arc value, on the strength of the same constructions and of exhaustive small-case checks run under the vertex test rather than inherited from the arc one. What it will not do is follow from the arc case, even once Conjecture A.21 is supplied, because an upper bound proved over the smaller family says nothing over the larger one. The vertex problem needs its own upper bound at $m \geq 3$, and Table 4.1 lists it as a separate conjecture rather than a corollary. The directed multigraph problem has its own two-branch story, parallel to the simple one. The conjectured value is

$$L_m^{\mathrm{dir}}(n) \;=\; (m-1)\,\max\!\Big(2(n-1),\ \Big\lfloor \frac{n^2}{4} \Big\rfloor\Big),$$

where the double star realises the linear branch and the one-directional bipartite multigraph, every arc at multiplicity $m-1$, realises the quadratic one.

The quadratic branch here uses the balanced partition $|A| = \lfloor n/2 \rfloor$, $|B| = \lceil n/2 \rceil$, giving $(m-1)\,\lfloor n^2/4 \rfloor$ arcs. The multigraph and the simple digraph disagree about the best partition, and the reason is worth spelling out. In a multigraph the $m-1$ routes per pair are supplied directly by $m-1$ parallel copies of every $A \to B$ arc, so nothing needs to be added inside B, and the arc count $(m-1)|A|\,|B|$ is largest at the balanced split. In a simple digraph each ordered pair carries at most one arc, so the extra routes must instead come from arcs inside B, and feeding B those in-arcs is what pushes its optimal size up to $|B| = \lceil (n+m-2)/2 \rceil$ once $m \geq 4$ (at $m \leq 3$ the balanced and shifted splits happen to give the same count, as Construction 1.13 records). One consequence is that the multigraph's two branches cross at $n = 7$ for every m, because the factor $m-1$ multiplies both branches and cancels, whereas the simple crossing point grows with m and already sits near $n = 9$ at $m = 3$.

On the linear side the certifier closes the problem, proving $L_m^{\mathrm{dir}}(n) = 2(n-1)(m-1)$ for $n \leq 6$ and every m at once (Theorem 2.2), and at $m = 2$ the whole problem collapses onto the simple digraph theorem for every n, because two parallel arcs are already two routes (Theorem A.31). On the quadratic side, a scaling-and-deletion induction proves the bipartite bound for even $n \geq 8$, conditional on the bound one vertex below. The odd case once reduced to a single minimum-degree statement (Conjecture A.29), which the program confirms in the purely saturated and purely unsaturated regimes but leaves open in the mixed one, where multiplicities strictly between 0 and $m-1$ defeat the integrality the two pure regimes lean on (Appendix A).

A sharper route now bypasses that conjecture at $m = 3$, and the way it does so is the cleanest instance in the thesis of the proof handing its remaining work to the machine. An *attachment lemma* (Lemma A.34) bounds the degree of a vertex glued onto the bipartite extremiser, and a *conditional odd step* built on it (Theorem A.36, for $m \in \{3, 4\}$) carries the value and extremal characterisation from each even level to the next odd one. At $m = 3$ that characterisation propagates through both parities (Remark A.39), so the entire problem for all $n \geq 8$ collapses onto two finite, integral statements at $n = 7$:

(a) $L_3^{\mathrm{dir}}(7) = 24$, and

(b) the only 24-arc feasible multigraphs on 7 vertices with maximum degree at most 8 are $2 \cdot B_{3,4}$, $2 \cdot B_{4,3}$, and the doubled bidirected path P_7.

Conjecture A.29 is thereby off the critical path for $m = 3$, replaced by two machine-checkable targets: a MILP infeasibility check for fact (a), and an exhaustive enumeration of degree-bounded multigraphs for fact (b). The enumeration has since closed fact (b): the sound generation enumerator of Chapter 2 completed the sweep and returns exactly the three predicted extremisers (Remark A.39). The value statement (a), $L_3^{\mathrm{dir}}(7) = 24$, is now the one computation still standing between the $m = 3$ problem and a complete theorem, and there the MILP remains runtime-limited on the bundled solver.

The linear branch turns out to be richer than the double star first suggested. Every doubled bidirected spanning tree is extremal, confirmed exhaustively at $n = 4$ and 5, and the saturated attachment lemma (Lemma A.37) explains why: the doubled trees are exactly the closure of a single doubled edge under maximal attachment. Several distinct extremisers tie at 24 arcs for

$n = 7$, and sifting and classifying them is precisely what the enumeration must do.

For $m \geq 4$ the conditional step still yields the value but uniqueness no longer propagates through odd levels, and for $m \geq 5$ even the value step stalls. These residual gaps are stated in full among the open problems below. They are finite, identified targets rather than mysteries, and natural goals for the next round of certified computation.

4.2 Two principal phenomena

Across all the variation, two phenomena stand out as the ones a first guess would not have predicted.

The first is the divergence at $m = 5$. The edge and vertex problems agree for $m \leq 4$, and it would be natural to expect them to agree forever. Instead they part company at the very next value, with $k_5(n) = \lfloor 8n/3 \rfloor - 3$ pulling strictly above $\ell_5(n)$, and beyond it the vertex problem becomes genuinely hard and has stayed open since 1974.

The second is the quadratic bipartite phenomenon. In an undirected graph every edge contributes to the degree of both its endpoints, and Mader's theorem turns that symmetry into a hard ceiling: $\lfloor m(n-1)/2 \rfloor$ edges, linear in n. A digraph breaks the symmetry. Place all n vertices into two equal parts A and B and draw every arc from A to B, one direction only. The result has $\lfloor n^2/4 \rfloor$ arcs, quadratic in n, yet $\lambda^{\max} = 1$: every pair has at most one arc-disjoint route, because every route must cross the A-to-B wall exactly once and no arc crosses the other way to offer a second path. A graph whose arc count grows like n^2 can sit at connectivity one, something no undirected graph can do past a handful of vertices.

The initial conjecture for the directed case was linear, matching the undirected intuition. The one-directional bipartite construction refutes it outright, already at $m = 2$: for $n = 8$ the wall holds 16 arcs while the linear hub holds only 14. At $m = 3$ the refutation is the thirty-arc augmented bipartite graph of Remark 1.12, which packs 30 arcs at $\lambda^{\max} = 2$ against the naive prediction of 27. The construction generalises to every m via Construction 1.13, thickening the larger part B slightly so that each vertex of B absorbs $m-2$ extra in-arcs from inside B: those extra arcs add exactly $m-2$ short detours from A to each $b \in B$, bringing $\lambda^{\max}$ from 1 to $m-1$ while the arc count climbs to $\lfloor (n+m-2)^2/4 \rfloor$.

The quadratic term is what makes the directed cases the deepest in the thesis. The upper bound argument must explain why no configuration beats the bipartite wall, which requires producing the partition in the first place, ruling out backward arcs, and controlling the internal structure of the large part B. That structural task is exactly what Conjecture A.21 has not yet resolved for $m \geq 3$. What Proposition A.22 shows is that the wall is at least the right shape: every feasible digraph is within $O_m(n^{3/2})$ arcs of being one, so the difficulty is the second-order term rather than the phenomenon. The undirected problem has no analogous obstacle, because Mader's degree-counting argument closes it in a few lines. The quadratic phenomenon is therefore not just a curiosity: it is the geometric reason the directed and undirected problems live on different levels of difficulty.

That same scarcity of extremal graphs is the thread through the remaining figures. Figure 4.3 shows where the frontier runs: it plots every labelled graph at small n as a single point, its binding connectivity $\lambda^{\max}$ on the horizontal axis and its edge count on the vertical, so the whole population of graphs is visible at once. The points crowd into the low-connectivity, low-edge corner and thin out upward, where the blue extremal envelope, an integer staircase carrying one dot per connectivity level, traces the densest graph available at each level. The empty region above that staircase is precisely the part the main theorems forbid a graph to occupy. The extremal problem is the question of where that staircase runs, and the figure shows it is a thin frontier with almost nothing pressed against it from below.

Figure 4.4 steps inside each graph to the level of individual pairs. For every graph in the full enumeration it records the connectivity of every vertex pair and pools all the observations, so the histogram counts not graphs but (graph, pair) observations. Each bar is split and stacked by the connectivity of the graph the pair came from: blue for pairs drawn from the lower-connectivity graphs, red for pairs from the higher-connectivity ones. The boundary between the two is set per variant at the midpoint of the connectivity range that variant's enumeration can reach, since the twelve variants span very different ranges and a single fixed boundary would leave some panels entirely one colour. A pair whose own connectivity exceeds the boundary can only come from a graph above it, so the bars to the right of the dashed line are wholly red, while to the left the two colours mix, since a high-connectivity graph still carries many low-connectivity pairs. The blue mass sits almost entirely at the lowest levels: high-connectivity pairs are rare even inside the graphs that have one.

Figure 4.5 then shows how the same graphs distribute by sheer edge count, with the same per-variant stacked colouring. At each edge count the blue part counts the lower-connectivity graphs and the red part the higher-connectivity ones, so the bar height is the total number of graphs there and the split shows how the two populations separate. The red mass sits to the right, since denser graphs tend to carry higher connectivity, and the dotted line marks the densest graph still on the low-connectivity side, landing far out in the sparse tail. The densest low-connectivity graphs, the ones the extremal problem is about, are exceptional in every variant.

Finally, Figure 4.6 lifts the whole study onto the (n, m) grid at once, drawing the best feasible edge count as a bar surface across every variant. The three proved variants rise as smooth linear or quadratic sheets that the formulas describe exactly, while the open variants show a visible step between the machine-proved blue bars and the search's purple lower bounds, so the surface makes the proved-versus-open divide of the trichotomy a single readable shape. Reading along the m axis recovers the linear scaling in the threshold, and reading along n recovers each variant's growth rate, which is the whole of Table 4.1 compressed into one object.

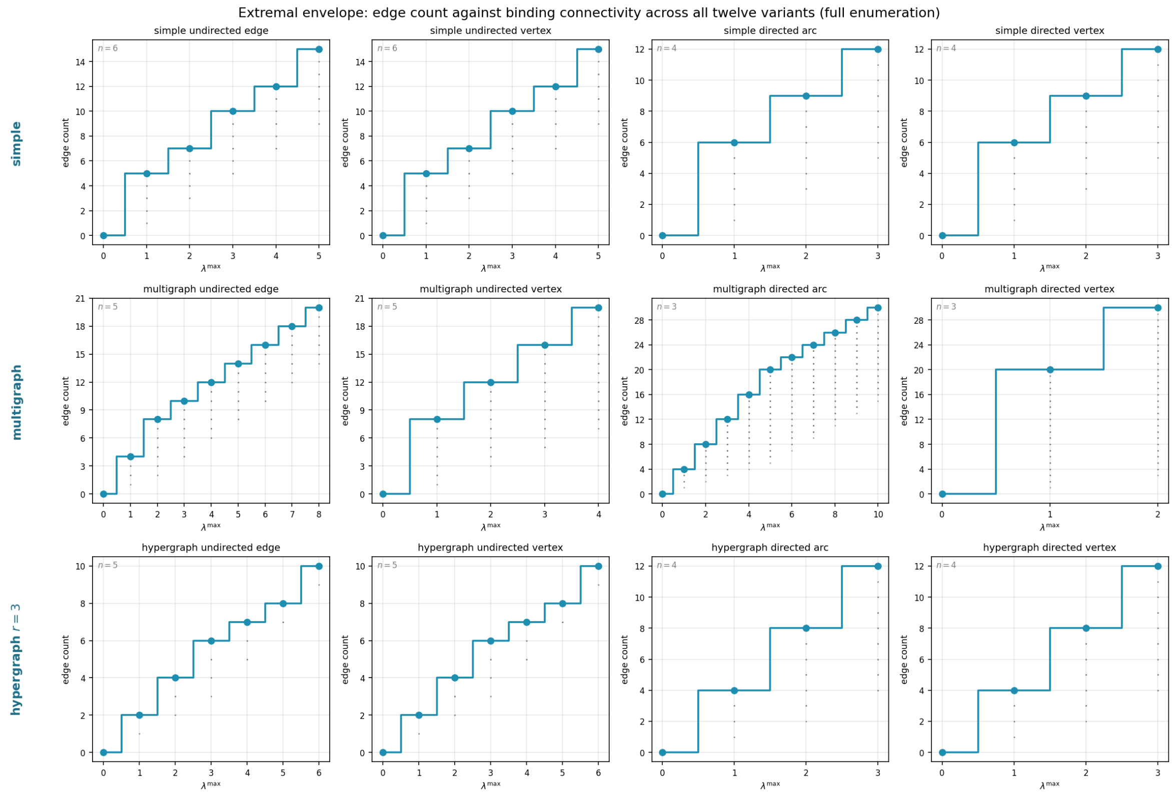


Figure 4.3. The complete landscape of all labelled graphs at small n (see panel annotations for each variant's n): every point is one graph, plotted at its binding connectivity $\lambda^{\max}$ horizontally and its edge count vertically. The blue staircase marking the densest graph at each connectivity level is the extremal envelope, the frontier the extremal problem asks about, with one dot per level so its exact value is legible. Graphs crowd the low-connectivity, low-edge region. The extremal graphs live at the top of the staircase. The emptiness above the envelope is exactly the region the main theorems forbid. The horizontal reach differs by panel because the connectivity that fits in a variant's enumeration n differs: arc-disjoint multigraph routes grow with edge multiplicity (the multigraph directed arc panel reaches $\lambda^{\max} = 10$ at $n = 3$), whereas vertex-disjoint routes are capped by the available internal vertices, so the multigraph directed *vertex* panel reaches only $\kappa^{\max} = 2$ at the same $n = 3$ (with a single internal vertex, a pair has at most the direct route plus one detour). This is a property of the separation, not of the search.

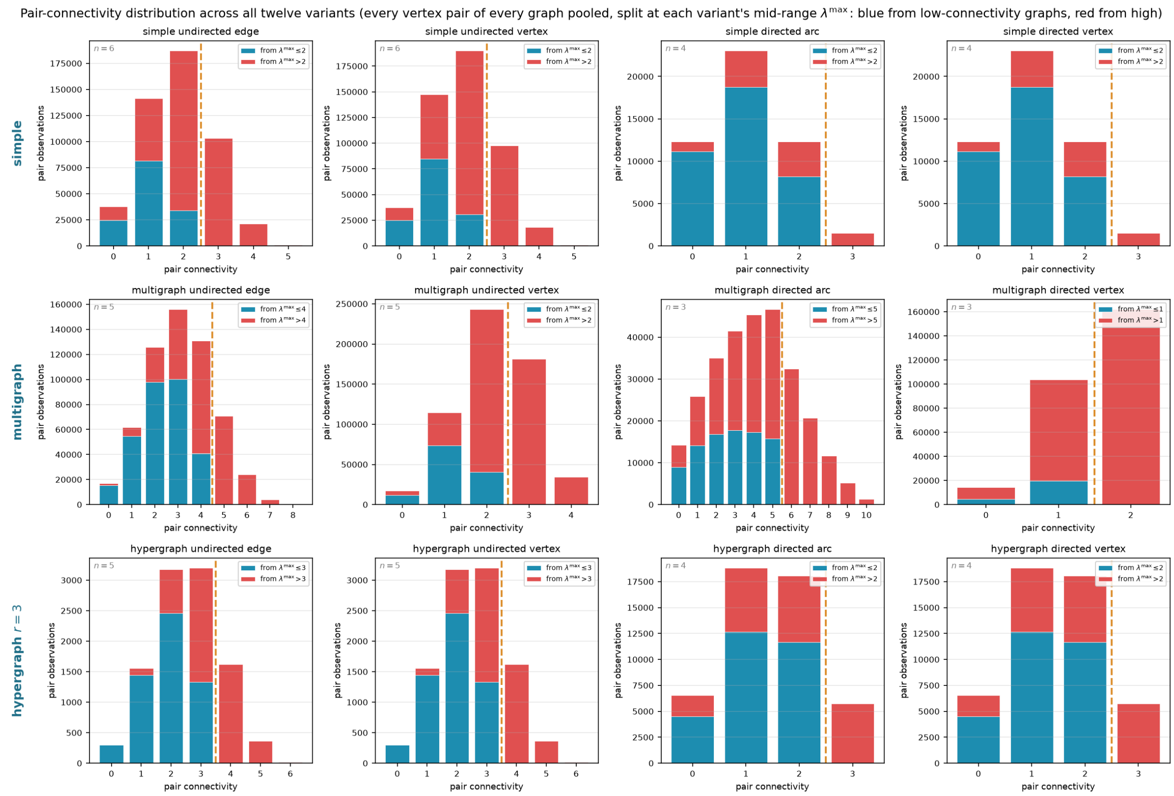


Figure 4.4. Pair-connectivity distribution across all twelve variants, pooling every vertex pair of every graph in the full enumeration. The horizontal axis is the connectivity of a single pair, and a bar's height is the number of (graph, pair) observations at that level. Each bar is split and stacked by the connectivity of the graph the pair was drawn from, blue from graphs at or below a per-panel boundary T and red from graphs above it, with the dashed line at T. The boundary is set per variant to the midpoint of the connectivity range that variant's enumeration reaches (each panel's legend gives its T), so the split stays informative in every panel instead of collapsing to one colour. Bars to the right of the dashed line are wholly red, since a pair of connectivity above T forces its graph above T. To the left the colours mix.

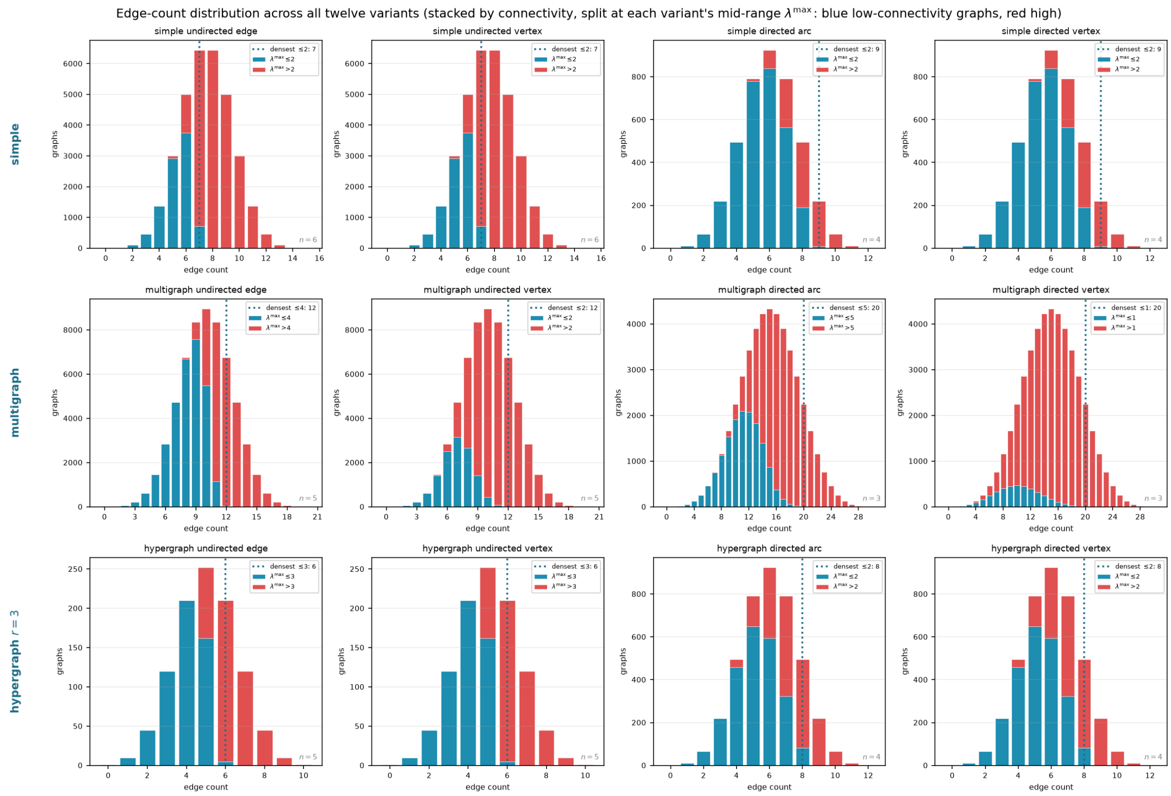


Figure 4.5. Edge-count distribution across all twelve variants, the same enumeration viewed by edge count rather than connectivity. Each bar is split and stacked by graph connectivity, blue for graphs at or below the per-panel boundary T and red above it (each panel's legend gives its T), so the full height is the total number of graphs at that edge count. The dotted vertical line marks the densest graph on the low-connectivity side. The reader can see directly how the two populations separate by edge count and how far the densest low-connectivity graphs sit from the bulk.

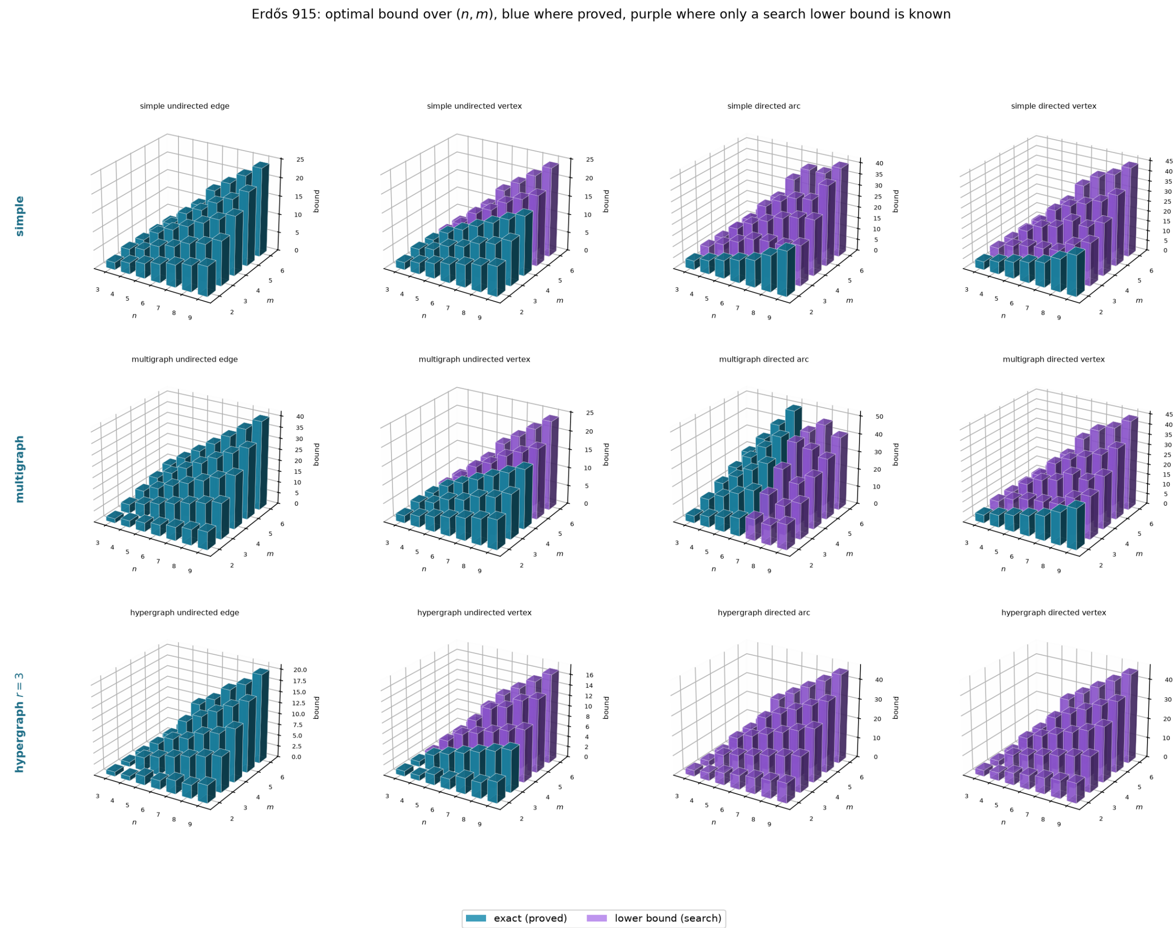


Figure 4.6. The optimal bound as a surface over the (n, m) grid for all twelve variants, every panel drawn from the same camera so the shapes compare directly. Each bar's height is the largest feasible edge count found by `solve` at that vertex count and forbidden threshold m. The single thing to read off is colour: a bar is *blue* exactly where the value is machine-proved and *purple* exactly where only a search lower bound is known, so each panel is a little map of what is settled. The three fully proved variants (Mader's undirected edge cases and the multigraph analogue) are solid blue sheets, rising linearly or quadratically as their formulas predict, while the open variants turn purple precisely where proof runs out. Reading along the m axis shows how the bound scales with the forbidden threshold, and reading along the n axis shows the growth rate for a fixed variant.

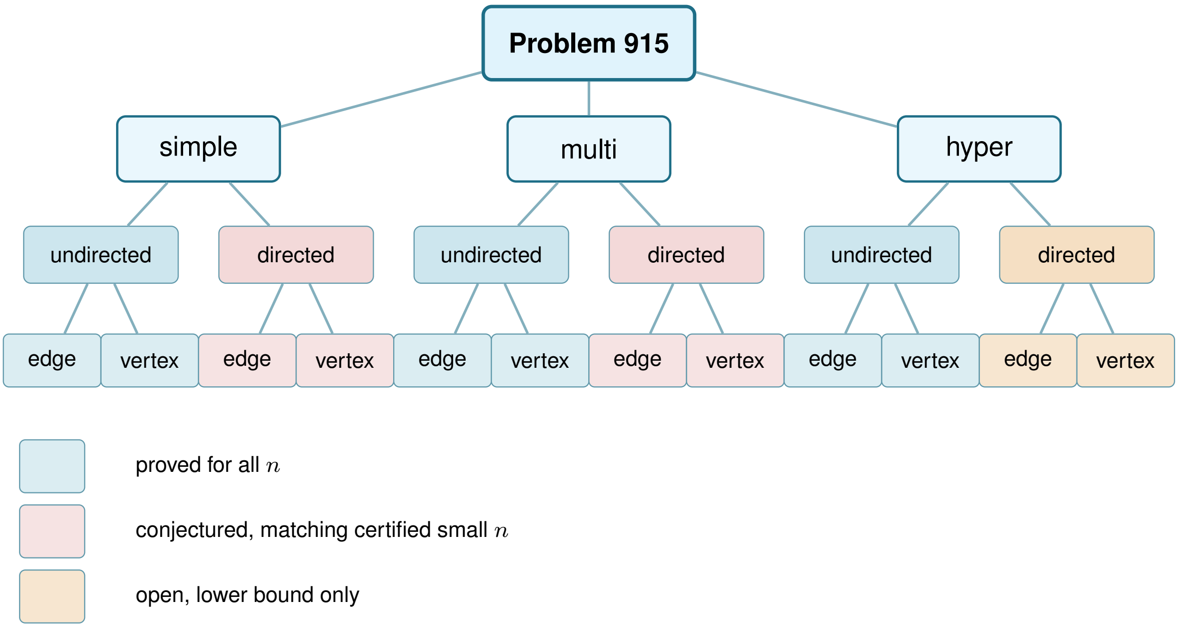


Figure 4.7. The same branching as Figure 1.7, now with every leaf resolved by the trichotomy of Section 3.6: blue is proved for all n, red is a conjectured formula backed by certified small cases and a matching search, and amber is open with only a lower bound known. Colour marks the status at general m, the harder regime this thesis mostly studies, and hides m-dependent exceptions and one convention that Table 4.1 states precisely. Every red leaf is a proved theorem at $m = 2$ and becomes conjectured only at $m \geq 3$, the directed edge leaves by Theorem 1.10 and the directed vertex leaves by the separate Theorem 1.11, which does not follow from it. Three blue leaves are proved only up to a finite m: the undirected vertex problem for simple graphs and for multigraphs ($m \leq 4$) and for hypergraphs ($m \leq 3$), open beyond. The two multigraph vertex leaves inherit whatever their simple counterparts have, because the counting convention of Remark 2.1 makes those two questions the same question.

4.3 Summary of the twelve variants

Table 4.1 records the status of all twelve structural variants. Each entry is marked by how it is known: proved by hand here or in Appendix A, cited to the literature, certified by the cut-counting optimisation for the stated sizes, or conjectured with a proved lower bound. The distinction is the honesty contract of Chapter 2 carried through to the summary. Figure 4.7 shows the same distinction as a picture before the table spells out every case.

Table 4.2 answers a different question about the same twelve variants: not what is known, but which routine of the program knows it. Most variants share machinery rather than each needing its own, so the table is grouped by family, and every name in it has its own `codecard` box where it is built in Chapters 2 and 3.

4.3.1 Orientation models for the directed hypergraph

The directed rows of Table 4.1 read a directed Berge edge in one way, the *forward* one of a single tail fanning to $r-1$ heads. It is not the only reading. In general a directed r-uniform hyperedge is a split of its vertices into a non-empty tail set T and head set H, a route entering at a tail and

Separation	Model	Value	Status
Undirected			
edge	simple	$\lfloor m(n-1)/2 \rfloor$	proved (Mader)
edge	multi	$(m-1)(n-1)$	proved
edge	hyper	$\lfloor (m-1)(n-1)/(r-1) \rfloor$	proved (cited, Gomory-Hu)
vertex	simple	$\lfloor m(n-1)/2 \rfloor$ $(m \le 4)$, $\lfloor 8n/3 \rfloor - 3$ $(m=5, n \ge 10)$	cited, open $m \ge 6$
vertex	multi	adjacency count, so the simple value $\lfloor m(n-1)/2 \rfloor$ $(m \le 4)$	convention (Remark 2.1)
vertex	hyper	$\left\lfloor \frac{(m-1)(n-1)}{r-1} \right\rfloor$ $(m \le 3)$, $\ge$ that for all m	proved $m \le 3$ (Theorems A.45 and A.48), open $m \ge 4$
Directed			
arc	simple	$\max(2(n-1), \lfloor n^2/4 \rfloor)$ $(m=2)$	proved, conj. $m \ge 3$
arc	multi	$(m-1)\max(2(n-1), \lfloor n^2/4 \rfloor)$	proved $n{\le}6$ and $m{=}2$, cond. $m{=}3$
arc	hyper	quadratic in n, exact value open	lower bd. (Proposition A.50)
vertex	simple	$\max(2(n-1), \lfloor n^2/4 \rfloor)$ $(m=2)$	proved separately (Theorem 1.11), conj. $m \ge 3$
vertex	multi	adjacency count, so the simple digraph value	convention (Remark 2.1)
vertex	hyper	quadratic in n, exact value open	lower bd. (Proposition A.50)

Table 4.1. The twelve structural variants of Problem 915 and their status. Every simple-graph closed form here is stated for $n \ge m$, the hypothesis Theorem 1.2, Theorem 1.5 and Conjecture 1.14 carry: below it the complete graph or digraph is itself feasible and the answer is the trivial maximum, which the formulas exceed. "Certified" means proved by the cut-counting optimisation for the stated sizes. "conj." means a conjecture with a proved matching lower bound. "cond." means proved conditional on a finite, identified statement (for $m = 3$ directed multigraphs, the one remaining $n = 7$ computation, fact (a) of Remark A.39). "lower bd." means only a proved lower bound is known, with the exact value open. "Convention" marks the two rows whose value is fixed by the counting convention of Remark 2.1 rather than by a theorem. The appearance threshold $p^* = m/n$ of Theorem 1.16 is deliberately not a column here: it is proved for $G(n,p)$, extends to the vertex and directed analogues by Remark A.59, and does *not* transfer to the hypergraph models, whose natural density scale is different (Remark A.60).

leaving at a head, and the gate checker of Chapter 2 measures any such split without change. Three models are worth naming, *forward* ($|T| = 1$), *backward* ($|H| = 1$), and *general* (any split, with genuinely mixed ones appearing once $r \ge 4$). The natural question, and one a reader of the directed figures tends to ask, is whether they multiply the table.

They do not, and two results say why. First, backward is not a new problem. Reversing every hyperarc turns a backward hypergraph into a forward one and reverses every route along the way, so the two models share every extremal number, edge and vertex, for all r, m and n (Lemma A.51). The backward column is the forward column read upside down. Second, the general model, though it admits more hyperedges on a single vertex set, obeys the same first-order bound. A hyperedge (T, H) occupies $|T|\,|H| \ge r-1$ ordered tail-head pairs, each usable by at most $m-1$ disjoint one-step routes, so $(r-1)|E| \le (m-1)\,n(n-1)$, exactly the forward cap of Proposition A.50 (Proposition A.52). The forward construction is itself a set of general hyperedges, so general is squeezed between the same lower and upper bounds as forward, and its value too is quadratic in n. Those two bounds differ by a factor of four, so what this fixes is the order and not the leading constant, and it does not by itself force the two models to agree asymptotically.

Where the models part company is only at small sizes, and the program shows exactly where. Table 4.3 lists the extremal hyperedge counts the branch-and-bound checker (Table 4.2) proves for the forward and general models. The method is the include-or-exclude search of Chapter 2 (Figure 2.9) walked over the candidate hyperedges instead of the arc cells: a hyperedge is

Variant family	Measure	Prove	Discover
Undirected, edge (all three models)	`max_edge_connectivity` `max_hyperedge_connectivity`	exhaustive enumeration (`solve`), Gomory–Hu cross-check	`search_for_dense_graph`
Undirected, vertex (all three models)	`max_vertex_connectivity` `max_hyper_vertex_connectivity`	exhaustive enumeration (`solve`)	`search_for_dense_graph`
Directed arc, simple	`max_edge_connectivity`	`_exhaustive_directed`: a pruned branch-and-bound	`tabu_search_for_dense_graph`
Directed arc, multigraph	`max_edge_connectivity`	the cut-counting optimisation (`prove_directed_multigraph`, `prove_integral_arc_bound`); the $n = 7$ classification (`enumerate_extremal_directed_multigraphs_via_generation`)	`tabu_search_for_dense_graph`
Directed vertex (simple and multi)	`max_vertex_connectivity`	reduces to the arc problem above	`tabu_search_for_dense_graph`
Directed hyper (arc and vertex)	`hyper_connectivity`	`max_feasible_hyperedges`: a pruned branch-and-bound	`search_for_dense_graph`

Table 4.2. The machinery of Chapters 2 and 3 indexed by variant family rather than by function name. This is the cross-reference from a variant to the routine that handles it; the routine's own signature, inputs, and outputs are in its `codecard` box.

included only while the checker confirms the growing hypergraph stays feasible, a branch is dropped once even including everything that remains cannot beat the best count already found, and a sweep that finishes has considered every candidate set, so the value it reports is proved exact rather than searched. The general one is strictly denser precisely when vertices are scarce, $n \approx r$, where the richer set of orientations lets a single vertex-set carry more hyperedges. At $n = r = 4$ it reaches eight hyperarcs against forward's four, and the surplus grows with the connectivity budget. Once n exceeds r the computed gap closes, proved equal already at $r = 3$, $n = 4$, and larger searches find no separation thereafter. The reading that suggests itself is that once the hypergraph spreads past a single edge-set, the one-tail model already saturates the Menger budget and extra orientations buy nothing. That reading is computational evidence and a conjecture, not a theorem: what is proved is that both models obey the same upper bound and admit the same quadratic construction (Proposition A.52), and those two bounds differ by a factor of four, which is too coarse to force the values to agree. On the strength of the evidence the orientation axis is treated as collapsing, which is why Problem 915 stays a twelve-variant question, and the general model enters as a refinement at the margin rather than a thirteenth column. Proving that it collapses, or exhibiting the first $n > r$ where it does not, is left open.

4.4 Contributions and limitations of the program

The program turned a per-case craft into a uniform pipeline. Before it, each variant came with its own one-off computation and its own hand argument. After it, one representation holds every variant, one exact checker measures connectivity, one certifier proves small-case upper bounds, and one cooled search discovers extremisers and the lower bounds they witness. The rediscovery of Chapter 3 is the evidence that the pipeline is faithful: pointed at solved cases, it returns the

r	m	n	forward	general
3	3	3	3	4
3	3	4	8	8
4	3	4	4	6
4	4	4	4	8

Table 4.3. Extremal hyperedge counts the branch-and-bound checker proves exact for the forward and general directed r-uniform models, at the small sizes where they can differ. The general model is strictly denser only when $n \approx r$, doubling the forward count at $n = r = 4$, $m = 4$, and the gap closes once n exceeds r (proved equal at $r = 3$, $n = 4$). Edge- and vertex-disjoint counts agree throughout, and backward equals forward by Lemma A.51.

solved answers.

Its limits are as clear as its reach, and stating them is part of the honesty the thesis insists on. The search proves no upper bounds. It only ever exhibits graphs. The certifier proves upper bounds, but only for a fixed size, and the directed multigraph encoding closed unconditionally only up to $n = 6$. Of the two $m = 3$ computations at $n = 7$, the generation enumerator has completed the classification, fact (b), while the value statement, fact (a), remains runtime-limited on the bundled solver and is the one certificate still outstanding. And no amount of either replaces the genuinely structural gaps the thesis leaves open: the decomposition of Conjecture A.21 that would turn Conjecture 1.14 into a theorem, of which the backward-arc clause is the hardest part but not the whole, the directed vertex problem at $m \geq 3$, which Whitney's inequality pointedly does not hand over with the arc case, and the undirected vertex-disjoint problem at $m \geq 6$, which has resisted for half a century and which the program can probe but not crack.

4.5 Open problems

- ☆ The decomposition of Conjecture A.21: prove that an extremal non-hub digraph admits a partition $A \cup B$ with every arc from A to B present, no arc inside A, and no arc from B back to A. That last clause is the backward-arc condition, and it is the one where the natural delete-and-compensate exchange fails to be monotone, but the other three are hypotheses too, and no argument here forces an arbitrary extremiser into that shape. The decomposition completes the upper bound in Conjecture 1.14 for $m \geq 3$. Unconditionally, Proposition A.22 already gives $\ell_m^{\mathrm{dir}}(n) = n^2/4 + O_m(n^{3/2})$, so what the decomposition is needed for is the second-order term, and sharpening the error there from $n^{3/2}$ to $O_m(n)$ by a stability argument is a smaller and possibly easier target than the decomposition itself.

- ☆ The directed *vertex* problem at $m \geq 3$, which is a separate question and not a corollary of the arc one. Whitney's inequality makes the vertex-feasible family the larger of the two, so an arc upper bound does not restrict it, and Conjecture A.21 would not settle it either. At $m = 2$ the two values agree (Theorem 1.11), proved by re-running the induction and its base cases under the stricter separation rather than by transfer. Whether they agree for $m \geq 3$ is open. The exhaustive search run under both tests at $m = 3$ returns the same value at every n it reaches, tabulated in Remark A.17, so there is no small counterexample, but that is evidence and not an argument. The useful next step is structural rather than numerical: find the first digraph in which some pair carries strictly more arc-disjoint routes than internally disjoint ones *and* that slack buys extra arcs, or show that it never can.

- The multigraph vertex problem under the other counting convention, worked out in Section A.8. Reading parallel copies as distinct routes makes this a genuinely new question, and the section settles $m = 2$ and exhibits several competing constructions. The natural guess from the $m \le 4$ data, that a thickened tree becomes optimal once $n \ge m + 2$, is false for every $m \ge 5$: Theorem A.56 shows that chaining thickened cliques through single bridge vertices beats the tree by an amount that grows linearly in n, and the true growth rate in m is $\Theta(m^2)$ rather than the $\Theta(m)$ the tree would give. What the exact value of $K_m^{\mathrm{multi}}(n)$ is, and whether same-size clique chains are the best construction, are open.

- The last finite $n = 7$ computation that closes the directed multigraph problem at $m = 3$. By the attachment lemma and the conditional odd step (Lemma A.34, Corollary A.35, Theorem A.36), the entire $m = 3$ problem (value *and* extremal uniqueness for all $n \ge 8$) reduces along a single induction to two integral statements about multigraphs with multiplicities in $\{0, 1, 2\}$ (Remark A.39): (a) $L_3^{\mathrm{dir}}(7) = 24$, and (b) every 24-arc feasible multigraph on 7 vertices with maximum degree at most 8 is $2 \cdot B_{3,4}$, $2 \cdot B_{4,3}$, or the doubled bidirected path P_7. Fact (b) is settled: the sound generation enumeration completed and returns exactly those three multigraphs, each re-verified by the exact checker (Remark A.39). What remains is fact (a) alone, where the MILP infeasibility check still reaches its time limit on the bundled solver, so the gap is a single finite computation rather than a structural one. Conjecture A.29 is thereby bypassed for $m = 3$, though it remains open and of independent interest as the natural minimum-degree statement for the directed multigraph problem.

- Extremal uniqueness at odd levels for directed multigraphs with $m \ge 4$. The conditional odd step (Theorem A.36) and the propagation of Remark A.39 are stated for $m \in \{3, 4\}$. For $m = 4$ the *value* step goes through, but at an odd extremal level the counting no longer forces a vertex of degree $(m-1)k$ (all degrees $\ge (m-1)k + 1$ are arithmetically consistent), so the uniqueness statement does not yet propagate. For $m \ge 5$ the same escape climbs one level and even the *value* step stalls: a surplus extremiser could be degree-regular (every vertex at one and the same degree) strictly above the deletion bound, so no vertex deletion is forced to land on an extremal multigraph. Closing either hole asks for an arithmetic or structural input the present averaging argument does not supply.

- The undirected vertex-disjoint problem for $m \ge 6$, open since 1974, where even the extremal structure is unknown.

- The hypergraph vertex problem at $m \ge 4$. The problem is now settled at $m = 2$ and $m = 3$ for every uniformity and all n, with $k_m^{(r)}(n) = \left\lfloor \frac{(m-1)(n-1)}{r-1} \right\rfloor = \ell_m^{(r)}(n)$ (Theorems A.45 and A.48). The incidence reduction stands for every m, but the rank bound that drives it (Lemma A.47) leans on the triconnected (Tutte/SPQR) decomposition exactly where $\kappa \leq 2$ lives. For $m = 4$ it would need a 4-connectivity analogue, where no comparably clean decomposition is available. Exhaustive computation finds no counterexample at $m = 4$ at any of eleven sizes, confirming both that $\lfloor 3(n-1)/(r-1) \rfloor$ is attained and that one more hyperedge is infeasible, so the agreement $k_m^{(r)} = \ell_m^{(r)}$ survives well past the theorem. That it must break eventually is not a guess: at $r = 2$ this problem is exactly the multigraph vertex problem of Section A.8, where Theorem A.56 breaks the formula for every $m \geq 5$. So the question is not whether the pattern fails but whether $r \geq 3$ postpones the failure past $m = 4$, which leaves the $m = 4$ row sitting on the boundary, and where it first breaks for $r \geq 3$ is unknown.

- ☆ A machine-checkable certificate for fact (a) of Remark A.39. It is the one computed claim in this thesis that would rest on a solver's infeasibility verdict rather than on a finished exhaustion (Section 2.9.1), so it is the one place where a replayable proof object, of the kind a pseudo-Boolean or SAT solver emits, would raise the standard rather than merely repeat the work. Encoding the question as a pseudo-Boolean constraint problem over the 42 multiplicities in $\{0, 1, 2\}$, with the cut conditions written out per ordered pair, is the natural route.

- ☆ The directed hypergraph variants. The tail-and-heads model has first bounds and a quadratic lower-bound family (Proposition A.50): counting how many hyperedges may serve the same ordered pair of vertices caps the total at $(m-1)n(n-1)/(r-1)$, and a bipartite construction reaches $\approx (m-1)n^2/(4(r-1))$, so the value is quadratic in n. The two bounds differ by a factor of four, so the order is settled and the leading constant $\lim_{n\to\infty} \mathrm{ex}(n)/n^2$ is not, for any of the orientation models. That constant is the right first target, ahead of exact small values, and Conjecture A.53 says which way it should go: the factor of four is exactly the gap between the n^2 ordered pairs the upper bound allows and the $n^2/4$ the bipartite construction uses, the same gap that appears in the simple directed problem, where Theorem A.24 now proves the bipartite pattern right to first order. So the upper bound is the side that should improve. The appendix records where the proof of that theorem fails to transfer, which is that a hyperedge carries $r-1$ heads at once, so two-step routes through different midpoints need not be edge-disjoint. The orientation axis does not visibly widen the gap: the backward model coincides with forward by reversal (Lemma A.51) and the general model obeys the same cap (Proposition A.52), beating forward only in the scarce-vertex regime $n \approx r$ (Section 4.3.1). That the three models share a leading constant is a conjecture supported by small-case computation, not a consequence of sharing two bounds four apart.

The map is filled in where honest mathematics could fill it, and where it could not, the blanks are marked precisely, with a machine standing ready at each of them. That, finally, is the use of building the microscope: not that it answers every question, but that it shows exactly which questions are left, and hands the next reader a clean place to start.

Appendix A

Full Proofs and Computational Transcripts

This appendix holds the arguments deferred from the body: the ones that are either standard machinery or long case-checking, and whose length, not whose difficulty, kept them out of the narrative. The body carries the ideas. Here they are carried out in full. Because graphs are visual objects, the longer arguments here are accompanied by figures: each is drawn rather than merely described, and the prose points the reader to the picture at the moment it is needed.

How to read this appendix. The sections divide into standard machinery and the thesis's own proofs, and a reader chasing one thread of the body need not read all of them. The first two sections, on Theorem A.1, Theorem A.2, and Mader's theorem, are classical results included only so the body can cite them in the exact form it uses, and a reader who already trusts Menger, Gomory–Hu, and Mader may skip them. Three sections carry the load-bearing original arguments. The directed arc value at $m = 2$ supplies the finite base cases the body's induction rests on. The directed multigraph problem at large n is the longest argument here and the one behind the main directed contribution. The hypergraph vertex problem proves the incidence-rank lemma that settles the $m = 3$ hypergraph case. The remaining sections, on the hypergraph edge bounds and the random threshold, each support a single claim of the body and can be read on their own when that claim is reached. Finally, everything the program asserts is reproduced verbatim in the computational transcripts and the self-test section, so the machine proofs can be audited without running anything.

A.1 Menger, Gomory–Hu, and the degree bound

The two classical theorems below are the foundation of everything the thesis measures, and we state them at the level of generality at which the program actually applies them. A *capacitated graph*, or *weighted graph*, is a graph, directed or not, carrying a nonnegative capacity $c(e)$ on each edge or arc. A plain graph is read as the *unit-capacity* case $c \equiv 1$, and a multigraph as the integer-capacity case, one unit per parallel copy. A *flow* from u to v sends units along edges without exceeding any capacity, and the *capacity of a cut* is the total capacity of the edges crossing it. Both theorems below are theorems about capacitated graphs, and the thesis uses them mostly at unit and integer capacities, where the maximum flow is exactly the connectivity λ.

Theorem A.1 (Menger, as max-flow min-cut [2]). *In any capacitated digraph, and for distinct* *vertices u, v, the maximum flow from u to v equals the minimum capacity of a cut separating u from v. At unit capacities this is the combinatorial statement we cite throughout: the maximum number of pairwise edge-disjoint u–v paths equals the minimum number of edges whose removal separates them,*

$$\lambda_G(u,v) = \min\{\, |C| : C \subseteq E(G),\ G - C \text{ has no } u\text{–}v \text{ path} \,\}.$$

A directed capacitated graph is the most general object here, and every graph model the thesis uses is a special case of it. An undirected graph is the symmetric case, each edge a pair of opposite unit-capacity arcs. A multigraph is the integer-capacity case, with $\mu(u,v)$ units on the pair uv. The hypergraph version (Theorem A.40) is this very theorem applied to the helper-point network of Figure 2.5, a capacitated digraph. The internally vertex-disjoint *version is obtained not by changing the theorem but by changing the graph it runs on, a device called* vertex splitting*: replace each internal vertex w by two copies joined by a single unit-capacity arc $w^- \to w^+$, send every arc that entered w into w^- and every arc that left w out of w^+. A flow may now pass through w at most once, so edge-disjoint paths in the split graph are exactly internally vertex-disjoint paths in the original, and a minimum edge cut there is a minimum vertex cut here. This split graph is literally what the checker of Chapter 2 builds, which is why a single maximum-flow routine measures all four separations.*

Theorem A.2 (Gomory–Hu [15])**.** *Every* undirected *capacitated graph G has a weighted tree T on the same vertex set, its* Gomory–Hu tree*, such that for all u, v the value $\lambda_G(u,v)$ equals the minimum edge weight on the u–v path in T, and each tree edge e corresponds to a minimum cut of G whose capacity is the weight of e. All $\binom{n}{2}$ pairwise values are thereby stored in the $n-1$ weights of a single tree.*

Two features of the statement fix how far it reaches, and both are used below. Being a theorem about capacities, it applies verbatim to a simple graph (unit weights), to a multigraph (integer weights, the multiplicity μ as capacity), and, in the form of Theorem A.41, to the hyperedge-connectivity of a hypergraph. But it needs the cut function to be *symmetric*, which forces G to be undirected: a digraph has $\lambda(u,v) \neq \lambda(v,u)$ in general and admits no such tree, and the vertex-connectivity function fails symmetry for the same reason. So the Gomory–Hu tree is precisely the tool for the undirected edge problems, where it drives Mader's theorem and its multigraph and hypergraph cousins (Theorems 1.2 and 1.3 and proposition 1.15) through one argument, and precisely the tool that vanishes once arcs turn one-way.

Primer: why a Gomory–Hu tree exists

The theorem is far from obvious, so here is why such a tree can always be built, in enough detail that nothing is taken on faith. Everything rests on one property of cuts. For a set of vertices S write $d(S)$ for the total capacity of the edges with exactly one endpoint in S, which is the capacity of the cut $(S, V \setminus S)$. The cut function is *submodular*: for any two vertex sets

A and B,

$$d(A) + d(B) \;\geq\; d(A \cap B) + d(A \cup B).$$

One checks this one edge at a time. An edge contributes to each side of the inequality according to where its two endpoints fall relative to A and B, and a short case check over the possible placements shows its contribution on the right never exceeds its contribution on the left. The only edges that could contribute more on the right have one endpoint in $A \setminus B$ and the other in $B \setminus A$, and those contribute 0 on the right and 2 on the left, so the inequality only gains.

Submodularity buys the single lemma the construction needs, the *uncrossing* lemma: if $(S, V \setminus S)$ is any cut and u, v lie on the same side of it, then some minimum u–v cut keeps all of S (or all of $V \setminus S$) on one side, that is, it does not cross S. The picture is that a cheapest u–v cut which sliced through S could be pushed clear of S without ever gaining capacity, and submodularity is exactly the inequality guaranteeing the push is free.

With that lemma the tree is built in $n - 1$ rounds, one minimum-cut computation each. Begin with the trivial tree, a single node holding all n vertices. In each round pick a node of the current tree that still holds two or more vertices, choose two of its vertices u and v, and compute a minimum u–v cut in the graph in which every neighbouring tree node, itself already a bag of several vertices, is shrunk to one vertex placed on whichever side the cut assigns it. Split the chosen node along the two sides of the cut and join the two halves by a new tree edge weighted by the cut's capacity. The uncrossing lemma is what makes the shrinking legitimate, because it says the minimum u–v cut can be chosen to respect every cut found in earlier rounds, so no earlier tree edge is spoiled. After $n - 1$ rounds every node holds a single vertex and T is a weighted tree on V with $n - 1$ edges.

Two facts finish the argument, and they are exactly what the body uses. Each tree edge is by construction the capacity of an actual cut of G, so its weight is a genuine minimum u–v cut value $\lambda_G(u, v)$ for the pair it separates. And for any u, v the value $\lambda_G(u, v)$ equals the lightest edge on the u–v path of T. One inequality holds because every tree edge on that path is itself a cut separating u from v, so none can be lighter than the global minimum u–v cut. The other holds because the lightest tree edge on the path is a genuine u–v cut of exactly that weight. The reader who wants the uncrossing induction spelled out in full will find it in Gomory and Hu [15]. The point here is only that the tree is an honest theorem, computed by $n - 1$ maximum-flow calls, and not a convenient fiction.

The most basic constraint on local connectivity is local: a pair cannot have more independent routes than either endpoint has edges. The constructions below use it constantly.

Lemma A.3 (Degree bound)**.** *For every pair of vertices of a graph G, $\lambda_G(u, v) \leq \min(\deg u, \deg v)$. In a digraph D, $\lambda_D(u, v) \leq \min(d^+(u), d^-(v))$.*

Proof. Every edge-disjoint u–v path uses a distinct edge at u, so there are at most $\deg u$ of them, and likewise at most $\deg v$. In the directed case each arc-disjoint path leaves u by a distinct out-arc and enters v by a distinct in-arc. □

A.2 Mader's theorem in full

The upper bound of Theorem 1.2 rests on three short lemmas about a Gomory–Hu tree T of G. For an edge $e = \{u, v\}$ of G, write $\mathrm{dist}_T(e)$ for the number of tree edges on the unique u–v path in T. Since T is a tree on the very same vertex set, $\mathrm{dist}_T(e) \geq 1$ for every edge. Figure A.1 shows a small graph beside one of its Gomory–Hu trees and marks which edges have $\mathrm{dist}_T(e) = 1$ and which have $\mathrm{dist}_T(e) \geq 2$. It is worth keeping in view, since the three lemmas below are exactly statements about that picture. The first lemma is the one place the proof uses that G is *simple*, and it is exactly where the simple-graph bound improves on the multigraph bound.

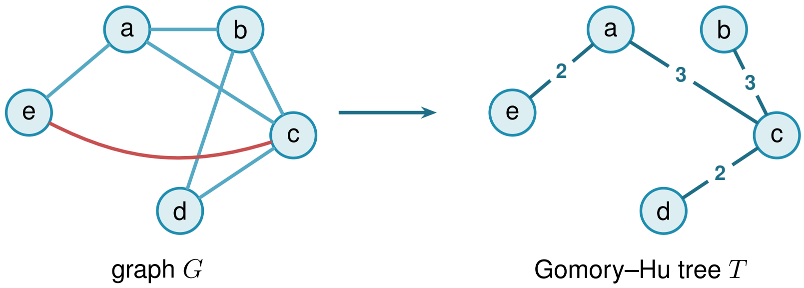


Figure A.1. A simple graph G (left) and a Gomory–Hu tree T of it (right). Both live on the same five vertices. The tree T has edges $ea\,(2)$, $ac\,(3)$, $cb\,(3)$, $cd\,(2)$, where each weight equals the local edge-connectivity of its endpoints in G. These four edges are also edges of G, so each has $\mathrm{dist}_T(e) = 1$: these are the E_1 edges of Lemma A.4, and a *simple* graph admits at most $n - 1 = 4$ of them, one per tree edge. The remaining edges ab, bd, and the red chord ec all join tree-distance-2 pairs: the path in T from e to c is $e{-}a{-}c$, so $\mathrm{dist}_T(ec) = 2$. Reading $\mathrm{dist}_T(e)$ as the number of tree cuts an edge crosses makes $\sum_e \mathrm{dist}_T(e) = \sum_t c(t)$ (Lemma A.6). Every edge crosses at least one cut, but only the $n-1$ tree edges cross exactly one, which is the surplus that drives Lemma A.5 and ultimately the factor of two in Mader's bound.

Lemma A.4 (Distance-one edges)**.** *At most $n-1$ edges of a simple graph G satisfy $\mathrm{dist}_T(e) = 1$.*

Proof. An edge $e \in E(G)$ has $\mathrm{dist}_T(e) = 1$ if and only if its endpoints are adjacent in T. The tree T has exactly $n - 1$ edges, and since G is simple each pair of adjacent tree vertices supports at most one edge of G. □

Lemma A.5 (Sum of tree distances)**.** $\displaystyle\sum_{e \in E(G)} \mathrm{dist}_T(e) \;\geq\; 2|E(G)| - (n-1).$

Proof. Partition $E(G)$ into $E_1 = \{e : \mathrm{dist}_T(e) = 1\}$ and $E_{\geq 2} = \{e : \mathrm{dist}_T(e) \geq 2\}$. Then

$$\sum_{e \in E(G)} \mathrm{dist}_T(e) \;\geq\; |E_1| + 2\,|E_{\geq 2}| \;=\; 2|E(G)| - |E_1| \;\geq\; 2|E(G)| - (n-1),$$

using Lemma A.4 in the last step. □

Lemma A.6 (Double-counting identity)**.** *Let G carry* unit capacities*, so that each weight $c(t)$ of a Gomory–Hu tree T is the plain* number *of edges of G crossing the cut that t induces. Then*

$$\sum_{t \in E(T)} c(t) \;=\; \sum_{e \in E(G)} \mathrm{dist}_T(e).$$

This is the form used for Mader's theorem, where G is a simple graph and so unit-capacitated by default.

Proof. By Theorem A.2 each tree edge t induces a cut of G of capacity $c(t)$, so $c(t) = \sum_{e \in E(G)} \mathbf{1}_{e \text{ crosses } t}$. Exchanging the order of summation,

$$\sum_{t \in E(T)} c(t) = \sum_{e \in E(G)} \sum_{t \in E(T)} \mathbf{1}_{e \text{ crosses } t} = \sum_{e \in E(G)} \mathrm{dist}_T(e),$$

where the last equality holds because an edge $\{u, v\}$ crosses exactly those tree cuts induced by the tree edges on the unique u–v path in T. $\square$

Proof of Theorem 1.2, upper bound. Let G be a simple graph on n vertices with $\lambda_G^{\max} \le m - 1$ and let T be a Gomory–Hu tree of G. Each tree edge $t = \{x, y\}$ has weight $c(t) = \lambda_G(x, y) \le m - 1$, so summing over the $n - 1$ tree edges, $\sum_t c(t) \le (m-1)(n-1)$. Combining with Lemmas A.5 and A.6,

$$2|E(G)| - (n-1) \;\le\; \sum_{e \in E(G)} \mathrm{dist}_T(e) \;=\; \sum_{t \in E(T)} c(t) \;\le\; (m-1)(n-1).$$

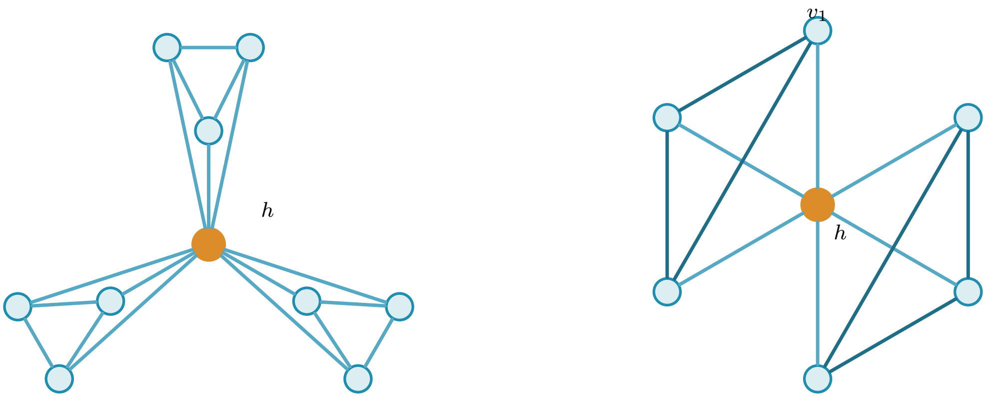
Rearranging gives $2|E(G)| \le m(n-1)$, and since $|E(G)|$ is an integer, $|E(G)| \le \left\lfloor \frac{m(n-1)}{2} \right\rfloor$. $\square$

Note where the factor of two came from: every edge crosses at least one tree cut, but a simple graph can afford at most $n - 1$ edges that cross only one, so on average each edge is charged closer to twice. For multigraphs that refinement is unavailable, since parallel edges between tree-adjacent pairs each have $\mathrm{dist}_T = 1$, and the same chain of inequalities stops at $|E| \le \sum_e \mathrm{dist}_T(e) \le (m-1)(n-1)$, which is exactly the multigraph bound of Theorem 1.3.

The lower bound is a pair of constructions, drawn side by side in Figure A.2. The first is the shape the bound was guessed from, and works when $m - 1$ divides $n - 1$. The second generalises it to every $n \ge m \ge 2$.

Construction A.7 (Shared-clique graph)**.** For $m \ge 3$ with $(m-1) \mid (n-1)$, take $k = (n-1)/(m-1)$ copies of the complete graph K_m and identify one vertex of each copy into a single shared hub vertex h. The result has $k(m-1) + 1 = n$ vertices and $k\binom{m}{2} = \frac{m(n-1)}{2}$ edges. Every non-hub vertex has degree $m - 1$, and any pair of vertices contains a non-hub vertex, so $\lambda^{\max} \le m - 1$ by Lemma A.3.

Construction A.8 (Hub-and-spokes graph)**.** For $n \geq m \geq 2$, let $H_{m,n}$ have vertex set $\{h, v_1, \ldots, v_{n-1}\}$ with (i) all $n-1$ *hub edges* $\{h, v_i\}$, and (ii) a *spoke graph* on $\{v_1, \ldots, v_{n-1}\}$ with $\left\lfloor \frac{(m-2)(n-1)}{2} \right\rfloor$ edges in which every v_i has spoke degree at most $m-2$. Such a spoke graph exists by Lemma A.9.



(a) Shared-clique graph: three K_4's glued at one hub h ($m = 4$, $k = 3$). Each non-hub vertex has degree $m - 1 = 3$.

(b) Hub-and-spokes $H_{m,n}$: the hub edges $\{h, v_i\}$ (light) plus a $(m-2)$-regular *spoke graph* (dark) in two $K_{m-1} = K_3$ cliques.

Figure A.2. The two extremal families. **(a)** The shared-clique graph of Construction A.7 works when $(m-1) \mid (n-1)$: every pair contains a non-hub vertex of degree exactly $m-1$, so $\lambda^{\max} \leq m-1$ by the degree bound, while the edge count hits $\frac{m(n-1)}{2}$. **(b)** The hub-and-spokes graph $H_{m,n}$ of Construction A.8 reaches the same bound for *every* $n \geq m$: each spoke vertex v_i has total degree $1 + (m-2) = m-1$, and choosing the spoke graph as disjoint cliques K_{m-1} recovers (a). The hub is drawn in orange in both.

Lemma A.9 (Near-regular graphs exist)**.** *For $N \geq 1$ and $0 \leq r \leq N-1$ there is a graph on N vertices with $\left\lfloor \frac{rN}{2} \right\rfloor$ edges and maximum degree at most r.*

Proof. If rN is even we build an r-regular circulant graph on vertex set $\{0, \ldots, N-1\}$: for even r connect each i to $i \pm j \pmod N$ for $j = 1, \ldots, r/2$, and for odd r (then N is even) use $j = 1, \ldots, (r-1)/2$ together with the diameter distance $N/2$, which contributes degree one. Both are r-regular with $rN/2$ edges. If rN is odd, then r and N are both odd: take the $(r-1)$-regular circulant with distances $1, \ldots, (r-1)/2$ and add the near-perfect matching that pairs vertex i with $i + (N+1)/2 \pmod N$ for $i = 0, \ldots, (N-3)/2$, which covers all vertices but one. The total is $\frac{(r-1)N}{2} + \frac{N-1}{2} = \frac{rN-1}{2} = \left\lfloor \frac{rN}{2} \right\rfloor$ edges, with one vertex of degree $r-1$ and all others of degree r. □

Theorem A.10 (Extremal graphs exist)**.** *For all $n \geq m \geq 2$ the graph $H_{m,n}$ has $\left\lfloor \frac{m(n-1)}{2} \right\rfloor$ edges and $\lambda^{\max} \leq m-1$.*

Proof. The edge count is $(n-1) + \left\lfloor \frac{(m-2)(n-1)}{2} \right\rfloor = \left\lfloor \frac{m(n-1)}{2} \right\rfloor$, since $(m-2)(n-1)$ and $m(n-1)$ have the same parity. For the connectivity, any two distinct vertices include at least one spoke vertex v_i, whose total degree is $1 + (m-2) = m-1$. Lemma A.3 then gives $\lambda(u, v) \leq m-1$. When $(m-1) \mid (n-1)$, choosing the spoke graph as k disjoint cliques K_{m-1} recovers Construction A.7, so the hub-and-spokes family genuinely generalises the shared-clique one. □

Together the upper bound and Theorem A.10 prove Theorem 1.2. The proof also reveals which graphs are extremal: equality forces both inequalities in the chain to be tight.

Theorem A.11 (Characterisation of equality). *Let G be simple with $\lambda_G^{\max} \leq m-1$ and $|E(G)| = \left\lfloor \frac{m(n-1)}{2} \right\rfloor$, and let T be a Gomory–Hu tree of G. If $m(n-1)$ is even, then every tree edge has weight exactly $m-1$, every tree edge is also an edge of G, and every edge of G joins vertices at tree distance at most 2. If $m(n-1)$ is odd, there is exactly one unit of slack, located in exactly one of three places: one tree edge has weight $m-2$, or one tree edge is not an edge of G, or one edge of G joins vertices at tree distance 3. Everything else is tight.*

Proof. Write the chain of the upper-bound proof as three nonnegative slacks. The first, $S_1 = (m-1)(n-1) - \sum_t c(t)$, counts tree weights below $m-1$. The second, $S_2 = \sum_{e:\, \mathrm{dist}_T(e)\geq 2}(\mathrm{dist}_T(e)-2)$, counts edges beyond tree distance two. The third, $S_3 = (n-1)-|E_1|$, counts tree edges that support no edge of G. Retracing the proof, $\sum_e \mathrm{dist}_T(e) = 2|E|-|E_1|+S_2$ and $\sum_e \mathrm{dist}_T(e) = \sum_t c(t) = (m-1)(n-1)-S_1$, so altogether $2|E| = m(n-1)-(S_1+S_2+S_3)$. If $m(n-1)$ is even, equality $|E| = m(n-1)/2$ forces $S_1 = S_2 = S_3 = 0$. If $m(n-1)$ is odd the floor absorbs exactly one unit, so $S_1 + S_2 + S_3 = 1$ and exactly one of the three slacks equals one. □

For $m = 2$ the characterisation says the extremal graphs are exactly the spanning trees: every tree edge must carry weight one and lie in G, so G contains its own Gomory–Hu tree and has only $n-1$ edges to spend. For $m = 4$ and $n \equiv 1 \pmod 3$ it is consistent with the trees of K_4 blocks glued at shared cut vertices, the case of Construction A.7 that the search of Chapter 3 rediscovers.

A.3 The directed arc value at $m = 2$

Three short lemmas feed the induction. The first shows a hub forces a linear bound for every m, and is the proof behind the hub branch of Conjecture 1.14. The three lemmas below and the proof of Theorem 1.10 that they support are the author's own, and its finite base cases $2 \leq n \leq 7$ were verified by the author's program, through the exhaustive search of Chapter 2 whose transcript is reproduced at the end of this appendix.

Lemma A.12 (Two-hop lemma). *Let D be a simple digraph with $\lambda_D^{\max} \leq m-1$ containing a vertex r with $d^+(r) = n-1$. Then every $v \neq r$ has at most $m-2$ in-arcs from $V \setminus \{r, v\}$.*

Proof. If $u_1, \dots, u_{m-1}$ were distinct in-neighbours of v other than r, that is, distinct vertices each sending an arc to v, the paths $r \to v$ and $r \to u_i \to v$ for $i = 1, \dots, m-1$ would be m arc-disjoint routes, since the u_i are distinct and $d^+(r) = n-1$ supplies every starting arc. That contradicts $\lambda_D(r, v) \leq m-1$. □

Theorem A.13 (Hub upper bound). *If a simple digraph D with $\lambda_D^{\max} \leq m-1$ has a vertex r with $d^+(r) = n-1$, then $|A(D)| \leq m(n-1)$.*

Proof. Split the arcs into those leaving r (exactly $n-1$), those entering r (at most $n-1$), and the internal arcs avoiding r. By Lemma A.12 each of the $n-1$ other vertices receives at most $m-2$ internal arcs, so there are at most $(m-2)(n-1)$ internal arcs, and the total is at most $m(n-1)$. $\square$

Lemma A.14 (No transitive triangle)**.** *A simple digraph with $\lambda^{\max} \leq 1$ contains no three vertices x, y, z with all of $(x, y), (y, z), (x, z)$ present, since $x \to z$ and $x \to y \to z$ would be two arc-disjoint routes.*

Lemma A.15 (Deletion monotonicity)**.** *Write $D - v_0$ for the digraph obtained from D by deleting the vertex v_0 together with all arcs incident to it. For any digraph D and vertex v_0,*

$$\lambda^{\max}(D - v_0) \leq \lambda^{\max}(D) \qquad \text{and} \qquad \kappa^{\max}(D - v_0) \leq \kappa^{\max}(D),$$

since arc-disjoint paths in $D - v_0$ are still arc-disjoint paths in D, and internally vertex-disjoint paths in $D - v_0$ are still internally vertex-disjoint paths in D. The two statements are proved by the same one-line observation because deleting a vertex only removes routes, never creates one, and it cannot make two surviving routes share anything they did not share before.

Proof of Theorem 1.10. Write $M(n) = \max(2(n-1), \lfloor n^2/4 \rfloor)$. A direct computation shows $2(n-1) \geq \lfloor n^2/4 \rfloor$ exactly for $n \leq 7$, with equality at $n = 7$, so $M(n) = 2(n-1)$ for $n \leq 7$ and $M(n) = \lfloor n^2/4 \rfloor$ for $n \geq 7$. The lower bound $\ell_2^{\mathrm{dir}}(n) \geq M(n)$ is witnessed by the directed hub construction (Construction 1.9 at $m = 2$, the bidirected star) and the one-directional bipartite construction (Construction 1.8).

For the upper bound we show by strong induction on n that $\lambda_D^{\max} \leq 1$ forces $|A(D)| \leq M(n)$.

Base cases $2 \leq n \leq 7$. All six are settled uniformly by the pruned exhaustive search of Chapter 2, which walks every simple digraph with $\lambda^{\max} \leq 1$ on up to seven vertices and reports the maxima $2, 4, 6, 8, 10, 12$, matching $M(n)$ at each size. The transcript is reproduced below, and the cut-counting certifier independently confirms the values within its reach through the $m = 2$ reduction of Theorem A.31. The two smallest are also immediate by hand, as a check on the machine rather than a separate argument: at $n = 2$ there are at most 2 arcs, and at $n = 3$ a fifth arc would leave at most one arc of the complete bidirected triangle missing, so three of the rest form a transitive triangle, which Lemma A.14 forbids. The point of the theorem is precisely that $n = 4, 5, 6, 7$ are *not* immediate by hand, which is what makes handing the case-check to the program the right move.

Inductive step $n \geq 8$. Let D be a simple digraph on n vertices with $\lambda_D^{\max} \leq 1$ and put $a := |A(D)|$. Choose v_0 minimising the total degree $d(v) = d^+(v) + d^-(v)$. Since $\sum_v d(v) = 2a$, averaging gives $d(v_0) \leq 2a/n$. By Lemma A.15 the digraph $D - v_0$ on $n - 1 \geq 7$ vertices satisfies the induction hypothesis, so $a - d(v_0) \leq M(n-1) = \left\lfloor \frac{(n-1)^2}{4} \right\rfloor$. Combining,

$$a \leq \frac{2a}{n} + \left\lfloor \frac{(n-1)^2}{4} \right\rfloor \qquad \Longrightarrow \qquad a \leq \frac{n}{n-2} \left\lfloor \frac{(n-1)^2}{4} \right\rfloor.$$

It remains to check that this right-hand side never exceeds $\lfloor n^2/4 \rfloor$, and the two parities behave differently enough to treat them in turn.

For even $n = 2k$ with $k \geq 4$, first simplify $\lfloor (n-1)^2/4 \rfloor$. Expanding $(2k-1)^2 = 4k^2 - 4k + 1$ and dividing by 4 gives $k^2 - k + \frac{1}{4}$, whose floor is $k(k-1)$. The bound then reads

$$a \;\leq\; \frac{2k}{2k-2}\, k(k-1) \;=\; \frac{k}{k-1}\, k(k-1) \;=\; k^2,$$

and since $\lfloor n^2/4 \rfloor = \lfloor (2k)^2/4 \rfloor = k^2$ as well, the bound lands exactly on the target with no slack left over.

For odd $n = 2k+1$ with $k \geq 4$, the same simplification gives $\lfloor (n-1)^2/4 \rfloor = \lfloor (2k)^2/4 \rfloor = k^2$, so the bound is $a \leq \frac{2k+1}{2k-1}\, k^2$. To compare this fraction with the target, split off its integer part by dividing $(2k+1)k^2$ by $2k-1$. The remainder is exactly k, because $(2k-1)(k^2+k) = 2k^3 + k^2 - k$ and $(2k+1)k^2 = 2k^3 + k^2$ differ by k, so

$$\frac{n}{n-2} \left\lfloor \frac{(n-1)^2}{4} \right\rfloor \;=\; \frac{(2k+1)k^2}{2k-1} \;=\; k(k+1) + \frac{k}{2k-1}.$$

The leftover fraction $\frac{k}{2k-1}$ lies strictly between 0 and 1 for every $k \geq 1$, so the bound puts a at most an integer $k(k+1)$ plus a positive amount below one. Since a is itself an integer it cannot reach the next integer up, so $a \leq k(k+1)$. The target $\lfloor n^2/4 \rfloor = \lfloor (2k+1)^2/4 \rfloor = k^2 + k = k(k+1)$ agrees, so for odd n it is integrality, not the raw arithmetic, that closes the last fraction of a unit. In both parities $a \leq \lfloor n^2/4 \rfloor \leq M(n)$, completing the induction. □

Two remarks on the shape of this proof. First, the minimum-degree deletion closes the step on its own, and no separate hub case is needed, because the averaging bound holds whether or not D contains a hub, although Theorem A.13 shows directly that a hub digraph never exceeds the linear branch. Second, for even n the bound is exactly tight against the one-directional bipartite construction, which is $\frac{n}{2}$-regular in total degree, so every step of the chain is achieved by the conjectured extremiser, and for odd n the slack $\frac{k}{2k-1} < 1$ is absorbed by integrality. The proof is long not because it is deep but because the two regimes of $M(n)$ must be carried through the arithmetic separately, and that bookkeeping is precisely what Chapter 2 hands to the machine at the base cases.

Remark A.16 (Which way Whitney runs)*.* It is worth pausing on the direction of Whitney's inequality before using it, because the tempting reading is the wrong one. From $\kappa_D(u,v) \leq \lambda_D(u,v)$ for every pair it follows that $\kappa_D^{\max} \leq \lambda_D^{\max}$, hence that

$$\{D : \lambda_D^{\max} \leq m-1\} \;\subseteq\; \{D : \kappa_D^{\max} \leq m-1\}.$$

Every digraph feasible for the *arc* problem is feasible for the *vertex* problem, and not the other way round. The vertex-feasible family is the larger of the two, so the inequality between the extremal numbers it hands over for free is

$$k_m^{\mathrm{dir}}(n) \;\geq\; \ell_m^{\mathrm{dir}}(n),$$

and an upper bound for the arc problem says nothing at all about the vertex problem. The gap is not vacuous: two routes that share an interior vertex but no arc are counted by λ and not by κ, so there really are digraphs in the difference of the two families. What Whitney gives is therefore exactly one thing, that an arc construction is an honest vertex *lower* bound, and the vertex upper bound has to be earned separately. The proof below earns it, and Chapter 4 keeps the same discipline for $m \geq 3$, where the vertex problem stays open even though the arc conjecture is stated.

Proof of Theorem 1.11. Write $M(n) = \max(2(n-1), \lfloor n^2/4 \rfloor)$ as before.

Lower bound. The directed hub and the one-directional bipartite construction both have $\lambda^{\max} = 1$, so by Remark A.16 both have $\kappa^{\max} \leq 1$ and are feasible for the vertex problem. Hence $k_2^{\mathrm{dir}}(n) \geq M(n)$.

Upper bound. We show by strong induction on n that $\kappa_D^{\max} \leq 1$ forces $|A(D)| \leq M(n)$. The induction is the one that proved Theorem 1.10, and every step of it survives the change of separation, because the only two properties of the ceiling that argument uses are that it is inherited by vertex deletion and that it holds at the base sizes.

The base cases $2 \leq n \leq 7$ are settled by the same pruned exhaustive search of Chapter 2, run with the vertex feasibility test in place of the arc one. It walks every simple digraph with $\kappa^{\max} \leq 1$ on up to seven vertices and reports the maxima $2, 4, 6, 8, 10, 12$, matching $M(n)$ at each size. The transcript is reproduced in Section A.10 beside the arc one.

For the inductive step $n \geq 8$, let D be a simple digraph on n vertices with $\kappa_D^{\max} \leq 1$ and put $a := |A(D)|$. Choose v_0 of minimum total degree, so $d(v_0) \leq 2a/n$ by averaging. By Lemma A.15, which holds for κ exactly as it does for λ, the digraph $D - v_0$ on $n-1 \geq 7$ vertices again satisfies $\kappa^{\max} \leq 1$, so the induction hypothesis gives $a - d(v_0) \leq M(n-1) = \lfloor (n-1)^2/4 \rfloor$. From here the two parity computations are verbatim those of Theorem 1.10: they use only the two displayed inequalities and the integrality of a, no property of the separation. They yield $a \leq \lfloor n^2/4 \rfloor \leq M(n)$, completing the induction.

Hence $k_2^{\mathrm{dir}}(n) = M(n) = \ell_2^{\mathrm{dir}}(n)$. The two problems have the same answer at $m = 2$, but this is a computed coincidence of the two searches meeting at every base size, not a formal consequence of Whitney's inequality. ☐

Remark A.17 (The directed vertex problem at $m \geq 3$). The proof above transfers the $m = 2$ induction but nothing more, and for $m \geq 3$ the vertex problem is genuinely separate: the arc conjecture, and the decomposition of Conjecture A.21 that would prove it, both concern the smaller family and leave $k_m^{\mathrm{dir}}(n)$ untouched. What can be said is computational. Running the exhaustive search of Chapter 2 under both feasibility tests at $m = 3$ gives

n	2	3	4	5	6
$\ell_3^{\mathrm{dir}}(n)$	2	6	9	12	15
$k_3^{\mathrm{dir}}(n)$	2	6	9	12	15

with every entry proved complete, so the two separations agree at every size the exhaustion reaches, and from $n = 3$ onwards both match the conjectured $\max(m(n-1), \lfloor (n+m-2)^2/4 \rfloor)$. At $n = 2$ the conjecture's formula gives 3 where only 2 arcs exist at all, the usual degeneracy of the formula below $n = m$. Agreement over five sizes is evidence and not a proof, and the natural next question is structural rather than numerical: to separate the two problems one needs a digraph where some pair carries strictly more arc-disjoint routes than internally disjoint ones *and* where that slack pays for extra arcs. The two demands pull against each other, which is perhaps why no small example does both.

A.4 Structural propositions on the directed frontier

These are the proved ingredients of the reduction discussed in Chapter 4. The section ends with what the reduction still assumes, stated as Conjecture A.21, and with the one quadratic bound that assumes nothing, Proposition A.22.

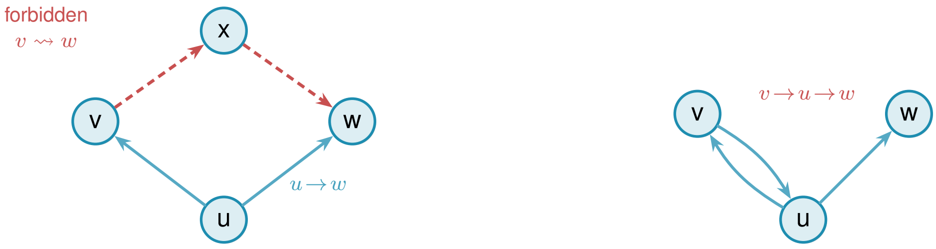
Proposition A.18 (Mutually unreachable out-neighbourhoods)**.** *Let D be a simple digraph with $\lambda_D^{\max} \le 1$. For any vertex u and distinct out-neighbours v, w of u, there is no directed path from v to w in $D - u$.*

Proof. If P were a directed v–w path avoiding u, then $u \to w$ and $u \to v \xrightarrow{P} w$ would be two arc-disjoint u-to-w routes: the first uses only the arc (u, w), the second starts with (u, v) and continues along P, none of whose arcs touch u. This contradicts $\lambda_D(u, w) \le 1$. The restriction to $D - u$ is not cosmetic, and Figure A.3 draws both the argument and the reason for the restriction: in the digraph with arcs (u, v), (v, u), (u, w) every local connectivity is 1, yet $v \to u \to w$ reaches w from v through u. ☐

Proposition A.19 (Disjoint second out-neighbourhoods)**.** *Let D be a simple digraph with $\lambda_D^{\max} \le 1$, let $u \in V$, and let $v_1 \neq v_2$ be out-neighbours of u. Then $(N^+(v_1) \setminus \{u\}) \cap (N^+(v_2) \setminus \{u\}) = \emptyset$.*

Proof. A common out-neighbour $w \neq u$ would give the two arc-disjoint routes $u \to v_1 \to w$ and $u \to v_2 \to w$, contradicting $\lambda_D(u, w) \le 1$. ☐

Proposition A.20 (Two-hop budget on bipartite partitions)**.** *Let D be a simple digraph with $\lambda_D^{\max} \le m - 1$. If V admits a partition $A \cup B$ with every arc from A to B present, then every $b \in B$ has in-degree at most $m - 2$ from inside B, and consequently the number of arcs inside B is at most $(m-2)|B|$.*



(a) If a directed $v \rightsquigarrow w$ path (red) existed in $D - u$, then $u \to w$ and $u \to v \rightsquigarrow w$ would be two arc-disjoint u–w routes, against $\lambda(u, w) \leq 1$.

(b) The counterexample $\{(u, v), (v, u), (u, w)\}$: $\lambda^{\max} = 1$, yet w *is* reachable from v, but only *through* u.

Figure A.3. Why Proposition A.18 must exclude paths through u. **(a)** For out-neighbours v, w of u in a digraph with $\lambda^{\max} \leq 1$, no $v \rightsquigarrow w$ path can avoid u: such a path would combine with the lone arc $u \to w$ into two arc-disjoint routes. **(b)** The restriction to $D - u$ is essential. In the three-vertex digraph $\{(u, v), (v, u), (u, w)\}$ we have $\lambda^{\max} = 1$, yet v reaches w via $v \to u \to w$, a route that reuses the vertex u, which the proposition allows.

Proof. Fix $a \in A$ and $b \in B$, and let $b'_1, \ldots, b'_d$ be the in-neighbours of b inside B, where $d = \deg^-_B(b)$. The direct arc $a \to b$ together with the two-step detours $a \to b'_i \to b$ are $1 + d$ arc-disjoint a-to-b routes, all present because $A \to B$ is complete. Feasibility forces $1 + d \leq m - 1$, so $d \leq m - 2$, and summing over B bounds the internal arcs by $(m - 2)|B|$. $\square$

We now combine Proposition A.20 with the best choice of partition. Suppose every arc of D either runs from A to B or stays inside B: the partition $A \to B$ is complete, no arc lies inside A, and crucially *no arc runs from B back to A*. Write $b = |B|$, so that $|A| = n - b$. The arcs then come in exactly two kinds. The arcs crossing the partition number $|A|\,|B| = (n - b)\, b$, one for each ordered A–B pair. The arcs inside B number at most $(m - 2)\, b$: since no route between two vertices of B ever needs to leave B (there is no arc back to A to leave by), Proposition A.20 applies to B on its own and caps the in-degree of each $b \in B$ from inside B at $m - 2$. There are no other arcs, so, adding the two kinds,

$$|A(D)| \;\leq\; (n - b)\, b \;+\; (m - 2)\, b \;=\; b\,\bigl(n + m - 2 - b\bigr).$$

The right-hand side is a downward parabola in b. Writing $s = n + m - 2$, it is $b(s - b) = -b^2 + sb$, maximised at $b = s/2$, so over integers the best partition takes $b = \lceil s/2 \rceil$ (equivalently $\lfloor s/2 \rfloor$), and the maximum value is

$$\max_b \; b\,(s - b) \;=\; \left\lfloor \frac{s^2}{4} \right\rfloor \;=\; \left\lfloor \frac{(n + m - 2)^2}{4} \right\rfloor.$$

This is the quadratic branch of Conjecture 1.14, reached uniformly in m. It is worth being exact about how much was assumed, because the count is often summarised as resting on the absence of backward arcs alone, and that summary is too generous. Four things were granted at once: that a partition $V = A \cup B$ exists at all, that every arc from A to B is present, that no arc lies inside A, and that no arc runs from B back to A. Only the last of the four is the backward-arc condition. The first three describe the shape of the extremiser, and nothing proved so far forces an arbitrary extremal digraph to have that shape. The honest statement of the gap is therefore a structural one:

Conjecture A.21 (The missing decomposition)**.** *Let n be large enough that the quadratic branch of Conjecture 1.14 strictly exceeds the linear one, that is, $\lfloor (n+m-2)^2/4 \rfloor > m(n-1)$. Then every extremal digraph D on n vertices with $\lambda_D^{\max} \leq m-1$ admits a partition $V(D) = A \cup B$ such that every arc from A to B is present, no arc lies inside A, and no arc runs from B to A.*

The hypothesis on n has to be there, and it is worth seeing why, because the obvious phrasing is too narrow. On the linear branch the extremisers are *not* just the hub. At $m = 2$, every bidirected spanning tree is feasible and has exactly $2(n-1)$ arcs, since between two vertices the only route follows the unique tree path, so the whole extremal set on that branch is the set of bidirected trees. An exhaustive classification confirms it: at $n = 5$ and $n = 6$ there are exactly 3 and 6 extremal digraphs up to isomorphism, which are precisely the numbers of trees on 5 and 6 vertices, and none of them admits the decomposition. Excluding "the hub" would leave all the other trees behind, so the clean hypothesis is on the size rather than on the shape.

Given Conjecture A.21 the count above closes Conjecture 1.14 for $m \geq 3$. The backward-arc clause is the part where the natural delete-and-compensate exchange fails to be monotone, which is why it is the clause usually named, but the partition itself is not free either, and Chapter 4 states the position that way. Testing the conjecture by exhaustion is harder than it looks: the crossover sits at $n = 8$ for $m = 2$ and at $n = 9$ for $m = 3$, so every size an exhaustive sweep can reach at $m = 3$ still lies on the linear branch, where the statement says nothing. That is the practical reason this conjecture has resisted a computational probe rather than a proof.

What can be proved without any of the four is the leading constant, and the argument is short enough to give in full. It also says something the conjecture does not: that every feasible digraph is within a lower-order number of arcs of being one-directional bipartite.

Proposition A.22 (Unconditional quadratic bound and stability)**.** *Let D be a simple digraph on $n \geq 2$ vertices with $\lambda_D^{\max} \leq m-1$. Then*

$$|A(D)| \;\leq\; \left\lfloor \frac{n^2}{4} \right\rfloor \;+\; \sqrt{m}\, n^{3/2}.$$

Moreover D can be made one-directional bipartite, meaning every remaining arc runs from one part to the other and none returns, by deleting at most $\sqrt{m}\, n^{3/2}$ arcs. Consequently, for each fixed m,

$$\ell_m^{\mathrm{dir}}(n) \;=\; \frac{n^2}{4} \;+\; O_m\!\left(n^{3/2}\right),$$

so the leading constant $\frac{1}{4}$ of Conjecture 1.14 holds unconditionally, with no hypothesis on the structure of the extremiser.

Proof. Step 1, a budget on two-step routes. Fix an ordered pair (u, v) with $u \neq v$, and let $p_2(u, v)$ count the vertices x with both $u \to x$ and $x \to v$ present. Such an x is automatically distinct from

u and from v, since D has no loops, so each one gives a genuine two-step route $u \to x \to v$. Two of these routes with different midpoints $x \neq x'$ share no arc, since the first uses (u, x) and (x, v) and the second uses (u, x') and (x', v), and neither uses the arc (u, v), which has no midpoint at all. So the direct arc, when present, together with all the two-step routes, is a family of pairwise arc-disjoint u-to-v routes, and feasibility caps its size:

$$\mathbf{1}_{(u,v)\in A(D)} + p_2(u, v) \ \leq\ \lambda_D(u, v) \ \leq\ m - 1.$$

Summing over all $n(n-1)$ ordered pairs gives $|A(D)| + \sum_{u\neq v} p_2(u, v) \leq (m-1)\, n(n-1)$.

Step 2, from pairs to degrees. Counting directed walks of length two by their midpoint, $\sum_{u,v} p_2(u, v) = \sum_x d^-(x)\, d^+(x)$, where this sum runs over *all* ordered pairs, the diagonal $u = v$ included. The diagonal terms count the closed walks $u \to x \to u$, that is, twice the number of digons of D. Distinct digons use disjoint pairs of arcs, so twice the number of digons is at most $|A(D)|$, and that is exactly the slack Step 1 left over. Combining the two,

$$\sum_{x\in V} d^+(x)\, d^-(x) \ \leq\ \big[(m-1)\, n(n-1) - |A(D)|\big] + |A(D)| \ =\ (m-1)\, n(n-1).$$

This is the one inequality the whole section runs on, and it is worth reading in words: a vertex with a large in-degree *and* a large out-degree is expensive, because it manufactures two-step routes for many pairs at once, and the budget for those routes is only quadratic in n.

Step 3, the smaller half-degree is small on average. Since $\min(a, b) \leq \sqrt{ab}$ for non-negative a, b, and by the Cauchy-Schwarz inequality in the form $\sum_x \sqrt{a_x} \leq \sqrt{n \sum_x a_x}$,

$$\begin{aligned} \sum_{x\in V} \min\big(d^+(x), d^-(x)\big) \ &\leq\ \sum_{x\in V} \sqrt{d^+(x)\, d^-(x)} \ \leq\ \sqrt{n \sum_{x\in V} d^+(x)\, d^-(x)} \\ &\leq\ \sqrt{n\cdot(m-1)\, n(n-1)} \ =\ n\sqrt{(m-1)(n-1)} \ \leq\ \sqrt{m}\, n^{3/2}. \end{aligned}$$

Step 4, deleting the smaller half of every vertex. Call x *source-like* if $d^+(x) \geq d^-(x)$ and *sink-like* otherwise, and write S and T for the two classes, so $V = S \cup T$. Delete every in-arc of a source-like vertex and every out-arc of a sink-like vertex. A source-like x loses $d^-(x) = \min(d^+(x), d^-(x))$ arcs and a sink-like x loses $d^+(x) = \min(d^+(x), d^-(x))$ arcs, so the number of deleted arcs is at most $\sum_x \min(d^+(x), d^-(x))$, which Step 3 bounds by $\sqrt{m}\, n^{3/2}$. An arc (a, b) survives only if a is not sink-like and b is not source-like, that is, only if $a \in S$ and $b \in T$. So what remains runs entirely from S to T, which is the one-directional bipartite shape, and it has at most $|S|\,|T| \leq \lfloor n^2/4 \rfloor$ arcs. Adding the two counts gives the bound.

The asymptotic statement follows because the augmented bipartite construction (Construction 1.13) already gives $\ell_m^{\mathrm{dir}}(n) \geq \lfloor (n+m-2)^2/4 \rfloor = n^2/4 + O_m(n)$, and $n^{3/2}$ dominates n. $\square$

The error term $n^{3/2}$ is larger than the $(m-2)n/2$ that Conjecture 1.14 predicts, so the proposition as it stands does not separate the conjectured formula from its near neighbours. It can be sharpened to the right order, and the extra idea is small enough to be worth giving, because it explains where the $\sqrt{n}$ was being wasted.

Step 3 is lossy exactly when every vertex carries a middling degree. The Cauchy-Schwarz step is tight only if $d^+(x)d^-(x)$ is about $(m-1)n$ at every vertex, which forces every degree down to about $\sqrt{mn}$. But a digraph with $\Theta(n^2)$ arcs has average degree $\Theta(n)$, and at a vertex of large degree the constraint $d^+(x)d^-(x) \le (m-1)n$ bites much harder: with $d^+ + d^-$ of order n, the smaller of the two must be of order m rather than $\sqrt{mn}$. The dense case and the tight case of Step 3 are therefore incompatible, and the way to exploit that is a dichotomy on the minimum degree, with vertex deletion to handle the sparse side.

Lemma A.23 (Small side of a high-degree vertex). *For any vertex x of a digraph with total degree $d(x) = d^+(x) + d^-(x) > 0$,*

$$\min\bigl(d^+(x), d^-(x)\bigr) \;\le\; \frac{2\, d^+(x)\, d^-(x)}{d(x)}.$$

Proof. Write $\mu = \min(d^+(x), d^-(x))$ and $M = \max(d^+(x), d^-(x))$, so $\mu + M = d(x)$ and $\mu M = d^+(x)d^-(x)$. Since $\mu \le M$ we have $M \ge d(x)/2$, hence $d^+(x)d^-(x) = \mu M \ge \mu\, d(x)/2$, which rearranges to the claim. $\square$

Theorem A.24 (The quadratic bound with a linear error). *For all $m \ge 2$ and $n \ge 2$, every simple digraph D on n vertices with $\lambda_D^{\max} \le m-1$ satisfies*

$$|A(D)| \;\le\; \left\lfloor \frac{n^2}{4} \right\rfloor \;+\; 4(m-1)(n-1),$$

so for each fixed m,

$$\ell_m^{\mathrm{dir}}(n) \;=\; \frac{n^2}{4} \;+\; \Theta_m(n),$$

and both the leading constant $\frac{1}{4}$ and the order *of the second-order term of Conjecture 1.14 hold unconditionally.*

Proof. Induction on n. For $n = 2$ a simple digraph has at most 2 arcs and the right-hand side is $1 + 4(m-1) \ge 5$, so the base holds. Let $n \ge 3$ and split on the minimum total degree.

Case 1: some vertex v has $d(v) < n/2$. Since $d(v)$ is an integer, $d(v) \le \lfloor n/2 \rfloor$. By Lemma A.15 the digraph $D - v$ on $n-1$ vertices is still feasible, so the induction hypothesis applies to it and

$$|A(D)| \;\le\; \left\lfloor \frac{(n-1)^2}{4} \right\rfloor + 4(m-1)(n-2) + \left\lfloor \frac{n}{2} \right\rfloor \;=\; \left\lfloor \frac{n^2}{4} \right\rfloor + 4(m-1)(n-2),$$

using Lemma A.27 for the last step, and this is below the claimed bound.

Case 2: every vertex has $d(x) \geq n/2$. By Lemma A.23 and Step 2 of Proposition A.22,

$$\sum_{x \in V} \min\bigl(d^+(x), d^-(x)\bigr) \;\leq\; \sum_{x \in V} \frac{2\, d^+(x)\, d^-(x)}{d(x)} \;\leq\; \frac{4}{n} \sum_{x \in V} d^+(x)\, d^-(x) \;\leq\; \frac{4}{n}\,(m-1)\, n(n-1),$$

which is $4(m-1)(n-1)$. Step 4 of Proposition A.22 deletes exactly that many arcs and leaves at most $\lfloor n^2/4 \rfloor$, so the bound holds in this case too.

For the two-sided asymptotic statement, Construction 1.13 gives

$$\ell_m^{\mathrm{dir}}(n) \;\geq\; \left\lfloor \frac{(n+m-2)^2}{4} \right\rfloor \;=\; \frac{n^2}{4} + \frac{(m-2)n}{2} + O_m(1),$$

so the second-order term is linear in n from both sides. □

What is left open is therefore a constant factor in one coefficient, not a shape. Conjecture 1.14 says the second-order coefficient is exactly $(m-2)/2$, and the theorem pins it between that and $4(m-1)$, a factor of roughly eight. Closing that gap is where the structure of Conjecture A.21 lives, and Remark A.25 rules out the most obvious way to try.

Remark A.25 (Case 2 cannot be sharpened without a new inequality)*.* One might hope to cut Case 2's constant using the observation that in the conjectured extremiser, one whole side has $\min(d^+, d^-) = 0$, so summing Lemma A.23's worst case at every vertex looks wasteful. It is not: Case 2's bound is already the exact maximum obtainable from the two facts it uses, the budget $\sum_x d^+(x)\, d^-(x) \leq (m-1)n(n-1)$ of Step 2 in Proposition A.22 and the floor $d(x) \geq n/2$, and no cleverer use of those two facts alone can do better.

At a vertex with $d(x) = s \geq n/2$, achieving $\min(d^+(x), d^-(x)) = t$ costs $d^+(x)\, d^-(x) = t(s-t)$ at least, cheapest at $s = n/2$, giving the cost function $c(t) = t(n/2 - t)$ on $t \in [0, n/4]$. Since c is concave with $c(0) = 0$, for any two vertices with values $0 < a \leq b$ strictly below the cap, shifting mass from a to b preserves $a + b$ while lowering $c(a) + c(b)$, because c' is decreasing. Repeating this exchange shows that the true maximum of $\sum_x t_x$ under a shared budget Q is attained by concentrating the budget on as few vertices as the cap allows, each at $t_x = n/4$, and the rest at $t_x = 0$, exactly the qualitative picture the conjectured extremiser suggests, and it already gives $4Q/n = 4(m-1)(n-1)$ once the number of vertices at the cap is below n. The reason the crude per-vertex bound loses nothing is that it is tight at exactly the same point: Lemma A.23 is an equality only when $d^+(x) = d^-(x)$, and the floor substitution $d(x) \to n/2$ is an equality only when $d(x) = n/2$, so both hold simultaneously exactly at $d^+(x) = d^-(x) = n/4$, the vertex profile the true optimum concentrates on. Cutting the constant therefore needs a second, independent inequality, one that constrains interference among the handful of expensive near-balanced vertices themselves rather than a sharper reading of the one already in hand.

A.5 The directed multigraph problem at large n

The body proves $L_m^{\mathrm{dir}}(n) = 2(n-1)(m-1)$ for $n \le 6$ (Theorem 2.2). This section records what is known beyond, where the one-directional bipartite construction takes over. Throughout, the scaling reduction of Chapter 2 is used in the following form.

Lemma A.26 (Scaling reduction)**.** *Let D be a directed multigraph with $\mu(u,v) \le m-1$ on every ordered pair and $\lambda^{\max}(D) \le m-1$. Then $w := \mu/(m-1)$ takes values in $[0,1]$, satisfies $\operatorname{maxflow}(s,t;w) \le 1$ for all $s \ne t$, and $|A(D)| = (m-1)\sum_{u\ne v} w(u,v)$.*

Proof. Set $w = \mu/(m-1)$. Every multiplicity obeys $0 \le \mu(u,v) \le m-1$, so each $w(u,v)$ lies in $[0,1]$. Dividing every capacity by the constant $m-1$ divides every flow value by $m-1$, and by Theorem A.1 the maximum flow under the capacities μ is $\lambda_D(s,t) \le m-1$, so under w it is at most 1. For the arc count, recall that the number of arcs is the total multiplicity, $|A(D)| = \sum_{u\ne v} \mu(u,v)$, since each parallel copy is one arc. Substituting $\mu = (m-1)w$,

$$|A(D)| \;=\; \sum_{u\ne v} \mu(u,v) \;=\; \sum_{u\ne v} (m-1)\, w(u,v) \;=\; (m-1)\sum_{u\ne v} w(u,v),$$

which is the claimed identity. □

Lemma A.27 (Arithmetic identity)**.** *For all $n \ge 2$, $\left\lfloor \frac{(n-1)^2}{4} \right\rfloor + \lfloor n/2 \rfloor = \left\lfloor \frac{n^2}{4} \right\rfloor$.*

Proof. For $n = 2k$ the left side is $k(k-1) + k = k^2$, and for $n = 2k+1$ it is $k^2 + k = k(k+1)$. Both match the right side. □

Proposition A.28 (Bipartite upper bound for even $n \ge 8$, conditional)**.** *Let $n = 2k \ge 8$ be even. Suppose every fractional weighting w' on $n-1$ vertices with values in $[0,1]$ and all pairwise maximum flows at most one has total weight at most $\left\lfloor \frac{(n-1)^2}{4} \right\rfloor$. Then every directed multigraph D on n vertices with $\lambda^{\max}(D) \le m-1$ satisfies $|A(D)| \le (m-1)\left\lfloor \frac{n^2}{4} \right\rfloor$.*

Proof. By Lemma A.26 the arc count is $|A(D)| = (m-1)M$ with $M := \sum_{u\ne v} w(u,v)$, so it suffices to bound $M \le \lfloor n^2/4 \rfloor$ for the scaled weighting w, all of whose pairwise maximum flows are at most 1.

Averaging. Give each vertex its weighted total degree $d(v) = \sum_u \big(w(u,v) + w(v,u)\big)$. Summing over all vertices counts the weight of each ordered pair exactly twice, once at its tail and once at its head, so $\sum_v d(v) = 2M$, and some vertex v_0 has $d(v_0) \le 2M/n$.

Deletion. The number $d(v_0)$ is exactly the total weight carried by pairs touching v_0, so deleting v_0 leaves a weighting on $n-1$ vertices of total $M - d(v_0)$. Removing a vertex cannot raise any maximum flow, so that restricted weighting still has all pairwise flows at most 1, and the hypothesis applies to it:

$$M - d(v_0) \;\le\; \left\lfloor \frac{(n-1)^2}{4} \right\rfloor \;=\; \left\lfloor \frac{(2k-1)^2}{4} \right\rfloor \;=\; k(k-1),$$

the last step because $(2k-1)^2/4 = k^2 - k + \frac{1}{4}$ floors to $k(k-1)$.

Combining. Since $n = 2k$ we have $d(v_0) \le 2M/n = M/k$, hence

$$M - \frac{M}{k} \le M - d(v_0) \le k(k-1), \qquad \text{that is} \qquad M \cdot \frac{k-1}{k} \le k(k-1).$$

Dividing by $(k-1)/k > 0$ gives $M \le k^2 = \lfloor n^2/4 \rfloor$, and multiplying back by $m-1$ gives $|A(D)| \le (m-1)\lfloor n^2/4 \rfloor$. □

The small cases $n \in \{4, 6\}$ genuinely do not satisfy the conclusion, and must not: there $2(n-1) > \lfloor n^2/4 \rfloor$ and the double star beats the bipartite weighting, which is why the certified values of Theorem 2.2 sit on the linear branch. For odd n the same deletion gives $M \le k(k+1) + \frac{k}{2k-1}$, an overshoot of less than one that integrality no longer absorbs, because fractional weights need not have integer totals. The induction would close if some vertex always had weighted degree at most $(m-1)k$ before scaling, which is the missing minimum-degree statement.

Conjecture A.29 (Minimum-degree lemma for odd n)**.** *In any directed multigraph D on $n = 2k+1$ vertices ($k \ge 3$) with $\lambda^{\max}(D) \le m-1$ and $m \ge 3$, some vertex has total degree $d(v) \le (m-1)k$, counting arcs with multiplicity.*

At $m = 2$ the statement is a proposition rather than a conjecture, through a reduction that also settles the whole multigraph problem at $m = 2$.

Proposition A.30 (Parallel-arc elimination at $m = 2$)**.** *A directed multigraph with $\lambda^{\max} \le 1$ has no parallel arcs, since two parallel arcs are already two arc-disjoint routes. Hence it is a simple digraph.*

Theorem A.31 (Directed multigraphs at $m = 2$)**.** *For all $n \ge 2$, $L_2^{\mathrm{dir}}(n) = \ell_2^{\mathrm{dir}}(n) = \max\left(2(n-1), \left\lfloor \frac{n^2}{4} \right\rfloor\right)$.*

Proof. Immediate from Proposition A.30 and Theorem 1.10. □

Proposition A.32 (Minimum degree at $m = 2$, odd n)**.** *In any directed multigraph on $n = 2k+1 \ge 7$ vertices with $\lambda^{\max} \le 1$, some vertex has $d(v) \le k$.*

Proof. By Proposition A.30 and theorem 1.10, $|A(D)| \le \max\left(2(n-1), \lfloor n^2/4 \rfloor\right) = \lfloor n^2/4 \rfloor = k(k+1)$, the maximum being the quadratic branch because $n \ge 7$. The average total degree is $\frac{2k(k+1)}{2k+1} < k+1$, and some integer degree is at most k. The hypothesis $n \ge 7$ is needed: on the linear branch the statement can fail, as the bidirected star on three vertices has minimum total degree $2 > 1$. □

For $m \ge 3$ the two extreme regimes of Conjecture A.29 are settled. In the *purely saturated* regime, where every multiplicity is 0 or $m-1$, the scaled weighting is the indicator of a simple digraph with $\lambda^{\max} \le 1$, its degrees are integers, and the argument of Proposition A.32 applies verbatim after multiplying back by $m-1$. In the *all-unsaturated* simple regime at $n=7$, $m=3$, the program certifies by a cut-counting infeasibility that no simple digraph satisfies all two-hop inequalities with every total degree at least 7. The open case is the *mixed* regime, where some multiplicity lies strictly between 0 and $m-1$: there the scaled degrees are genuinely fractional, the averaging bound $\frac{2k(k+1)}{2k+1}$ lands strictly between k and $k+1$, and no integrality rescues it. This is the single identified gap that separates the certified values of Theorem 2.2 from the full conjecture

$$L_m^{\mathrm{dir}}(n) \;=\; (m-1)\,\max\!\Big(2(n-1),\;\Big\lfloor\frac{n^2}{4}\Big\rfloor\Big),$$

whose lower bound is unconditional: the double star realises $2(n-1)(m-1)$ and the one-directional bipartite multigraph, every arc at multiplicity $m-1$, realises $(m-1)\lfloor n^2/4\rfloor$ with $\lambda^{\max}=m-1$.

Remark A.33 (Why the mixed regime resists, and its kinship with the back-arc gap). Two facts explain why the mixed regime is genuinely hard rather than merely unfinished, and both are worth recording so the conjecture is not mistaken for a routine averaging exercise.

First, averaging alone cannot close it, even granted the strongest input available. Work in the scaled weighting $w=\mu/(m-1)$ of Lemma A.26, with total weight $M=\sum_{u\neq v} w(u,v)$ and weighted total degree $d(v)$, so the conjecture asks for a vertex with $d(v)\le k$. Granting the fractional even bound that drives Proposition A.28, the odd deletion already gives $M\le k(k+1)+\frac{k}{2k-1}=\frac{(2k+1)k^2}{2k-1}$. Since the minimum degree never exceeds the average $\frac{2M}{2k+1}$,

$$\min_v d(v)\;\le\;\frac{2M}{2k+1}\;\le\;\frac{2k^2}{2k-1}\;=\;k+\frac{k}{2k-1}\;<\;k+1.$$

In the saturated and $m=2$ regimes the degrees are integers, so this rounds down to k and the conjecture follows, exactly as in Proposition A.32. In the mixed regime $d(v)$ is genuinely fractional and the leftover $\frac{k}{2k-1}$ cannot be rounded away. The obstruction is not the particular bound chosen. To force $\min_v d(v)\le k$ by averaging one would need $M\le k^2+\frac{k}{2}$, yet the one-directional bipartite weighting with $|A|=k$ and $|B|=k+1$ already carries $M=k(k+1)=k^2+k$, so no averaging argument can finish for any m. Closing the gap needs a structural step that exhibits a specific low-degree vertex, which is precisely what Lemma A.34 supplies at $m=3$ in order to *bypass* the conjecture rather than prove it.

Second, the conjecture is tight, and the natural way to break it fails for a reason that ties this gap to the back-arc obstruction of Conjecture 1.14. At the bipartite extremiser the minimum degree is exactly k, attained on the B side. The obvious attempt to raise it bolts a little backward weight onto the saturated bipartite point, sending $w(b,a_0)=\epsilon$ from every $b\in B$ to one fixed source $a_0\in A$, which would lift each B-degree to $k+\epsilon$. It is infeasible for every $\epsilon>0$. With the forward layer at full weight the backward arcs open the recirculating routes $a\to b\to a_0\to b'$, so the flow from a source $a\neq a_0$ to a sink b' rises to $1+k\epsilon>1$. A single backward arc laid on a saturated forward layer manufactures an extra route, the same mechanism that keeps the upper bound of Conjecture 1.14 open and the reason the bipartite extremisers are locally rigid under the searches of Chapter 3. The minimum-degree gap and the back-arc gap are two faces of one obstruction. A systematic fractional hill-climb (`fractional_search`), a search that repeatedly nudges one weight at a time and keeps a change only when the objective improves while every pairwise flow stays at most one, run with three initialisation types and eight seeds each at $n = 9$, 11, and 13, finds no feasible weighting with minimum weighted degree above k, confirming the bipartite point's rigidity for $k = 4$, 5, and 6. Feasibility of a fractional weighting is checked by `fractional_flows_feasible`, which calls `_float_maxflow_value`, a textbook maximum-flow routine (Edmonds–Karp) run directly on the fractional capacities, since scipy's `csgraph` max-flow accepts only integer capacities.

Lemma A.34 (Attachment lemma)**.** *Let $m \geq 2$ and let D_0 be the one-directional complete bipartite multigraph with parts A and B, $|A| = p \geq 2$, $|B| = q \geq 2$, and $\mu(a, b) = m - 1$ for every $a \in A$, $b \in B$, with no other arcs. Let D be obtained from D_0 by adding one new vertex v together with any multiset of arcs incident to v, a multiset because parallel copies are allowed (multiplicities at most $m - 1$). If $\lambda_D^{\max} \leq m - 1$, then $d(v) \leq (m - 1)\max(p, q)$.*

The proof is a bookkeeping argument on the arcs at v, and Figure A.4 is its scaffold. Sort those arcs into the four classes i_A, o_A, i_B, o_B of the figure, by direction and by which part they touch. Three combinations are impossible, because each completes an m-th arc-disjoint route between two old vertices on top of the $m - 1$ parallel arcs they already share. An i_A arc with an o_B arc makes $a \to v \to b$. An i_A arc with an o_A arc to a *different* source makes $a \to v \to a' \to b$. An i_B arc with an o_B arc to a *different* sink makes $a \to b \to v \to b'$. Forbidding these forces v into one of a few rigid shapes, a pure sink touching a single source, a pure source touching a single sink, or a small mixed remnant, and in each shape a count of two-step routes through v caps its degree. The three cases below are exactly those shapes, and the bound $(m - 1)\max(p, q)$ is whichever of them is largest.

Proof. The strategy is to sort the arcs at the new vertex v into four groups by their direction and which side they touch, rule out the combinations that would manufacture an m-th route between two old vertices, and then bound whatever survives. Write $\mathrm{i}_A = \sum_a \mu(a, v)$, $\mathrm{i}_B = \sum_b \mu(b, v)$, $\mathrm{o}_A = \sum_a \mu(v, a)$, $\mathrm{o}_B = \sum_b \mu(v, b)$ for the total multiplicities into v from A, into v from B, out of v to A, and out of v to B, so that $d(v) = \mathrm{i}_A + \mathrm{i}_B + \mathrm{o}_A + \mathrm{o}_B$. Three route patterns are forbidden outright, because each adds an m-th arc-disjoint route on top of the $m - 1$ parallel arcs of a bipartite pair:

(1) $\mu(a, v) \geq 1$ and $\mu(v, b) \geq 1$: the path $a \to v \to b$ together with the $m - 1$ arcs $a \to b$ gives $\lambda(a, b) \geq m$.

(2) $\mu(a, v) \geq 1$ and $\mu(v, a') \geq 1$ with $a' \neq a$: the path $a \to v \to a' \to b$ (any $b \in B$) gives $\lambda(a, b) \geq m$.

(3) $\mu(b, v) \geq 1$ and $\mu(v, b') \geq 1$ with $b' \neq b$: the path $a \to b \to v \to b'$ (any $a \in A$) gives $\lambda(a, b') \geq m$.

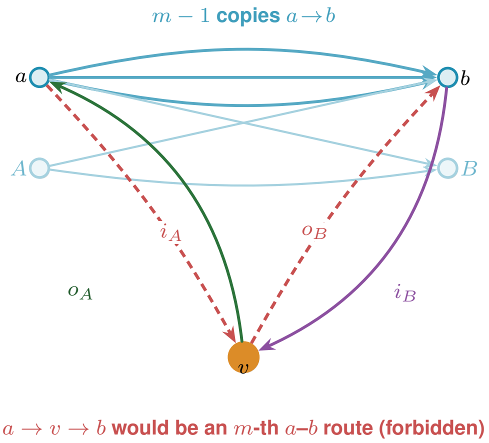


Figure A.4. The attachment lemma (Lemma A.34): a new vertex v is glued onto $(m-1) \cdot B_{p,q}$, the one-directional complete bipartite multigraph in which every $a \in A$ sends $m-1$ parallel arcs to every $b \in B$ (the bundle along the top, faint for the pairs other than a, b). The arcs at v fall into four classes by direction and side: i_A into v from A, o_A out of v to A, i_B into v from B, and o_B out of v to B. The two red legs i_A and o_B are drawn together because they are the worst case: the route $a \to v \to b$ laid on top of the $m-1$ existing $a \to b$ arcs is an m-th arc-disjoint a–b route, so $\lambda(a,b) \geq m$, which feasibility forbids. Ruling this out, with two siblings like it, caps $d(v) \leq (m-1)\max(p,q)$.

Each bound below rests on one principle. The local connectivity $\lambda(s,t)$ is at least the number of pairwise arc-disjoint s-to-t routes we can exhibit, because any such family of routes is an integral flow from s to t. Feasibility says $\lambda \leq m-1$, so the moment we can name m arc-disjoint routes between two vertices we have a contradiction.

Case I: $\mathrm{i}_A \geq 1$. Some arc enters v from a vertex $a \in A$. Pattern (1) then forces $\mathrm{o}_B = 0$, because any arc $v \to b$ would complete the route $a \to v \to b$, an m-th route on the pair (a,b) laid on top of its $m-1$ parallel arcs.

Suppose first that $\mathrm{o}_A \geq 1$, so v also sends an arc to some vertex of A. Pattern (2) makes this rigid: an arc into v from a and an arc out of v to a different $a' \neq a$ would give the route $a \to v \to a' \to b$ for any b, again an m-th route. So every arc between v and A, in either direction, must touch one single vertex a^*. Counting arc-disjoint routes from a^* to v, there are the $\mu(a^*,v)$ direct arcs, and for each $b \in B$ the two-step route $a^* \to b \to v$, which can be run $\min(\mu(a^*,b),\mu(b,v))$ times on disjoint arcs. Different b use different arcs, and none reuse a direct arc, so

$$\lambda(a^*,v) \;\geq\; \mu(a^*,v) + \sum_b \min(\mu(a^*,b),\mu(b,v)) \;=\; \mu(a^*,v) + \mathrm{i}_B,$$

the last equality because $\mu(a^*,b) = m-1$ is at least $\mu(b,v)$, so each minimum is $\mu(b,v)$ and the sum is i_B. Feasibility caps the left side at $m-1$, so $\mu(a^*,v) + \mathrm{i}_B \leq m-1$. Every arc at v now touches either a^* or B, so $d(v) = \mu(a^*,v) + \mathrm{i}_B + \mu(v,a^*)$, and with the first two summing to at most $m-1$ and the last at most $m-1$ we get $d(v) \leq 2(m-1)$.

Suppose instead $\mathrm{o}_A = 0$. With $\mathrm{o}_B = 0$ already, v is now a pure sink. The same two-step count applies to *every* $a \in A$ at once, giving $\lambda(a,v) \geq \mu(a,v) + \mathrm{i}_B$, hence $\mu(a,v) \leq (m-1) - \mathrm{i}_B$ for each of the p vertices of A. Summing over A bounds the in-flow from A by $\mathrm{i}_A \leq p(m-1) - p\,\mathrm{i}_B$, so

$$d(v) = \mathrm{i}_A + \mathrm{i}_B \le p(m-1) - (p-1)\,\mathrm{i}_B \le p(m-1),$$

the final step because $(p-1)\,\mathrm{i}_B \ge 0$.

Case II: $\mathrm{i}_A = 0$ *and* $\mathrm{o}_B \ge 1$*.* This is Case I read backwards. Reversing every arc turns $(m-1)\cdot B_{p,q}$ into $(m-1)\cdot B_{q,p}$, swaps in-arcs with out-arcs, and sends pattern (2) to pattern (3), so the same argument applies with the roles of A and B exchanged. It yields either $d(v) \le 2(m-1)$, or v is a pure source for which $\lambda(v,b) \ge \mu(v,b) + \mathrm{o}_A \le m-1$ holds for every b, and summing over the q vertices of B gives $d(v) \le q(m-1) - (q-1)\,\mathrm{o}_A \le q(m-1)$.

Case III: $\mathrm{i}_A = \mathrm{o}_B = 0$*.* Now v receives nothing from A and sends nothing to B, so its only arcs are the i_B in-arcs from B and the o_A out-arcs to A. For any $a \in A$ the two-step routes $a \to b \to v$ over all b give $\lambda(a,v) \ge \mathrm{i}_B$, so $\mathrm{i}_B \le m-1$, and for any $b \in B$ the routes $v \to a \to b$ over all a give $\lambda(v,b) \ge \mathrm{o}_A$, so $\mathrm{o}_A \le m-1$. Hence $d(v) = \mathrm{i}_B + \mathrm{o}_A \le 2(m-1)$.

Every case bounds $d(v)$ by either $2(m-1)$ or $\max(p,q)(m-1)$. Since $p, q \ge 2$ we have $2(m-1) \le (m-1)\max(p,q)$, so in all cases $d(v) \le (m-1)\max(p,q)$. □

Corollary A.35 (Attachment equality)**.** *In the setting of Lemma A.34 with* $\max(p,q) \ge 3$*, suppose* $d(v) = (m-1)\max(p,q)$*. If* $q > p$*, then* v *is a pure source with* $\mu(v,b) = m-1$ *for every* $b \in B$*, so* $D = (m-1)\cdot B_{p+1,q}$ *with* v *joining the source side. If* $p > q$*, then* v *is symmetrically a pure sink at full multiplicity, and if* $p = q$*, it is one of the two.*

Proof. Rerun the cases of Lemma A.34. The mixed cases cap $d(v)$ at $2(m-1) < (m-1)\max(p,q)$. The pure-sink case gives $d(v) \le p(m-1) - (p-1)\mathrm{i}_B$, which reaches $(m-1)\max(p,q)$ only when $p \ge q$, $\mathrm{i}_B = 0$ and every $\mu(a,v) = m-1$. The pure-source case symmetrically forces $\mathrm{o}_A = 0$ and every $\mu(v,b) = m-1$ when $q \ge p$. Feasibility of the resulting digraph is the bipartite construction itself. □

Theorem A.36 (Conditional odd step, $m \in \{3,4\}$)**.** *Fix* $k \ge 4$ *and* $m \in \{3,4\}$*, and assume (i)* $L_m^{\mathrm{dir}}(2k) = (m-1)k^2$*, and (ii) every directed multigraph attaining it is the one-directional complete bipartite multigraph* $(m-1)\cdot B_{k,k}$*. Then* $L_m^{\mathrm{dir}}(2k+1) = (m-1)k(k+1)$*.*

Proof. The lower bound is $(m-1)\cdot B_{k,k+1}$. The upper bound is a counting squeeze. If the arc count were even one over the target, then deleting any single vertex would still leave a feasible multigraph on $2k$ vertices, which by (i) cannot exceed $(m-1)k^2$, so every vertex would have to carry a large degree. Yet the average degree is only barely above that floor, so some vertex must sit exactly on it, and deleting that one lands precisely on the even extremiser $(m-1)\cdot B_{k,k}$ of (ii). The attachment lemma then caps how much that vertex could have carried, below what the surplus requires, and the contradiction forces the surplus to vanish. We now make this precise.

For the upper bound let D on $n = 2k+1$ vertices have $\lambda^{\max} \le m-1$ and $|A(D)| = (m-1)k(k+1) + j$ with $j \ge 1$. Deleting any vertex v leaves a feasible multigraph on $2k$ vertices, so by (i) $d(v) \geq |A(D)| - (m-1)k^2 = (m-1)k + j$ for *every* v. The average degree is

$$\frac{2|A(D)|}{2k+1} = (m-1)k + j + \frac{(m-1)k - (2k-1)j}{2k+1},$$

by direct computation. If $(m-1)k - (2k-1)j < 0$ the average lies below the minimum, a contradiction. Otherwise the correction term is nonnegative and, because $m \leq 4$ and $j \geq 1$, strictly below one: $(m-1)k - (2k-1)j \leq (m-3)k + 1 \leq k + 1 < 2k + 1$. The average therefore lies in $[(m-1)k + j,\ (m-1)k + j + 1)$ while every degree is at least $(m-1)k + j$, so some vertex v has $d(v) = (m-1)k + j$ exactly, and $D - v$ has exactly $(m-1)k^2$ arcs. By (ii), $D - v = (m-1) \cdot B_{k,k}$, and Lemma A.34 caps $d(v) \leq (m-1)k < (m-1)k + j$, the final contradiction. Hence $j \leq 0$. □

The attachment lemma bounds the degree of a vertex glued onto the *bipartite* extremiser. The linear branch has an exact analogue, worth stating once for the whole family it governs. Call a directed multigraph *everywhere-saturated* at level m when every ordered pair of distinct vertices has local connectivity exactly $m-1$. The doubled bidirected spanning trees at multiplicity $m-1$, the extremisers of the linear branch, are exactly such graphs: the $m-1$ routes between two vertices run along the doubled tree path, and cutting one doubled edge of that path is a cut of $m-1$ arcs.

Lemma A.37 (Saturated attachment)**.** *Let $m \geq 2$ and let D_0 be a directed multigraph on at least two vertices with $\lambda_{D_0}(x,y) = m-1$ for every ordered pair of distinct vertices. Let D be obtained from D_0 by adding one new vertex v together with any multiset of arcs incident to v. If $\lambda_D^{\max} \leq m-1$, then $d(v) \leq 2(m-1)$. If $d(v) = 2(m-1)$, then every arc at v joins v to one single vertex u, with $\mu(v,u) = \mu(u,v) = m-1$, and D is again everywhere-saturated.*

Proof. *No mixed pair.* Suppose v has an in-arc from u and an out-arc to w with $u \neq w$. By Menger and saturation D_0 carries $m-1$ pairwise arc-disjoint u-to-w routes, and the route $u \to v \to w$ uses only arcs incident to v, so it is arc-disjoint from all of them and $\lambda_D(u,w) \geq m$, a contradiction. Hence v is a pure source, a pure sink, or every arc at v touches one single vertex u.

A pure source has $d(v) \leq m-1$. Let $t = \sum_x \mu(v,x)$ and fix u with $\mu(v,u) \geq 1$. Any cut $(S, \bar{S})$ with $v \in S$ and $u \in \bar{S}$ has capacity $\sum_{x \in \bar{S}} \mu(v,x) + e_{D_0}(S \setminus \{v\}, \bar{S})$. If S contains a vertex x of D_0, the second term is at least $\lambda_{D_0}(x,u) = m-1$ and the first is at least $\mu(v,u)$. If $S = \{v\}$, the capacity is t. Every cut therefore has capacity at least $\mu(v,u) + \min\bigl(t - \mu(v,u),\ m-1\bigr)$, and by max-flow min-cut so does $\lambda_D(v,u)$, which feasibility caps at $m-1$. Were $t - \mu(v,u) \geq m-1$, this would force $\mu(v,u) = 0$, a contradiction, so $t - \mu(v,u) < m-1$ and the bound reads $t \leq m-1$.

A pure sink has $d(v) \leq m-1$. Reverse every arc: the reverse of an everywhere-saturated multigraph is everywhere-saturated, a pure sink becomes a pure source, and the previous step applies.

A single partner. If every arc at v touches one vertex u, then parallel arcs are already that many arc-disjoint one-step routes, so $\mu(v,u) \leq m-1$ and $\mu(u,v) \leq m-1$, giving $d(v) \leq$

$2(m-1)$.

Equality. Since $2(m-1) > m-1$, a degree of $2(m-1)$ rules out the pure cases, so v has a single partner at full multiplicity both ways. The result is feasible and again everywhere-saturated: a route through v must enter and leave through u, so it adds nothing to any flow between old vertices, and $\lambda_D(x,v) = \min(\lambda_{D_0}(x,u), \mu(u,v)) = m-1$ for every old x, symmetrically for $\lambda_D(v,x)$. □

Remark A.38 (The linear branch grows one leaf at a time). The equality case of Lemma A.37 attaches a new leaf to an arbitrary vertex at full multiplicity, and that is the only attachment reaching $2(m-1)$. Iterating from a single doubled edge, itself everywhere-saturated on two vertices, generates exactly the doubled bidirected spanning trees, so the extremal family of the linear branch is closed under maximal attachment and grows one leaf at a time. The program confirms the lemma exhaustively at $m=3$: over every doubled tree on five and six vertices and every one of the 3^{2n} possible attachment patterns, the densest feasible attachment has degree exactly $4 = 2(m-1)$, and the degree-4 patterns are precisely the single-partner attachments at full multiplicity, one per choice of partner.

Remark A.39 (The $m=3$ chain, sharpened). Theorem A.36 removes Conjecture A.29 from the critical path: the odd case no longer needs a minimum-degree statement, only the extremal *characterisation* one level down, and for $m=3$ that characterisation propagates along the same induction in both parities.

Odd levels ($n = 2k+1 \geq 9$): an extremiser has $|A| = 2k(k+1)$ arcs and every degree at least $2k$ (deletion bound). All degrees $\geq 2k+1$ is impossible since $(2k+1)^2 > 4k(k+1) = \sum_v d(v)$, so some $d(v) = 2k$ exactly, $D - v$ is the even extremiser $2 \cdot B_{k,k}$, and Corollary A.35 (the $p = q$ case) forces v to rejoin as a full-multiplicity pure source or sink: $D = 2 \cdot B_{k+1,k}$ or $2 \cdot B_{k,k+1}$.

Even levels ($n = 2k \geq 10$): equality in the averaging step forces $2k$-regularity, every vertex at degree exactly $2k$, hence $D - v$ extremal at $2k-1$ for *every* v. Odd uniqueness one level down leaves $2 \cdot B_{k,k-1}$ or $2 \cdot B_{k-1,k}$, and Corollary A.35 with $d(v) = 2k = 2\max(k-1,k)$ (the $q > p$ case for $2 \cdot B_{k-1,k}$, its $p > q$ mirror for $2 \cdot B_{k,k-1}$) rebuilds exactly $2 \cdot B_{k,k}$ either way.

The seam at $n = 8$: the same regularity argument runs from $n = 7$, where the branches tie at 24 arcs and several extremisers coexist. On the linear branch the extremisers are richer than the double star alone. Every *bidirected spanning tree at full multiplicity* $m-1$ is feasible with $2(n-1)(m-1)$ arcs: deleting a tree edge splits the vertices in two, the only arcs from one side to the other are that edge's own $m-1$ copies, and the $m-1$ routes along the tree path meet that cut, so every local connectivity is exactly $m-1$ and the doubled trees are everywhere-saturated in the sense of Lemma A.37. Exhaustive enumeration confirms that at $m=3$ these doubled trees are the *only* extremisers for $n = 4, 5, 6$ alike, with 2, 3, and 6 classes matching the tree shapes exactly: the in-house enumerator (`enumerate_extremal_directed_multigraphs`) settles $n \leq 5$, and the sound generation enumerator has now completed the uncapped $n = 6$ classification as well. A deleted vertex of the 8-regular D leaves a 24-arc extremiser with maximum degree at most 8: among doubled trees only the doubled path qualifies (tree degree ≤ 2), the double star (hub degree 24) being excluded. The doubled path P_7 is everywhere-saturated, so Lemma A.37 caps any feasible attachment to it at $d(v) \leq 2(m-1) = 4$, while 8-regularity demands $d(v) = 8$. Figure A.5 draws what a degree-8 attachment would have to be, weight 4 on each path end, closing the path into the doubled bidirected cycle C_8 with $\lambda = 4 > 2$: the collapse the lemma rules out wholesale. So every deletion is bipartite-type, the attachment equality applies, and $D = 2 \cdot B_{4,4}$ again.

The whole $m = 3$ problem therefore rests on two finite integral statements about multigraphs with multiplicities in $\{0, 1, 2\}$:

(a) $L_3^{\mathrm{dir}}(7) = 24$, which $M^*(7) = 12$ would imply but which is not implied by it in reverse, so it is posed to the integral program directly, and

(b) every 24-arc feasible multigraph on 7 vertices with maximum degree at most 8 is $2 \cdot B_{3,4}$, $2 \cdot B_{4,3}$, or the doubled bidirected path P_7.

Both targets sit at sizes the certifier and the enumerator can realistically reach. Fact (a) is the single feasibility query `prove_integral_arc_bound(7, 3, 25)`: the cut-counting model of Chapter 2 with the weights read as integer multiplicities and the maximisation replaced by the fixed arc total 25, so that an INFEASIBLE verdict is a proof that no 25-arc feasible multigraph on seven vertices exists. The model is run with two extra families of rows (`use_deletion_cuts`), and each is a valid inequality, satisfied by every feasible weighting, so adding them speeds the solver without ever excluding a witness. The deletion bounds say that restricting a feasible weighting to any five or six of the seven vertices leaves a feasible weighting, whose total is therefore capped by the certified $M^*(5)$ or $M^*(6)$. The degree-pair inequality $d^+(s) + d^-(t) - w(s,t) \leq n - 1$ holds for every ordered pair because the direct arc and the two-step detours are arc-disjoint routes, so $\mathrm{maxflow}(s,t) \geq w(s,t) + \sum_x \min\big(w(s,x), w(x,t)\big) \geq d^+(s) + d^-(t) - w(s,t) - (n-2)$, the last step using $\min(a,b) \geq a + b - 1$ for weights in $[0,1]$, and feasibility caps the left side at one. Fact (b) is the run `enumerate_extremal_directed_multigraphs_via_generation(7, 3, 24, max_degree=8)` on the generation enumerator of Chapter 2, which prunes with proved bounds only and is therefore a sound complete search at $n = 7$, where the in-house depth-first twin is not. That run has now completed, in about seven hours across ten processor cores, and it returns exactly three isomorphism classes: $2 \cdot B_{3,4}$, $2 \cdot B_{4,3}$, and the doubled bidirected path P_7, each re-verified by the exact checker. *Fact (b) is therefore settled by machine*, and the whole $m = 3$ directed multigraph problem now rests on fact (a) alone. The $n = 6$ companion run (target 20, maximum degree 8) completed the same way and returns exactly one class, the doubled bidirected path P_6, the pattern the seam argument uses one level up. The uncapped $n = 6$ classification also pays directly into fact (a): a hypothetical 25-arc feasible multigraph on 7 vertices has every degree at least 5 by the deletion bound against $L_3^{\mathrm{dir}}(6) = 20$, and a vertex of degree exactly 5 would delete to a 20-arc extremiser on six vertices, a doubled tree, to which Lemma A.37 forbids any attachment of more than 4 arcs. Any witness against fact (a) therefore has minimum total degree at least 6.

For $m \geq 4$ one hole remains: at an odd extremal level the counting no longer forces a vertex of degree $(m-1)k$ (all degrees $\geq (m-1)k+1$ is arithmetically consistent once $k(m-3) \geq 1$), so the uniqueness statement does not yet propagate through odd levels, although the *value* at odd levels follows from even uniqueness whenever that is available. For $m \geq 5$ the same escape climbs one level: a hypothetical extremiser carrying $j \geq 1$ surplus arcs could have *every* degree strictly above the deletion bound (all degrees $\geq (m-1)k+j+1$ is consistent exactly when $(2k-1)j \leq (m-1)k-2k-1$, which admits $j=1$ once $m \geq 5$), so no vertex deletion is forced to land on an extremal multigraph and even the *value* step stalls. This is why Theorem A.36 is stated for $m \in \{3,4\}$.

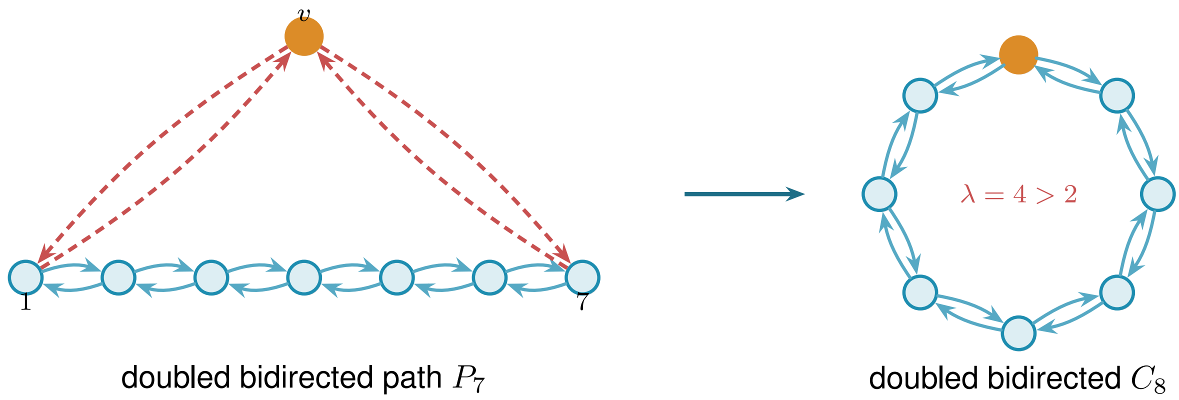


Figure A.5. The seam argument of Remark A.39. A degree count forces a deleted vertex of the 8-regular extremiser D to leave the doubled bidirected path P_7 (left, each path edge a pair of opposing arcs). To reach degree 8 the new vertex v must attach with full multiplicity to the two *ends* 1 and 7 (red), bidirected. That closes the path into the doubled bidirected cycle C_8 (right), which is *infeasible*: the two directions around the cycle give two arc-disjoint routes each way, so $\lambda = 4 > 2$ between any pair. The contradiction rules out the path branch, forcing every deletion to be bipartite-type and hence $D = 2 \cdot B_{4,4}$.

A.6 The hypergraph bounds

The body's hypergraph claims rest on two theorems: a Menger theorem for Berge routes, which also certifies the helper-point checker of Chapter 2, and a hypergraph Gomory–Hu theorem from the recent literature.

Theorem A.40 (Menger for hypergraphs)**.** *For a hypergraph $\mathcal{H}$ and vertices u, v, the maximum number of hyperedge-disjoint Berge u–v paths equals the minimum size of a set C of hyperedges whose removal separates u from v.*

Proof. Build the helper-point network of Figure 2.5: for each hyperedge e a helper arc $e^- \to e^+$ of capacity one, and arcs $x \to e^-$ and $e^+ \to x$ of unbounded capacity for each member $x \in e$. We match the two sides by translating in each direction.

Berge paths become flow. A Berge path $u, e_1, v_1, \ldots, e_k, v$ becomes the directed walk $u \to e_1^- \to e_1^+ \to v_1 \to \cdots \to v$, crossing one helper arc per hyperedge it uses. The e_i along a single

Berge path are distinct, and two hyperedge-disjoint Berge paths share no hyperedge at all, so they use disjoint sets of helper arcs and translate to *arc-disjoint* flow paths, paths that share no arc, each carrying one unit of flow.

Flow becomes Berge paths. Conversely, because every capacity is an integer, a maximum flow may be taken integral, and an integral flow of value k splits into k arc-disjoint unit paths. Reading a helper crossing $\cdots x \to e^- \to e^+ \to y \cdots$ back as the single hyperedge step x, e, y, that is, *suppressing* the two helper nodes, turns each unit path into a Berge path, and no two of them can share a hyperedge, since that would send two units across some capacity-one helper arc.

Cuts. Finally, the member arcs are uncapped, so any cut of finite capacity uses helper arcs only, and a set of helper arcs separates u from v in the network exactly when the corresponding hyperedges separate them in $\mathcal{H}$. The max-flow min-cut theorem equates the largest number of arc-disjoint flow paths with the smallest helper-arc cut, which is precisely the claim. □

Theorem A.41 (Hypergraph Gomory–Hu, Bérczi et al. [9])**.** *Every r-uniform hypergraph $\mathcal{H}$ on vertex set V has a weighted tree T^* on V, its* hypergraph Gomory–Hu tree*, such that $\lambda_{\mathcal{H}}(u, v)$ equals the minimum weight on the u–v tree path, and each tree edge t induces a vertex partition $(S_t, V \setminus S_t)$ with $|\delta_{\mathcal{H}}(S_t)| = c^*(t)$, where $\delta_{\mathcal{H}}(S)$ is the set of hyperedges incident to both sides, that is, having at least one vertex in S and at least one outside it.*

This is the exact hypergraph echo of Theorem A.2, the same tree-of-cuts summary of all pairwise connectivities, with the single change that the capacity of a cut now counts the hyperedges straddling it, $|\delta_{\mathcal{H}}(S)|$, in place of the edges. Bérczi, Chandrasekaran, Király and Kulkarni [9] obtain the tree by the same uncrossing strategy sketched for the graph case above, applied now to the submodular hyperedge-cut function $|\delta_{\mathcal{H}}(\cdot)|$. We use only the two properties in the statement, the path-minimum reading of $\lambda_{\mathcal{H}}$ and the identity $|\delta_{\mathcal{H}}(S_t)| = c^*(t)$.

Proof of Proposition 1.15. Upper bound. Let $\mathcal{H}$ be r-uniform with $\lambda_{\mathcal{H}}^{\max} \le m - 1$, and let T^* be its hypergraph Gomory–Hu tree (Theorem A.41), a weighted tree on the same n vertices with weights c^*. The plan is to charge each hyperedge to tree edges and count the charges two ways, exactly as in the Mader proof, with the one change that a hyperedge spans r vertices rather than two.

Step 1: each hyperedge crosses at least $r - 1$ tree cuts. Fix a hyperedge e, a set of r vertices, and let $\mathrm{ST}(e)$ be its *smallest connected subtree*, the minimal subtree of T^* that contains all r of them (its Steiner tree). A tree spanning at least r vertices has at least $r - 1$ edges, so $\mathrm{ST}(e)$ has at least $r - 1$ of them. Each such edge t is a genuine cut for e: deleting t from T^* splits the vertices into the two sides $(S_t, V \setminus S_t)$, and because t lies on $\mathrm{ST}(e)$ the hyperedge e has at least one vertex on each side. So e touches both sides, that is, $e \in \delta_{\mathcal{H}}(S_t)$. Therefore

$$\left|\{\, t \in E(T^*) : e \in \delta_{\mathcal{H}}(S_t) \,\}\right| \;\ge\; r - 1 \qquad \text{for every hyperedge } e.$$

Step 2: count the incidences two ways. Sum that inequality over all hyperedges, then exchange the order of summation. This is legitimate because both of the middle sums below simply count the same pairs (e,t) with $e \in \delta_{\mathcal{H}}(S_t)$, once grouped by the hyperedge and once by the tree edge:

$$(r-1)\,|E(\mathcal{H})| \;\le\; \sum_e \bigl|\{\,t : e \in \delta_{\mathcal{H}}(S_t)\,\}\bigr| \;=\; \sum_t \bigl|\{\,e : e \in \delta_{\mathcal{H}}(S_t)\,\}\bigr| \;=\; \sum_t |\delta_{\mathcal{H}}(S_t)|.$$

Step 3: read off the bound. By the Gomory–Hu property each tree edge has $|\delta_{\mathcal{H}}(S_t)| = c^*(t)$, and each weight $c^*(t)$ is a local connectivity $\lambda_{\mathcal{H}}$ of its endpoints, hence at most $m-1$ by feasibility. Summing over the $n-1$ tree edges, $\sum_t |\delta_{\mathcal{H}}(S_t)| = \sum_t c^*(t) \le (m-1)(n-1)$. Chaining Steps 2 and 3,

$$(r-1)\,|E(\mathcal{H})| \;\le\; (m-1)(n-1),$$

and dividing by $r-1$, then rounding down because $|E(\mathcal{H})|$ is an integer, gives $|E(\mathcal{H})| \le \left\lfloor \frac{(m-1)(n-1)}{r-1} \right\rfloor$. *Lower bound.* When $(r-1) \mid (n-1)$, take a hub h and split the other $n-1$ vertices into groups $G_1, G_2, \ldots$ of size $r-1$ each. Form the star hypertree whose hyperedges are $\{h\} \cup G_i$, one per group, and give each hyperedge multiplicity $m-1$. Every non-hub vertex lies in exactly $m-1$ hyperedges, which form a cut isolating it, so by Theorem A.40 every pair has Berge connectivity at most $m-1$, while the count is $\frac{(m-1)(n-1)}{r-1}$. For general n under the hypothesis $m-1 \le \binom{n-2}{r-2}$, Theorem A.42 below attains the floor exactly, without even needing repeated hyperedges. $\square$

The multiplicities in the lower bound can in fact be avoided: for $r \ge 3$, *simple* r-uniform hypergraphs already attain the same bound, in sharp contrast to the graph case $r = 2$, where simplicity costs the factor that separates Mader's $\lfloor m(n-1)/2 \rfloor$ from the multigraph value $(m-1)(n-1)$.

Theorem A.42 (Simplicity is free for $r \ge 3$)**.** *Let $r \ge 3$, $n \ge r$, and $2 \le m$ with $m - 1 \le \binom{n-2}{r-2}$. Then the maximum number of hyperedges in a* simple *r-uniform hypergraph on n vertices with Berge edge-connectivity at most $m-1$ is exactly $\left\lfloor \frac{(m-1)(n-1)}{r-1} \right\rfloor$.*

Proof. The upper bound is Proposition 1.15, since a simple hypergraph is a multihypergraph. For the lower bound, fix a hub h and let $V' = V \setminus \{h\}$. By Lemma A.44 applied with rank $r-1$ and degree ceiling $d = m-1$ on the $n-1$ vertices of V', there is an $(r-1)$-uniform simple hypergraph $\mathcal{G}^*$ on V' with maximum degree at most $m-1$ and exactly $\left\lfloor \frac{(m-1)(n-1)}{r-1} \right\rfloor$ hyperedges. Adding h to each hyperedge produces distinct r-sets, so the result $\mathcal{H}$ is simple, and every non-hub vertex w lies in $\deg_{\mathcal{G}^*}(w) \le m-1$ hyperedges, which form a cut isolating w. By Theorem A.40, $\lambda_{\mathcal{H}}^{\max} \le m-1$. $\square$

The sparse hypergraph the construction needs is a corollary of a classical partition theorem, which we state before using it.

Theorem A.43 (Baranyai [16])**.** *If r divides n, the $\binom{n}{r}$ distinct r-subsets of an n-element set can be partitioned into $\binom{n-1}{r-1}$ classes, each a* perfect matching*, meaning a family of n/r disjoint r-sets that together cover every vertex exactly once. More generally, for any prescribed class sizes $a_1+\cdots+a_t = \binom{n}{r}$, and with no divisibility assumed, the r-subsets can be partitioned into classes of exactly those sizes so that in the class of size a_i every vertex lies in either $\lfloor ra_i/n \rfloor$ or $\lceil ra_i/n \rceil$ of the sets, as evenly balanced as the size permits.*

The first statement is the memorable one. When r divides n the complete r-uniform hypergraph, the one holding every r-set, comes apart into $\binom{n-1}{r-1}$ perfect matchings, the exact hypergraph analogue of splitting a regular bipartite graph into perfect matchings by König's theorem. The proof is an induction that keeps the partial partition as balanced as possible at every step, an integer-flow argument in the spirit of Hall's marriage theorem rather than an explicit construction, and its full form is in Baranyai [16]. We need only the general equitable form, and only its conclusion about degrees.

Lemma A.44 (Sparse uniform hypergraphs exist)**.** *Let $r \geq 2$, $n \geq r$, and $0 \leq d \leq \binom{n-1}{r-1}$. There exists an r-uniform simple hypergraph on n vertices with maximum degree at most d and exactly $\left\lfloor \frac{dn}{r} \right\rfloor$ hyperedges.*

Proof. Put $e := \left\lfloor \frac{dn}{r} \right\rfloor \leq \frac{n}{r}\binom{n-1}{r-1} = \binom{n}{r}$. Apply the equitable form of Theorem A.43 with two classes, of sizes e and $\binom{n}{r} - e$. Within each class every vertex is covered either $\lfloor re/n \rfloor$ or $\lceil re/n \rceil$ times, so the class of size e is a simple r-uniform hypergraph with $e = \left\lfloor \frac{dn}{r} \right\rfloor$ hyperedges and maximum degree at most $\lceil re/n \rceil \leq \lceil d \rceil = d$, the last step because $re/n \leq r(dn/r)/n = d$ and d is an integer. □

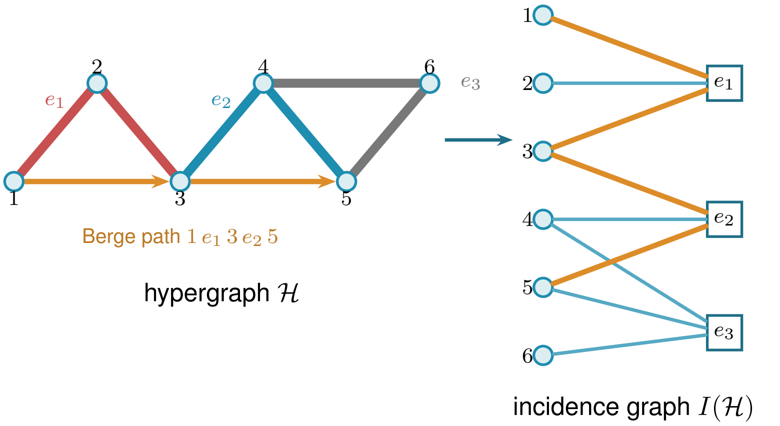
A.7 The hypergraph vertex problem

The whole vertex problem translates into a graph problem through one observation, the one the rest of this section rests on and which Figure A.6 draws in full. Write $I(\mathcal{H})$ for the bipartite *incidence graph* of a hypergraph $\mathcal{H}$: one node per vertex, one node per hyperedge, an edge when the vertex lies in the hyperedge. A Berge path with distinct hyperedges is exactly a path of $I(\mathcal{H})$ between two vertex-class nodes. The interior of such a path consists of both kinds of node, the hyperedges it rides and the vertices where it changes hyperedge, so two paths of $I(\mathcal{H})$ are internally disjoint in the ordinary graph sense exactly when the two Berge routes share neither a hyperedge nor an intermediate vertex. That is precisely the separation fixed in Chapter 1, and it is the reason both halves were required there. Hence

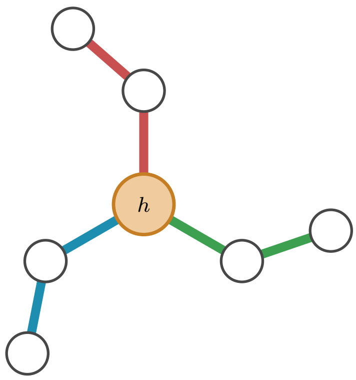
$$\kappa_{\mathcal{H}}(u, v) \;=\; \kappa_{I(\mathcal{H})}(u, v),$$

the maximum number of internally disjoint u–v paths in $I(\mathcal{H})$, and the hypergraph vertex problem asks for the maximum number of degree-r nodes on one side of a bipartite graph whose other side holds n nodes, subject to no two of those n nodes being joined by m internally disjoint paths.

Figure A.6a traces a single Berge path through both pictures, and Figure A.6b shows the star hypertree of Theorem A.45, whose incidence graph is a tree.



(a) A 3-uniform hypergraph $\mathcal{H}$ (left) and its bipartite incidence graph $I(\mathcal{H})$ (right): circles for vertices, hollow squares for hyperedges, an edge when the vertex lies in the hyperedge. The Berge path $1\,e_1\,3\,e_2\,5$ is traced in orange in both pictures. In $\mathcal{H}$ it rides the red line e_1 and the blue line e_2, changing at the shared vertex 3, while the grey line e_3 lies off it. In $I(\mathcal{H})$ it is the ordinary path $1-e_1-3-e_2-5$ alternating sides.



(b) A star hypertree: every hyperedge $\{h\} \cup G_i$ shares only the hub h (orange), and the other $r-1=2$ vertices of each block sit in a single hyperedge. Its incidence graph is a tree, so $\kappa^{\max} \leq 1$.

Figure A.6. The translation the vertex problem rests on. Internally vertex-disjoint Berge paths in $\mathcal{H}$ correspond exactly to internally disjoint paths in $I(\mathcal{H})$, so $\kappa_{\mathcal{H}}(u,v) = \kappa_{I(\mathcal{H})}(u,v)$. **(top)** A hypergraph and its incidence graph with one Berge path drawn in both. **(bottom)** The star hypertree of Theorem A.45, whose incidence graph is a tree.

Theorem A.45 (Hypergraph vertex value at $m=2$)**.** *For $r \geq 2$ and $n \geq 1$, the maximum number of hyperedges of an r-uniform multihypergraph on n vertices with $\kappa^{\max} \leq 1$ is*

$$k_2^{(r)}(n) = \left\lfloor \frac{n-1}{r-1} \right\rfloor,$$

attained by the star hypertree. In particular $k_2^{(r)}(n) = \ell_2^{(r)}(n)$: the vertex and edge problems agree at $m=2$ for every uniformity, and repeated hyperedges never help.

Proof. We claim $\kappa_{\mathcal{H}}^{\max} \leq 1$ holds if and only if $I(\mathcal{H})$ is a forest. If $I(\mathcal{H})$ contains a cycle $v_1, e_1, v_2, e_2, \ldots, v_\ell, e_\ell, v_1$ (it alternates classes, so $\ell \geq 2$, all v_i and e_i distinct), then v_1, e_1, v_2 and $v_1, e_\ell, v_\ell, e_{\ell-1}, \ldots, e_2, v_2$ are two Berge v_1–v_2 paths with disjoint hyperedge sets and disjoint interior vertex sets, so $\kappa(v_1, v_2) \geq 2$. (At $\ell = 2$ this is the repeated pair: two hyperedges sharing two vertices.) Conversely, two such paths between u and v concatenate to a cycle of $I(\mathcal{H})$. This proves the claim. A forest on $n+q$ nodes has at most $n+q-1$ edges, while $I(\mathcal{H})$ has exactly rq edges, so $rq \leq n+q-1$, i.e. $q \leq (n-1)/(r-1)$, and q is an integer. The star hypertree has $\lfloor (n-1)/(r-1) \rfloor$ hyperedges and a tree as incidence graph (each block vertex lies in one hyperedge, every cycle would need two hyperedges sharing two vertices), so the floor is attained. ☐

Proposition A.46 (General lower bound)**.** *For $r \geq 3$, $m \geq 2$ and $m-1 \leq \binom{n-2}{r-2}$,*

$$k_m^{(r)}(n) \;\geq\; \left\lfloor \frac{(m-1)(n-1)}{r-1} \right\rfloor,$$

attained by a simple hypergraph.

Proof. The simple construction of Theorem A.42 has Berge edge-connectivity at most $m-1$, and Whitney's inequality $\kappa \leq \lambda$ holds pairwise (an internally vertex-disjoint family is in particular hyperedge-disjoint under our definition), so the same hypergraph is feasible for the vertex problem. ☐

The $m = 3$ case also closes completely. The engine is a rank bound for graphs in which one side of a partition must stay below local 3-connectivity. The hypergraph theorem then falls out of the incidence translation. Throughout, $\operatorname{rank}(G) = |E(G)| - |V(G)| + 1$ denotes the cycle rank of a connected multigraph, that is, the number of independent cycles it carries. A tree has cycle rank 0, and each edge added beyond a spanning tree closes exactly one new cycle and raises the rank by one, so bounding the rank is the same as bounding how many independent cycles the graph can hold.

Primer: global connectivity and the pieces of a graph

The lemma below speaks of a graph being 2-connected or 3-connected and of taking it apart at small separators. These are the global cousins of the local $\kappa(u, v)$ from Chapter 1, so we fix the words once. A graph on more than k vertices is *k-connected* if it stays connected after the removal of any $k-1$ vertices. By Menger this is the same as asking that every pair of vertices be joined by at least k internally vertex-disjoint paths, so k-connected means $\kappa(u, v) \geq k$ for all pairs at once, the uniform version of the local measure. Thus 2-connected means no single vertex disconnects the graph, and 3-connected means no two vertices do. Three named obstructions go with this. A *cut vertex* is a vertex whose removal disconnects the graph, so a connected graph on at least three vertices is 2-connected exactly when it has no cut vertex. A *separating pair* is a set of two vertices whose removal disconnects the graph, the obstruction a 2-connected graph must lack in order to be 3-connected. Finally a *block* is a maximal piece with no cut vertex of its own, hence either a single bridge edge or a maximal 2-connected subgraph. Two blocks overlap in at most one vertex, always a cut vertex, and the blocks and cut vertices hang together in a tree, the *block-cut tree*, with one node per block and one per cut vertex. Step 1 of the proof is precisely an induction over that tree.

Lemma A.47 (Incidence rank lemma)**.** *Let G be a connected multigraph with vertex set $X \cup Z$ such that (i) Z is independent, (ii) no edge incident to Z is parallel to another edge, (iii) every $z \in Z$ has degree at least 2, and (iv) $\kappa_G(x, x') \le 2$ for all distinct $x, x' \in X$. Then $\operatorname{rank}(G) \le |X| - 1$.*

Proof. Induction on $|V| + |E|$. The proof peels G down to a core that cannot exist. Step 1 handles a cut vertex by arguing block by block. Steps 2 and 3 remove a degree-two Z-vertex or X-vertex without changing the bound. What survives is 2-connected with minimum degree at least three, and Step 4 shows that such a graph already violates the connectivity ceiling (iv), so it never arises. *Base* $|V| \le 2$. A single vertex lies in X by (iii), and $0 \le |X| - 1$. On two vertices, (i)–(iii) force both into X, parallel edges are internally disjoint paths, so (iv) caps the multiplicity at 2 and $\operatorname{rank} \le 1 = |X| - 1$.

Step 1: a cut vertex. Suppose G has a cut vertex, so it is not yet 2-connected. Break it into its *blocks* $B_1, \dots, B_c$ with $c \ge 2$, a block being a maximal piece with no cut vertex of its own, that is, either a single bridge edge or a maximal 2-connected subgraph. Two blocks meet in at most one vertex, and any shared vertex is a cut vertex of G. No cycle can pass through two different blocks, so the cycle rank adds up over blocks, $\operatorname{rank}(G) = \sum_i \operatorname{rank}(B_i)$, and it is enough to bound each $\operatorname{rank}(B_i)$ by $|X \cap B_i| - 1$ and sum.

Each block inherits the hypotheses. A bridge block is one edge, so its rank is 0, and by (i) at least one endpoint lies in X, giving $0 \le |X \cap B| - 1$. Every other block is 2-connected, hence has minimum degree at least 2, so (i) to (iv) hold inside it and the induction hypothesis gives $\operatorname{rank}(B_i) \le |X \cap B_i| - 1$.

Adding these requires care, because a cut vertex is shared by several blocks and would be counted several times. Write $b(v)$ for the number of blocks containing v, so $b(v) = 1$ except at cut vertices. Counting each X-vertex once for every block it lies in,

$$\sum_i |X \cap B_i| = |X| + \sum_{x \in X \text{ cut}} (b(x) - 1).$$

The number of blocks satisfies the block–cut tree identity $c = 1 + \sum_{v \text{ cut}} (b(v) - 1)$, which holds because the blocks and cut vertices form a tree in which each cut vertex v is joined to the $b(v)$ blocks that contain it. Subtracting this count of blocks from the previous display and separating the cut vertices into their X- and Z-parts leaves

$$\operatorname{rank}(G) = \sum_i \operatorname{rank}(B_i) \le \sum_i (|X \cap B_i| - 1) = |X| - 1 - \sum_{z \in Z \text{ cut}} (b(z) - 1) \le |X| - 1,$$

where the last inequality holds because each $b(z) - 1 \ge 0$, so the Z-cut sum only pushes the bound further below $|X| - 1$.

Step 2: a Z-vertex of degree two. From now on G is 2-connected, every cut vertex having been handled. Suppose some $z_0 \in Z$ has degree exactly 2. Both its neighbours lie in X by (i), and they are distinct, since two edges to the same vertex would be parallel at a Z-vertex, which (ii) forbids. *Suppress* z_0 by deleting it and joining its two neighbours x, x' with one new edge. This loses two edges and one vertex and gains one edge, so $|E|$ and $|V|$ each drop by one and $\mathrm{rank} = |E| - |V| + 1$ is unchanged. The new edge runs X to X, so $|X|$ is unchanged and (i), (ii), (iii) still hold for the remaining Z-vertices. Replacing the path x, z_0, x' by a direct edge alters no route among the surviving vertices, so every $\kappa(x_1, x_2)$ is preserved and (iv) holds. The graph is strictly smaller, so the induction hypothesis applies and returns the bound for G.

Step 3: an X-vertex of degree two. Now G is 2-connected and, Step 2 being used up, every Z-vertex has degree at least 3. Suppose some $x_0 \in X$ has degree 2, and delete it. Deleting one vertex from a 2-connected graph leaves it connected. Stripping a degree-2 vertex removes two edges and one vertex, so rank drops by exactly one. Each neighbour of x_0 loses one edge, and a neighbour in Z only falls from degree at least 3 to degree at least 2, so (iii) survives, while deleting a vertex cannot raise a connectivity, so (iv) survives. Here $|X|$ drops by one, and $|X| \geq 2$ to begin with, for if $|X| \leq 1$ then by (i) and (ii) every z would send all its edges to one X-vertex without repeats, hence have degree at most 1, against (iii). The smaller graph satisfies the hypotheses, so induction gives $\mathrm{rank}(G) - 1 \leq (|X| - 1) - 1$, that is $\mathrm{rank}(G) \leq |X| - 1$.

Step 4: the remaining case is impossible. Now G is 2-connected with $|V| \geq 3$ and every degree at least 3. First, G is simple: a parallel class lies inside X by (ii), and if $\mu(x, x') \geq 2$ then a third internally disjoint route exists: walk from x to any third vertex w inside $G - x'$, then from w to x' inside $G - x$. The concatenation visits x only first and x' only last, so it contains an x–x' path with at least one interior vertex, giving $\kappa(x, x') \geq 3$, against (iv). If G is 3-connected, Menger puts every pair at $\kappa \geq 3$, so $|X| \leq 1$ and every z has degree at most 1 by (i), absurd.

Otherwise G is 2-connected but not 3-connected, so it can be taken apart at its 2-vertex separators.

Primer: the triconnected (SPQR) decomposition

A 2-connected graph that is not 3-connected must have a separating pair, and cutting along every such pair sorts any 2-connected graph into a few standard shapes. This is a classical theorem of Tutte [17], made into a linear-time algorithm by Hopcroft and Tarjan [18].

Theorem (triconnected decomposition). Every 2-connected graph G on at least three vertices decomposes into a *tree of pieces*, unique up to relabelling, in which each piece is one of three kinds: a *cycle* of length at least three (an S-piece, for *series*), a *bond*, meaning two vertices joined by three or more parallel edges (a P-piece, for *parallel*), or a 3-connected simple graph on at least four vertices (an R-piece, for *rigid*). Two adjacent pieces share exactly one *virtual edge* $\{a, b\}$, a placeholder that records a separating pair of G at which the two pieces were split apart. The three initials S, P, R, together with Q for a trivial single-edge piece, give the structure its usual name, the *SPQR tree*.

The tree is built by the recursion that also defines it. If G is already a cycle, a bond, or 3-connected, it is a single piece and the tree is one node. Otherwise choose a separating pair $\{a, b\}$, split G into the parts that this pair separates, add the virtual edge $\{a, b\}$ to each part so the split is remembered, and recurse on the parts. Every split makes the parts strictly smaller, so the recursion halts, and Tutte's theorem is that the resulting collection of pieces does not depend on the order in which the separating pairs are taken. The one fact the proof draws from all this is that a virtual edge $\{a, b\}$ always stands for a genuine a–b path through the rest of G, so an argument that must cross from a to b outside a given piece can always do so on the far side of that piece's virtual edge.

The decomposition just stated is sketched in Figure A.7, which also marks the leaf at which the argument bites. Two properties are all the proof needs from it, and both are visible in the figure. First, a vertex of a piece that touches none of that piece's virtual edges belongs to that piece alone, so it keeps in G exactly the degree it has in the piece. Second, for any virtual edge $\{a, b\}$ the part of G on the far side of the cut contains a genuine a–b path whose interior avoids the piece, so the virtual edge can always be replaced by a real route.

Now G itself is none of the three piece types, since it is not a single cycle (its degrees are at least three), not a bond (it is simple), and not 3-connected (the case just handled), so the tree has at least two leaves. Let L be one, with unique virtual edge $\{a, b\}$. If L is a cycle, a vertex of L off $\{a, b\}$ has degree 2 in G, absurd. If L is a bond, it holds at least two real parallel edges, absurd in a simple graph. If L is 3-connected, then any two of its vertices are joined by three internally disjoint paths inside L, at most one through the virtual edge, which the far-side path replaces: so $\kappa_G(u, v) \geq 3$ for *all* $u, v \in V(L)$, forcing $|X \cap V(L)| \leq 1$ by (iv). But $|V(L)| \geq 4$ leaves a Z-vertex $z^* \notin \{a, b\}$. Its neighbours lie in $X \cap V(L)$ by (i), at most one vertex, reached by at most one edge since G is simple, giving degree at most 1, which is absurd. □

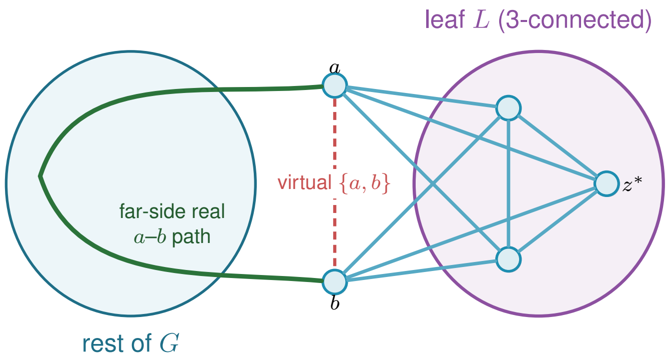


Figure A.7. The triconnected (Tutte/SPQR) decomposition in Step 4 of Lemma A.47, schematic: G splits into a tree of pieces (blobs) glued along virtual edges, and the analysis works at a *leaf* L with its single virtual edge $\{a, b\}$ (dashed red). The far side of the tree supplies a *real* a–b path (green) that internally avoids L. If L is 3-connected, any two of its vertices are joined by three internally disjoint paths inside L, at most one using the virtual edge, and the far-side path replaces it, so $\kappa_G \geq 3$ for every pair in L. With $|V(L)| \geq 4$ that forces a hyperedge-node $z^* \notin \{a, b\}$ of degree at most one, which is the contradiction that closes the lemma.

Theorem A.48 (Hypergraph vertex value at $m = 3$)**.** *For $r \geq 2$, every r-uniform multihypergraph on n vertices with $\kappa^{\max} \leq 2$ has at most $\left\lfloor \frac{2(n-1)}{r-1} \right\rfloor$ hyperedges. For $r \geq 3$ with $2 \leq \binom{n-2}{r-2}$ the bound is attained by a simple hypergraph, so*

$$k_3^{(r)}(n) = \left\lfloor \frac{2(n-1)}{r-1} \right\rfloor = \ell_3^{(r)}(n).$$

Proof. Apply Lemma A.47 to each connected component of the incidence graph $I(\mathcal{H})$, with X the vertex class and Z the hyperedge class: Z is independent, incidences are simple even when hyperedges repeat, hyperedge-nodes have degree $r \geq 2$, components without an X-node are impossible, and $\kappa_I(u, v) = \kappa_{\mathcal{H}}(u, v) \leq 2$ for vertex pairs. Sum $\operatorname{rank} \leq |X_c| - 1$ over the C components. The incidence graph $I(\mathcal{H})$ has rq edges (each of the q hyperedges contributes r incidences) and $n + q$ nodes, so $\sum_c \operatorname{rank} = rq - (n + q) + C$, while $\sum_c(|X_c| - 1) = n - C$. The inequality reads $rq - (n + q) + C \leq n - C$, hence $q(r - 1) \leq 2(n - 1) + 2(1 - C) \leq 2(n - 1)$, the last step using $C \geq 1$. The lower bound is Proposition A.46 at $m = 3$. ☐

Remark A.49 (Edges as gates at $r = 2$, and the boundary of the method). At $r = 2$ the theorem speaks of multigraphs under the hyperedge-as-gate convention, in which parallel edges are distinct one-step routes, giving the multigraph value $2(n - 1)$ of Theorem 1.3, complementing the body's convention for graphs, where parallel adjacencies count once. The proof of Lemma A.47 leans on the triconnected decomposition exactly where $\kappa \leq 2$ lives. For $m = 4$, the analogous question asks for a rank bound relative to $\kappa \leq 3$, where no comparably clean decomposition exists. That the pattern $k_m^{(r)} = \ell_m^{(r)}$ must fail eventually is not merely an analogy with the graph case ($m = 5$, Sørensen-Thomassen), which is stated for simple graphs under the body's convention. It already fails inside this convention, at $r = 2$. The identification in the first sentence of this remark runs both ways: the $r = 2$ case of the present problem *is* the multigraph vertex problem of Section A.8, since a route there is a family of edge-disjoint and internally disjoint paths, which is exactly Lemma A.54. So Theorem A.56 applies to it, and the value exceeds $\lfloor(m - 1)(n - 1)/(r - 1)\rfloor = (m - 1)(n - 1)$ for every $m \geq 5$ and every $n \geq 3$, by a margin that grows with n. The formula is therefore not a candidate for all m and r together, and the live question is only whether $r \geq 3$ postpones the failure past $m = 4$.

For $m = 4$ it does not fail anywhere within reach. An exhaustive search over r-uniform multihypergraphs confirms both halves of the value, that $\lfloor 3(n - 1)/(r - 1)\rfloor$ is attained and that one more hyperedge is infeasible, at $(n, r) = (4, 3)$, $(5, 3)$, $(5, 4)$, $(6, 4)$, $(6, 5)$, $(7, 5)$, $(7, 6)$, $(8, 6)$, $(8, 7)$, $(9, 7)$ and $(9, 8)$. That is a wider net than the single cell $k_4^{(3)}(5) = 6$, and it is evidence rather than proof: the missing ingredient remains the 4-connectivity rank bound, and the $r = 2$ failure at $m = 5$ shows the surrounding claim has a genuine boundary rather than extending indefinitely.

Proposition A.50 (Directed hypergraphs: first bounds)**.** *Model a directed r-uniform hyperedge as a tail vertex together with $r - 1$ head vertices, a Berge route entering at the tail and leaving at a head (the program's convention, drawn in Figure A.8). If every pair of vertices has at most*

$m-1$ *arc-disjoint directed Berge routes, then*

$$|E(\mathcal{H})| \;\leq\; \frac{(m-1)\,n(n-1)}{r-1},$$

and for $m-1 \leq \binom{n-\lceil n/2\rceil-1}{r-2}$ *there are feasible constructions with* $\max_{1\leq\alpha\leq n-1}\,\alpha\left\lfloor\frac{(m-1)(n-\alpha)}{r-1}\right\rfloor \approx (m-1)n^2/(4(r-1))$ *hyperedges, so the value is quadratic in* n.

Proof. Upper bound: a hyperedge with tail u serves the $r-1$ ordered pairs (u,h), h a head. If some ordered pair (u,v) were served by m distinct hyperedges, those would be m arc-disjoint one-step routes, so each ordered pair is served at most $m-1$ times and $(r-1)|E| \leq (m-1)n(n-1)$. Lower bound: split V into tails A, $|A|=\alpha$, and heads B. By Lemma A.44 there is a simple $(r-1)$-uniform hypergraph $\mathcal{G}^*$ on B with maximum degree $m-1$ and $\lfloor(m-1)|B|/(r-1)\rfloor$ edges. Give every tail $a\in A$ the hyperedges (a,H), $H\in\mathcal{G}^*$. No vertex of B is a tail, so every route is a single step and $\lambda(a,b)=\deg_{\mathcal{G}^*}(b)\leq m-1$, while pairs inside A or starting in B have no routes. □

Figure A.8. How the thesis draws a single hyperedge on its r vertices, in the metro colours of Figure 1.3. *Left:* an undirected hyperedge is one metro line threading all its members, with no arrows. *Right:* a directed hyperedge, a tail with $r-1$ heads, is a star, the tail sending one thick arc to each head, so a Berge route enters at the tail and leaves at any head. The hypergraph panels of Figure 1.4 use this same star. The shape is a drawing convention, not a path. Each coloured line or star is a *single* edge on several vertices, not a sequence of ordinary edges, so the line on the left is one hyperedge and not three. One colour means one hyperedge, and where two of them meet they cross at a shared station rather than joining into a longer line. What might be mistaken for a hyperedge is a *route*, which does run edge by edge from one vertex to another, but a route is drawn as a thin thread laid over the lines, so the thick coloured hyperedge and the thin route never look alike.

The forward model is one of three ways to orient a Berge edge. In general a directed r-uniform hyperedge is a split of its r vertices into a non-empty *tail set* T and a non-empty *head set* H, a route entering at a tail and leaving at a head. The *forward* model is $|T|=1$, the *backward* model is $|H|=1$, and the *general* model allows any split (genuinely mixed splits, both parts at least two, exist from $r\geq 4$). The next two results place all three. The first is that backward needs no separate study, and the second is that the general model, despite admitting more hyperedges on a single vertex set, obeys the very same first-order bound as forward.

Lemma A.51 (Backward is the reverse of forward)**.** *Reversing every hyperarc,* $(T,H)\mapsto(H,T)$*, is an edge-count-preserving bijection between the forward directed* r*-uniform hypergraphs and the backward ones on the same vertices, and the reverse* $\mathcal{H}^R$ *satisfies* $\lambda_{\mathcal{H}}(u,v) = \lambda_{\mathcal{H}^R}(v,u)$

and $\kappa_{\mathcal{H}}(u,v) = \kappa_{\mathcal{H}^R}(v,u)$. Hence $\lambda^{\max}$ and $\kappa^{\max}$ are invariant under reversal, and the backward edge and vertex extremal numbers equal the forward ones, for every r, m and n.

Proof. Reversal sends a backward hyperarc ($|T| = r-1$, $|H| = 1$) to a forward one ($|T| = 1$) and is its own inverse, so it is an edge-count-preserving bijection of the two families. Let $u = x_0, e_1, x_1, \ldots, e_k, x_k = v$ be a directed Berge route in $\mathcal{H}$, where x_{i-1} is a tail and x_i a head of e_i. The reversed edge $e_i^R = (H_i, T_i)$ has x_i as a tail and x_{i-1} as a head, so $v = x_k, e_k^R, x_{k-1}, \ldots, e_1^R, x_0 = u$ is a directed Berge route from v to u in $\mathcal{H}^R$ on the same edges and the same internal vertices. This is a bijection between u–v routes in $\mathcal{H}$ and v–u routes in $\mathcal{H}^R$ that preserves which edges and which internal vertices any two routes share, so it carries an edge-disjoint (respectively internally vertex-disjoint) family to one of equal size. Therefore $\lambda_{\mathcal{H}}(u,v) = \lambda_{\mathcal{H}^R}(v,u)$ and $\kappa_{\mathcal{H}}(u,v) = \kappa_{\mathcal{H}^R}(v,u)$, and maximising over ordered pairs gives $\lambda^{\max}(\mathcal{H}) = \lambda^{\max}(\mathcal{H}^R)$ and the same for κ. Since reversal is a bijection preserving feasibility and edge count, the two models share every extremal number. ☐

Proposition A.52 (The general model obeys the forward bound)**.** *Let a directed r-uniform hyperedge be any split into a non-empty tail set T and head set H. If every ordered pair has at most $m-1$ arc-disjoint, or at most $m-1$ internally vertex-disjoint, directed Berge routes, then*

$$|E(\mathcal{H})| \;\le\; \frac{(m-1)\,n(n-1)}{r-1},$$

the same bound as the forward model of Proposition A.50. The forward construction of that proposition is itself a set of general hyperedges, so the general value obeys the same upper bound and inherits the same quadratic lower bound, and in particular is $\Theta(n^2)$.

The two models are therefore trapped between the same pair of bounds, which differ by a factor of 4. That is enough to fix the *order* of both values and not enough to fix either leading constant, so it does not follow that the forward and general models agree asymptotically, only that any disagreement between them lives inside that factor. Determining $\lim_{n\to\infty} \mathrm{ex}(n)/n^2$ for even one of the two, and then asking whether the two limits coincide, is the open question Chapter 4 records.

It is worth saying which end of that gap is the likely culprit, because the two bounds are not equally crude and the analogy with the simple digraph case is informative. The upper bound charges each hyperedge to the ordered tail-head pairs it occupies and then allows all $n(n-1)$ ordered pairs to be used to capacity. The construction uses only the $\alpha(n-\alpha) \le n^2/4$ pairs that run from the tail class to the head class, and leaves the pairs inside each class, and those running backwards, entirely idle. The factor of four is exactly that difference, n^2 against $n^2/4$, and it is the same difference that appears in the simple directed problem, where Construction 1.13 also uses only a one-directional bipartite pattern. There, Theorem A.24 now proves that the bipartite pattern is right to first order, that no arrangement recovers the idle pairs. If that phenomenon is a feature of directedness rather than of graphs specifically, then it is the upper bound here that should come down by a factor of four, not the construction that should climb, which suggests the following.

Conjecture A.53 (The directed hypergraph constant)**.** *For fixed $r \geq 3$ and $m \geq 2$, the maximum number of hyperedges of a directed r-uniform hypergraph on n vertices with $\lambda^{\max} \leq m-1$ is $\big(1+o(1)\big)\,(m-1)n^2/\big(4(r-1)\big)$, so the bipartite construction of Proposition A.50 is asymptotically optimal.*

The proof of Theorem A.24 does not transfer directly, and the place it breaks is worth recording for anyone who takes this up. There, two-step routes through distinct midpoints were automatically arc-disjoint. Here a single hyperedge carries $r-1$ heads at once, so two two-step routes with different midpoints can enter through the very same hyperedge, and the family of two-step routes is no longer automatically edge-disjoint. Recovering a packing of them, or replacing the count by one that tolerates the sharing, is the technical step this conjecture waits on.

Proof. Fix an ordered pair (u,v). A hyperedge (T,H) with $u \in T$ and $v \in H$ carries the one-step route $u \to v$ that enters at the tail u and leaves at the head v. Distinct such hyperedges give routes that share no edge and no internal vertex, so if k hyperedges had u in their tail set and v in their head set then $\lambda(u,v) \geq k$ and $\kappa(u,v) \geq k$, so feasibility forces $k \leq m-1$. Each hyperedge (T,H) is counted by exactly the $|T|\,|H|$ ordered pairs (u,v) with $u \in T$ and $v \in H$, so summing the per-pair bound over all pairs gives $\sum_{(T,H)} |T|\,|H| \leq (m-1)\,n(n-1)$. As $|T|+|H| = r$ with both parts at least one, $|T|\,|H| \geq 1 \cdot (r-1) = r-1$, whence $(r-1)|E| \leq (m-1)n(n-1)$. The forward family of Proposition A.50 consists of general hyperedges, so it witnesses the same $\approx (m-1)n^2/(4(r-1))$ lower bound. ☐

A.8 The multigraph vertex problem under the other convention

Remark 2.1 collapses parallel copies in vertex mode, which makes the two multigraph vertex variants restatements of their simple counterparts and is why they carry no theorem of their own. This section takes the other reading seriously, because under it the question is genuinely new, and records what can be proved and what the machine finds. Throughout, q parallel copies of the edge uv count as q internally vertex-disjoint u-v routes, since a single edge is a path with no interior. Write $K_m^{\text{multi}}(n)$ for the largest total multiplicity of a multigraph on n vertices with $\kappa^{\max} \leq m-1$ under this reading.

The first thing to notice is that the two kinds of route never interfere, which splits the measure cleanly in two.

Lemma A.54 (Splitting the vertex measure)**.** *Let G be a multigraph with multiplicity function μ and underlying simple graph G_0. For any two vertices $u \neq v$, write $\pi(u,v)$ for the largest number of pairwise internally disjoint u-v paths of length at least two in G_0. Then, under the convention above,*

$$\kappa_G(u,v) \;=\; \mu(u,v) \;+\; \pi(u,v).$$

Proof. A parallel copy of uv is a path with empty interior, so any two copies are internally disjoint from each other and from every longer route. A maximum family may therefore be assumed to contain all $\mu(u,v)$ copies, and what it can add to them is a family of pairwise internally disjoint routes of length at least two, whose interiors are disjoint sets of vertices. Since parallel copies are irrelevant to a route of length at least two, which is determined by its vertex sequence, the largest such family has size $\pi(u,v)$, computed in G_0. □

So feasibility says exactly that $\mu(u,v) \le m-1-\pi(u,v)$ on every edge, and that $\pi(u,v) \le m-1$ on every pair. At $m=2$ this is already decisive.

Theorem A.55 (The other convention at $m=2$). $K_2^{\text{multi}}(n) = n-1$, *attained exactly by the spanning trees.*

Proof. By Lemma A.54, $\kappa^{\max} \le 1$ forces $\mu(u,v)+\pi(u,v) \le 1$ on every edge. If G_0 contained a cycle, an edge uv of that cycle would have $\mu(u,v) \ge 1$ and $\pi(u,v) \ge 1$ from the rest of the cycle, a contradiction, so G_0 is a forest and every multiplicity is one. A forest has at most $n-1$ edges, and a spanning tree attains it. □

For $m \ge 3$ at least three constructions compete, and which one wins depends on how n compares with m, exactly as in every other variant of this thesis.

- *The thickened tree.* Take a spanning tree and give every edge multiplicity $m-1$. Tree edges have $\pi=0$ and non-adjacent pairs have $\pi=1$, so this is feasible for every $m \ge 2$, and it carries $(m-1)(n-1)$ edges, the multigraph *edge* value of Theorem 1.3.
- *The thickened complete graph.* Give every edge of K_n the same multiplicity q. Here $\pi(u,v) = n-2$, so feasibility asks $q \le m+1-n$, and the count is $\binom{n}{2}(m+1-n)$, positive only when $n \le m+1$.
- *The thickened theta.* Choose two *poles* and join each of the $n-2$ remaining vertices to both of them at multiplicity $m-2$, then join the poles to each other at multiplicity $m+1-n$ when that is positive. A pole and a middle vertex have $\pi=1$, two middle vertices have $\pi=2$, and the two poles have $\pi=n-2$, so this is feasible exactly when $n \le m+1$, and it carries $2(n-2)(m-2)+\max(0,\,m+1-n)$ edges.

The theta family is the one that makes this problem genuinely different from the edge problem, and it was found by the search rather than by hand. Whenever it is feasible, which is to say for $n \le m+1$, it beats the thickened tree for every $m \ge 4$, and it is the extremiser in every such case the exhaustion has settled. Comparing the three counts, the tree leads once $n \ge m+2$, where the other two die for lack of route budget.

Exhaustive search over all multigraphs with multiplicities in $\{0,\ldots,m-1\}$, run under this convention, gives the values in Table A.1. The routine is `max_multigraph_vertex_standard(n,` `m`), which is the same include-or-exclude branch and bound as everywhere else in Chapter 2, with the checker's vertex mode switched to the second convention by one flag, `parallel_routes`, on `_split_capacity_matrix`. Every value below was computed twice, once by that routine and once by a separately written script sharing no code with it, and on the sixteen cells both have finished they agree exactly.

		$n = 2$	3	4	5	6	7
$m = 2$	value	1	2	3	4	5	6
	tree	1	2	3	4	5	6
$m = 3$	value	2	4	6	8	10	12
	tree	2	4	6	8	10	12
$m = 4$	value		6	9	12	15	
	tree		6	9	12	15	
	theta		6	9	12		
$m = 5$	value			**14**	**19**		
	tree			12	16		
	theta			**14**	**19**		
$m = 6$	value			**19**			
	tree			15			
	theta			**19**			

Table A.1. The multigraph vertex problem under the convention that parallel copies are distinct routes, computed exhaustively by `max_multigraph_vertex_standard`. Entries in bold are the cases where the value exceeds the multigraph edge value $(m-1)(n-1)$, and there the thickened theta is the extremiser. A blank cell is a size not computed, or one where the construction in that row does not apply. Cells with $n \leq m+1$ are the regime in which the theta and complete constructions are feasible at all.

Two things stand out. First, the problem is *not* the edge problem in disguise: at $m = 5$, $n = 5$ the value is 19 against the edge value 16, so the separation axis genuinely buys something for multigraphs under this reading. Second, the small-n regime $n \leq m + 1$, where the complete graph is feasible for the simple problems too, is tempting to read as the *only* place a cycle pays for itself, with the thickened tree optimal once $n \geq m + 2$. It is not: a single triangle grafted anywhere onto the thickened tree already beats it, for every $m \geq 5$ and every such n, which the next theorem makes precise.

By Lemma A.54, a feasible multigraph on top of a fixed connected simple graph G_0 maximizes its total multiplicity by setting $\mu(u,v) = m - 1 - \pi(u,v)$ on every edge, giving total $(m-1)\,|E(G_0)| - \sum_{uv \in E(G_0)} \pi(u,v)$. Writing $|E(G_0)| = n - 1 + \operatorname{rank}(G_0)$, the thickened tree is beaten by any G_0 with

$$\sum_{uv \in E(G_0)} \pi(u,v) \; < \; (m-1)\operatorname{rank}(G_0).$$

The obvious guess is that cycles must always pay for themselves against this bound, each independent cycle costing $m-1$ units of route capacity. The next theorem shows a single triangle already fails to pay, for $m \geq 5$, and packing many of them compounds the failure into a gap that grows with n.

Theorem A.56 (Clique chains beat the thickened tree, for every $m \geq 5$)**.** *Fix $3 \leq r \leq m$ and set $q = m + 1 - r \geq 1$, the multiplicity of a thickened K_r that saturates the vertex ceiling on r vertices (the "thickened complete graph" construction above, restricted to a block of size r rather than all of n). For $n \geq r$, partition $k = \lfloor n/r \rfloor$ disjoint r-vertex blocks out of the n vertices, thicken each into a K_r at multiplicity q, join a fixed vertex of each block to a fixed vertex of the next by a single edge at multiplicity $m-1$, and attach any leftover vertices as a pendant path at the same multiplicity. This multigraph is feasible, $\kappa^{\max} \leq m - 1$, and its total multiplicity is exactly*

$$(m-1)(n-1) \;+\; \left\lfloor \frac{n}{r} \right\rfloor \cdot \mathrm{gain}(r,m), \qquad \mathrm{gain}(r,m) := \frac{(r-1)(r-2)(m-1-r)}{2}.$$

Proof. Write G_0 for the underlying simple graph.

Feasibility. Every bridge and pendant edge of G_0 is a genuine bridge, since blocks are joined only by single edges with no alternate route, so no path of length ≥ 2 between two vertices of the same block can leave it and return, which would cross a bridge edge twice. Hence for an edge uv inside a block, every route counted by $\pi(u,v)$ stays inside K_r, where deleting uv leaves u, v joined by exactly $r-2$ pairwise internally-disjoint two-step routes, one through each of the other block vertices, and by Menger no more, since those same $r-2$ vertices are a cut of that size. So $\pi(u,v) = r-2$ and, by Lemma A.54, $\kappa_G(u,v) = q + (r-2) = m-1$ exactly, never over. For u, v in different blocks, or either endpoint on a bridge or pendant edge, the port vertex adjacent to u's side (or to v's side, if u is itself that port) is a single cut vertex separating them, so $\kappa_G(u,v) \leq 1$.

Count. Since $q \geq 1$, every block edge is genuinely present, so G_0 is connected and $|E(G_0)| = n-1+\mathrm{rank}(G_0)$. This is where the hypothesis $r \leq m$ earns its keep: at $r = m+1$ the multiplicity q would drop to zero, the blocks would carry no edges at all, and G_0 would fall apart into isolated vertices. Only the k blocks contribute to $\mathrm{rank}(G_0)$ or to $\sum \pi$, since bridges and pendant edges have $\pi = 0$ and lie in no cycle. Each K_r has rank $(r-1)(r-2)/2$ and contributes $\binom{r}{2}(r-2)$ to the sum, since each of its $\binom{r}{2}$ edges has $\pi = r-2$. Substituting into $(m-1)(n-1+\mathrm{rank}(G_0)) - \sum \pi$,

$$(m-1)\Big[n-1+k\tfrac{(r-1)(r-2)}{2}\Big] - k\tfrac{r(r-1)(r-2)}{2} \;=\; (m-1)(n-1) + k\,\mathrm{gain}(r,m),$$

using $(m-1)\frac{(r-1)(r-2)}{2} - \frac{r(r-1)(r-2)}{2} = \frac{(r-1)(r-2)\big[(m-1)-r\big]}{2} = \mathrm{gain}(r,m)$. □

At $r = 3$, $\mathrm{gain}(3,m) = m-4$: zero at $m = 4$, matching the exhaustive finding that $m \leq 4$ has no counterexample, and strictly positive for every $m \geq 5$. So the guess that the thickened tree is optimal once $n \geq m+2$ is false for every $m \geq 5$: it held only within the reach of the exhaustive table, $m \leq 4$. The smallest witness is small enough to check by hand. Take $m = 5$ and $n = 7$, the first size the guess covers. Put a triangle on $\{u_1, u_2, u_3\}$ at multiplicity 3 and hang a path of four further vertices off u_1 at multiplicity 4. Two triangle vertices have three parallel copies plus one route through the third vertex, so $\kappa = 4$, and every other pair is separated by a single cut vertex or joined only by its own parallel copies, so nothing exceeds $4 = m-1$. The total multiplicity is $3 \cdot 3 + 4 \cdot 4 = 25$, one more than the $(m-1)(n-1) = 24$ the guess allows. Packing $k = \lfloor n/r \rfloor$ blocks instead of one multiplies the excess by k, so the gap over the thickened tree grows linearly in n, and optimizing r widens it further. Treated as continuous, $\mathrm{gain}(r,m)/r$ peaks near $r \sim m/2$, where it is $\Theta(m^2)$, so $K_m^{\mathrm{multi}}(n)/n \to (m-1) + \Theta(m^2)$ as $n \to \infty$ at fixed large m, a different order in m from the guess above. Whether same-size clique chains are optimal among all feasible multigraphs, and what the true value of $K_m^{\mathrm{multi}}(n)$ is, are open again rather than reduced to a single inequality about cycle rank.

A.9 The random threshold

The probabilistic ingredients are two Chernoff bounds, quoted in the form we use them.

Primer: what a Chernoff bound says

Both proofs below need to say that a sum of many independent yes-or-no trials almost never lands far from its average. That is the content of a *Chernoff bound*, and the intuition is worth stating even though we only quote the result. Let $X = X_1 + \cdots + X_N$ count the successes among independent trials, with mean $\mu = \mathbb{E}[X]$. A Chernoff bound says the chance that X overshoots μ by a factor $1+\delta$, or undershoots it by a factor $1-\delta$, decays *exponentially* in μ, not merely polynomially as Chebyshev's inequality would give. The mechanism is a single trick. Rather than bound $\Pr[X \geq a]$ directly, bound $\Pr[e^{tX} \geq e^{ta}]$ by Markov's inequality for a parameter $t > 0$, use independence to factor $\mathbb{E}[e^{tX}] = \prod_i \mathbb{E}[e^{tX_i}]$ into a product over the trials, and then choose t to make the resulting bound as small as possible. The exponential in the answer is exactly the e^{tX} that was fed through Markov. We quote the two tails in the shape the threshold proof consumes.

Theorem A.57 (Chernoff bounds, see [19, Cor. 4.4, Thm. 4.4])**.** *Let X be a sum of independent $\{0,1\}$-valued random variables with mean μ. For $\delta > 0$, $\Pr[X \geq (1+\delta)\mu] \leq \exp\left(-\frac{\min(\delta,\delta^2)\,\mu}{4}\right)$, and for $\delta \in (0,1)$, $\Pr[X \leq (1-\delta)\mu] \leq \exp\left(-\frac{\delta^2 \mu}{2}\right)$.*

Corollary A.58. *Let $X \sim \mathrm{Binomial}(n-1,p)$ with $p = c\,m/n$ for a constant $c \in (0,1)$. Then for all sufficiently large m, $\Pr[X \geq m] \leq e^{-\alpha m}$ with $\alpha = \alpha(c) > 0$.*

Proof. The mean is $\mu = (n-1)p \leq cm$, so $m \geq (1+\delta)\mu$ with $\delta \geq \frac{1-c}{c} > 0$, and Theorem A.57 applies with $\mu \geq cm/2$ for $n \geq 2$, giving $\alpha = \min\left(\frac{1-c}{c}, \left(\frac{1-c}{c}\right)^2\right)\frac{c}{8}$. ☐

Proof of Theorem 1.16. Recall the hypotheses: $m = m(n) \geq 2$ with $m/\ln n \to \infty$ and $m = o(n)$, and $p = c\,m/n$ for a constant $c > 0$.

Case $c > 1$. The edge count $|E|$ of $G(n,p)$ is binomial with mean $\mu = \binom{n}{2}p = \frac{c\,m(n-1)}{2}$. With $\delta = 1 - \frac{1}{c}$ the lower-tail bound of Theorem A.57 gives

$$\Pr\Big[\,|E| \leq \tfrac{m(n-1)}{2}\Big] \;\leq\; \exp\Big(-\tfrac{(c-1)^2\,m(n-1)}{4c}\Big) \;\to\; 0,$$

so with high probability $|E| > \left\lfloor \frac{m(n-1)}{2} \right\rfloor$, and the contrapositive of Mader's theorem (Theorem 1.2) forces a pair with $\lambda \geq m$. Hence $\Pr[\lambda^{\max} \geq m] \to 1$.

Case $c < 1$. Each degree is $\mathrm{Binomial}(n-1, p)$, so by Corollary A.58 and a union bound over the n vertices,

$$\Pr[\Delta(G) \geq m] \;\leq\; n\, e^{-\alpha m} \;=\; e^{\ln n - \alpha m} \;\to\; 0,$$

since $m/\ln n \to \infty$. By the degree bound (Lemma A.3) $\lambda^{\max} \leq \Delta(G)$, so $\Pr[\lambda^{\max} \geq m] \to 0$. □

Remark A.59 (The vertex and directed analogues). Below the threshold, $\kappa^{\max} \leq \lambda^{\max}$ gives the vertex statement for free, and in the random digraph $D(n,p)$, where each ordered pair's arc is tossed independently with probability p, the out-degree version of the same union bound applies. Above the threshold, the minimum degree exceeds m with high probability by the lower-tail bound. Mader's theorem alone does not carry this to the vertex problem, since $\kappa^{\max} \leq \lambda^{\max}$ runs the wrong way to deduce $\kappa^{\max} \geq m$ from the edge bound. What supplies it instead are the theorems of Bollobás [20, Ch. 7] for $G(n,p)$, in hitting-time form due to Bollobás and Thomason [21], and its directed analogue [22], which state that in this regime the global connectivities equal the minimum degree (respectively the minimum semi-degree, the smaller of a vertex's in- and out-degree) with high probability, so $\kappa^{\max}$ and the directed $\lambda^{\max}$ also cross m at $p^* = m/n$. The growth hypothesis $m/\ln n \to \infty$ is what lets the union bound beat the factor n. For fixed m, the bound is vacuous and finer tools (Poisson approximation of low-degree vertex counts) would be needed.

Remark A.60 (How far the threshold reaches, and where it stops). Theorem 1.16 is a statement about $G(n,p)$, and Remark A.59 carries it to the vertex separation and to the directed model $D(n,p)$. It does not reach the hypergraph models, and the reason is a change of scale rather than a gap in the argument. The threshold sits where a vertex's expected degree reaches m, since below that the degree bound caps every local connectivity and above it the extremal bound forces a high pair. In $G(n,p)$ a vertex has expected degree $p(n-1)$, so the balance point is $p^* = m/n$ as stated. In the binomial r-uniform hypergraph, where each of the $\binom{n}{r}$ possible hyperedges is included independently with probability p, a vertex lies in an expected $p\binom{n-1}{r-1}$ hyperedges, so the balance point is

$$p^* \;=\; \frac{m}{\binom{n-1}{r-1}} \;=\; \Theta\!\left(\frac{m}{n^{r-1}}\right),$$

which for $r \geq 3$ is smaller than m/n by a factor of order n^{r-2}. Reading m/n as the hypergraph threshold would therefore place the transition far above where it happens: at $r = 3$ and $n = 8$, for instance, m/n is $3/8$ while $m/\binom{7}{2}$ is $3/21$, and by the former density the sampled hypergraph is long past the transition. Figure B.5 accordingly sweeps each model in units of its own p^* rather than a shared p. A second limit of scope is worth repeating here: the theorem assumes $m/\ln n \to \infty$, so any picture at a fixed small m illustrates the shape of the transition without being an instance of the theorem.

A.10 Computational transcripts

The cut-counting optimisation formulation that certifies Theorem 2.2 is given in Chapter 2. Here are the solver transcripts that back it, together with the exhaustive-search output that settles the directed base cases. The complete commented source accompanies the document as a single self-contained file, `erdos915_unified.py` in the `program/` directory. Its core runs on `numpy` and `scipy` alone, the certifier adds `pulp`, and `matplotlib` and `networkx` are optional and used only for the figures. An optional compiled helper accelerates selected small hot paths but is not needed for correctness.

```
Cut-MILP certification of the directed multigraph problem
M*(n) is the scaled optimum; L_m^dir(n) = (m-1) M*(n).

n=3: status=OPTIMAL, M*=4.000, target=2(n-1)=4, proof=True, time=0.0s, L_3^dir(n)=8
n=4: status=OPTIMAL, M*=6.000, target=2(n-1)=6, proof=True, time=0.3s, L_3^dir(n)=12
n=5: status=OPTIMAL, M*=8.000, target=2(n-1)=8, proof=True, time=5.9s, L_3^dir(n)=16
n=6: status=OPTIMAL, M*=10.000, target=2(n-1)=10, proof=True, time=1259.9s,  L_3^dir(n)=20
```

Listing A.1. Solver output: $M^*(n) = 2(n-1)$, optimal with zero gap, for $n = 3, 4, 5, 6$.

The cut-counting optimisation closes the gap up to $n = 5$ inside a short budget and reaches $n = 6$ in about twenty minutes. Past that it stalls, because a solver handles the yes-or-no cut labels by case-splitting, trying a label both ways and solving each half, and the number of cases it must open grows too fast at $n = 7$. The base cases of the directed induction (Theorem 1.10) run to $n = 7$, and those are settled instead by the pruned exhaustive search over all simple digraphs with $\lambda^{\max} \leq 1$. That search is exact and complete at these sizes, and it agrees with $\ell_2^{\mathrm{dir}}(n) = M(n)$ throughout.

```
Exhaustive search over all simple digraphs with lambda^max <= 1 (directed, m=2).
Branch and bound; reported max arc count = ell_2^dir(n) = max(2(n-1), floor (n^2/4)).

  n=2:  max= 2,  formula= 2,  proved (complete),  nodes=5,         0.0s
  n=3:  max= 4,  formula= 4,  proved (complete),  nodes=22,        0.0s
  n=4:  max= 6,  formula= 6,  proved (complete),  nodes=537,       0.0s
  n=5:  max= 8,  formula= 8,  proved (complete),  nodes=17823,     0.1s
  n=6:  max=10,  formula=10,  proved (complete),  nodes=788101,    9.1s
  n=7:  max=12,  formula=12,  proved (complete),  nodes=45122129,  718.1s

The cut-MILP certifier of the multigraph problem closes n <= 6 (n = 6 in about
21 minutes); the directed m = 2 base cases through n = 7 that the induction
```

```
requires are settled by this exhaustive search instead.
```

Listing A.2. Exhaustive-search output for the base cases $2 \leq n \leq 7$: the maximum arc count equals $M(n) = 2, 4, 6, 8, 10, 12$, proved complete at each n.

The vertex problem needs its own base cases, since Remark A.16 shows they cannot be inherited from the arc ones, and the same driver supplies them with only the inner feasibility test swapped. The two separations agree at every size, which is what closes Theorem 1.11.

```
Exhaustive search over all simple digraphs with kappa^max <= 1 (directed, m =2).
Branch and bound; reported max arc count = k_2^dir(n) = max(2(n-1), floor(n ^2/4)).

  n=2:  max= 2,  formula= 2,  proved (complete),  nodes=5,            0.0s
  n=3:  max= 4,  formula= 4,  proved (complete),  nodes=22,           0.0s
  n=4:  max= 6,  formula= 6,  proved (complete),  nodes=537,          0.0s
  n=5:  max= 8,  formula= 8,  proved (complete),  nodes=17823,        1.1s
  n=6:  max=10,  formula=10,  proved (complete),  nodes=788101,      77.0s
  n=7:  max=12,  formula=12,  proved (complete),  nodes=45122129,  3124.8s

Same driver, same prunes, same sizes as the arc run above. Only the
inner test changes, from arc-disjoint to internally vertex-disjoint.
The two separations agree at every base case, which is what the
vertex induction needs and what Whitney's inequality cannot supply.
```

Listing A.3. The same search run with the internally vertex-disjoint feasibility test, settling the base cases of Theorem 1.11: the maximum arc count is again $2, 4, 6, 8, 10, 12$, proved complete at each n.

A.11 How the program checks itself

The invariants are checked by the self-test at the foot of `erdos915_unified.py`, run simply with `python erdos915_unified.py`. It confirms the checker against the proven $m = 2$ values $2, 4, 6, 8$ and against the first three vertex base cases of Theorem 1.11, the fast vertex-flow test against the exact checker it stands in for, the counting step and both bounds of Proposition A.22 and Theorem A.24 on sampled feasible digraphs, including the high-degree case of the latter's proof, two values of the alternative-convention multigraph vertex problem of Section A.8 including the one that separates it from the edge problem, the named constructions against their predicted sizes and connectivities, the Gomory–Hu shortcut against brute-force max-flow, the Berge connectivities of the hypergraph checker, the Monte Carlo appearance threshold and Whitney's inequality on every sample, the cut-counting optima for small n, the search reaching the certified optimum, and the driver returning the proven value or a feasible witness in each mode. Every check passes. A single companion script, `make_figures.py`, regenerates every figure in the thesis from the same file, with the mapping listed in `program/README.md`.

Appendix B

Supplementary Figures

This appendix collects the supplementary figures that the program produces but that the chapters reference rather than reproduce in full. Each one is named where it belongs in the body, and the file name printed in its caption matches the output of `make_figures.py` and the figure table in `program/README.md`. Figures already shown in the running text are not repeated here.

B.1 A gallery of extremal graphs

The extremal graphs are not abstractions. Figure B.1 draws nine of the named families the body argues about, each at a deliberately large representative size rather than at the smallest case, so the structure is visible. These are the special, structured extremisers, not the trivial small trees: the directed double star and bidirected star (hub at the centre, with arcs in both directions to every leaf), the doubled bipartite wall $2 \cdot B_{3,4}$ that ties the double star at the crossover, the dense bidirected complete graph, the multigraph star at full multiplicity, and the star hypertrees, the last drawn as metro maps in which each hyperedge is one coloured line through its members. The program builds every one of these directly and, at the small sizes the enumeration reaches, also proves them optimal rather than merely feasible, so the gallery is the analysis made concrete at a size a reader can take in.

B.2 Search traces across the variants

Figure 3.2 in Chapter 3 follows one cooling run in detail. The two runs below repeat that experiment for two further, deliberately dissimilar variants, one undirected and simple, one directed and multigraph, to show the behaviour is the same across the family rather than special to one case. Each plots every visited graph by its binding connectivity (horizontal) against its size (vertical), colouring each point by the step at which it was visited, from the hot early exploration in warm tones to the converged tail in blue. In both the walk strays past the feasibility boundary while hot and is driven back onto the densest feasible graph as the temperature falls, exactly as in the body figure. The seeds are fixed, so each picture reproduces verbatim.

B.3 The variant landscape at $m = 6$ and in three dimensions

This appendix shows the per-graph connectivity distribution at the fixed forbidden threshold $m = 6$, alongside a three-dimensional view of the appearance threshold (Figure B.5). They con-

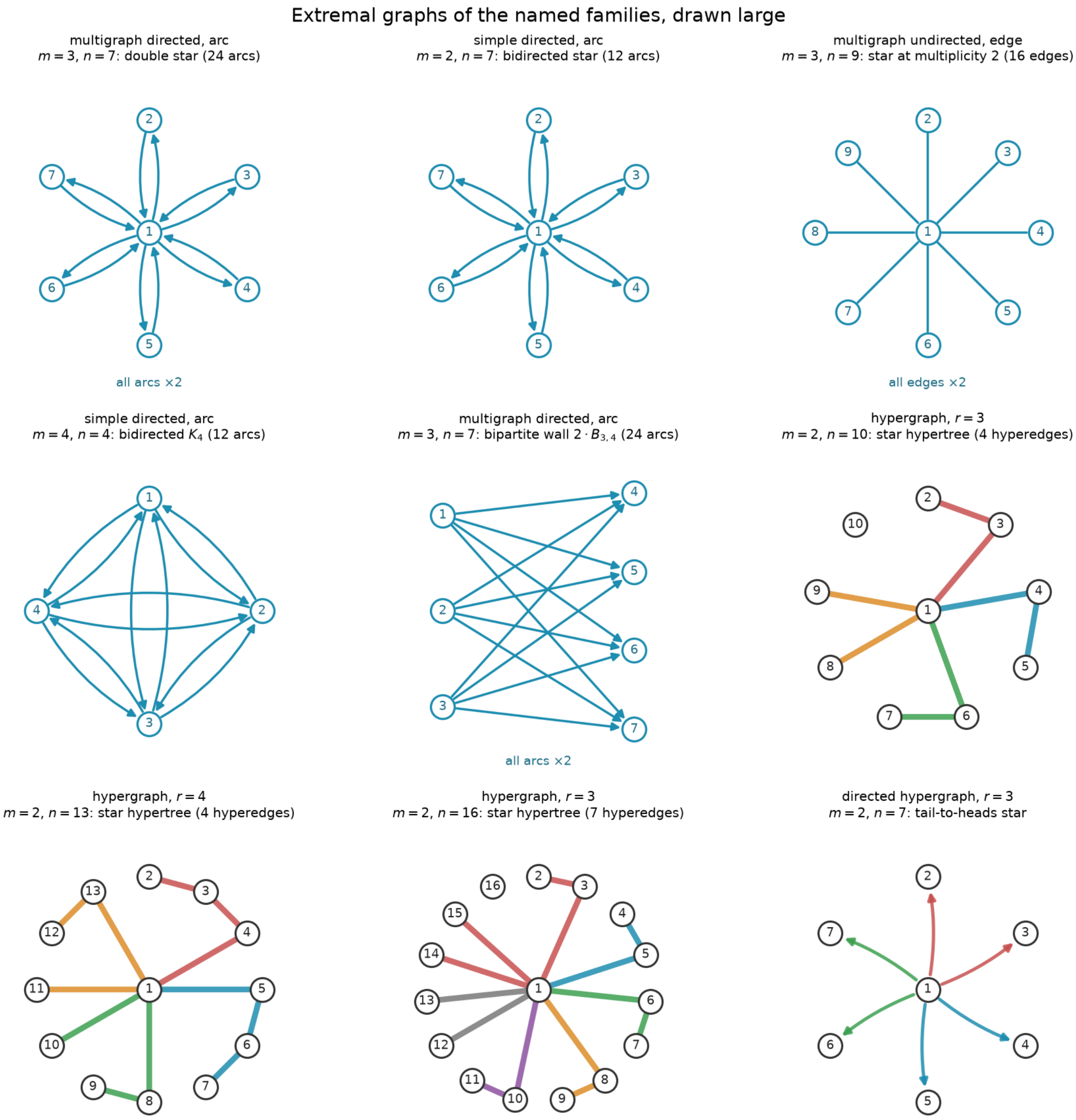


Figure B.1. Extremal graphs of the named families, drawn at a large representative size. *Top and middle rows:* matrix variants, with a central hub where the family has one. Every family here carries one uniform multiplicity, stated once per panel as "all arcs $\times k$" rather than repeated on each arc or drawn as parallel curves, and directed arcs carry arrowheads. *Hypergraph panels:* each hyperedge is a metro line through its member vertices, in its own colour, with the hub at the centre, and the directed hypergraph fans coloured arrows from each tail to its heads. Every graph shown is feasible at the stated m. The matrix and undirected-hypergraph families are extremal for their variant, the two directed stars drawn at the crossover $n = 7$, the largest size at which the linear star still ties the one-directional wall before the wall overtakes it. The bipartite wall $2 \cdot B_{3,4}$ is drawn at that same size to show the other side of the tie, matching the double star at 24 arcs as one of the three families the $n = 7$ classification of Remark A.39 identifies. The final panel is the directed-hypergraph analogue, a tail-to-heads star, shown for the one variant whose exact value is still open (Proposition A.50). File `extremal_graphs_gallery.png`.

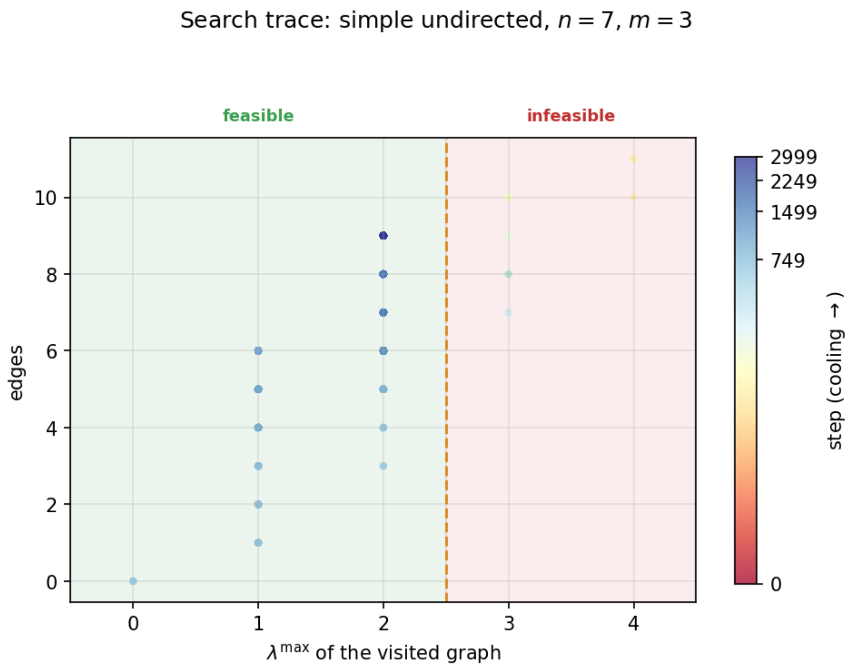


Figure B.2. Search trace for the simple undirected edge problem at $n = 7$, $m = 3$. File `trace_simple_undirected_n7_m3.png`.

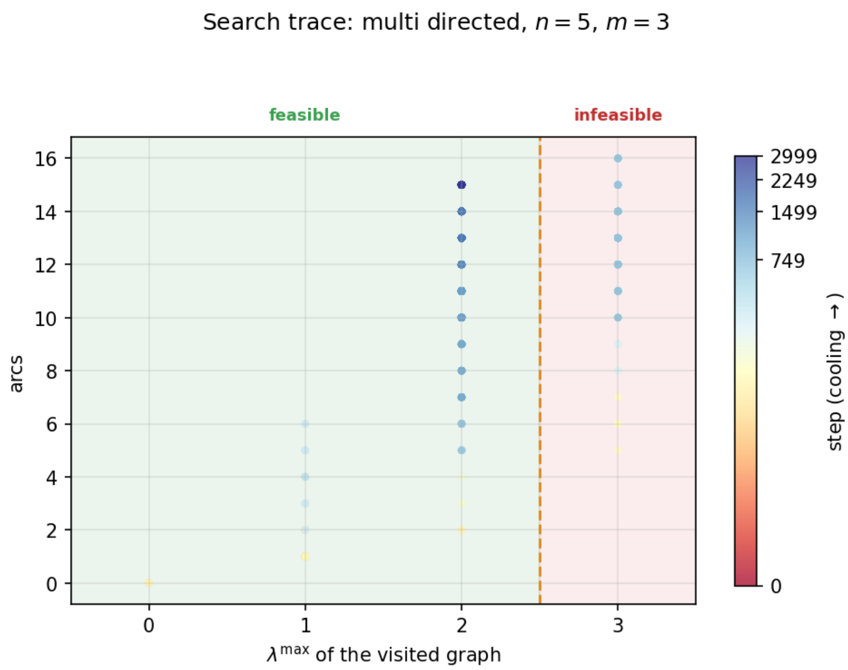


Figure B.3. Search trace for the multigraph directed arc problem at $n = 5$, $m = 3$. File `trace_multi_directed_n5_m3.png`.

firm that raising the forbidden threshold relocates the feasibility boundary without changing the qualitative picture. The pooled pair-connectivity and edge-count grids of Chapter 4 (Figures 4.4 and 4.5) already split each variant at its own mid-range connectivity, so they carry no separate $m = 6$ companion.

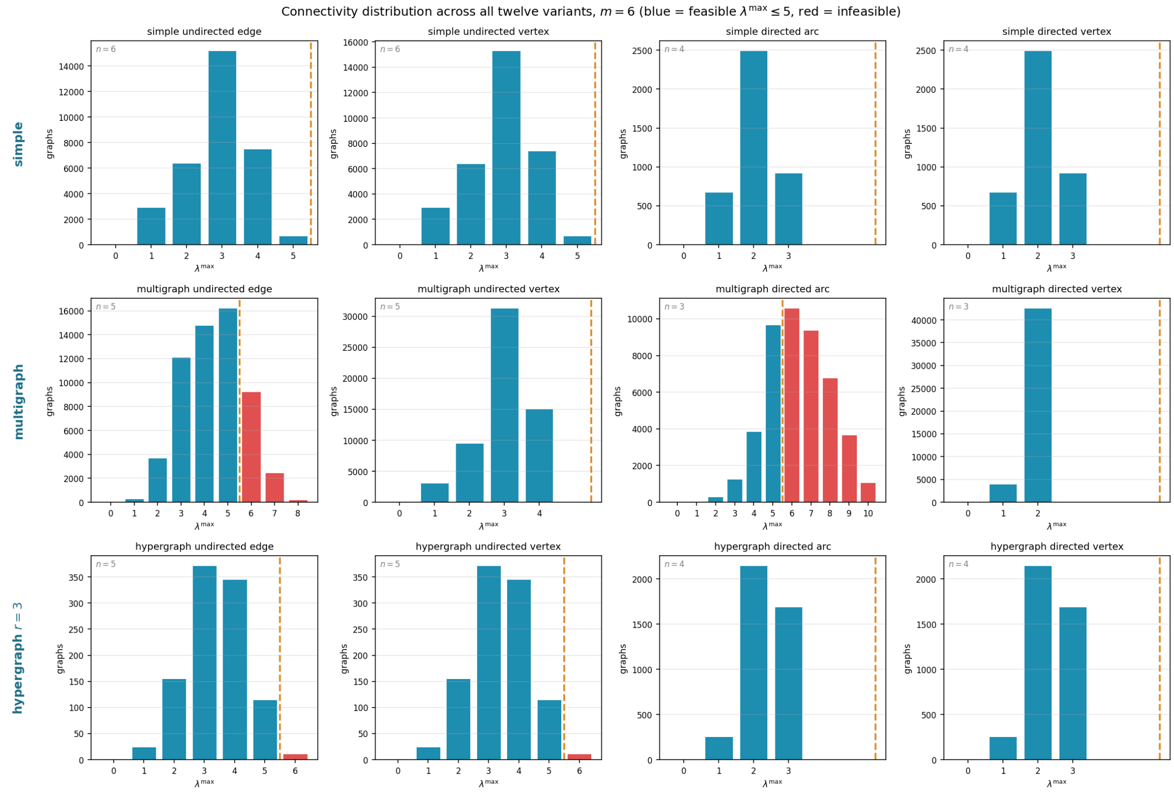


Figure B.4. How the full small-n enumeration distributes by binding connectivity $\lambda^{\max}$ across all twelve variants at the fixed forbidden threshold $m = 6$. Blue bars are feasible graphs ($\lambda^{\max} \leq 5$), red bars infeasible ones, and the dashed line marks the feasibility boundary. At this higher threshold most of the small graphs the enumeration reaches are feasible, so the blue mass dominates: raising m moves the boundary out past the bulk of the distribution. File `conn_dist_m6.png`.

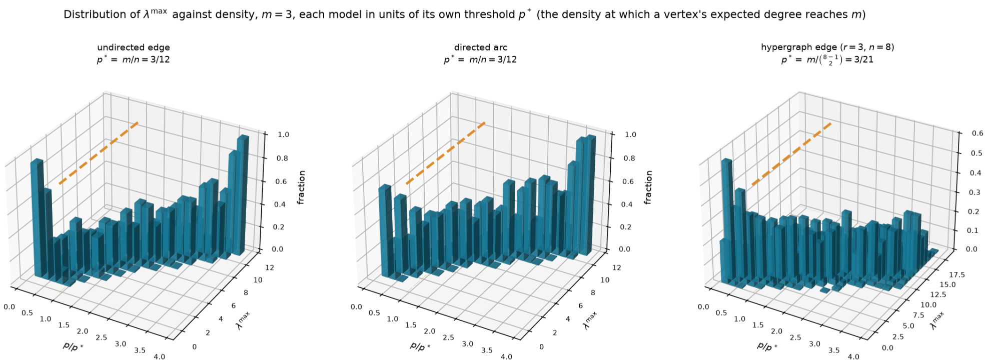


Figure B.5. The appearance threshold as a three-dimensional bar chart, for three representative variants (simple undirected edge and simple directed arc at $n = 12$, three-uniform hypergraph edge at $n = 8$), all at $m = 3$. Each panel plots the fraction of samples (height) reaching each binding connectivity $\lambda^{\max}$ (depth) against density (width), and the mass migrates from low to high connectivity as the density crosses the dashed line. The three models do *not* share a density scale, so each panel is swept in units of its *own* threshold p^*, the density at which a vertex's expected degree reaches m: that is m/n for the two graph models and $m/\binom{n-1}{r-1}$ for the hypergraph, smaller by a factor of order n^{r-2} (Remark A.60). Each panel title gives its own p^*. Only the two graph panels fall under Theorem 1.16, and even they sit outside its asymptotic regime $m/\ln n \to \infty$ at this fixed $m = 3$, so the figure illustrates the shape of the transition rather than instantiating the theorem. File `threshold_3d.png`.

Appendix C

The Complete Source Code

Every number in this thesis that was not proved by hand came out of the program printed here, and this appendix prints all of it, with nothing summarised away. The point is that a reader who doubts a computed value can find the exact lines that produced it, rather than taking a description of them on trust.

There is no need to read this front to back. It is laid out so that you can jump to whichever part you want to check. The listing is broken into the same four movements as the thesis itself, because the program is organised that way on purpose: it defines the objects, then measures them, then proves things about them, then searches for examples and draws the results. Each part below opens with a short account of what it does and which chapter it belongs to, and every listing carries line numbers so that a reference like “line 1412” has one unambiguous meaning. Where a line of code was too long for the page it is wrapped, and the continuation is marked with a ↪ arrow rather than being silently cut.

Four files make up the whole of it. The main program, `erdos915_unified.py`, is the one the chapters discuss throughout and the only one that computes a reported result. `make_figures.py` redraws every picture in the thesis by calling into that program, so a figure can never drift away from the numbers behind it. `_erdos_fast.c` is a small optional accelerator: it speeds up two inner loops and changes no answer, and the program runs correctly without it. The `tests/` directory holds the suite that checks all of the above.

C.1 The main program

C.1.1 How the file opens

The program begins by saying what it is for, then imports what it needs and fixes a few constants. Two decisions made here shape everything after. The first is that the heavy scientific libraries are optional: only `numpy` and `scipy` are required, and `matplotlib`, `networkx` and `pulp` are each wrapped so that the file still loads without them, with the features that need them disabled rather than the whole program failing. The second is that every import is hoisted to the top, so there is one place to look to see what the program depends on.

```
1 """
2 erdos915_unified: the complete unified program for Erdos Problem 915, in one file.
3
4 This single module is the whole program, kept in one namespace so it can be read
```

```
top to bottom as a continuous story.  It is meant to be read, not just run: every
non-trivial step carries an inline comment explaining *why* it is there, not
merely what it does.  Running it (``python erdos915_unified.py``) executes the
invariant self-check at the foot of the file; ``make_figures.py`` next to it
regenerates the thesis figures.

---------------------------------------------------------------------------
The problem, in one breath
---------------------------------------------------------------------------
Erdos Problem 915 asks, in its many guises, for the densest graph on ``n``
vertices in which no pair of vertices has ``m`` independent routes between them.
"Independent" can mean edge-disjoint or internally vertex-disjoint; the object
can be a simple graph, a multigraph, a digraph, or a hypergraph; and forbidding
``m`` routes is exactly the constraint that the largest *local* connectivity
stays at or below ``m - 1``.  Twelve concrete variants fall out of these
choices, and this program treats all of them with one representation and one
search.

---------------------------------------------------------------------------
The epistemic spine: measure / prove / discover
---------------------------------------------------------------------------
The whole architecture is organised around three different *kinds of claim* a
program can make about extremal values, and it keeps them scrupulously apart so
they can never silently masquerade as one another:

  * MEASURE.  The checker computes exact local connectivities by max-flow
    (Menger's theorem).  Whatever it reports is a fact about a concrete graph,
    not an estimate.  This is the trusted core; everything else is checked
    against it.

  * PROVE.   The cut-counting method proves *upper* bounds for a fixed number of
    vertices: it shows that no graph on ``n`` vertices can be denser, with a
    zero optimality gap, so an optimal solution is a genuine proof.

  * DISCOVER. The random search finds concrete dense graphs, hence
    *lower* bounds.  A construction it returns is a witness that the extremal
    value is at least so large; it never proves that no denser graph exists.

The closed-form ``bounds`` library sits to the side as the theory's predictions
(proved, cited, or merely conjectured, each labelled honestly), so that "what
the program found" can always be held against "what the theory predicts" without
the two drifting apart.  The Monte Carlo module sits in the DISCOVER/observe
corner: a sampled estimate is an estimate, used to make a proved threshold
*visible*, never to prove it.

---------------------------------------------------------------------------
Chapter map (sections are grouped below by the thesis chapter they support)
---------------------------------------------------------------------------
  CHAPTER 1  The problem and its base cases
        VARIANT, BOUNDS, HYPERGRAPH, CONSTRUCTIONS
  CHAPTER 2  Proving bounds by machine
        CHECKER, GOMORY-HU, PROVE (the brute-force and exhaustive provers
        live inside the SOLVE driver, in Chapter 3)
  CHAPTER 3  Discovering bounds by search
        SENSITIVITY, SEARCH, SOLVE (the one driver: prove or discover)
  CHAPTER 4  Synthesis and results
        MONTE CARLO, FIGURES, __main__ (the invariant self-check suite)

```

```
A bold CHAPTER divider precedes each group, so as you scroll you always know
which chapter's machinery you are reading. Each section keeps its own banner.
"""

# ================================================================
# Hoisted imports
# ----------------------------------------------------------------
# All external dependencies, gathered into a single block.  ``from __future__
# import annotations`` must be (and is) the very first statement so that
# string-form type hints work uniformly.  Everything in this program lives in one
# namespace, so there are no intra-module imports to follow.
# ================================================================
from __future__ import annotations

import copy
import json
import math
import pickle
import random
import shutil
import statistics
import subprocess
import time
import warnings
from collections import deque
from collections.abc import Iterable
from dataclasses import dataclass, field
from itertools import combinations, combinations_with_replacement, permutations,
    ↪ product
from pathlib import Path
from typing import Callable, Iterator

import concurrent.futures

import numpy as np
from scipy.sparse import csr_matrix as _csr
from scipy.sparse.csgraph import maximum_flow as _csgraph_maxflow

# networkx is OPTIONAL.  Every connectivity measure the thesis reports is computed
# through scipy's integer max-flow on a capacity matrix, so the core checker,
# search, provers, and enumeration run on numpy + scipy alone.  networkx is needed
# only for the Gomory-Hu tree (one figure/analysis helper), which raises a clear
# message if called without it installed.
try:
    import networkx as nx
    NETWORKX_AVAILABLE = True
except ImportError:
    nx = None
    NETWORKX_AVAILABLE = False


def _require_networkx(feature: str):
    """Raise a clear error when an optional networkx-only feature is used headless
        ↪ ."""
    if not NETWORKX_AVAILABLE:
        raise ImportError(
            f"{feature} needs networkx (pip install networkx). The core
                ↪ connectivity "
```

```python
            f"measures do not: they run on numpy + scipy alone."
        )

# pulp is OPTIONAL.  It backs the MILP certifier (prove_directed_multigraph and
# prove_integral_arc_bound): one solver-agnostic cut-counting model that CBC (pulp's
# bundled solver) runs by default and Gurobi runs by a one-line switch.  The model,
# checker, search, enumeration, and sampling do not need it.
try:
    import pulp
    # pulp 3.x emits migration notices for its 4.0 API (LpVariable construction,
    # the PULP_CBC_CMD name).  We use the stable 3.x API on purpose and pin it in
    # requirements.txt, so silence that forward-looking noise.
    import warnings as _warnings
    _warnings.filterwarnings("ignore", message=r".*PuLP 4\.0.*",
                             category=DeprecationWarning)
    PULP_AVAILABLE = True
except ImportError:
    pulp = None
    PULP_AVAILABLE = False


def _require_pulp():
    """Raise a clear error when the MILP certifier is used without pulp installed."""
    if not PULP_AVAILABLE:
        raise ImportError(
            "the MILP certifier needs pulp (pip install pulp). The checker, search, "
            "and enumeration do not: they run on numpy + scipy alone."
        )

import ctypes as _ct
_C = None
_so_path = Path(__file__).with_name("_erdos_fast.so")
if _so_path.exists():
    _C = _ct.CDLL(str(_so_path))
    _C.tiny_maxflow.restype = _ct.c_int
    _C.tiny_maxflow.argtypes = [
        _ct.POINTER(_ct.c_int), _ct.c_int, _ct.c_int, _ct.c_int, _ct.c_int,
    ]
    _C.max_connectivity_exceeds.restype = _ct.c_int
    _C.max_connectivity_exceeds.argtypes = [
        _ct.POINTER(_ct.c_int), _ct.c_int, _ct.c_int, _ct.c_int,
    ]
    _C.canonical_form_min.restype = None
    _C.canonical_form_min.argtypes = [
        _ct.POINTER(_ct.c_int), _ct.c_int, _ct.POINTER(_ct.c_int),
    ]
C_EXTENSION_LOADED = _C is not None

# matplotlib is OPTIONAL: it is needed only to render the figures (the plot_*
# functions in the FIGURES section).  It needs its backend chosen before pyplot is
# imported, so that the figure code runs head-less (writes PNG files, never opens a
# window).  When it is absent the model, checker, provers, search, and enumeration
# all still run; only figure generation is unavailable.
```

```python
try:
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt  # noqa: E402  (after backend selection)
    from matplotlib.ticker import MaxNLocator  # noqa: E402  (integer-only axis ticks)
    from mpl_toolkits.mplot3d import Axes3D  # noqa: F401  (registers the '3d' projection)
    MATPLOTLIB_AVAILABLE = True
except ImportError:
    plt = None
    MaxNLocator = None
    MATPLOTLIB_AVAILABLE = False
```

C.1.2 The objects: graphs, hypergraphs, and the values we already know

This part corresponds to Chapter 1. It defines what a graph is for the purposes of this thesis, which is a single square table of numbers: entry (u, v) records how many parallel connections run from u to v. That one table covers all eight of the matrix variants at once, because a `Variant` record says whether the graph is directed and whether it is simple, and those two switches are the only difference between them. Hypergraphs need a different shape, so they are stored as a list of the sets of vertices each hyperedge joins.

After the objects come the closed-form values the thesis proves or conjectures, each written as a small function whose docstring records whether it is proved, conjectured, or a bound only. Then come the named constructions, the specific graphs that achieve those values, so that a claimed bound can always be met with a concrete example rather than an existence argument.

```python
######################################################################
##
##   CHAPTER 1 -- THE PROBLEM AND ITS BASE CASES
##   Objects, variants, and the closed-form values we already know.
##
######################################################################

# --- VARIANT: mu[u,v] = multiplicity from u to v; simple = {0,1}; undirected = symmetric ---


@dataclass(frozen=True)
class Variant:
    """Which kind of graph object we are working with.

    Attributes:
        directed: ``True`` for digraphs (arcs), ``False`` for graphs (edges).
        simple: ``True`` caps every multiplicity at one. ``False`` allows
            arbitrarily many parallel edges or arcs (a multigraph).
        name: a short human-readable label, used in figures and reports.
    """

    directed: bool
    simple: bool
    #loops: bool
    name: str = ""
```

```python

    def describe(self) -> str:
        """Return a readable description such as ``"simple directed"``."""
        # Compose the label from the two boolean axes; used in plot titles.
        words = ["simple" if self.simple else "multi",
                 "directed" if self.directed else "undirected"]
        return " ".join(words)


# The four matrix-representable variants, named for convenience.  Random graphs
# reuse these (a sampled simple undirected graph is SIMPLE_UNDIRECTED).
SIMPLE_UNDIRECTED = Variant(directed=False, simple=True, name="simple undirected")
MULTI_UNDIRECTED = Variant(directed=False, simple=False, name="multi undirected")
SIMPLE_DIRECTED = Variant(directed=True, simple=True, name="simple directed")
MULTI_DIRECTED = Variant(directed=True, simple=False, name="multi directed")


class Graph:
    """A graph or digraph stored as an integer multiplicity matrix.

    The single source of truth is ``self.mu``, an ``n by n`` array of
    non-negative integers with a zero diagonal (no self-loops).  For an
    undirected variant the matrix is kept symmetric at all times, so the
    invariant ``mu[u, v] == mu[v, u]`` may be relied upon everywhere.

    All mutating methods respect the variant: they keep an undirected graph
    symmetric and never let a simple graph exceed multiplicity one.
    """

    def __init__(self, num_vertices: int, variant: Variant):
        if num_vertices < 0:
            raise ValueError("number of vertices must be non-negative")
        self.variant = variant
        # The whole graph is this one matrix; integer dtype keeps it exact.
        self.mu = np.zeros((num_vertices, num_vertices), dtype=int)

    # -- basic queries -------------------------------------------------------

    @property
    def num_vertices(self) -> int:
        """The number of vertices ``n``."""
        return self.mu.shape[0]

    def vertices(self) -> range:
        """The vertex set ``{0, 1, ..., n - 1}`` as a range."""
        return range(self.num_vertices)

    def multiplicity(self, u: int, v: int) -> int:
        """Number of parallel edges or arcs from ``u`` to ``v``."""
        return int(self.mu[u, v])

    def has_edge(self, u: int, v: int) -> bool:
        """Whether at least one edge or arc joins ``u`` to ``v``."""
        return self.mu[u, v] > 0

    def edge_count(self) -> int:
        """Total number of edges or arcs, counted with multiplicity.

```

```
        For an undirected graph each edge is stored twice in the matrix, so we
        count the strict upper triangle. For a digraph every ordered pair
        counts once.
        """
        if self.variant.directed:
            return int(self.mu.sum())
        # Strict upper triangle (k=1) avoids double-counting the symmetric pair.
        return int(np.triu(self.mu, k=1).sum())

    def out_degree(self, v: int) -> int:
        """Number of edges or arcs leaving ``v`` (with multiplicity)."""
        return int(self.mu[v, :].sum())

    def in_degree(self, v: int) -> int:
        """Number of edges or arcs entering ``v`` (with multiplicity)."""
        return int(self.mu[:, v].sum())

    def degree(self, v: int) -> int:
        """Total degree of ``v``.

        For a digraph this is the sum of in- and out-degree. For an undirected
        graph in- and out-degree coincide, so we report the out-degree.
        """
        if self.variant.directed:
            return self.out_degree(v) + self.in_degree(v)
        return self.out_degree(v)

    def edges(self) -> Iterator[tuple[int, int, int]]:
        """Yield ``(u, v, multiplicity)`` for every present edge or arc.

        For an undirected graph each edge is yielded once, with ``u < v``.
        """
        n = self.num_vertices
        for u in range(n):
            # Undirected: only scan v > u so each edge is emitted once.
            # Directed: scan all v so both arc directions are emitted.
            start = u + 1 if not self.variant.directed else 0
            for v in range(start, n):
                if u != v and self.mu[u, v] > 0:
                    yield u, v, int(self.mu[u, v])

    # -- mutation ------------------------------------------------------------

    def add_edge(self, u: int, v: int, count: int = 1) -> None:
        """Add ``count`` parallel edges or arcs from ``u`` to ``v``.

        A simple graph silently saturates at multiplicity one. An undirected
        graph updates both matrix entries to stay symmetric.
        """
        self._require_distinct(u, v)
        new_value = self.mu[u, v] + count
        if self.variant.simple:
            new_value = min(new_value, 1)  # enforce the 0/1 simple-graph cap
        self._assign(u, v, new_value)

    def remove_edge(self, u: int, v: int, count: int = 1) -> None:
        """Remove ``count`` parallel edges or arcs (never below zero)."""
        self._require_distinct(u, v)
```

```python
        new_value = max(0, self.mu[u, v] - count)
        self._assign(u, v, new_value)

    def set_multiplicity(self, u: int, v: int, value: int) -> None:
        """Set the multiplicity from ``u`` to ``v`` directly."""
        self._require_distinct(u, v)
        if value < 0:
            raise ValueError("multiplicity must be non-negative")
        if self.variant.simple and value > 1:
            raise ValueError("a simple graph cannot have multiplicity above one")
        self._assign(u, v, value)

    def copy(self) -> "Graph":
        """Return an independent copy with the same variant and edges."""
        clone = Graph(self.num_vertices, self.variant)
        clone.mu = self.mu.copy()  # deep-copy the matrix so edits don't alias
        return clone

    # -- internal helpers ---------------------------------------------------

    def _assign(self, u: int, v: int, value: int) -> None:
        """Write ``value`` into the matrix, mirroring it when undirected."""
        self.mu[u, v] = value
        # Maintain the symmetry invariant for undirected graphs in one place.
        if not self.variant.directed:
            self.mu[v, u] = value

    def _require_distinct(self, u: int, v: int) -> None:
        if u == v:
            raise ValueError("self-loops are not allowed in this model")

    def __repr__(self) -> str:
        return (f"Graph(n={self.num_vertices}, variant={self.variant.describe()!r}, "
                f"edges={self.edge_count()})")


# --- BOUNDS: closed-form extremal values; docstrings record proved/conjectured status ---


def simple_undirected_edge(n: int, m: int) -> int:
    """``ell_m(n) = floor(m (n-1) / 2)``.  Proved (Mader, 1973)."""
    return (m * (n - 1)) // 2


def multigraph_undirected_edge(n: int, m: int) -> int:
    """``L_m(n) = (m-1)(n-1)``.  Proved (multi-tree construction and bound)."""
    return (m - 1) * (n - 1)


def simple_undirected_vertex_le4(n: int, m: int) -> int:
    """``k_m(n) = floor(m (n-1) / 2)`` for ``m <= 4``.  Cited (Leonard, 1972)."""
    if m > 4:
        raise ValueError("this equality is only known for m <= 4")
    # For m <= 4 the vertex problem coincides with the edge problem.
    return simple_undirected_edge(n, m)

```

```python

def simple_undirected_vertex_m5(n: int) -> int:
    """``k_5(n) = floor(8n/3) - 3`` for large ``n``.  Cited (Sorensen-Thomassen)."""
    return (8 * n) // 3 - 3


def directed_arc_m2(n: int) -> int:
    """``ell_2^dir(n) = max(2(n-1), floor(n^2/4))``.  Proved (induction on n)."""
    # The max picks whichever of the two competing constructions wins at this n:
    # the linear hub (double star) versus the quadratic one-directional wall.
    return max(2 * (n - 1), (n * n) // 4)


def directed_arc_lower_bound(n: int, m: int) -> int:
    """Lower bound and conjectured value for ``ell_m^dir(n)``, ``m >= 2``.

    ``max(m(n-1), floor((n+m-2)^2/4))``.  The two branches are the hub
    construction (``const:directed-hub``) and the shifted-partition augmented
    bipartite construction (``const:augmented-bipartite``).  Proved as a lower
    bound for all ``m``; conjectured to be tight for ``m >= 3``
    (``conj:dir-arc``), proved tight for ``m = 2``.
    """
    hub_branch = m * (n - 1)
    bipartite_branch = (n + m - 2) ** 2 // 4
    return max(hub_branch, bipartite_branch)


def hypergraph_edge(n: int, m: int, r: int) -> int:
    """``floor((m-1)(n-1)/(r-1))`` hyperedges for the ``r``-uniform edge problem.

    Proved modulo the hypergraph Gomory-Hu theorem; the star hypertree attains
    the ``m = 2`` case and a sparse hypergraph attains the general bound.
    """
    return ((m - 1) * (n - 1)) // (r - 1)


def directed_multigraph_arc(n: int, m: int) -> int:
    """``L_m^dir(n) = 2(n-1)(m-1)``.  Proved for ``n <= 6`` (by counting cuts).

    Conjectured to continue to hold while the double star dominates the
    one-directional bipartite branch (small ``n``); for larger ``n`` the
    quadratic branch takes over.
    """
    double_star_branch = 2 * (n - 1) * (m - 1)
    bipartite_branch = (m - 1) * ((n * n) // 4)
    return max(double_star_branch, bipartite_branch)


# --- HYPERGRAPH: stored as incidence list; Berge-path connectivity via helper-node max-flow ---


class Hypergraph:
    """A hypergraph stored by its incidence (a list of hyperedges).

    An undirected hyperedge is a set of vertices, crossed in either direction.  A
    directed hyperedge (``directed=True``) splits an ``r``-set into a non-empty
```

```
    tail set ``T`` and a non-empty head set ``H``: a Berge route may enter it only
    at a tail and leave only at a head.  Two storage forms are accepted, and the
    checker treats them identically:

    * the legacy *forward* form ``(tail, frozenset(heads))`` with a single integer
      tail and ``r - 1`` heads (``|T| = 1``), the one-tail / many-head model the
      rest of the thesis uses; and
    * the *general* form ``(frozenset(tails), frozenset(heads))`` for any split,
      which subsumes forward (``|T| = 1``), backward (``|H| = 1``) and, from
      ``r >= 4``, genuinely mixed (``|T|, |H| >= 2``) hyperarcs.

    For ``r = 2`` every form collapses to a single arc, so a directed graph is the
    two-vertex special case.
    """

    def __init__(self, num_vertices: int, hyperedges: Iterable = (),
                 *, directed: bool = False):
        self.num_vertices = num_vertices
        self.directed = directed
        # Each entry is a frozenset, or a ``(tail, frozenset(heads))`` pair.
        self.hyperedges: list = []
        for edge in hyperedges:
            self.add_hyperedge(edge)

    def add_hyperedge(self, edge) -> None:
        """Add an undirected vertex set, or a directed ``(tail, heads)`` pair.

        A directed edge may be the forward form ``(tail:int, heads)`` or the
        general form ``(tails:frozenset, heads:frozenset)``; the general form is
        kept verbatim, the forward form is kept verbatim too (no normalisation),
        so existing forward results are unchanged byte-for-byte.
        """
        if self.directed:
            first, raw_heads = edge
            if isinstance(first, (set, frozenset)):     # general (tails, heads)
                tails, heads = frozenset(first), frozenset(raw_heads)
                members = tails | heads
                if (tails & heads) or not tails or not heads or len(members) < 2:
                    raise ValueError("a directed hyperedge needs disjoint, non-empty tail and head sets")
                if any(v < 0 or v >= self.num_vertices for v in members):
                    raise ValueError("hyperedge refers to a vertex outside the graph")
                self.hyperedges.append((tails, heads))
                return
            tail, heads = first, frozenset(raw_heads)    # legacy forward (tail, heads)
            members = heads | {tail}
            if tail in heads or len(members) < 2:
                raise ValueError("a directed hyperedge needs a tail and a distinct head")
            if any(v < 0 or v >= self.num_vertices for v in members):
                raise ValueError("hyperedge refers to a vertex outside the graph")
            self.hyperedges.append((tail, heads))
        else:
            vertices = frozenset(edge)  # a hyperedge is a set of distinct vertices
            if len(vertices) < 2:
```

```
                raise ValueError("a hyperedge must contain at least two vertices")
            if any(v < 0 or v >= self.num_vertices for v in vertices):
                raise ValueError("hyperedge refers to a vertex outside the graph")
            self.hyperedges.append(vertices)

    def members(self, edge) -> frozenset:
        """The vertex set of a stored hyperedge, directed or not."""
        if self.directed:
            tails, heads = _dir_tails_heads(edge)
            return tails | heads
        return edge

    def vertices(self) -> range:
        return range(self.num_vertices)

    def edge_count(self) -> int:
        """Number of hyperedges."""
        return len(self.hyperedges)

    def incident_hyperedges(self, v: int) -> list[int]:
        """Indices of the hyperedges containing vertex ``v``."""
        return [i for i, edge in enumerate(self.hyperedges) if v in self.members(edge)]


def _dir_tails_heads(edge) -> tuple[frozenset, frozenset]:
    """Tail and head sets of a stored directed hyperedge, as frozensets.

    Accepts the legacy forward form ``(tail:int, heads)`` and the general form
    ``(tails:frozenset, heads:frozenset)``, so every consumer of a directed
    hyperedge can read one ``(tails, heads)`` shape regardless of how it was
    built.
    """
    first, heads = edge
    if isinstance(first, (set, frozenset)):
        return frozenset(first), frozenset(heads)
    return frozenset((first,)), frozenset(heads)


def _hyper_capacity_matrix(hypergraph: Hypergraph, *, vertex_split: bool = False):
    """Integer capacity matrix of the hypergraph flow network (one per variant).

    Each hyperedge becomes an in/out gate joined by a capacity-1 arc, so at most
    one disjoint Berge route may traverse it: this is the hyperedge-as-node trick.
    With ``vertex_split`` each ORIGINAL vertex is *also* split into an in/out pair
    of capacity one, so the flow counts internally vertex-disjoint Berge routes
    instead of edge-disjoint ones.  For a directed hypergraph the gate is entered
    only from a tail and left only toward a head (one tail for the forward model,
    several for the general one); for an undirected one every member links to the
    gate both ways.  The hypergraph variants are thus the same construction with a
    boolean or two flipped, not separate measures.

    Vertices index ``0..base-1`` (``base = 2n`` split, else ``n``); hyperedge gate
    ``i`` indexes ``base+2i`` (in) and ``base+2i+1`` (out).  ``leave(v)``/``enter(v)``
    return the index a route leaves/enters vertex ``v`` through, so the same scipy
    max-flow that serves plain graphs serves hypergraphs too.
    """
```

```python
    n = hypergraph.num_vertices
    base = 2 * n if vertex_split else n
    size = base + 2 * len(hypergraph.hyperedges)
    cap = np.zeros((size, size), dtype=int)
    leave = (lambda v: 2 * v + 1) if vertex_split else (lambda v: v)
    enter = (lambda v: 2 * v) if vertex_split else (lambda v: v)
    if vertex_split:
        for v in range(n):
            cap[2 * v, 2 * v + 1] = 1            # one route through each vertex
    for index, edge in enumerate(hypergraph.hyperedges):
        gate_in, gate_out = base + 2 * index, base + 2 * index + 1
        cap[gate_in, gate_out] = 1               # one route through each hyperedge
        if hypergraph.directed:
            tails, heads = _dir_tails_heads(edge)
            for tail in tails:                   # enter the gate from any tail
                cap[leave(tail), gate_in] = _UNBOUNDED
            for head in heads:                   # leave the gate toward any head
                cap[gate_out, enter(head)] = _UNBOUNDED
        else:
            for vertex in edge:
                cap[leave(vertex), gate_in] = _UNBOUNDED
                cap[gate_out, enter(vertex)] = _UNBOUNDED
    return cap, size, leave, enter


def hyper_connectivity(hypergraph: Hypergraph, source: int, target: int,
                       *, vertex_split: bool = False) -> int:
    """Edge- or vertex-disjoint Berge routes from ``source`` to ``target``.

    A single scipy integer max-flow on the matrix above.  An isolated endpoint
    (in no hyperedge) has no incident arcs, so the flow is then zero.
    """
    cap, _, leave, enter = _hyper_capacity_matrix(hypergraph, vertex_split=vertex_split)
    if vertex_split:
        # Endpoints must not be the bottleneck: only interior vertices are capped.
        cap[2 * source, 2 * source + 1] = _UNBOUNDED
        cap[2 * target, 2 * target + 1] = _UNBOUNDED
    return int(_csgraph_maxflow(_csr(cap, dtype=int), leave(source), enter(target)).flow_value)


def max_hyper_connectivity(hypergraph: Hypergraph, *, vertex_split: bool = False) -> int:
    """``lambda^max`` (edge) or ``kappa^max`` (vertex) over all pairs."""
    return max((hyper_connectivity(hypergraph, source, target, vertex_split=vertex_split)
                for source, target in _pairs(hypergraph)), default=0)


def hyperedge_connectivity(hypergraph: Hypergraph, source: int, target: int) -> int:
    """Maximum number of edge-disjoint Berge ``source``-``target`` paths."""
    return hyper_connectivity(hypergraph, source, target, vertex_split=False)


def max_hyperedge_connectivity(hypergraph: Hypergraph) -> int:
```

```
    """``lambda^max`` for the undirected edge hypergraph problem."""
    return max_hyper_connectivity(hypergraph, vertex_split=False)


# --- CONSTRUCTIONS: named extremal graphs; each states its edge count and lambda^max ---


def double_star(n: int, m: int, directed: bool = True) -> Graph:
    """The double star: one hub joined to every leaf with multiplicity ``m-1``.

    For a digraph the hub carries ``m-1`` arcs in *both* directions to each
    leaf, attaining ``2(n-1)(m-1)`` arcs with ``lambda^max = m-1``.  For an
    undirected multigraph it attains ``(m-1)(n-1)`` edges.  This is the
    small-``n`` extremiser of the multigraph problems.
    """
    variant = MULTI_DIRECTED if directed else MULTI_UNDIRECTED
    graph = Graph(n, variant)
    hub = 0  # vertex 0 is the central hub; everything else is a leaf
    for leaf in range(1, n):
        graph.set_multiplicity(hub, leaf, m - 1)
        if directed:
            graph.set_multiplicity(leaf, hub, m - 1)  # arcs in both directions
    return graph


def one_directional_bipartite(n: int) -> Graph:
    """All arcs from one part to the other: ``floor(n^2/4)`` arcs, ``lambda^max = 1``.

    The vertices split into a smaller part ``A`` and a larger part ``B`` and
    every arc runs ``A -> B``.  No two vertices have two arc-disjoint routes, so
    this single-direction wall of arcs is feasible for ``m = 2`` while holding
    quadratically many arcs: the phenomenon that has no undirected analogue.
    """
    graph = Graph(n, SIMPLE_DIRECTED)
    size_a = n // 2  # smaller part; the n^2/4 maximum is at the balanced split
    part_a = range(size_a)
    part_b = range(size_a, n)
    for a in part_a:
        for b in part_b:
            graph.add_edge(a, b)  # every arc runs A -> B only (one direction)
    return graph


def augmented_bipartite(n: int, m: int) -> Graph:
    """The conjectured directed extremiser for ``m >= 2``.

    Set ``|B| = ceil((n+m-2)/2)`` and ``|A| = n - |B|``.  Fill every arc
    ``A -> B`` (the one-directional wall), then give each ``B``-vertex
    ``m - 2`` extra in-arcs from its circulant predecessors inside ``B``.
    Because no arc leaves ``B`` toward ``A``, a route between two ``B``
    vertices never escapes ``B``, and a route from ``A`` to ``b`` uses the
    direct arc plus one short detour through each of ``b``'s ``m - 2``
    in-arcs, giving ``lambda^max = m - 1``.  The arc count is
    ``floor((n+m-2)^2/4)`` (``const:augmented-bipartite`` in the thesis).

    Requires ``n >= m`` so that ``|A| >= 1`` and the circulant offsets stay
```

```
    distinct.  At ``m = 2`` or ``m = 3`` the partition equals the balanced
    ``(floor(n/2), ceil(n/2))`` split and the formula reduces to
    ``floor(n^2/4) + (m-2)ceil(n/2)``.  At ``m >= 4`` the shifted partition
    gives strictly more arcs.  For ``m = 3, n = 10`` this is the 30-arc
    counterexample.
    """
    size_b = (n + m - 2 + 1) // 2   # ceil((n+m-2)/2)
    size_a = n - size_b
    if size_a < 1:
        raise ValueError("augmented_bipartite needs n >= m")

    graph = Graph(n, SIMPLE_DIRECTED)
    part_a = range(size_a)
    part_b = list(range(size_a, n))

    # Complete wall of arcs from the small part to the large part.
    for a in part_a:
        for b in part_b:
            graph.add_edge(a, b)

    # Thicken B by a circulant giving every B-vertex in-degree exactly m - 2.
    # Each offset adds a full cycle of arcs within B; m - 2 offsets give each
    # B-vertex m - 2 extra in-arcs from other B-vertices (the short detours).
    for offset in range(1, m - 1):
        for i in range(size_b):
            source = part_b[i]
            target = part_b[(i + offset) % size_b]
            graph.add_edge(source, target)
    return graph


def clique_tree(clique_size: int, blocks_added: int) -> Graph:
    """A tree of cliques (a ``k``-tree): the chordal scaffold of the vertex case.

    Begin with a complete graph ``K_r`` on ``r = clique_size`` vertices, then
    repeatedly attach a new vertex to the previous ``r - 1`` vertices, which
    always form a clique.  The result is a maximal chordal graph of treewidth
    ``r - 1`` and *global* vertex connectivity ``r - 1``.

    A caution that the thesis makes explicit: the controlled quantity here is the
    global connectivity, not ``kappa^max``.  Attaching a vertex to a triangle
    raises the local connectivity of that triangle's vertices above ``r - 1``, so
    a ``k``-tree is not itself feasible for the ``kappa^max`` problem.  It appears
    in the thesis to illustrate the chordal structure underlying the
    Sorensen-Thomassen analysis of the ``m = 5`` divergence, not as a feasible
    extremiser.
    """
    if clique_size < 2:
        raise ValueError("clique_size must be at least two")
    n = clique_size + blocks_added
    graph = Graph(n, SIMPLE_UNDIRECTED)

    # The seed clique on the first r vertices.
    for u in range(clique_size):
        for v in range(u + 1, clique_size):
            graph.add_edge(u, v)

    # Each new vertex joins the r - 1 most recently added vertices.
```

```
    # Those r - 1 always form a clique, which keeps the graph chordal (a k-tree).
    for new_vertex in range(clique_size, n):
        attach_to = range(new_vertex - (clique_size - 1), new_vertex)
        for old_vertex in attach_to:
            graph.add_edge(new_vertex, old_vertex)
    return graph


def complete_uniform_hypergraph(n: int, r: int) -> Hypergraph:
    """The complete ``r``-uniform hypergraph: every ``r``-subset is a hyperedge.

    It is dense and highly connected; for example $K_4^{(3)}$ has Berge
    edge-connectivity three between every pair, exceeding the two-hyperedge count
    of the pairs because longer Berge routes also contribute.
    """
    # One hyperedge per r-subset of the vertex set.
    return Hypergraph(n, (set(subset) for subset in combinations(range(n), r)))


def star_hypertree(n: int, r: int) -> Hypergraph:
    """A hub joined to disjoint blocks: the feasible extremiser at ``m = 2``.

    Vertex ``0`` is the hub, and the remaining ``n - 1`` vertices are split into
    blocks of size ``r - 1``; each block together with the hub forms one
    hyperedge.  No two vertices share more than one hyperedge and the only routes
    between blocks pass through the single hub, so the Berge edge-connectivity is
    one everywhere, and the construction holds $\\lfloor (n-1)/(r-1)\\rfloor$
    hyperedges, the ``m = 2`` case of the hypergraph edge bound.
    """
    if r < 2:
        raise ValueError("hyperedge size r must be at least two")
    hub = 0
    others = list(range(1, n))
    hypergraph = Hypergraph(n)
    # Walk the non-hub vertices in blocks of r - 1; each block plus the hub is
    # one hyperedge (a "star" of r-sets all sharing the single hub).
    for start in range(0, len(others) - (r - 2), r - 1):
        block = others[start:start + (r - 1)]
        if len(block) == r - 1:  # drop a trailing partial block
            hypergraph.add_hyperedge([hub, *block])
    return hypergraph
```

C.1.3 Measuring, and proving

This part corresponds to Chapter 2. Measuring comes first. The question "how many independent routes join these two vertices" is answered exactly, by turning it into a maximum-flow problem and solving that, which is Menger's theorem used as an algorithm. The same routine answers all four versions of the question, because whether routes must avoid shared edges or shared intermediate vertices is controlled by one flag that decides how the flow network is built, and hypergraphs are handled by giving each hyperedge a gate of its own.

Proving comes second and is a different kind of work. To show that no graph of a given size can beat a bound, it is not enough to fail to find one. The program instead writes the whole question as a single optimisation problem whose variables describe an arbitrary graph and whose constraints say that every pair of vertices admits a small cut. If that problem has no solution, no such graph exists, and that is a proof rather than an absence of evidence.

```
######################################################################
##
##  CHAPTER 2 -- PROVING BOUNDS BY MACHINE
##  Measure connectivity exactly, then prove small-case upper bounds.
##
######################################################################

# --- CHECKER: exact max-flow connectivity; vertex_split=True gives kappa^max ---

# A capacity large enough to never be the bottleneck of any cut we build.
_UNBOUNDED = 10 ** 9


# ------------------------------------------------------------------
# One flow network and one measure, parameterised by edge vs vertex
# ------------------------------------------------------------------
# Every measure here is a single scipy integer max-flow on a capacity matrix
# (Menger).  EDGE mode runs directly on the multiplicity matrix mu: each unit of
# capacity is one parallel edge, so a max-flow of value f is f edge-disjoint
# routes.  VERTEX mode runs on the 2n x 2n split matrix (see _split_capacity_matrix)
# where each vertex becomes a capacity-one in/out gate, so a route uses a vertex at
# most once.  Edge vs vertex is one boolean, not a second code path.

def local_connectivity(graph: Graph, source: int, target: int,
                       *, vertex_split: bool = False) -> int:
    """Disjoint ``source``-``target`` routes by a single max-flow (Menger).

    Edge-disjoint with ``vertex_split=False``; internally vertex-disjoint with
    ``vertex_split=True``.  An isolated endpoint yields zero (no augmenting path).
    """
    if not vertex_split:
        # scipy.sparse.csgraph.maximum_flow is a C implementation; faster than
        # NetworkX for the small integer capacity matrices we use.
        return int(_csgraph_maxflow(_csr(graph.mu, dtype=int), source, target).flow_value)
    # Vertex mode: the same scipy max-flow on the split matrix.  Uncapping the two
    # endpoints' own in->out gates makes Menger count internally vertex-disjoint
    # routes (only interior vertices stay capacity-capped at one).
    cap, _ = _split_capacity_matrix(graph)
    cap[2 * source, 2 * source + 1] = _UNBOUNDED
    cap[2 * target, 2 * target + 1] = _UNBOUNDED
    return int(_csgraph_maxflow(_csr(cap, dtype=int), 2 * source + 1, 2 * target).flow_value)


def max_connectivity(graph: Graph, *, vertex_split: bool = False) -> int:
    """The largest local connectivity over all pairs: ``lambda^max`` (edge) or
    ``kappa^max`` (vertex).  For a digraph every ordered pair is examined, for an
    undirected graph each unordered pair once.
    """
    if not vertex_split:
        # Build the CSR matrix once and reuse it for every pair.
        csr = _csr(graph.mu, dtype=int)
```

```
        return max((int(_csgraph_maxflow(csr, s, t).flow_value)
                    for s, t in _pairs(graph)), default=0)
    return max((local_connectivity(graph, s, t, vertex_split=True)
                for s, t in _pairs(graph)), default=0)


# ------------------------------------------------------------------
# Named measures: thin views on the single path above (so the rest of the
# program can pass ``max_edge_connectivity`` / ``max_vertex_connectivity`` around
# by name -- just as ``hyperedge_connectivity`` names one branch of
# ``hyper_connectivity`` in the hypergraph section).
# ------------------------------------------------------------------

def local_edge_connectivity(graph: Graph, source: int, target: int) -> int:
    """Maximum number of edge-disjoint ``source``-``target`` paths."""
    return local_connectivity(graph, source, target, vertex_split=False)


def max_edge_connectivity(graph: Graph) -> int:
    """``lambda^max(G)``: the largest local edge-connectivity over all pairs."""
    return max_connectivity(graph, vertex_split=False)


def local_vertex_connectivity(graph: Graph, source: int, target: int) -> int:
    """Maximum number of internally vertex-disjoint ``source``-``target`` paths."""
    return local_connectivity(graph, source, target, vertex_split=True)


def max_vertex_connectivity(graph: Graph) -> int:
    """``kappa^max(G)``: the largest local vertex-connectivity over all pairs."""
    return max_connectivity(graph, vertex_split=True)


def min_vertex_connectivity(graph: Graph) -> int:
    """The global vertex connectivity ``kappa(G)``: the smallest over all pairs.

    This is the classical connectivity of a graph (the least number of vertices
    whose removal disconnects it).  It is distinct from ``kappa^max``: a ``k``-tree,
    for instance, is globally ``k``-connected yet has pairs with higher local
    connectivity.
    """
    pairs = list(_pairs(graph))
    if not pairs:
        return 0
    # Global connectivity is the MINIMUM local connectivity over all pairs.
    return min(local_vertex_connectivity(graph, source, target)
               for source, target in pairs)


# ------------------------------------------------------------------
# Shared helpers
# ------------------------------------------------------------------

def _pairs(obj):
    """Yield the vertex pairs to test: ordered if directed, unordered if not.

```

```python
    Works for both a ``Graph`` (directedness on ``variant``) and a ``Hypergraph``
    (directedness on the object), so one pair sweep serves every variant.
    """
    directed = obj.directed if hasattr(obj, "directed") else obj.variant.directed
    n = obj.num_vertices
    for source in range(n):
        # Directed objects need both (s, t) and (t, s); undirected need each
        # unordered pair once, so we start the inner loop above the source.
        start = 0 if directed else source + 1
        for target in range(start, n):
            if source != target:
                yield source, target


# ----------------------------------------------------------------------
# Capped feasibility predicate (used inside the search, Section 7)
# ----------------------------------------------------------------------
# The exact measures above answer "what is lambda^max / kappa^max?".  The search
# only ever needs the cheaper YES/NO question "is it greater than k?", because
# its energy depends on the value solely through the excess max(0, value-(m-1)).
# A capped flow answers exactly that and stops the moment the answer is known, so
# this predicate is equivalent to ``max_connectivity(graph, ...) > k`` but usually
# far cheaper: each pair's flow halts after k+1 augmenting paths, and the pair
# loop halts at the first violating pair.  The exact value is recomputed only on
# the rare step where the search genuinely needs it (an infeasible proposal).
# DEPENDENCIES: _pairs (above), _tiny_maxflow (Section ENUMERATE), _UNBOUNDED.

def _split_capacity_matrix(graph: Graph,
                           parallel_routes: bool = False) -> tuple[np.ndarray, int]:
    """The ``2n x 2n`` capacity matrix of the vertex-split network.

    The vertex-mode network as a plain matrix, consumed by both the exact measure
    (:func:`local_connectivity` via scipy max-flow) and the capped search predicate
    (:func:`exceeds_bound` via :func:`_tiny_maxflow`).  Vertex ``v`` becomes an
    in-copy ``2v`` and an out-copy ``2v+1`` joined by an internal arc of capacity
    one (so any route uses ``v`` at most once), and each adjacency ``u -> v``
    becomes an arc ``(2u+1) -> (2v)``.  The caller uncaps the two endpoints' own
    in->out gates so Menger counts internally disjoint routes (see
    :func:`exceeds_bound`).

    ``parallel_routes`` selects the counting convention for parallel copies, the
    choice discussed at length in the thesis (sec:parallel-convention).  The
    default False caps every adjacency at one, so a bundle of parallel edges is a
    single direct route and a multigraph measures as its underlying simple graph:
    that is the convention the twelve variants use.  Setting it True gives the
    adjacency capacity ``mu(u,v)``, so ``q`` parallel copies count as ``q``
    internally disjoint routes, which is the alternative convention explored in
    sec:multi-vertex-standard.
    """
    n = graph.num_vertices
    size = 2 * n
    cap = np.zeros((size, size), dtype=int)
    for v in range(n):
        cap[2 * v, 2 * v + 1] = 1            # internal gate: one route through v
    for u in range(n):
        for v in range(n):
```

```
            if u != v and graph.mu[u, v] > 0:
                cap[2 * u + 1, 2 * v] = (int(graph.mu[u, v]) if parallel_routes
                                         else 1)
    return cap, size


def max_multigraph_vertex_standard(n: int, m: int,
                                   deadline: float | None = None) -> tuple[int, Graph | None, bool]:
    """Exhaustive maximum for the multigraph vertex problem, OTHER convention.

    Maximises the total multiplicity of an undirected multigraph on ``n``
    vertices subject to ``kappa^max <= m - 1`` when parallel copies are counted
    as distinct internally vertex-disjoint routes.  Every multiplicity is capped
    at ``m - 1`` because ``m`` parallel copies already exceed the ceiling on
    their own.  Branch and bound over the pairs in a fixed order, trying the
    largest multiplicity first, pruning on feasibility (which is monotone) and
    on the incumbent.  Returns ``(best_total, witness, completed)``.
    """
    pairs = list(combinations(range(n), 2))
    cap = m - 1
    graph = Graph(n, MULTI_UNDIRECTED)
    best = [0, None]
    timed_out = [False]

    def feasible() -> bool:
        return not exceeds_bound(graph, m - 1, separation="vertex",
                                 parallel_routes=True)

    def recurse(i: int, total: int) -> None:
        if deadline is not None and time.time() > deadline:
            timed_out[0] = True
            return
        if total + cap * (len(pairs) - i) <= best[0]:
            return
        if i == len(pairs):
            if total > best[0]:
                best[0] = total
                best[1] = graph.copy()
            return
        u, v = pairs[i]
        for q in range(cap, -1, -1):
            graph.set_multiplicity(u, v, q)
            if q == 0 or feasible():
                recurse(i + 1, total + q)
            if timed_out[0]:
                break
        graph.set_multiplicity(u, v, 0)

    recurse(0, 0)
    return best[0], best[1], not timed_out[0]


def exceeds_bound(graph: Graph, k: int, *, separation: str = "edge",
                  parallel_routes: bool = False) -> bool:
    """``True`` iff ``lambda^max(G) > k`` (edge) or ``kappa^max(G) > k`` (vertex).

    Equivalent to ``max_connectivity(graph, vertex_split=(separation=='vertex')) >
```

```
    k``
    but with two early exits: each pair's capped flow stops after ``k+1`` augmenting
    paths, and the pair loop stops at the first violating pair.  On an infeasible
    graph this typically returns after a single pair.  The pair set is exactly the
    one :func:`max_connectivity` iterates, so the predicate matches it pair for pair.

    ``parallel_routes`` is passed through to :func:`_split_capacity_matrix` and
    only affects vertex mode: it selects the convention in which parallel copies
    are distinct routes, used by :func:`max_multigraph_vertex_standard`.
    """
    n = graph.num_vertices
    if separation == "edge":
        mu = graph.mu
        if _C is not None and n <= 16:
            flat, ptr = _c_flat(mu)
            directed = int(graph.variant.directed)
            return bool(_C.max_connectivity_exceeds(ptr, n, k, directed))
        for s, t in _pairs(graph):
            if _tiny_maxflow(mu, n, s, t, k):
                return True
        return False
    if separation == "vertex":
        cap, size = _split_capacity_matrix(graph, parallel_routes)
        for s, t in _pairs(graph):
            # Uncap the endpoints' own in->out gates so Menger counts INTERNAL
            # routes (mirrors local_connectivity's vertex mode); flow leaves s's
            # out-copy (2s+1) and arrives at t's in-copy (2t).
            cap[2 * s, 2 * s + 1] = _UNBOUNDED
            cap[2 * t, 2 * t + 1] = _UNBOUNDED
            hit = _tiny_maxflow(cap, size, 2 * s + 1, 2 * t, k)
            cap[2 * s, 2 * s + 1] = 1          # restore the gates for the next pair
            cap[2 * t, 2 * t + 1] = 1
            if hit:
                return True
        return False
    raise ValueError("separation must be 'edge' or 'vertex'")


# --- GOMORY-HU: encode all pairwise edge-connectivities in n-1 flows (undirected only) ---


def _undirected_capacity_graph(graph: Graph) -> nx.Graph:
    """Build the undirected capacitated graph that Gomory-Hu consumes."""
    if graph.variant.directed:
        raise ValueError("Gomory-Hu trees are defined for undirected graphs only")
    capacity_graph = nx.Graph()
    capacity_graph.add_nodes_from(graph.vertices())
    # One undirected edge per adjacency, capacity = multiplicity (its edge count).
    for u, v, multiplicity in graph.edges():
        capacity_graph.add_edge(u, v, capacity=float(multiplicity))
    return capacity_graph


def gomory_hu_tree(graph: Graph) -> nx.Graph:
```

```python
    """Return a Gomory-Hu tree of ``graph`` as a weighted ``networkx`` tree.

    Each tree edge carries a ``weight`` equal to the capacity of the minimum cut
    it represents.  This is the one place that genuinely needs networkx's
    Gomory-Hu routine; it backs a figure, not any reported bound.
    """
    _require_networkx("gomory_hu_tree")
    return nx.gomory_hu_tree(_undirected_capacity_graph(graph), capacity="capacity")


def max_edge_connectivity_via_tree(graph: Graph) -> int:
    """``lambda^max(G)`` read off the Gomory-Hu tree as its heaviest edge.

    Returns ``0`` for graphs with fewer than two vertices or no edges.
    """
    if graph.num_vertices < 2:
        return 0
    tree = gomory_hu_tree(graph)
    # READ-OFF: lambda(u, v) = lightest tree edge on the u-v path, so the LARGEST
    # such bottleneck over all pairs is simply the heaviest edge in the tree.
    weights = [data["weight"] for _, _, data in tree.edges(data=True)]
    if not weights:
        return 0
    return int(round(max(weights)))


# --- PROVE: MILP for M*(n) via cut-counting (scipy/HiGHS or Gurobi backend) ---
# L_m^dir(n) = (m-1)*M*(n) where M*(n) = max sum(w) s.t. maxflow(s,t;w)<=1 all pairs.
# Flow constraint encoded exactly: choose a cut x in {0,1}^n per pair, cap crossing
# weight via p; zero MIP gap = proof.  Shared helpers build constraints for both backends.


@dataclass
class ProofResult:
    """The outcome of a proof run."""

    n: int
    status: str                 # OPTIMAL, LIMIT, INFEASIBLE, UNBOUNDED, ...
    scaled_optimum: float       # M*(n) = max total weight, exact if status OPTIMAL
    relative_gap_zero: bool     # whether the solver closed the gap to zero
    solve_seconds: float
    weight_matrix: np.ndarray | None  # a witnessing matrix w, if one was found

    def value_for(self, m: int) -> int:
        """The proved directed multigraph value ``L_m^dir(n) = (m-1) M*(n)``."""
        # Undo the (m-1) scaling: one proved M*(n) yields every m.
        return (m - 1) * int(round(self.scaled_optimum))

    def is_proof(self) -> bool:
        """Whether this run is a genuine upper-bound proof (optimal, gap zero)."""
        # Only an OPTIMAL solve with a closed gap counts as a proof; a LIMIT
        # (timed out) result is merely a feasible lower bound, never a proof.
        return self.status == "OPTIMAL" and self.relative_gap_zero
```

```
def _ordered_pairs(n: int) -> list[tuple[int, int]]:
    return [(u, v) for u in range(n) for v in range(n) if u != v]


def _two_hop_triples(n: int) -> list[tuple[int, int, int]]:
    # All (source, middle, target) with three distinct vertices: the s -> x -> t
    # detours that the two-hop inequality reasons about.
    return [(s, x, t)
            for s in range(n) for t in range(n) if s != t
            for x in range(n) if x != s and x != t]


# Proved values of M*(k) for the deletion cuts below (proved: M*(k) =
# 2(k-1) for k <= 6, proved by this very optimisation with zero gap).
_PROVEN_MSTAR = {2: 2, 3: 4, 4: 6, 5: 8, 6: 10}


# -----------------------------------------------------------------
# The cut-counting MILP: one PuLP model for both provers, any solver
# -----------------------------------------------------------------
# The two provers below are the SAME cut-counting optimisation over two weight
# domains: the fractional one (prove_directed_multigraph, weights w in [0, 1], cut
# cap 1) maximises the total weight M*(n); the integral one (prove_integral_arc_
# bound, multiplicities mu in {0..m-1}, cut cap m-1) decides feasibility at a fixed
# arc target.  One builder serves both, and any MILP solver runs it: CBC (bundled
# with pulp) by default, Gurobi by a one-line switch.  Because the cut formulation
# is exact and not a relaxation, an OPTIMAL or INFEASIBLE verdict is a genuine proof.

_PULP_STATUS = {"Optimal": "OPTIMAL", "Infeasible": "INFEASIBLE",
                "Unbounded": "UNBOUNDED"}


def _pick_solver(time_limit: float, show_log: bool, use_gurobi: bool | None):
    """Choose the MILP solver.  ``gapRel=0`` demands a closed gap, which is what
    makes an OPTIMAL verdict a proof rather than a heuristic.

    ``use_gurobi``: ``True`` forces Gurobi (error if it is not usable), ``False``
    forces CBC, ``None`` uses Gurobi when its licence is active and CBC otherwise.
    Switching to Gurobi is the whole change needed to attack the open n=7 facts.
    """
    if use_gurobi is not False:
        gurobi = pulp.GUROBI_CMD(msg=show_log, timeLimit=time_limit,
                                 options=[("MIPGap", 0)])
        if gurobi.available():
            return gurobi
        if use_gurobi is True:
            raise RuntimeError(
                "use_gurobi=True but no usable Gurobi was found. Install gurobipy "
                "and activate a licence, then retry."
            )
    return pulp.PULP_CBC_CMD(msg=show_log, timeLimit=time_limit, gapRel=0.0)


def _cut_counting_model(n: int, *, cap: float, integer: bool, two_hop: bool,
```

```
                        symmetry: bool, deletion: bool, degree_pair: bool):
    """Build the shared cut-counting MILP as a PuLP model (no objective set yet).

    For every ordered pair (s, t) the model CHOOSES one cut: the side indicators
    fix s on the source side (constant 1) and t on the sink side (constant 0), and
    every other vertex takes a binary side.  The helper p picks up each crossing
    arc's weight through ``p >= w + cap*(x_u - x_v) - cap``, which forces ``p >= w``
    exactly on a crossing arc (x_u = 1, x_v = 0) and leaves p free at 0 otherwise.
    Capping the crossing total at ``cap`` says maxflow(s, t) <= cap, and letting the
    optimisation pick the cut means it picks the MINIMUM cut, so this is exactly
    maxflow(s, t) <= cap, the true feasible region.  ``cap=1`` with continuous
    weights is the scaled prover; ``cap=m-1`` with integer weights the arc one.

    The optional families are all proved-valid inequalities that tighten the
    relaxation without ever moving the optimum: ``two_hop`` (a two-arc-disjoint
    detour bound via z = min of the two hops), ``degree_pair`` (a flow lower bound
    on each pair), ``deletion`` (an induced-subgraph bound from proved smaller
    M*(k)), and ``symmetry`` (a degree ordering that prunes relabelled duplicates).
    Returns ``(prob, w)`` with ``w`` the weight variables keyed by ordered pair.
    """
    pairs = _ordered_pairs(n)
    prob = pulp.LpProblem("erdos915", pulp.LpMaximize)
    category = "Integer" if integer else "Continuous"
    w = {(u, v): pulp.LpVariable(f"w_{u}_{v}", 0, cap, cat=category)
         for (u, v) in pairs}

    for (s, t) in pairs:
        # s is always on the source side, t always on the sink side: constants, not
        # variables (which also avoids pulp resetting a Binary's bounds to [0, 1]).
        side = {u: 1 if u == s else 0 if u == t
                else pulp.LpVariable(f"x_{s}_{t}_{u}", cat="Binary")
                for u in range(n)}
        crossing = []
        for (u, v) in pairs:
            p_uv = pulp.LpVariable(f"p_{s}_{t}_{u}_{v}", 0, cap)
            prob += p_uv >= w[(u, v)] + cap * (side[u] - side[v]) - cap
            crossing.append(p_uv)
        prob += pulp.lpSum(crossing) <= cap          # crossing weight <= cap

    if two_hop:
        z = {}
        for (s, x, t) in _two_hop_triples(n):
            z[(s, x, t)] = pulp.LpVariable(f"z_{s}_{x}_{t}", 0, cap)
            selector = pulp.LpVariable(f"b_{s}_{x}_{t}", cat="Binary")
            # z = min(w[s,x], w[x,t]): two upper bounds, and the selector forces z
            # up to whichever argument is the minimum (big-M with M = cap).
            prob += z[(s, x, t)] <= w[(s, x)]
            prob += z[(s, x, t)] <= w[(x, t)]
            prob += z[(s, x, t)] >= w[(s, x)] - cap * (1 - selector)
            prob += z[(s, x, t)] >= w[(x, t)] - cap * selector
        for (s, t) in pairs:
            prob += w[(s, t)] + pulp.lpSum(
                z[(s, x, t)] for x in range(n) if x != s and x != t) <= cap
```

```python

    if degree_pair:
        # d+(s) + d-(t) - w[s,t] <= (n-1)*cap.  This is the two-hop bound
        # aggregated: w[s,t] + sum_x min(w[s,x], w[x,t]) <= cap, plus each middle
        # max(w[s,x], w[x,t]) <= cap, sums to (n-1)*cap.  It is a genuine cut, NOT
        # trivially satisfied: the box bound alone allows the LHS up to (2n-3)*cap,
        # and a two-route witness reaches 2(n-1)*cap (e.g. n=4: s->x1->t, s->x2->t
        # gives LHS 4 > 3).  Dominated by two_hop when that family is on, but
        # standalone-useful when it is off; valid in both weight domains.
        for (s, t) in pairs:
            prob += (pulp.lpSum(w[(s, x)] for x in range(n) if x != s)
                     + pulp.lpSum(w[(x, t)] for x in range(n) if x != t)
                     - w[(s, t)]) <= (n - 1) * cap

    if deletion:
        for k, holes in ((n - 1, 1), (n - 2, 2)):
            if k not in _PROVEN_MSTAR:
                continue
            bound = cap * _PROVEN_MSTAR[k]       # induced subgraph on k vertices
            for gone in combinations(range(n), holes):
                prob += pulp.lpSum(w[(a, b)] for (a, b) in pairs
                                   if a not in gone and b not in gone) <= bound

    if symmetry:
        degree = [pulp.lpSum(w[(u, v)] for (u, v) in pairs if v0 in (u, v))
                  for v0 in range(n)]
        for v0 in range(n - 1):
            prob += degree[v0] <= degree[v0 + 1]

    return prob, w


def prove_directed_multigraph(
    n: int,
    *,
    time_limit: float = 1500.0,
    use_gurobi: bool | None = None,
    use_two_hop: bool = True,
    use_symmetry_breaking: bool = True,
    use_deletion_cuts: bool = False,
    show_solver_log: bool = False,
) -> ProofResult:
    """Prove ``M*(n)`` for the directed multigraph arc problem.

    Returns a :class:`ProofResult`.  When its :meth:`is_proof` is true the value
    is a proven upper bound, and ``value_for(m)`` gives ``L_m^dir(n)`` for every
    ``m``.  ``use_gurobi`` picks the solver (see :func:`_pick_solver`); the default
    uses Gurobi when its licence is active and CBC otherwise.
    """
    _require_pulp()
    prob, w = _cut_counting_model(
        n, cap=1.0, integer=False, two_hop=use_two_hop,
        symmetry=use_symmetry_breaking, deletion=use_deletion_cuts,
        degree_pair=use_deletion_cuts,
    )
    prob += pulp.lpSum(w.values())                    # maximise total weight = M*(n
```

```
)

    start = time.time()
    prob.solve(_pick_solver(time_limit, show_solver_log, use_gurobi))
    elapsed = time.time() - start
    status = _PULP_STATUS.get(pulp.LpStatus[prob.status], "LIMIT")

    weight_matrix = None
    if status == "OPTIMAL":
        weight_matrix = np.zeros((n, n))
        for (u, v), variable in w.items():
            weight_matrix[u, v] = variable.value() or 0.0

    return ProofResult(
        n=n, status=status,
        scaled_optimum=(pulp.value(prob.objective) if status == "OPTIMAL"
                        else float("nan")),
        relative_gap_zero=(status == "OPTIMAL"),
        solve_seconds=elapsed, weight_matrix=weight_matrix,
    )
```

C.1.4 Searching

This part corresponds to Chapter 3. Where proving is out of reach, the program looks for good examples instead. It starts from a graph, repeatedly makes a small random change, and keeps the change if it helps. Keeping only improvements gets stuck in the first decent shape it finds, so early on it also accepts some changes that make things worse, and it does this less and less as it goes, in the manner of hot metal cooling into an ordered state. A second engine, tabu search, walks the same landscape with a different rule: it always takes the best available move but refuses to revisit recent positions, which is what stops it circling.

Two ideas make this fast enough to be useful. The search is told which edges are load-bearing, so that it changes those rather than wasting moves on edges that carry nothing. And it almost never measures connectivity fully: adding one connection can raise the count by at most one and removing one can never raise it at all, so most steps can be settled by arithmetic on a bound the search already carries, with an exact measurement only when the answer is genuinely in doubt. This part ends with `solve`, the single entry point that either proves or searches depending on one argument.

```
###########################################################################
##
##  CHAPTER 3 -- DISCOVERING BOUNDS BY SEARCH
##  The temperature search, and the one driver that proves or discovers.
##
###########################################################################

# --- SENSITIVITY: sigma(e) = lambda^max(G) - lambda^max(G-e); dial for the cooling schedule ---

# A connectivity measure maps a graph to its lambda^max or kappa^max.
ConnectivityMeasure = Callable[[Graph], int]
```

```python

def edge_sensitivity(
    graph: Graph,
    u: int,
    v: int,
    connectivity: ConnectivityMeasure = max_edge_connectivity,
) -> int:
    """Return ``sigma(e)`` for the edge or arc ``e = (u, v)``.

    The edge is deleted in full (all parallel copies) before remeasuring, so the
    result reflects the structural role of the adjacency rather than of a single
    parallel copy.  If the edge is absent the sensitivity is zero by convention.
    """
    if not graph.has_edge(u, v):
        return 0  # absent edge: nothing to remove, sensitivity is zero
    before = connectivity(graph)
    reduced = graph.copy()
    reduced.set_multiplicity(u, v, 0)  # delete the WHOLE adjacency (all copies)
    after = connectivity(reduced)
    # How much lambda^max dropped: positive means e was load-bearing.
    return before - after


def sensitivity_map(
    graph: Graph,
    connectivity: ConnectivityMeasure = max_edge_connectivity,
) -> dict[tuple[int, int], int]:
    """Return ``{(u, v): sigma((u, v))}`` over all present edges or arcs.

    Useful for visualising an extremiser: high-sensitivity edges are the load
    bearers, low-sensitivity edges are slack.

    The baseline connectivity ``before = connectivity(graph)`` is the SAME for
    every edge, so it is measured once here rather than re-measured inside each
    :func:`edge_sensitivity` call.  This changes no value (the result equals
    ``{e: edge_sensitivity(graph, *e) for e in graph.edges()}``); it only avoids
    recomputing the shared baseline once per edge.
    """
    before = connectivity(graph)
    result: dict[tuple[int, int], int] = {}
    for u, v, _ in graph.edges():
        reduced = graph.copy()
        reduced.set_multiplicity(u, v, 0)   # delete the WHOLE adjacency (all copies)
        result[(u, v)] = before - connectivity(reduced)
    return result


# --- SEARCH: simulated annealing; E = -|E(G)| + penalty*max(0,lambda^max-(m-1)) ---
# Metropolis rule with geometric cooling; removal proposals biased by edge sensitivity.


@dataclass
class SearchStep:
    """One step of the search, recorded for plotting the temperature trace."""

```

```python
    step: int
    temperature: float
    energy: float
    edge_count: int
    connectivity: int
    accepted: bool
    seconds: float = 0.0        # wall-clock since the search began (for timed traces)
    best_feasible: int = 0      # densest feasible graph seen so far (for convergence)


@dataclass
class SearchResult:
    """The outcome of a search: the best feasible graph and the full trace."""

    best_graph: Graph
    best_edge_count: int
    feasible_found: bool
    history: list[SearchStep] = field(default_factory=list)

    def acceptance_rate(self) -> float:
        """Fraction of proposed moves that were accepted."""
        if not self.history:
            return 0.0
        return sum(record.accepted for record in self.history) / len(self.history)


def _connectivity_measure(separation: str):
    """Return the connectivity function named by ``separation``."""
    if separation == "edge":
        return max_edge_connectivity
    if separation == "vertex":
        return max_vertex_connectivity
    raise ValueError("separation must be 'edge' or 'vertex'")


def _energy(graph: Graph, m: int, measure, penalty: float) -> float:
    """The energy ``-|E| + penalty * excess connectivity``."""
    # Excess is how far connectivity exceeds the allowed m-1; zero when feasible.
    excess = max(0, measure(graph) - (m - 1))
    # Lower energy is better: more edges (negative term) and no excess.
    return -graph.edge_count() + penalty * excess


def _proposal_energy(
    proposal: Graph,
    *,
    is_add: bool,
    feasible_current: bool,
    lam_ub: int,
    m: int,
    penalty: float,
    measure_full,
    separation: str,
) -> tuple[float, bool]:
    """Return ``(_energy(proposal), proposal_is_feasible)`` doing the least work.

```

```
    The energy ``-|E| + penalty * max(0, connectivity - (m-1))`` depends on the
    connectivity only through the excess, and by monotonicity (Facts M1/M2 in the
    thesis) that excess is provably zero on most steps, so no flow is needed:

    - a removal from a feasible graph stays feasible (M1), excess 0;
    - an addition while ``lam_ub <= m-2`` stays feasible (M2), excess 0.

    Only when neither shortcut applies do we run one capped predicate, and only when
    that reports an infeasible proposal do we spend the exact ``measure_full`` to get
    the penalty term right.  The float returned is byte-for-byte the one ``_energy``
    would return (the same arithmetic on the same excess), so the search trajectory
    is unchanged.  The boolean is the EXACT feasibility of ``proposal``.
    """
    edges = proposal.edge_count()
    # Decide excess = max(0, measure(proposal) - (m-1)) with the least work.
    if (is_add and lam_ub <= m - 2) or (not is_add and feasible_current):
        excess = 0                          # M2 (add) or M1 (remove): provably feasible
    elif not exceeds_bound(proposal, m - 1, separation=separation):
        excess = 0                          # at the boundary but the capped flow clears it
    else:
        excess = measure_full(proposal) - (m - 1)   # infeasible: exact penalty term
    # Identical arithmetic to _energy, so the returned float matches it exactly.
    return -edges + penalty * excess, excess == 0


def _candidate_pairs(graph: Graph) -> list[tuple[int, int]]:
    """All ordered pairs (directed) or unordered pairs (undirected)."""
    n = graph.num_vertices
    pairs = []
    for u in range(n):
        start = 0 if graph.variant.directed else u + 1
        for v in range(start, n):
            if u != v:
                pairs.append((u, v))
    return pairs


def _propose_removal(
    graph: Graph,
    temperature: float,
    rng: random.Random,
    cached_sensitivity: dict[tuple[int, int], int] | None,
) -> tuple[int, int]:
    """Choose a present edge to thin out, biased toward free (low sigma) edges.

    Without a sensitivity cache the choice is uniform.  With one, edge ``e`` is
    drawn with weight ``exp(-sigma(e) / T)``: at high ``T`` the weights flatten
    (uniform exploration), at low ``T`` they concentrate on ``sigma = 0`` edges.
    """
    present = [(u, v) for u, v, _ in graph.edges()]
    if cached_sensitivity is None:
```

```
        return rng.choice(present)  # no guidance: uniform removal
    # exp(-sigma/T): big sigma (load-bearing) is suppressed, and the suppression
    # sharpens as T -> 0.  max(T, eps) guards against division by zero.
    weights = [
        math.exp(-cached_sensitivity.get((u, v), 0) / max(temperature, 1e-9))
        for (u, v) in present
    ]
    return rng.choices(present, weights=weights, k=1)[0]


def search_for_dense_graph(
    variant: Variant,
    n: int,
    m: int,
    *,
    separation: str = "edge",
    steps: int = 4000,
    initial_temperature: float = 3.0,
    cooling: float = 0.9985,
    penalty: float = 6.0,
    max_multiplicity: int | None = None,
    sensitivity_guided: bool = True,
    bias_refresh: int = 200,
    seed: int = 0,
    deadline: float | None = None,
    record_exact_connectivity: bool = False,
    reference_mode: bool = False,
) -> SearchResult:
    """Find the densest feasible graph by a guided random search (simulated annealing).

    Args:
        variant: which graph model to search in.
        n: number of vertices.
        m: forbidden value. Feasibility means ``connectivity <= m - 1``.
        separation: ``"edge"`` for arc/edge-disjoint, ``"vertex"`` for
            internally vertex-disjoint paths.
        steps: number of proposed moves.
        initial_temperature: starting temperature ``T_0``.
        cooling: factor (< 1) the temperature is multiplied by each step.
        penalty: weight of the connectivity violation in the energy.
        max_multiplicity: cap on parallel edges (defaults to ``m - 1``, a simple
            variant is always capped at one regardless).
        sensitivity_guided: bias removals toward free edges (the temperature
            dial described in the module docstring).
        bias_refresh: recompute the sensitivity cache every this many steps.
        seed: random seed, fixed so figures are reproducible.
        record_exact_connectivity: log the EXACT ``lambda^max`` per step in
            ``SearchStep.connectivity`` (the figure-trace needs it).  When False,
            the cheaper maintained upper bound ``lam_ub`` is logged instead, which
            does not affect the search at all (the energy never reads this field).
        reference_mode: INTERNAL, test only.  Force the slow, exact per-step
            energy (call :func:`_energy` and :func:`max_connectivity` directly)
            instead of the capped/monotone fast path.  Used by the equivalence
            test to prove the fast path reproduces this reference step for step.

    Returns:
        An :class:`SearchResult` with the best feasible graph found and the
```

```
        full step-by-step history.

    The fast path is *trajectory-preserving*: it computes the identical per-step
    energy as ``reference_mode`` (proven by ``test_search.test_fast_path_matches_exact``),
    so every accept/reject decision, the random-number draw order, and the returned
    witness are unchanged.  It only avoids flows whose result the energy cannot
    depend on (Facts M1/M2: a removal from a feasible graph stays feasible, an
    addition while ``lam_ub <= m-2`` stays feasible), and resyncs the upper bound
    ``lam_ub`` to the exact value at the ``bias_refresh`` cadence.
    """
    rng = random.Random(seed)
    measure = _connectivity_measure(separation)
    # Multiplicity cap: simple graphs are always 0/1; multigraphs default to m-1.
    cap = 1 if variant.simple else (max_multiplicity if max_multiplicity else m - 1)
    pairs = None  # built lazily once we know we need it

    current = Graph(n, variant)  # start from the empty graph: feasible, energy 0
    current_energy = _energy(current, m, measure, penalty)

    best_graph = current.copy()
    best_edges = 0
    feasible_found = True  # the empty graph is feasible

    # State carried across steps so the energy can avoid recomputing connectivity
    # from scratch (see the trajectory-preserving note in the docstring):
    #   feasible -- is `current` feasible (lambda^max <= m-1)?  Exact at all times.
    #   lam_ub   -- a SOUND upper bound on lambda^max(current).  Facts M1/M2 keep it
    #               sound: +1 on an accepted addition, unchanged on a removal.
    feasible = True             # the empty graph is feasible
    lam_ub = 0                  # lambda^max(empty graph) = 0

    temperature = initial_temperature
    cached_sensitivity = None
    history: list[SearchStep] = []
    clock_start = time.perf_counter()  # for the timed convergence trace only

    for step in range(steps):
        # Periodically recompute the sensitivity map that guides removals.  It
        # is expensive, so we cache it and refresh only every bias_refresh steps.
        if sensitivity_guided and step % bias_refresh == 0 and current.edge_count() > 0:
            cached_sensitivity = sensitivity_map(current, measure)
        # Resync the upper bound to the truth at the same cadence (the bound can
        # only drift upward over a run of removals, which costs extra capped checks
        # but never correctness).  This step already pays for flows above, and the
        # resync touches neither the energy nor the RNG, so the trajectory is safe.
        if step % bias_refresh == 0:
            lam_ub = measure(current)
            feasible = lam_ub <= m - 1

        proposal = current.copy()
```

```
        # Flip a coin: remove an edge (only if some exist) or add one.
        removing = current.edge_count() > 0 and rng.random() < 0.5
        if removing:
            # Sensitivity-guided removal: prefers free edges, especially cold.
            u, v = _propose_removal(current, temperature, rng, cached_sensitivity)
            proposal.remove_edge(u, v)
        else:
            if pairs is None:
                pairs = _candidate_pairs(current)
            # Only pairs that have room under the multiplicity cap can grow.
            addable = [(u, v) for (u, v) in pairs if proposal.multiplicity(u, v) < cap]
            if not addable:
                continue  # nothing to add; skip this step
            u, v = rng.choice(addable)
            proposal.add_edge(u, v)
        is_add = not removing

        # The proposal's energy and exact feasibility.  reference_mode is the slow ,
        # exact path the equivalence test compares against; the fast path returns
        # the IDENTICAL energy by Facts M1/M2 (see _proposal_energy).
        if reference_mode:
            proposal_energy = _energy(proposal, m, measure, penalty)
            proposal_feasible = measure(proposal) <= m - 1
        else:
            proposal_energy, proposal_feasible = _proposal_energy(
                proposal, is_add=is_add, feasible_current=feasible, lam_ub=lam_ub,
                m=m, penalty=penalty, measure_full=measure, separation=separation)
        change = proposal_energy - current_energy
        # ACCEPT OR NOT: always take an improvement (change <= 0); take a worsening
        # move only with probability exp(-change / T).  As T falls this probability
        # shrinks, so the walk increasingly refuses to go uphill.
        accept = change <= 0 or rng.random() < math.exp(-change / max(temperature, 1e-9))

        if accept:
            current, current_energy = proposal, proposal_energy
            # Carry the exact feasibility and update the bound by M2 (an accepted
            # addition can raise lambda^max by at most 1; a removal cannot raise it).
            feasible = proposal_feasible
            if is_add:
                lam_ub += 1
            # Track the best FEASIBLE graph found (within bound and denser than any
            # seen so far): this is the lower-bound witness.  `feasible` is exact
            # here, so the witness is certified with no extra flow.
            if feasible and current.edge_count() > best_edges:
                best_graph, best_edges = current.copy(), current.edge_count()
                feasible_found = True

        # The connectivity field feeds fig:trace only.  Log the exact value when the
        # figure asks for it (or in reference_mode), else the maintained upper bound.
```

```python
        if record_exact_connectivity or reference_mode:
            recorded_connectivity = measure(current)
            # Cheap audit in the exact paths: the maintained bound must never
            # underestimate, and the feasibility flag must match the truth.
            assert lam_ub >= recorded_connectivity, (lam_ub, recorded_connectivity
                ↪ )
            assert feasible == (recorded_connectivity <= m - 1)
        else:
            recorded_connectivity = lam_ub
        history.append(SearchStep(
            step=step,
            temperature=temperature,
            energy=current_energy,
            edge_count=current.edge_count(),
            connectivity=recorded_connectivity,
            accepted=accept,
            seconds=time.perf_counter() - clock_start,
            best_feasible=best_edges,
        ))
        temperature *= cooling  # cool down: T <- cooling * T each step

        # Respect the deadline when supplied; check every 100 steps to amortise
        # the time.time() call.
        if deadline is not None and step % 100 == 99 and time.time() > deadline:
            break

    return SearchResult(best_graph, best_edges, feasible_found, history)


def _neighbour_moves(graph: Graph, cap: int) -> list[tuple[int, int, int]]:
    """Every single add (``+1``) or remove (``-1``) of one edge/arc under the cap.

    The shared neighbourhood of both discoverers: one ordered pair for a digraph,
    one unordered pair for an undirected graph, addable while below the
    multiplicity ``cap`` and removable while above zero.
    """
    n = graph.num_vertices
    directed = graph.variant.directed
    moves: list[tuple[int, int, int]] = []
    for u in range(n):
        for v in range(n):
            if u == v or (not directed and v < u):
                continue
            mult = graph.multiplicity(u, v)
            if mult < cap:
                moves.append((u, v, +1))
            if mult > 0:
                moves.append((u, v, -1))
    return moves


def tabu_search_for_dense_graph(
    variant: Variant,
    n: int,
    m: int,
    *,
    separation: str = "edge",
    steps: int = 4000,
```

```
    penalty: float = 6.0,
    tenure: int = 7,
    stall_limit: int = 40,
    max_multiplicity: int | None = None,
    seed: int = 0,
    deadline: float | None = None,
    record_exact_connectivity: bool = True,
) -> SearchResult:
    """Find the densest feasible graph by tabu search, the deterministic twin of
    :func:`search_for_dense_graph`.

    Same energy ``-|E| + penalty * max(0, connectivity - (m-1))`` and the same
    neighbourhood (:func:`_neighbour_moves`) as the annealer, but a different
    engine.  Each step scans the whole neighbourhood and takes the best move whose
    pair is not on the tabu list, forbids that pair for ``tenure`` steps, and
    perturbs from the incumbent once it has stalled for ``stall_limit`` steps.
    Aspiration overrides the tabu flag for any move that beats the global best.
    There is no temperature and no accept-worse coin: the only randomness is the
    seed driving the perturbation kicks, so two runs from one seed are identical.

    Returns the same :class:`SearchResult` as the annealer (the best feasible graph
    and the full timed history), so the two engines are interchangeable through the
    ``method`` argument of :func:`best_of_searches` and :func:`solve`.
    """
    rng = random.Random(seed)
    measure = _connectivity_measure(separation)
    cap = 1 if variant.simple else (max_multiplicity if max_multiplicity else m - 1)

    current = Graph(n, variant)             # start empty: feasible, energy 0
    best_graph, best_edges = current.copy(), 0
    tabu: dict[tuple[int, int], int] = {}
    stall = 0
    history: list[SearchStep] = []
    clock_start = time.perf_counter()

    def trial_energy_and_feasibility(graph: Graph) -> tuple[float, bool]:
        """The annealer's energy and exact feasibility, by the same capped trick.

        ``exceeds_bound`` decides feasibility with an early-exit capped flow, and
        the full ``measure`` runs only on the rare infeasible trial that needs its
        penalty term, so most moves cost one cheap predicate instead of a full
        all-pairs flow.  The returned float equals ``_energy`` exactly.
        """
        edges = graph.edge_count()
        if not exceeds_bound(graph, m - 1, separation=separation):
            return float(-edges), True               # feasible: excess is zero
        return -edges + penalty * (measure(graph) - (m - 1)), False

    for step in range(steps):
        if deadline is not None and time.time() > deadline:
            break
        best_move: tuple[int, int, int] | None = None
        best_move_energy: float | None = None
        best_move_global = False
        for (u, v, d) in _neighbour_moves(current, cap):
```

```
            trial = current.copy()
            trial.add_edge(u, v) if d > 0 else trial.remove_edge(u, v)
            trial_energy, trial_feasible = trial_energy_and_feasibility(trial)
            improves_global = trial_feasible and trial.edge_count() > best_edges
            if tabu.get((u, v), 0) > step and not improves_global:
                continue                    # tabu, with no aspiration to override
            if (best_move_energy is None or trial_energy < best_move_energy
                    or (improves_global and not best_move_global)):
                best_move, best_move_energy = (u, v, d), trial_energy
                best_move_global = improves_global
        if best_move is None:               # every move tabu: clear the list, retry
            tabu.clear()
            continue

        u, v, d = best_move
        current.add_edge(u, v) if d > 0 else current.remove_edge(u, v)
        tabu[(u, v)] = step + tenure

        connectivity = measure(current)     # computed anyway for best-tracking
        if connectivity <= m - 1 and current.edge_count() > best_edges:
            best_graph, best_edges = current.copy(), current.edge_count()
            stall = 0
        else:
            stall += 1
        history.append(SearchStep(
            step=step, temperature=0.0, energy=best_move_energy,
            edge_count=current.edge_count(),
            connectivity=connectivity if record_exact_connectivity else -1,
            accepted=True,                  # tabu always takes its best legal move
            seconds=time.perf_counter() - clock_start,
            best_feasible=best_edges,
        ))

        if stall >= stall_limit:            # perturb from the incumbent and reset
            current = best_graph.copy()
            for _ in range(rng.randint(2, 4)):
                kicks = _neighbour_moves(current, cap)
                if kicks:
                    a, b, dd = rng.choice(kicks)
                    current.add_edge(a, b) if dd > 0 else current.remove_edge(a, b)
            tabu.clear()
            stall = 0

    return SearchResult(best_graph, best_edges, feasible_found=True, history=history)


def _run_one_search(args: tuple) -> SearchResult:
    """Module-level worker for parallel restarts (must be picklable for ProcessPoolExecutor)."""
    variant, n, m, seed, method, search_options = args
    engine = tabu_search_for_dense_graph if method == "tabu" else search_for_dense_graph
    return engine(variant, n, m, seed=seed, **search_options)


```

```python
def best_of_searches(
    variant: Variant,
    n: int,
    m: int,
    *,
    restarts: int = 6,
    seed: int = 0,
    parallel: bool = True,
    method: str = "sa",
    **search_options,
) -> SearchResult:
    """Run several independent discovery schedules and keep the densest result.

    A single search can settle into a local optimum, so for reliable
    rediscovery we restart from a fresh random seed a handful of times and return
    the run that found the most edges.  Any keyword understood by the chosen
    engine (``steps``, ``penalty``, ``separation``, and so on) is forwarded
    unchanged.

    Args:
        parallel: when ``True`` (default), run the restarts concurrently using
            :class:`~concurrent.futures.ProcessPoolExecutor`.  Each restart has
            a distinct seed so the runs are genuinely independent.  Falls back
            to sequential execution if multiprocessing is unavailable.
        method: ``"sa"`` (default, simulated annealing,
            :func:`search_for_dense_graph`) or ``"tabu"``
            (:func:`tabu_search_for_dense_graph`).  Both optimise the same energy
            over the same neighbourhood and return the same result type.
    """
    if method not in ("sa", "tabu"):
        raise ValueError("method must be 'sa' or 'tabu'")
    task_args = [(variant, n, m, seed + r, method, search_options) for r in range(restarts)]

    if parallel and restarts > 1:
        try:
            with concurrent.futures.ProcessPoolExecutor() as executor:
                results = list(executor.map(_run_one_search, task_args))
        except (OSError, concurrent.futures.BrokenExecutor) as exc:
            # Multiprocessing is genuinely unavailable here (a sandbox with no
            # fork/spawn, or a worker that could not start).  Fall back to a
            # sequential run, but say so rather than masking the loss of cores.
            # A bug inside the worker is NOT caught here, so it fails loudly.
            warnings.warn(
                f"parallel restarts unavailable ({exc!r}); running sequentially",
                RuntimeWarning, stacklevel=2,
            )
            results = [_run_one_search(a) for a in task_args]
    else:
        results = [_run_one_search(a) for a in task_args]

    return max(results, key=lambda r: r.best_edge_count)


# --- SOLVE: unified driver; exhaustive=True proves, exhaustive=False discovers ---


```

```python
@dataclass
class SolveResult:
    """The outcome of a solve, carrying an honest bound label."""

    n: int
    m: int
    variant: str            # human label, e.g. "simple directed"
    separation: str         # "edge" or "vertex"
    value: int              # the extremal count we report
    bound: str              # "exact" | "lower" | "upper"
    method: str             # how the value was obtained
    seconds: float
    complete: bool          # an exhaustive run that actually finished
    witness: object | None  # a Graph/Hypergraph attaining ``value``, if any
    note: str = ""

    @property
    def proven(self) -> bool:
        """Whether ``value`` is a proved exact extremal value."""
        return self.bound == "exact"

    def describe(self) -> str:
        """One readable line summarising the result."""
        relation = {"exact": "= ", "lower": ">= ", "upper": "<= "}[self.bound]
        line = (f"{self.variant} ({self.separation}), n={self.n}, m={self.m}: "
                f"value {relation}{self.value}  "
                f"[{self.bound}, {self.method}, {self.seconds:.1f}s]")
        return line if not self.note else f"{line}\n    note: {self.note}"


def _variant_for(directed: bool, simple: bool) -> Variant:
    """Pick the matching named Variant constant from the two booleans."""
    if directed:
        return SIMPLE_DIRECTED if simple else MULTI_DIRECTED
    return SIMPLE_UNDIRECTED if simple else MULTI_UNDIRECTED


def _matrix_cells(n: int, directed: bool) -> list[tuple[int, int]]:
    """The off-diagonal matrix cells a graph of this kind may fill.

    Directed graphs vary every ordered pair; undirected graphs vary only the
    upper triangle, since ``set_multiplicity`` mirrors the lower half for us.
    """
    if directed:
        return [(u, v) for u in range(n) for v in range(n) if u != v]
    return [(u, v) for u in range(n) for v in range(n) if u < v]


def _directed_witness(n: int, m: int, simple: bool) -> Graph | None:
    """The densest named directed construction that is feasible for ``(n, m)``.

    Used to exhibit a concrete graph attaining the proved value: the better
    of the double star and the (augmented) one-directional bipartite, or
    ``None`` if neither applies to this case.
    """
    candidates: list[Graph] = []
    if not (simple and m > 2):              # a simple star needs m == 2
        candidates.append(double_star(n, m, directed=True))   # 2(n-1)(m-1) arcs
```

```
    if m == 2:
        candidates.append(one_directional_bipartite(n))       # floor(n^2/4) arcs
    elif (m - 2) < (n - n // 2):            # augmented needs m-2 < ceil(n/2)
        candidates.append(augmented_bipartite(n, m))
    feasible = [g for g in candidates if max_edge_connectivity(g) <= m - 1]
    return max(feasible, key=lambda g: g.edge_count()) if feasible else None


def _brute_force_matrix(
    variant: Variant, n: int, m: int, separation: str, deadline: float,
) -> tuple[int, Graph | None, bool]:
    """Exhaustively maximise edges over every graph of ``variant`` on ``n``.

    Returns ``(best_edge_count, witness, completed)``.  ``completed`` is False if
    the budget ran out first, in which case the value is only a lower bound.
    A simple cell ranges over ``{0, 1}``; a multigraph cell over ``{0, ..., m-1}``
    because multiplicity ``m`` already gives ``m`` parallel disjoint routes.
    """
    measure = _connectivity_measure(separation)
    cells = _matrix_cells(n, variant.directed)
    span = 2 if variant.simple else m       # cell value lives in range(span)
    best_count, best_graph, completed = 0, None, True
    base = Graph(n, variant)
    for tick, values in enumerate(product(range(span), repeat=len(cells))):
        if tick % 256 == 0 and time.time() > deadline:
            completed = False               # ran out before exhausting the space
            break
        candidate = base.copy()
        for (u, v), value in zip(cells, values):
            if value:
                candidate.set_multiplicity(u, v, value)
        if measure(candidate) <= m - 1 and candidate.edge_count() > best_count:
            best_count, best_graph = candidate.edge_count(), candidate.copy()
    return best_count, best_graph, completed


def _search_within_budget(
    variant: Variant, n: int, m: int, separation: str,
    deadline: float, seed: int, method: str = "sa",
) -> SearchResult:
    """Repeated search restarts until the deadline; keep the densest run."""
    engine = tabu_search_for_dense_graph if method == "tabu" else search_for_dense_graph
    best: SearchResult | None = None
    restart = 0
    while True:
        # Pass the deadline through so a single long restart can be cut short.
        outcome = engine(variant, n, m, separation=separation,
                         steps=4000, seed=seed + restart, deadline=deadline)
        if best is None or outcome.best_edge_count > best.best_edge_count:
            best = outcome
        restart += 1
        if time.time() > deadline or restart >= 200:
            break
    assert best is not None
    return best
```

```python
def _hyperedge_candidates(n: int, r: int, directed: bool, *, kind: str = "forward") -> list:
    """Every possible ``r``-uniform hyperedge.

    Undirected: every ``r``-subset.  Directed: the ``kind`` selects the
    orientation model.

    * ``"forward"`` (default): one tail, ``r - 1`` heads, stored in the legacy
      ``(tail:int, heads)`` form so existing forward results are untouched.
    * ``"backward"``: ``r - 1`` tails, one head (the arc-reversal dual of
      forward), stored in the general ``(tails, heads)`` form.
    * ``"general"``: every split of an ``r``-subset into a non-empty tail set and
      a non-empty head set, i.e. forward, backward, and (from ``r >= 4``) mixed.
    """
    if not directed:
        return [frozenset(s) for s in combinations(range(n), r)]
    if kind == "forward":
        # One tail and r-1 heads chosen from the rest (legacy form).
        return [(tail, frozenset(heads))
                for tail in range(n)
                for heads in combinations([v for v in range(n) if v != tail], r - 1)]
    if kind == "backward":
        # r-1 tails and one head: the reverse of every forward arc.
        return [(frozenset(tails), frozenset((head,)))
                for head in range(n)
                for tails in combinations([v for v in range(n) if v != head], r - 1)]
    if kind == "general":
        # Every ordered (tails, heads) split of each r-subset, both sides non-empty.
        out: list = []
        for subset in combinations(range(n), r):
            for mask in range(1, (1 << r) - 1):     # exclude all-heads and all-tails
                tails = frozenset(subset[i] for i in range(r) if mask & (1 << i))
                heads = frozenset(subset[i] for i in range(r) if not (mask & (1 << i)))
                out.append((tails, heads))
        return out
    raise ValueError(f"unknown directed hyperedge kind {kind!r}")


def _brute_force_hypergraph(
    n: int, r: int, m: int, deadline: float,
    *, directed: bool = False, vertex_split: bool = False, kind: str = "forward",
) -> tuple[int, Hypergraph | None, bool]:
    """Exhaustively maximise hyperedges over all ``r``-uniform hypergraphs.

    Each possible hyperedge is present or absent, so the search visits
    ``2 ** (#candidates)`` hypergraphs; only tiny ``(n, r)`` finish.  The
    ``directed`` and ``vertex_split`` flags select which of the four hypergraph
    measures decides feasibility, exactly as ``solve`` passes them through, and
    ``kind`` selects the directed orientation model (forward/backward/general).
    """
    candidates = _hyperedge_candidates(n, r, directed, kind=kind)
    best_count, best_h, completed = 0, None, True
    for tick, mask in enumerate(product((0, 1), repeat=len(candidates))):
```

```python
        if tick % 256 == 0 and time.time() > deadline:
            completed = False
            break
        chosen = [candidates[i] for i, on in enumerate(mask) if on]
        hypergraph = Hypergraph(n, chosen, directed=directed)
        if (max_hyper_connectivity(hypergraph, vertex_split=vertex_split) <= m - 1
                and hypergraph.edge_count() > best_count):
            best_count, best_h = hypergraph.edge_count(), hypergraph
    return best_count, best_h, completed


def _random_hypergraph_search(
    n: int, r: int, m: int, deadline: float, seed: int,
    *, directed: bool = False, vertex_split: bool = False, kind: str = "forward",
) -> tuple[int, Hypergraph | None]:
    """Greedy randomised growth: add random hyperedges while feasible, restart.

    A discovery heuristic for the hypergraph model (search is matrix-only):
    each pass shuffles the candidate hyperedges and adds each one that keeps the
    Berge connectivity within ``m - 1``; the densest pass within budget wins.  The
    feasible hypergraph it returns is the easy construction behind a lower bound.
    ``kind`` selects the directed orientation model.
    """
    rng = random.Random(seed)
    candidates = _hyperedge_candidates(n, r, directed, kind=kind)
    best_count, best_h = 0, None
    while time.time() < deadline:
        order = candidates[:]
        rng.shuffle(order)
        hypergraph = Hypergraph(n, directed=directed)
        for edge in order:
            hypergraph.add_hyperedge(edge)
            if max_hyper_connectivity(hypergraph, vertex_split=vertex_split) > m - 1:
                hypergraph.hyperedges.pop()   # this one broke feasibility; undo
        if hypergraph.edge_count() > best_count:
            best_count, best_h = hypergraph.edge_count(), hypergraph
    return best_count, (best_h if best_h is not None else Hypergraph(n, directed=directed))


def _arc_flow_at_least(out_adj: list[set[int]], n: int, s: int, t: int,
                       k: int) -> bool:
    """True if there are at least ``k`` arc-disjoint ``s``-``t`` paths.

    A plain Ford--Fulkerson with unit capacities: find an augmenting path in the
    residual (forward arcs and the reverse arcs left by earlier augmentations),
    augment, and repeat at most ``k`` times.  Used as the inner feasibility test
    of the exhaustive digraph search, where the question is only whether the
    local connectivity has reached the forbidden value ``m`` (so ``k = m``).
    """
    cap: dict[tuple[int, int], int] = {}
    for a in range(n):
        for b in out_adj[a]:
            cap[(a, b)] = 1
    for _ in range(k):
        prev = {s: s}
        stack = [s]                          # DFS for any residual augmenting path
```

```
        while stack:
            x = stack.pop()
            if x == t:
                break
            for y in range(n):
                if y not in prev and cap.get((x, y), 0) > 0:
                    prev[y] = x
                    stack.append(y)
        if t not in prev:
            return False                  # no further path: fewer than k exist
        node = t                          # walk back, pushing one unit of flow
        while node != s:
            p = prev[node]
            cap[(p, node)] -= 1
            cap[(node, p)] = cap.get((node, p), 0) + 1
            node = p
    return True


def _vertex_flow_at_least(out_adj: list[set[int]], n: int, s: int, t: int,
                          k: int) -> bool:
    """True if there are at least ``k`` internally vertex-disjoint ``s``-``t`` paths.

    The vertex twin of :func:`_arc_flow_at_least`, and the same Ford--Fulkerson
    loop, run on the split network instead of the plain one: every vertex ``x``
    becomes an entry copy ``2x`` and an exit copy ``2x+1`` joined by a single
    unit-capacity arc, so a path may pass through ``x`` at most once.  The two
    endpoints keep an uncapped gate, since a route is allowed to start at ``s``
    and finish at ``t``.  A direct arc ``s -> t`` therefore counts as one route
    with no interior, exactly as the thesis's separation axis intends.
    """
    cap: dict[tuple[int, int], int] = {}
    for x in range(n):
        cap[(2 * x, 2 * x + 1)] = k if (x == s or x == t) else 1
    for a in range(n):
        for b in out_adj[a]:
            cap[(2 * a + 1, 2 * b)] = 1
    source, sink = 2 * s + 1, 2 * t
    for _ in range(k):
        prev = {source: source}
        stack = [source]
        while stack:
            x = stack.pop()
            if x == sink:
                break
            for y in range(2 * n):
                if y not in prev and cap.get((x, y), 0) > 0:
                    prev[y] = x
                    stack.append(y)
        if sink not in prev:
            return False
        node = sink
        while node != source:
            p = prev[node]
            cap[(p, node)] -= 1
            cap[(node, p)] = cap.get((node, p), 0) + 1
            node = p
```

```python
    return True


def _exhaustive_directed(
    n: int, m: int, separation: str, deadline: float,
    stats: dict[str, int] | None = None,
) -> tuple[int, Graph | None, bool]:
    """Prove the simple directed maximum by a pruned exhaustive search.

    This is the honest version of the thesis's base-case search: branch and bound
    over every simple digraph on ``n`` vertices, keeping the densest one whose
    connectivity stays within ``m - 1``.  Two prunes make it finish where naive
    enumeration cannot.  Feasibility is monotone, so the include branch adds an
    arc only when the digraph is still feasible (an infeasible prefix can never be
    rescued by adding more arcs), and the bound prune drops a subtree once even
    taking every remaining arc could not beat the best found.  Both separations
    use the same incremental feasibility test: a new arc can only lift the
    connectivity of a pair whose source reaches its tail and whose sink is
    reachable from its head, so only those pairs are re-measured, with
    :func:`_arc_flow_at_least` or :func:`_vertex_flow_at_least` according to the
    separation.  Returns ``(max_count, witness, completed)``; ``completed`` is
    False if the time budget ran out first, leaving the value a lower bound.
    Pass ``stats`` to receive the number of search-tree nodes visited.
    """
    pairs = [(u, v) for u in range(n) for v in range(n) if u != v]
    total = len(pairs)
    out: list[set[int]] = [set() for _ in range(n)]
    inc: list[set[int]] = [set() for _ in range(n)]
    # Seed the best at one below a known construction so the bound prune bites
    # immediately, and the search still records an actual witness when it ties it.
    # The same seed is valid in both separations: by Whitney's inequality
    # kappa <= lambda, a digraph feasible for the arc problem is feasible for the
    # vertex problem too, so an arc construction is an honest vertex lower bound.
    seed = _directed_witness(n, m, simple=True)
    seed_value = seed.edge_count() if seed is not None else 0
    best_count = [max(0, seed_value - 1)]
    best_arcs: list[tuple[int, int]] = []
    timed_out = [False]

    def reaches_to(target: int) -> set[int]:
        seen, stack = {target}, [target]   # reverse reachability (can reach t)
        while stack:
            x = stack.pop()
            for y in inc[x]:
                if y not in seen:
                    seen.add(y); stack.append(y)
        return seen

    def reaches_from(source: int) -> set[int]:
        seen, stack = {source}, [source]   # forward reachability (s can reach)
        while stack:
            x = stack.pop()
            for y in out[x]:
                if y not in seen:
                    seen.add(y); stack.append(y)
        return seen

    flow_at_least = (_arc_flow_at_least if separation == "edge"
```

```
                     else _vertex_flow_at_least)

    def feasible_after(u: int, v: int) -> bool:
        for s in reaches_to(u):
            for t in reaches_from(v):
                if s != t and flow_at_least(out, n, s, t, m):
                    return False           # this pair reached m disjoint paths
        return True

    def recurse(idx: int, count: int) -> None:
        if stats is not None:
            stats["nodes"] = stats.get("nodes", 0) + 1
        if time.time() > deadline:
            timed_out[0] = True
            return
        if count + (total - idx) <= best_count[0]:
            return                         # cannot beat the incumbent: prune
        if idx == total:
            if count > best_count[0]:
                best_count[0] = count
                best_arcs[:] = [(a, b) for a in range(n) for b in out[a]]
            return
        u, v = pairs[idx]
        out[u].add(v); inc[v].add(u)       # include the arc, if still feasible
        if feasible_after(u, v):
            recurse(idx + 1, count + 1)
        out[u].discard(v); inc[v].discard(u)
        if timed_out[0]:
            return
        recurse(idx + 1, count)            # exclude the arc

    recurse(0, 0)
    witness = None
    if best_arcs:
        witness = Graph(n, SIMPLE_DIRECTED)
        for (a, b) in best_arcs:
            witness.mu[a, b] = 1
    elif seed is not None:
        witness = seed                     # max equalled the seed construction
    # A completed search always reaches at least the seed's count, so the max
    # below only matters on a timeout before any recorded leaf: there it lifts
    # the reported lower bound from seed_value - 1 (the pruning incumbent) to
    # the seed witness's own arc count, keeping value and witness consistent.
    return max(best_count[0], seed_value), witness, not timed_out[0]


def solve(
    n: int, m: int, *,
    directed: bool = False, simple: bool = True,
    hypergraph: bool = False, r: int = 3,
    exhaustive: bool = False, separation: str = "edge",
    max_seconds: float = 60.0, seed: int = 0, method: str = "tabu",
) -> SolveResult:
    """The single driver: prove the exact value, or discover a dense example.

    Discovery defaults to ``method="tabu"``: tabu search is the stronger engine on
    the harder directed problems and the one used to solve the problems in this
    work.  Simulated annealing (``method="sa"``) is kept for the self-check and
```

```
    verification, where the lighter engine is enough.

    Args:
        n, m: the vertices and the forbidden number of independent routes.
        directed, simple: the two matrix axes (ignored when ``hypergraph``).
        hypergraph, r: switch to the ``r``-uniform hypergraph model instead.
        exhaustive: ``True`` to PROVE the optimum, ``False`` to DISCOVER one.
        separation: ``"edge"`` or ``"vertex"`` disjointness (matrix models).
        max_seconds: wall-clock budget; always respected.
        seed: random seed for the discovery searches.

    Returns:
        A :class:`SolveResult` whose ``bound`` is ``"exact"`` (proved),
        ``"lower"`` (a witness), or ``"upper"`` (proved, no matching witness).
    """
    start = time.time()
    deadline = start + max_seconds

    # ----- the hypergraph model --------------------------------------------
    # Same driver, a different internal measure: the edge/vertex separation and
    # the direction pick which of the four Berge measures decides feasibility.
    if hypergraph:
        vertex_split = (separation == "vertex")
        label = f"{r}-uniform {'directed ' if directed else ''}hypergraph"
        if exhaustive:
            value, witness, done = _brute_force_hypergraph(
                n, r, m, deadline, directed=directed, vertex_split=vertex_split)
            bound = "exact" if done else "lower"
            method = "brute-force enumeration"
            note = "" if done else "budget ran out; value is only a lower bound"
        else:
            value, witness = _random_hypergraph_search(
                n, r, m, deadline, seed, directed=directed, vertex_split=
                    vertex_split)
            bound, done, method = "lower", True, "randomised greedy search"
            note = "discovery only ever yields a lower bound"
        return SolveResult(n, m, label, separation, value, bound, method,
                           time.time() - start, done, witness, note)

    # ----- the matrix models -----------------------------------------------
    variant = _variant_for(directed, simple)
    label = variant.describe()

    # DISCOVER: search within the budget; the witness found is a lower bound.
    if not exhaustive:
        if method not in ("sa", "tabu"):
            raise ValueError("method must be 'sa' or 'tabu'")
        result = _search_within_budget(variant, n, m, separation, deadline, seed,
            method)
        method_label = "tabu search" if method == "tabu" else "random search"
        return SolveResult(
            n, m, label, separation, result.best_edge_count, "lower",
            method_label, time.time() - start, True,
            result.best_graph, "discovery only ever yields a lower bound")

    # EXHAUSTIVE, simple directed: a pruned exhaustive digraph search is exact.
    # This is the prover for the m=2 base cases of the directed theorem, and it
    # reaches n = 6, 7 where the cut-counting cannot close the gap in any sane
```

```
        time.
    if directed and simple:
        value, witness, done = _exhaustive_directed(n, m, separation, deadline)
        bound = "exact" if done else "lower"
        method = "exhaustive digraph search (branch and bound)"
        note = "" if done else "budget ran out; value is only a lower bound"
        return SolveResult(n, m, label, separation, value, bound, method,
                           time.time() - start, done, witness, note)

    # EXHAUSTIVE, directed MULTIGRAPH arc problem: the cut-counting is the prover.
    if directed and separation == "edge":
        budget = max(1.0, deadline - time.time())
        proof = prove_directed_multigraph(n, time_limit=budget)
        if proof.is_proof():
            # We only reach here for a multigraph, where (m-1) M*(n) IS the value.
            value = proof.value_for(m)          # (m-1) * M*(n)
            witness = _directed_witness(n, m, simple)
            return SolveResult(n, m, label, separation, value, "exact",
                               "cut-counting (gap zero)", time.time() - start,
                               True, witness, "")
        # Timed out: fall back to the best construction as a lower bound.
        witness = _directed_witness(n, m, simple)
        low = witness.edge_count() if witness else 0
        return SolveResult(
            n, m, label, separation, low, "lower",
            "cut-counting hit the time limit; reporting a construction",
            time.time() - start, False, witness,
            "raise max_seconds to let the cut-counting close the gap")

    # EXHAUSTIVE otherwise (undirected, or vertex separation): brute force.
    value, witness, done = _brute_force_matrix(
        variant, n, m, separation, deadline)
    bound = "exact" if done else "lower"
    method = "brute-force enumeration"
    note = ("" if done else "budget ran out; value is only a lower bound "
            "(no cut-counting exists for this case)")
    return SolveResult(n, m, label, separation, value, bound, method,
                       time.time() - start, done, witness, note)
```

C.1.5 The random model

Here begins the material behind Chapter 4. This first piece builds random graphs and measures them, which is how the thesis looks at the typical case rather than the extreme one. Each of the six models gets its own sampler, and the multigraph samplers use a two-stage rule: decide whether a pair is connected at all, then decide how many parallel copies it gets, which recovers the ordinary random graph when the second stage is switched off.

```
######################################################################
##
##   CHAPTER 4 -- SYNTHESIS AND RESULTS
##   Sampling the random model, the thesis figures, and the self-check.
##
######################################################################

# --- MONTE CARLO: sample G(n,p) graphs, measure with exact checker, average ---
```

```python


def sample_random_graph(n: int, p: float, directed: bool, rng: random.Random) -> Graph:
    """Draw one sample of $G(n, p)$ (or the random digraph $D(n, p)$).

    Each possible edge is included independently with probability ``p``: unordered
    pairs for an undirected graph, ordered pairs for a digraph.
    """
    variant = SIMPLE_DIRECTED if directed else SIMPLE_UNDIRECTED
    graph = Graph(n, variant)
    for u in range(n):
        # Undirected: scan v > u (each pair once). Directed: scan all v.
        start = 0 if directed else u + 1
        for v in range(start, n):
            if u != v and rng.random() < p:  # include this edge with prob p
                graph.add_edge(u, v)
    return graph


def estimate_appearance_probability(
    n: int, p: float, m: int, *, trials: int = 200,
    separation: str = "edge", directed: bool = False, seed: int = 0,
) -> float:
    """Estimate the probability that a sample has $\\lambda^{\\max} \\ge m$."""
    rng = random.Random(seed)
    measure = _connectivity_measure(separation)
    # Count the fraction of samples that already exhibit m independent routes
    # somewhere (lambda^max >= m): the empirical appearance probability.
    appearances = sum(
        measure(sample_random_graph(n, p, directed, rng)) >= m
        for _ in range(trials)
    )
    return appearances / trials


def connectivity_distribution(
    n: int, p: float, *, trials: int = 200,
    separation: str = "edge", directed: bool = False, seed: int = 0,
) -> list[int]:
    """Return the list of $\\lambda^{\\max}$ (or $\\kappa^{\\max}$) over samples.

    Same arguments as the estimators above. The returned list has one entry per
    sample, so its histogram shows how the binding connectivity is distributed
    for that kind of graph. Calling this with the same ``seed`` and ``(n, p)``
    but different ``separation`` reuses the very same random graphs, which is why
    the edge and vertex distributions can be compared sample by sample.
    """
    rng = random.Random(seed)
    measure = _connectivity_measure(separation)
    # Same seed + same (n, p) reproduces the identical sequence of graphs, so an
    # edge run and a vertex run can be compared sample by sample (Whitney check).
    return [measure(sample_random_graph(n, p, directed, rng)) for _ in range(trials)]


@dataclass
class ThresholdCurve:
```

```python
    """The appearance probability sampled across a range of densities."""

    n: int
    m: int
    probabilities: list[tuple[float, float]]  # (p, estimated probability)

    @property
    def predicted_threshold(self) -> float:
        """The proved location $p^{*} = m/n$."""
        return self.m / self.n


def threshold_curve(
    n: int, m: int, p_values: list[float], *, trials: int = 200,
    separation: str = "edge", directed: bool = False, seed: int = 0,
) -> ThresholdCurve:
    """Sweep ``p`` and return the appearance probability at each value."""
    points = [
        (p, estimate_appearance_probability(
            n, p, m, trials=trials, separation=separation, directed=directed, seed=seed))
        for p in p_values
    ]
    return ThresholdCurve(n=n, m=m, probabilities=points)


# --- edge against vertex disjointness in the random model -------------------
#
# Whitney's inequality kappa^max <= lambda^max leaves open whether the two
# binding connectivities of a graph actually differ.  For the EXTREMAL problem
# they first part company at m = 5 (Sorensen-Thomassen).  The function below lets
# the same split be watched in the random model: it draws samples of G(n, p) and
# measures both maxima on each one.  On small graphs, where the binding
# connectivity stays at or below four, the two coincide on every sample; on
# larger, denser graphs, where it climbs past five, a pair can carry more
# edge-disjoint routes than internally vertex-disjoint ones, and kappa^max drops
# below lambda^max on a growing fraction of samples.


def edge_vertex_distribution(
    n: int, p: float, *, trials: int = 300, seed: int = 0,
) -> tuple[list[int], list[int]]:
    """Return paired (lambda^max, kappa^max) over the same samples of G(n, p).

    Both lists are measured on the identical sequence of random graphs, because
    connectivity_distribution reuses the seed, so the two can be compared sample
    by sample: the fraction of entries with kappa^max < lambda^max is exactly how
    often vertex disjointness is strictly more demanding than edge disjointness
    at that size and density.
    """
    lambda_max = connectivity_distribution(
        n, p, trials=trials, separation="edge", seed=seed)
    kappa_max = connectivity_distribution(
        n, p, trials=trials, separation="vertex", seed=seed)
    return lambda_max, kappa_max
```

```python
# --- SAMPLING ALL VARIANTS: Bernoulli for simple/hypergraph, hurdle-geometric for multigraph ---


def sample_random_multigraph(
    n: int, p: float, alpha: float, directed: bool, rng: random.Random,
    *, cap: int | None = None,
) -> Graph:
    """Random multigraph with hurdle-geometric edge multiplicities.

    Each cell is empty with probability ``1 - p``; otherwise it carries one copy
    plus a Geometric(``alpha``) number of further copies, so the multiplicity
    decays exponentially with rate ``alpha`` and ``alpha = 0`` recovers simple
    ``G(n, p)``.  With ``cap`` set (use ``m - 1``) a single fat edge cannot
    trivially break feasibility, so the panel stays a structural question.
    """
    variant = MULTI_DIRECTED if directed else MULTI_UNDIRECTED
    graph = Graph(n, variant)
    for u in range(n):
        start = 0 if directed else u + 1
        for v in range(start, n):
            if u == v or rng.random() >= p:
                continue
            k = 1
            while rng.random() < alpha:
                k += 1
            graph.set_multiplicity(u, v, k if cap is None else min(k, cap))
    return graph


def sample_random_hypergraph(
    n: int, p: float, r: int, directed: bool, rng: random.Random,
) -> Hypergraph:
    """Random ``r``-uniform hypergraph: each candidate hyperedge included w.p. ``p``."""
    chosen = [c for c in _hyperedge_candidates(n, r, directed) if rng.random() < p]
    return Hypergraph(n, chosen, directed=directed)


def _sample_variant(n, p, *, directed, simple, hypergraph, r, alpha, cap, rng):
    """Draw one sample of whichever generative model the flags select."""
    if hypergraph:
        return sample_random_hypergraph(n, p, r, directed, rng)
    if simple:
        return sample_random_graph(n, p, directed, rng)
    return sample_random_multigraph(n, p, alpha, directed, rng, cap=cap)


def _measure_variant(obj, *, separation, hypergraph):
    """Binding connectivity of a sampled object under the chosen separation."""
    if hypergraph:
        return max_hyper_connectivity(obj, vertex_split=(separation == "vertex"))
    return _connectivity_measure(separation)(obj)


def _mean_binding_degree(obj, *, directed, hypergraph):
```

```python
    """The degree that the degree bound caps connectivity by, averaged over
        ↪ vertices.

    Undirected: total incident multiplicity.  Directed: ``min(d^+, d^-)`` (the
    quantity bounding an arc-disjoint count).  Hypergraph: the Berge degree, the
    number of hyperedges through a vertex.
    """
    n = obj.num_vertices
    if hypergraph:
        deg = [0] * n
        for edge in obj.hyperedges:
            for v in obj.members(edge):
                deg[v] += 1
        return sum(deg) / n
    if directed:
        return sum(min(obj.out_degree(v), obj.in_degree(v)) for v in range(n)) / n
    return sum(obj.degree(v) for v in range(n)) / n


def _p_for_target_degree(target_deg, n, *, directed, simple, hypergraph, r, alpha)
    ↪ :
    """Presence probability ``p`` that puts the expected degree near ``target_deg
        ↪ ``.

    Only an aiming heuristic -- the figures plot the *measured* mean degree -- so
    the exact value of the cap or of the geometric tail does not need inverting.
    """
    if hypergraph:
        per_vertex = math.comb(n - 1, r - 1)
        if directed:
            per_vertex += (n - 1) * math.comb(n - 2, r - 2)
        return min(1.0, target_deg / max(per_vertex, 1))
    copies = 1.0 if simple else 1.0 / (1.0 - alpha)
    return min(1.0, target_deg / ((n - 1) * copies))


# Twelve sampled panels, mirroring _VARIANT_ENUM_CONFIGS but at sampling sizes
# well past the enumeration wall (n=6).  ``sample_n`` is chosen per variant so
# the binding-connectivity sweep stays a few minutes overall.
_VARIANT_SAMPLE_CONFIGS: list[dict] = [
    dict(key="simple_undirected_edge",   title="simple undirected edge",
         directed=False, simple=True,  hypergraph=False, r=3, separation="edge",
            ↪ sample_n=26),
    dict(key="simple_undirected_vertex", title="simple undirected vertex",
         directed=False, simple=True,  hypergraph=False, r=3, separation="vertex",
            ↪  sample_n=18),
    dict(key="simple_directed_edge",     title="simple directed arc",
         directed=True,  simple=True,  hypergraph=False, r=3, separation="edge",
            ↪ sample_n=14),
    dict(key="simple_directed_vertex",   title="simple directed vertex",
         directed=True,  simple=True,  hypergraph=False, r=3, separation="vertex",
            ↪  sample_n=14),
    dict(key="multi_undirected_edge",    title="multigraph undirected edge",
         directed=False, simple=False, hypergraph=False, r=3, separation="edge",
            ↪ sample_n=22),
    dict(key="multi_undirected_vertex",  title="multigraph undirected vertex",
         directed=False, simple=False, hypergraph=False, r=3, separation="vertex",
            ↪  sample_n=16),
```

```
    dict(key="multi_directed_edge",     title="multigraph directed arc",
         directed=True,  simple=False, hypergraph=False, r=3, separation="edge", sample_n=12),
    dict(key="multi_directed_vertex",   title="multigraph directed vertex",
         directed=True,  simple=False, hypergraph=False, r=3, separation="vertex", sample_n=12),
    dict(key="hyper_undirected_edge",   title="hypergraph undirected edge",
         directed=False, simple=True,  hypergraph=True,  r=3, separation="edge", sample_n=12),
    dict(key="hyper_undirected_vertex", title="hypergraph undirected vertex",
         directed=False, simple=True,  hypergraph=True,  r=3, separation="vertex", sample_n=12),
    dict(key="hyper_directed_edge",     title="hypergraph directed arc",
         directed=True,  simple=True,  hypergraph=True,  r=3, separation="edge", sample_n=9),
    dict(key="hyper_directed_vertex",   title="hypergraph directed vertex",
         directed=True,  simple=True,  hypergraph=True,  r=3, separation="vertex", sample_n=9),
]
```

C.1.6 Drawing the figures

Every picture in the thesis is produced by the routines below, from the same data structures the rest of the program uses. They are collected here rather than in the figure script next door so that a plot and the computation behind it stay in one file.

```
# --- FIGURES: headless matplotlib PNGs; each function takes a path and writes a file ---

_KUL_BLUE = "#1D8DB0"     # feasible / proved
_KUL_DARK = "#1E6E87"     # row labels, known-max lines
_KUL_LIGHT = "#52BDEC"    # named branches
_WARM = "#DC8C28"         # feasibility boundary, certain interval
_RED = "#E05050"          # infeasible bars
_GREEN = "#3CA050"        # machine-checked exact markers
_VIOLET = "#9B5DE5"       # search lower-bound markers
_GUESS = "#E6B800"        # "guess" interpolation: a clear yellow/gold, kept well
                          # apart from the red conjecture curve (proved=blue,
                          # conjectured=red, guess=yellow)

# Four-level sensitivity palette: cool (sigma=0) -> hot (sigma=max).
_SIGMA_PALETTE = ["#52BDEC", "#F9C74F", "#F4914B", "#DC8C28"]  # 0,1,2,3+


def plot_directed_crossover(m: int, max_n: int, path: str | Path) -> None:
    """Plot the two competing directed branches and their maximum versus ``n``.

    Shows how the linear hub branch ``m(n-1)`` is overtaken by the quadratic
    augmented-bipartite branch near ``n ~ 2m``: the heart of the directed story.
    """
    ns = list(range(2, max_n + 1))
    hub = [m * (n - 1) for n in ns]                          # linear hub branch
    # Quadratic branch: the shifted-partition augmented bipartite count
    # floor((n+m-2)^2/4) of const:augmented-bipartite (at m <= 3 it equals the
    # balanced-partition count floor(n^2/4) + (m-2)ceil(n/2)).
    bipartite = [((n + m - 2) ** 2) // 4 for n in ns]
```

```
    envelope = [directed_arc_lower_bound(n, m) for n in ns]  # their pointwise max

    plt.figure(figsize=(7, 4.3))
    plt.plot(ns, hub, "--", color=_KUL_DARK, label=r"hub branch $m(n-1)$")
    plt.plot(ns, bipartite, "--", color=_WARM,
             label=r"bipartite branch $\lfloor (n+m-2)^2/4\rfloor$")
    plt.plot(ns, envelope, "-", color=_KUL_BLUE, linewidth=2.4, label="their maximum")

    # Mark the crossover: the first n at which the quadratic branch overtakes the
    # linear one, the order (linear in m) where the directed story changes hands.
    cross_n = next((n for n, h, b in zip(ns, hub, bipartite) if b > h), None)
    if cross_n is not None:
        y_cross = directed_arc_lower_bound(cross_n, m)
        plt.axvline(cross_n, color="gray", linestyle=":", linewidth=1.3, alpha=0.8)
        plt.scatter([cross_n], [y_cross], color=_KUL_BLUE, zorder=5,
                    s=45, edgecolor="white", linewidth=0.8)
        plt.annotate(f"crossover at $n = {cross_n}$",
                     xy=(cross_n, y_cross),
                     xytext=(cross_n - 2.8, max(envelope) * 0.48),
                     fontsize=9.5, color="black", ha="center",
                     arrowprops=dict(arrowstyle="->", color="gray", lw=1.1))

    plt.xlabel("number of vertices $n$")
    plt.ylabel("arcs")
    plt.title(f"Directed lower bound at $m = {m}$")
    plt.legend(loc="upper left")
    plt.grid(True, alpha=0.3)
    _save(path)


def plot_edge_vertex_divergence(max_n: int, path: str | Path) -> None:
    """Show agreement through m<=4 and divergence at m=5, all starting at the same n.

    All curves begin at n_start so the reader can compare directly.
    For m=2,3,4 the two problems coincide (one line each, blue shades).
    At m=5 they part: edge below, vertex above, with the gap shaded.
    """
    n_start = 4  # common start for all curves
    ns = list(range(n_start, max_n + 1))

    plt.figure(figsize=(8, 5))

    # m=2,3,4: edge == vertex -- one curve per m
    agree_palette = ["#AED9EE", "#5CB4D9", _KUL_DARK]
    for m_val, color in zip([2, 3, 4], agree_palette):
        vals = [simple_undirected_edge(n, m_val) for n in ns]
        plt.plot(ns, vals, "-", color=color, linewidth=1.8, alpha=0.9,
                 label=f"$m={m_val}$: edge $=$ vertex")

    # m=5: edge and vertex (same starting point, diverge from n=6 onwards)
    edge5 = [simple_undirected_edge(n, 5) for n in ns]
    vert5 = [simple_undirected_vertex_m5(n) for n in ns]
    plt.plot(ns, edge5, "--", color=_KUL_BLUE, linewidth=2.4,
             label=r"$m=5$: edge $\ell_5(n)=\lfloor 5(n-1)/2\rfloor$")
    plt.plot(ns, vert5, "-",  color=_WARM,     linewidth=2.4,
```

```python
             label=r"$m=5$: vertex $k_5(n)=\lfloor 8n/3\rfloor-3$")
    # shade only where the gap is positive
    plt.fill_between(ns, edge5, vert5, where=[v > e for v, e in zip(vert5, edge5)],
                     alpha=0.15, color=_WARM)

    plt.xlabel("number of vertices $n$")
    plt.ylabel("maximum edges")
    plt.title("Agreement through $m \\leq 4$, first divergence at $m = 5$")
    plt.legend(fontsize=9, loc="upper left")
    plt.grid(True, alpha=0.3)
    _save(path)


# NOTE: plot_degree_threshold generated the 2-D random-sampling threshold figure
# (the old Figure 4.2) removed from the thesis on 2026-06-20.  Kept here so the
# threshold phenomenon can be restored; see
# research_notes/removed_threshold_phenomenon.md for the recipe.  (Its 3-D
# companion plot_conn_threshold_3d is still used, for the appendix.)
def plot_degree_threshold(
    path: str | Path, *, n: int = 12, m: int = 4, alpha: float = 0.5,
    trials: int = 70, seed: int = 7,
    targets=(0.3, 0.5, 0.65, 0.8, 0.95, 1.1, 1.3, 1.6, 2.0),
) -> None:
    """Appearance probability against mean binding-degree, across every model.

    Six random models -- simple, multigraph and $3$-uniform hypergraph, each
    undirected and directed -- are swept in density and plotted by their
    *measured* mean binding degree (in units of ``m``) against the probability
    that a sample already contains a pair of (Berge) edge-connectivity at least
    ``m``.  The vertical line at degree ``= m`` is the asymptotic threshold the
    degree argument of thm:gnp-threshold predicts; at the fixed small ``m`` a
    computation can reach the curves are ORDERED rather than coincident -- a
    directed multigraph forces the pair at the least degree (heavy parallel edges
    concentrate connectivity), a directed hypergraph at the most (a hyperedge
    serves only one Berge route), with the simple cases between.  Universality is
    the ``m/ln n -> infinity`` limit; the spread here is the finite-m gap.
    """
    models = [
        ("simple, undirected",       dict(directed=False, simple=True, hypergraph=False, r=3), _KUL_LIGHT, "o", "-"),
        ("simple, directed",         dict(directed=True,  simple=True, hypergraph=False, r=3), _KUL_BLUE,  "s", "-"),
        ("multigraph, undirected",   dict(directed=False, simple=False, hypergraph=False, r=3), _WARM,      "o", "--"),
        ("multigraph, directed",     dict(directed=True,  simple=False, hypergraph=False, r=3), "#C0392B",  "s", "--"),
        ("$3$-uniform hyper, undir.", dict(directed=False, simple=True, hypergraph=True,  r=3), _VIOLET,    "^", ":"),
        ("$3$-uniform hyper, dir.",   dict(directed=True,  simple=True, hypergraph=True,  r=3), _GREEN,     "^", ":"),
    ]
    plt.figure(figsize=(7.8, 5.0))
    for label, spec, colour, mk, ls in models:
        xs, ys = [], []
        for t in targets:
            p = _p_for_target_degree(t * m, n, alpha=alpha, **spec)
            rng = random.Random(seed)
```

```
            degs, hits = [], 0
            for _ in range(trials):
                obj = _sample_variant(n, p, alpha=alpha, cap=m - 1, rng=rng, **spec)
                degs.append(_mean_binding_degree(
                    obj, directed=spec["directed"], hypergraph=spec["hypergraph"]))
                if _measure_variant(obj, separation="edge",
                                    hypergraph=spec["hypergraph"]) >= m:
                    hits += 1
            xs.append(sum(degs) / len(degs) / m)
            ys.append(hits / trials)
        plt.plot(xs, ys, ls, marker=mk, color=colour, markersize=4.5,
                 linewidth=1.8, label=label)
    plt.axvline(1.0, color="black", linestyle="--", linewidth=1.6,
                label=r"asymptotic threshold (degree $= m$)")
    plt.axhline(0.5, color="gray", linestyle=":", linewidth=1.0, alpha=0.6)
    plt.xlabel(r"mean binding degree, in units of $m$")
    plt.ylabel(r"$\hat{P}\,[\,\text{binding connectivity} \geq m\,]$")
    plt.title(f"Appearance vs expected degree, by model "
              f"($n = {n}$, $m = {m}$, multiplicities capped at $m-1$)")
    plt.ylim(-0.03, 1.03)
    plt.legend(fontsize=8.5, loc="lower right")
    plt.grid(True, alpha=0.3)
    _save(path)


def plot_connectivity_distribution(series: dict, n: int, p: float, path: str | Path) -> None:
    """Plot overlaid histograms of the binding connectivity for several models.

    ``series`` maps a label to a list of connectivity values (one per random
    sample), as produced by :func:`montecarlo.connectivity_distribution`.  The
    integer-valued histograms are drawn on shared integer bins so the shapes can
    be compared directly across the kinds of graph.
    """
    # Shared integer bins across every series so the shapes line up exactly.
    all_values = [value for values in series.values() for value in values]
    low, high = min(all_values), max(all_values)
    levels = list(range(low, high + 1))
    colours = [_KUL_BLUE, _WARM, _KUL_DARK, _KUL_LIGHT]
    group_count = len(series)
    bar_width = 0.8 / group_count  # split each integer slot among the series

    plt.figure(figsize=(7, 4.3))
    for index, ((label, values), colour) in enumerate(zip(series.items(), colours)):
        counts = [values.count(level) for level in levels]
        # Offset each series' bars so grouped bars sit side by side per level.
        offset = (index - (group_count - 1) / 2) * bar_width
        positions = [level + offset for level in levels]
        plt.bar(positions, counts, width=bar_width, color=colour, label=label,
                edgecolor="white", linewidth=0.4)
    plt.xticks(levels)
    plt.xlabel("binding connectivity")
    plt.ylabel("number of samples")
    plt.title(f"Connectivity over random graphs, $n = {n}$, $p = {p}$")
    plt.legend()
```

```
    plt.grid(True, axis="y", alpha=0.3)
    _save(path)


_FRACTION_LABEL = {0.25: "1/4", 0.5: "1/2", 0.75: "3/4"}


def plot_edge_vertex_histograms(
    data: dict[float, tuple[list[int], list[int]]],
    n: int,
    p_values: list[float],
    path: str | Path,
) -> None:
    """Six histograms of the binding connectivity in $G(n, p)$, at one size $n$.

    ``data`` maps each density ``p`` to a pair ``(lambda_max_values,
    kappa_max_values)`` measured on the same samples, from
    :func:`montecarlo.edge_vertex_distribution`.  The grid is two rows by three
    columns: the top row is the edge-connectivity ``lambda^max`` and the bottom
    row the vertex-connectivity ``kappa^max``, one column per density.  Reading a
    column top to bottom compares the two distributions at a fixed density, so
    Whitney's inequality shows up as the vertex histogram sitting at or to the
    left of the edge one.  The fraction of samples with ``kappa^max <
    lambda^max`` is printed on each vertex panel.  All six share one axis range.
    """
    rows = [(r"edge $\lambda^{\max}$", 0, _KUL_BLUE),
            (r"vertex $\kappa^{\max}$", 1, _WARM)]
    all_values = [v for p in p_values for series in data[p] for v in series]
    lo, hi = min(all_values), max(all_values)
    bins = range(lo, hi + 2)  # one integer-wide bin per connectivity value
    trials = len(data[p_values[0]][0])

    fig, axes = plt.subplots(2, len(p_values), figsize=(11, 6),
                             sharex=True, sharey=True)
    for col, p in enumerate(p_values):
        lam, kap = data[p]
        gap = 100 * sum(k < l for k, l in zip(kap, lam)) / len(lam)
        plabel = _FRACTION_LABEL.get(p, f"{p:.2f}")
        for label, ridx, colour in rows:
            axis = axes[ridx][col]
            values = lam if ridx == 0 else kap
            axis.hist(values, bins=bins, align="left", color=colour,
                      edgecolor="white", linewidth=0.4, rwidth=0.9)
            axis.axvline(statistics.mean(values), color="black", linestyle="--",
                         linewidth=1.2)
            if ridx == 0:
                axis.set_title(f"$p = {plabel}$")
            else:
                axis.set_xlabel("binding connectivity")
                axis.text(0.03, 0.92, fr"$\kappa\!<\!\lambda$: {gap:.0f}%",
                          transform=axis.transAxes, ha="left", va="top",
                          fontsize=9, color=_WARM)
            if col == 0:
                axis.set_ylabel(f"{label}\nsamples")
            axis.grid(True, axis="y", alpha=0.3)

    fig.suptitle(fr"Distribution of the binding connectivity in $G({n},p)$ "
                 fr"over {trials} samples", fontsize=12)
```

```
    _save(path)


def plot_search_trace(result: SearchResult, path: str | Path,
                      optimum: int | None = None,
                      ceiling: int | None = None,
                      *,
                      show_cooling: bool = True,
                      show_histogram: bool = True,
                      connectivity_label: str = r"$\lambda^{\max}$",
                      edge_label: str = "arcs",
                      title: str = "Cooling toward an extremal graph") -> None:
    """Plot the search trajectory; optionally include the cooling schedule.

    Layout: [cooling schedule (optional)] | [scatter] | [visit histogram].

    The scatter colours points by step number with a heavily nonlinear
    PowerNorm (gamma=0.2, RdYlBu colourmap) so that only the brief hot
    exploration phase (early steps) stands out in warm reds/yellows, while
    the long converged tail is uniformly blue.  When ``ceiling`` is given the
    feasibility boundary is marked and "feasible"/"infeasible" labels are
    placed just above the axes box using a blended data/axes transform, so
    they sit centred over their respective shaded regions regardless of zoom.

    The rightmost panel is an optional horizontal density histogram showing how
    many search steps landed at each edge-count level; it shares the y-axis with
    the scatter so the bars align with the dots.

    New keyword arguments (all optional, all have sensible defaults):
    - ``show_cooling``: set False to omit the left cooling-schedule panel.
    - ``show_histogram``: set False to omit the right visit-density panel.
    - ``connectivity_label``: x-axis label for the scatter (default lambda^max).
    - ``edge_label``: y-axis label (default "arcs"; use "edges" for undirected).
    - ``title``: figure suptitle.
    """
    from collections import Counter
    from matplotlib.colors import PowerNorm
    from matplotlib.transforms import blended_transform_factory
    from matplotlib.gridspec import GridSpec

    steps = [record.step for record in result.history]
    temperature = [record.temperature for record in result.history]
    edges = [record.edge_count for record in result.history]
    connectivity = [record.connectivity for record in result.history]

    # Build layout: cooling (optional) | scatter | histogram (optional).
    # Histogram axes use no sharey -- that avoids a tight_layout warning, so the
    # y-limits are synced to the scatter manually after it is fully drawn.
    if show_cooling and show_histogram:
        fig = plt.figure(figsize=(13.5, 4.8))
        gs = GridSpec(1, 3, figure=fig, width_ratios=[1.0, 1.6, 0.28],
                      wspace=0.40)
        ax_cool = fig.add_subplot(gs[0])
        ax_scat = fig.add_subplot(gs[1])
        ax_hist = fig.add_subplot(gs[2])
    elif show_cooling and not show_histogram:
        fig = plt.figure(figsize=(11.5, 4.8))
        gs = GridSpec(1, 2, figure=fig, width_ratios=[1.0, 1.6], wspace=0.40)
```

```
        ax_cool = fig.add_subplot(gs[0])
        ax_scat = fig.add_subplot(gs[1])
        ax_hist = None
    elif (not show_cooling) and show_histogram:
        fig = plt.figure(figsize=(8.5, 4.8))
        gs = GridSpec(1, 2, figure=fig, width_ratios=[1.6, 0.28], wspace=0.12)
        ax_cool = None
        ax_scat = fig.add_subplot(gs[0])
        ax_hist = fig.add_subplot(gs[1])
    else:
        fig = plt.figure(figsize=(6.6, 4.8))
        ax_cool = None
        ax_scat = fig.add_subplot(1, 1, 1)
        ax_hist = None

    # Left: cooling schedule.
    if ax_cool is not None:
        ax_cool.plot(steps, temperature, color=_WARM, linewidth=1.8)
        ax_cool.set_xlabel("step")
        ax_cool.set_ylabel("temperature")
        ax_cool.set_title("Cooling schedule")
        ax_cool.grid(True, alpha=0.3)

    # Middle: scatter of (connectivity, edge_count), coloured by step.
    ax = ax_scat
    lo = min(connectivity) - 0.5
    hi = max(connectivity) + 0.5
    if ceiling is not None:
        boundary = ceiling + 0.5
        ax.axvspan(lo, boundary, color=_GREEN, alpha=0.10)
        ax.axvspan(boundary, hi, color=_RED, alpha=0.10)
        ax.axvline(boundary, color=_WARM, linestyle="--", linewidth=1.4)
        ax.set_xlim(lo, hi)
        # Labels centred over each shaded region, placed just above the axes box.
        # blended_transform mixes data-x (tracks the region) with axes-fraction-y
        # (1.03 is always just above the top of the box regardless of zoom level).
        xt = blended_transform_factory(ax.transData, ax.transAxes)
        ax.text((lo + boundary) / 2, 1.03, "feasible",
                transform=xt, ha="center", va="bottom",
                fontsize=8.5, color=_GREEN, fontweight="bold", clip_on=False)
        ax.text((boundary + hi) / 2, 1.03, "infeasible",
                transform=xt, ha="center", va="bottom",
                fontsize=8.5, color="#C03030", fontweight="bold", clip_on=False)

    # Nonlinear colour norm: gamma < 1 squashes large step values toward 1,
    # mapping the long converged tail (most points) to the BLUE end of RdYlBu
    # while only the first few hot steps appear in warm reds and yellows.
    max_step = max(steps) if steps else 1
    norm = PowerNorm(gamma=0.2, vmin=0, vmax=max_step)
    sc = ax.scatter(connectivity, edges, c=steps, cmap="RdYlBu",
                    norm=norm, s=16, alpha=0.75, linewidths=0)

    if optimum is not None:
        ax.axhline(optimum, color=_KUL_DARK, linestyle="--", linewidth=1.4,
                   label=f"certified optimum = {optimum}")
    if optimum is not None and ceiling is not None:
        ax.plot([ceiling], [optimum], "*", color="#D11A2A", markersize=20,
                markeredgecolor="white", markeredgewidth=0.9, zorder=5,
```

```python
                label="densest feasible graph")
    ax.set_xlabel(connectivity_label + " of the visited graph")
    ax.set_ylabel(edge_label)
    # No panel title: the suptitle and the feasible/infeasible labels describe
    # the panel; a panel title would overlap with the labels above the axes box.
    ax.xaxis.set_major_locator(MaxNLocator(integer=True))
    ax.grid(True, alpha=0.3)
    handles, labels = ax.get_legend_handles_labels()
    if labels:
        ax.legend(loc="upper left", fontsize=8, framealpha=0.9)
    cbar = fig.colorbar(sc, ax=ax, shrink=0.92)
    cbar.set_label(r"step (cooling $\rightarrow$)")
    cbar.set_ticks([0, max_step // 4, max_step // 2,
                    3 * max_step // 4, max_step])

    # Right: horizontal histogram -- visit density per edge-count level.
    # Sync y-limits with the scatter manually (avoids tight_layout warning
    # that sharey axes cause inside _save's plt.tight_layout() call).
    if ax_hist is not None:
        visit_counts = Counter(edges)
        arc_vals = sorted(visit_counts)
        ax_hist.barh(arc_vals, [visit_counts[a] for a in arc_vals],
                     height=0.72, color=_KUL_BLUE, alpha=0.75, linewidth=0)
        ax_hist.set_ylim(ax.get_ylim())
        ax_hist.set_xlabel("visits", fontsize=8)
        ax_hist.set_title("Distribution", fontsize=9)
        ax_hist.tick_params(axis="y", left=False, labelleft=False)
        ax_hist.xaxis.set_major_locator(MaxNLocator(integer=True, nbins=3))
        ax_hist.tick_params(labelsize=7)
        ax_hist.grid(True, axis="x", alpha=0.3)

    fig.suptitle(title, fontsize=12, y=1.04)
    _save(path)


def draw_graph_with_sensitivity(
    graph: Graph,
    path: str | Path,
    layout: dict | None = None,
    show_sigma: bool = False,
    node_labels: dict | None = None,
) -> None:
    """Draw a directed multigraph with every parallel arc drawn, coloured by sigma.

    Each parallel copy of an adjacency is rendered as its own curved arc, so the
    multiplicity ``mu(u, v)`` is read by *counting* arcs rather than from a label,
    and the whole adjacency is coloured by its sensitivity ``sigma(e)`` -- how far
    ``lambda^max`` falls when the adjacency is deleted -- on a four-level
    cool-to-hot palette.  An over-provisioned adjacency (several parallel copies
    but a downstream bottleneck) therefore shows as a bundle of arcs in a cool
    colour: the visual point that multiplicity and load-bearing role differ.

    Pass a ``layout`` dict mapping vertex id to (x, y) and a ``node_labels`` dict
    to override the numeric labels.  ``show_sigma`` is accepted for backward
    compatibility and ignored (sigma is always shown).
    """
    from matplotlib.patches import FancyArrowPatch
```

```python

    sensitivities = sensitivity_map(graph)
    if layout is None:
        # Evenly spaced on the unit circle (the same picture nx.circular_layout
        # draws), computed directly so drawing needs no networkx.
        verts = list(graph.vertices())
        layout = {v: (math.cos(2 * math.pi * i / len(verts)),
                      math.sin(2 * math.pi * i / len(verts)))
                  for i, v in enumerate(verts)}
    labels = (node_labels if node_labels is not None
              else {v: str(v) for v in graph.vertices()})

    fig, ax = plt.subplots(figsize=(8.6, 6.2))

    for (u, v), sigma in sensitivities.items():
        mu = graph.multiplicity(u, v)
        colour = _SIGMA_PALETTE[min(sigma, len(_SIGMA_PALETTE) - 1)]
        # Symmetric fan of curvatures, one per parallel copy.
        if mu <= 1:
            rads = [0.0]
        else:
            spread = 0.13 * (mu - 1)
            rads = [-spread + 2 * spread * k / (mu - 1) for k in range(mu)]
        for rad in rads:
            ax.add_patch(FancyArrowPatch(
                layout[u], layout[v], connectionstyle=f"arc3,rad={rad}",
                arrowstyle="-|>", mutation_scale=13, lw=2.3, color=colour,
                shrinkA=13, shrinkB=13, zorder=1))
        # One sigma label per adjacency, at the chord midpoint.
        (x0, y0), (x1, y1) = layout[u], layout[v]
        ax.text((x0 + x1) / 2, (y0 + y1) / 2, f"$\\sigma={sigma}$",
                fontsize=9, ha="center", va="center", zorder=3,
                bbox={"boxstyle": "round,pad=0.18", "fc": "white",
                      "ec": colour, "alpha": 0.9})

    xs = [p[0] for p in layout.values()]
    ys = [p[1] for p in layout.values()]
    ax.scatter(xs, ys, s=620, c=_KUL_DARK, edgecolors="white", linewidths=1.2,
               zorder=2)
    for v, (x, y) in layout.items():
        ax.text(x, y, str(labels.get(v, v)), color="white", fontweight="bold",
                ha="center", va="center", zorder=4)

    mx = (max(xs) - min(xs)) * 0.12 + 0.3
    my = (max(ys) - min(ys)) * 0.12 + 0.3
    ax.set_xlim(min(xs) - mx, max(xs) + mx)
    ax.set_ylim(min(ys) - my, max(ys) + my)
    ax.set_aspect("equal")
    ax.axis("off")
    ax.set_title("Parallel arcs drawn explicitly, coloured by sensitivity "
                 "$\\sigma$:\ncool ($\\sigma=0$, free) $\\to$ warm (load-bearing)",
                 fontsize=12)
    _save(path)


def _running_best_trace(result: SearchResult) -> tuple[list[float], list[int]]:
    """Best feasible edge count so far against wall-clock seconds, from a history.
```

```
    Reads the timed :class:`SearchStep` log (its ``best_feasible`` field already
    carries the densest feasible graph seen up to each step) and returns the timed
    convergence curve both engines record in their native fast modes.
    """
    xs = [record.seconds for record in result.history]
    ys = [record.best_feasible for record in result.history]
    return xs, ys


def _settle_time(xs: list[float], ys: list[int]) -> float:
    """Wall-clock second at which a trace first reached its own final best value.

    The convergence curves plateau long before the time budget runs out, so this
    is what the x-axis should track: cropping to the budget would leave a long
    flat tail and the cropped view would barely change when the budget does.
    """
    if not xs:
        return 0.0
    final = ys[-1]
    return next((x for x, y in zip(xs, ys) if y == final), xs[-1])


def plot_sa_vs_tabu_convergence(
    path: str | Path, *,
    cases: tuple[tuple[int, int], ...] = ((5, 3), (7, 3)),
    budget: float = 8.0,
    seed: int = 0,
    xmaxes: tuple[float, ...] | None = None,
    settle_margin: float = 1.3,
) -> None:
    """Best-feasible-value against wall-clock for annealing vs tabu search.

    For each ``(n, m)`` it runs one simulated-annealing schedule
    (:func:`search_for_dense_graph`) and one tabu schedule
    (:func:`tabu_search_for_dense_graph`) on the directed multigraph at an equal
    wall-clock ``budget``, and overlays their timed best-so-far traces against the
    conjectured optimum.  The chosen cases are where the two engines diverge: tabu
    assembles the bipartite extremiser and reaches the optimum while annealing
    plateaus a few arcs short.  The time axis is wall-clock, so this one figure is
    a representative timed run on the reference machine rather than a seed-exact
    reproduction like the others.

    By default each panel's x-axis stops shortly after the slower engine settles
    (``settle_margin`` times the later of the two plateau times), so the view
    tracks the actual run instead of a fixed window: cropping to a hard-coded
    second count leaves a long flat tail because both engines converge well
    inside one second.  Pass ``xmaxes`` to force fixed per-panel limits instead.
    """
    if not MATPLOTLIB_AVAILABLE:
        raise RuntimeError("matplotlib is required for figures")
    fig, axes = plt.subplots(1, len(cases), figsize=(11, 4.3))
    if len(cases) == 1:
        axes = [axes]
    for i, (ax, (n, m)) in enumerate(zip(axes, cases)):
        sa = search_for_dense_graph(MULTI_DIRECTED, n, m, seed=seed, steps=10**7,
                                    deadline=time.time() + budget)
        tb = tabu_search_for_dense_graph(MULTI_DIRECTED, n, m, seed=seed,
```

```
                                               steps=10**7, deadline=time.time() + budget)
        settle = 0.0
        for res, label, colour in ((sa, "simulated annealing", _KUL_BLUE),
                                   (tb, "tabu search", _WARM)):
            xs, ys = _running_best_trace(res)
            ax.step(xs, ys, where="post", label=label, color=colour, linewidth=2.0)
            settle = max(settle, _settle_time(xs, ys))
        opt = directed_multigraph_arc(n, m)
        ax.axhline(opt, linestyle=":", color=_KUL_DARK, linewidth=1.6,
                   label=f"optimum $L_3^{{\\mathrm{{dir}}}}({n}) = {opt}$")
        ax.set_title(f"directed multigraph, $n = {n}$, $m = {m}$", fontsize=11)
        ax.set_xlabel("wall-clock seconds", fontsize=9.5)
        ax.set_ylabel("densest feasible arc count", fontsize=9.5)
        # Crop the long flat tail: stop just after the slower engine settles, so
        # the rise fills the panel instead of being squeezed against the y-axis.
        # A fixed second count cannot do this, since the settle time shifts run to
        # run; an explicit xmaxes overrides for a reproducible fixed window.
        if xmaxes is not None and i < len(xmaxes):
            ax.set_xlim(0, xmaxes[i])
        else:
            ax.set_xlim(0, max(settle * settle_margin, 0.5))
        ax.grid(True, alpha=0.3)
        ax.legend(fontsize=8.5, loc="lower right")
        # Fewer x-ticks declutters the time axis: the action concentrates early
        # and dense ticks add noise.
        if MaxNLocator is not None:
            ax.xaxis.set_major_locator(MaxNLocator(nbins=5))
    _save(path)


def _save(path: str | Path, *, tight: bool = True) -> None:
    """Tighten the layout, write the file, and close the figure.

    ``tight=False`` skips ``tight_layout`` for the 3-D figures, where it
    misbehaves with the projected axes (they manage their own spacing).
    """
    import warnings
    Path(path).parent.mkdir(parents=True, exist_ok=True)
    if tight:
        # tight_layout can warn when figures contain colorbars or shared axes;
        # bbox_inches="tight" in savefig handles the actual bounding box so the
        # layout warning is cosmetic and safe to suppress.
        with warnings.catch_warnings():
            warnings.simplefilter("ignore", UserWarning)
            plt.tight_layout()
    plt.savefig(path, dpi=150, bbox_inches="tight")
    plt.close()  # release the figure so head-less runs don't leak memory


def plot_complexity_growth(path: str | Path) -> None:
    """The combinatorial explosion: number of graphs to enumerate, by direction.

    Two panels, directed (left) and undirected (right), sharing one axis range.
    Each plots, on a log scale, how many graphs blind enumeration must visit
    against the vertex count ``n``, for the simple model, two multigraph
    connectivity caps, and (dotted) the 3-uniform hypergraph, with a dashed line
```

```python
    at a generous budget of 10^9 candidates.  The count does not depend on the
    edge-versus-vertex separation, because both search the very same set of
    graphs, so one pair of panels covers all of those variants at once.
    Direction doubles the number of cells to fill, which is the whole difference
    between the two panels.
    """
    ns = list(range(2, 13))

    def log10_pow(base: float, exponent: float) -> float:
        return exponent * math.log10(base)

    def curves(directed: bool):
        # Each model fills a number of off-diagonal cells; the candidate count is
        # (values per cell) ^ (number of cells).  Labels stay plain -- the precise
        # exponential forms live in the caption, not the legend.  Direction
        # multiplies the cell count: by 2 for graphs (ordered vs unordered pairs,
        # n(n-1) vs C(n,2)), but by 3 for 3-uniform hypergraphs, since a directed
        # hyperedge is a tail plus 2 heads (n*C(n-1,2) = 3*C(n,3) candidates).
        cells = (lambda n: n * (n - 1)) if directed else (lambda n: n * (n - 1) / 2)
        hyper_cells = ((lambda n: n * (n - 1) * (n - 2) / 2) if directed
                       else (lambda n: n * (n - 1) * (n - 2) / 6))
        return [
            ("simple / random", _KUL_BLUE, "-",
             [log10_pow(2, cells(n)) for n in ns]),
            ("multigraph $m=3$", _KUL_DARK, "-",
             [log10_pow(3, cells(n)) for n in ns]),
            ("multigraph $m=4$", _WARM, "-",
             [log10_pow(4, cells(n)) for n in ns]),
            ("$3$-uniform hypergraph", _VIOLET, ":",
             [log10_pow(2, hyper_cells(n)) for n in ns]),
        ]

    fig, axes = plt.subplots(1, 2, figsize=(11, 4.6), sharey=True)
    for ax, directed, title in [(axes[0], True, "directed graphs"),
                                (axes[1], False, "undirected graphs")]:
        for label, colour, style, vals in curves(directed):
            ax.plot(ns, vals, style, color=colour, marker="o", markersize=3.5,
                    label=label)
        ax.axhline(9, color="gray", linestyle="--", linewidth=1,
                   label=r"$10^{9}$ budget")
        ax.set_title(title)
        ax.set_xlabel("vertices $n$")
        ax.grid(True, alpha=0.3)
    # Cap the shared vertical axis at 100 so the slowest curve (the directed
    # 3-uniform hypergraph) runs off the top instead of compressing every other
    # curve into a thin band near the bottom.  sharey=True propagates the limit.
    axes[0].set_ylim(0, 100)
    axes[0].annotate(r"$3$-uniform directed" + "\n" + r"continues off the top",
                     xy=(9.55, 99.3), xytext=(7.5, 92), fontsize=8, color=_VIOLET,
                     ha="center", va="center",
                     arrowprops=dict(arrowstyle="->", color=_VIOLET, lw=1.0))
    axes[0].set_ylabel(r"$\log_{10}$ (number of graphs)")
    axes[0].legend(fontsize=8.5, loc="upper left", title="model")
    fig.suptitle("Number of graphs blind enumeration must visit",
                 fontsize=12)
    _save(path)
```

```python

def _circle_layout(n, hub=None):
    """Vertex positions for a panel: all on a unit circle, or the ``hub`` vertex
    at the centre and the rest on the circle, which draws stars and hub graphs
    far more clearly than a plain circle does."""
    if hub is None:
        angs = [math.pi / 2 - 2 * math.pi * k / n for k in range(n)]
        return [(math.cos(a), math.sin(a)) for a in angs]
    pos = [(0.0, 0.0) for _ in range(n)]
    others = [k for k in range(n) if k != hub]
    for idx, k in enumerate(others):
        a = math.pi / 2 - 2 * math.pi * idx / len(others)
        pos[k] = (math.cos(a), math.sin(a))
    return pos


def _draw_extremal_panel(ax, matrix, *, directed: bool, hub=None) -> None:
    """Draw one extremal multiplicity matrix on ``ax`` as a graph.

    Vertices sit on a circle (or, when ``hub`` is given, that vertex sits at the
    centre and the rest on the circle).  An adjacency of multiplicity one is a
    single line (an arrowed line when ``directed``), and a higher multiplicity is
    shown by a small count label rather than parallel curves, so the picture
    stays legible.  Opposite arcs of a bidirected pair are bent apart.

    Every named family the gallery draws is *uniform*: all its adjacencies carry
    one and the same multiplicity.  Repeating that count on every arc buries the
    structure under labels (worst on the bidirected ``K_4``, twelve labels over
    six crossing arcs), so a uniform multiplicity above one is stated once in a
    corner note, and the per-arc labels are kept only for the genuinely mixed
    case.  ``hub`` may also be an explicit list of ``(x, y)`` positions, used
    for the bipartite wall panel, where two columns read far better than a circle.
    """
    from matplotlib.patches import FancyArrowPatch
    n = len(matrix)
    pos = hub if isinstance(hub, list) else _circle_layout(n, hub)
    mults = {int(matrix[i][j]) for i in range(n) for j in range(n)
             if i != j and matrix[i][j] > 0}
    uniform = mults.pop() if len(mults) == 1 else None
    per_arc_labels = uniform is None

    def edge(i, j, rad, arrow):
        ax.add_patch(FancyArrowPatch(
            pos[i], pos[j], connectionstyle=f"arc3,rad={rad}",
            arrowstyle=("-|>" if arrow else "-"), mutation_scale=12,
            lw=1.7, color=_KUL_BLUE, shrinkA=10, shrinkB=10, zorder=1))

    def mlabel(i, j, rad, mu):
        if mu <= 1:
            return
        mx, my = (pos[i][0] + pos[j][0]) / 2, (pos[i][1] + pos[j][1]) / 2
        dx, dy = pos[j][0] - pos[i][0], pos[j][1] - pos[i][1]
        length = math.hypot(dx, dy) or 1.0
        mx += (rad * 0.9 + 0.06) * (-dy) / length
        my += (rad * 0.9 + 0.06) * (dx) / length
        ax.text(mx, my, f"$\\times{mu}$", fontsize=8, ha="center", va="center",
                color=_KUL_DARK, zorder=3,
                bbox=dict(boxstyle="round,pad=0.08", fc="white", ec="none"))
```

```python

    if directed:
        for i in range(n):
            for j in range(n):
                if i == j or matrix[i][j] == 0:
                    continue
                rad = 0.16 if matrix[j][i] else 0.0
                edge(i, j, rad, True)
                if per_arc_labels:
                    mlabel(i, j, rad, matrix[i][j])
    else:
        for i in range(n):
            for j in range(i + 1, n):
                if matrix[i][j] == 0:
                    continue
                edge(i, j, 0.0, False)
                if per_arc_labels:
                    mlabel(i, j, 0.0, matrix[i][j])
    for k, (x, y) in enumerate(pos):
        ax.scatter([x], [y], s=300, c="white", edgecolors=_KUL_BLUE,
                   linewidths=1.6, zorder=2)
        ax.text(x, y, str(k + 1), ha="center", va="center", fontsize=9,
                zorder=3, color=_KUL_DARK)
    if uniform is not None and uniform > 1:
        word = "arcs" if directed else "edges"
        ax.text(0.0, -1.34, f"all {word} $\\times{uniform}$", ha="center",
                va="center", fontsize=9, color=_KUL_DARK)
    ax.set_xlim(-1.45, 1.45)
    ax.set_ylim(-1.45, 1.45)


# metro-line palette for hypergraph panels: red, blue, green, orange, purple, grey
_GALLERY_METRO = ["#C85050", "#1D8DB0", "#3CA050", "#DC8C28", "#8C50A0", "#787878"]


def _draw_hyper_panel(ax, hyperedges, n, *, directed: bool, hub=None) -> None:
    """Draw a hypergraph extremiser as a metro map: vertices on a circle (or a
    hub at the centre), each hyperedge a thick coloured line through its members
    when undirected, or a fan of coloured arrows from its tail when directed."""
    from matplotlib.patches import FancyArrowPatch
    pos = _circle_layout(n, hub)
    for idx, he in enumerate(hyperedges):
        col = _GALLERY_METRO[idx % len(_GALLERY_METRO)]
        if directed:
            tail, heads = he
            tx, ty = pos[tail]
            for h in heads:
                hx, hy = pos[h]
                ax.add_patch(FancyArrowPatch(
                    (tx, ty), (hx, hy), arrowstyle="-|>", mutation_scale=11,
                    lw=2.4, color=col, alpha=0.85, shrinkA=11, shrinkB=11,
                    zorder=1, connectionstyle="arc3,rad=0.10"))
        else:
            pts = [pos[v] for v in sorted(he)]
            cx = sum(p[0] for p in pts) / len(pts)
            cy = sum(p[1] for p in pts) / len(pts)
            pts.sort(key=lambda p: math.atan2(p[1] - cy, p[0] - cx))
```

```python
            ax.plot([p[0] for p in pts], [p[1] for p in pts], "-", color=col,
                    lw=4.2, solid_capstyle="round", solid_joinstyle="round",
                    alpha=0.85, zorder=1)
    for k, (x, y) in enumerate(pos):
        ax.scatter([x], [y], s=300, c="white", edgecolors="#333333",
                   linewidths=1.5, zorder=2)
        ax.text(x, y, str(k + 1), ha="center", va="center", fontsize=9,
                zorder=3, color="#222222")
    ax.set_xlim(-1.45, 1.45)
    ax.set_ylim(-1.45, 1.45)


def plot_extremal_gallery(path: str | Path, *,
                          gallery_json: str | Path | None = None) -> None:
    """A gallery of the named extremal graphs, drawn at a large representative size.

    These are the structured extremisers the analysis identifies, not the
    trivial small trees: the directed double and bidirected stars, the doubled
    bipartite wall, the dense bidirected complete graph, the multigraph star at
    full multiplicity, and the star hypertrees (drawn as metro maps). The program both builds these and,
    at small ``n``, proves them optimal by the enumeration of
    :func:`gallery_extremal_graphs`. The ``gallery_json`` argument is accepted for
    backward compatibility and ignored.
    """
    if not MATPLOTLIB_AVAILABLE:
        raise RuntimeError("matplotlib is required for figures")
    # dense bidirected complete graphs, built directly
    bidir_k4 = Graph(4, SIMPLE_DIRECTED)
    for i in range(4):
        for j in range(4):
            if i != j:
                bidir_k4.set_multiplicity(i, j, 1)
    # 2.B_{3,4}: the doubled one-directional bipartite wall, one of the three
    # proved 24-arc extremisers at n = 7, m = 3 (rem:odd-step-roadmap, fact (b)).
    # It ties the double star at the crossover, so the gallery shows both branches.
    bip_2b34 = Graph(7, MULTI_DIRECTED)
    for a in range(3):
        for b in range(3, 7):
            bip_2b34.set_multiplicity(a, b, 2)
    bip_pos = [(-0.85, 0.85), (-0.85, 0.0), (-0.85, -0.85),
               (0.85, 1.05), (0.85, 0.35), (0.85, -0.35), (0.85, -1.05)]
    multi_star = Graph(9, MULTI_UNDIRECTED)   # undirected star, mult 2
    for i in range(1, 9):
        multi_star.set_multiplicity(0, i, 2)
    dir_hypertree = [(0, frozenset({1, 2})), (0, frozenset({3, 4})),
                     (0, frozenset({5, 6}))]   # directed star hypertree on 7 vertices

    # ("g"/"h"/"hd", object, directed, hub, title)
    panels = [
        ("g", double_star(7, 3, directed=True), True, 0,
         "multigraph directed, arc\n$m=3$, $n=7$: double star ($24$ arcs)"),
        ("g", double_star(7, 2, directed=True), True, 0,
         "simple directed, arc\n$m=2$, $n=7$: bidirected star ($12$ arcs)"),
        ("g", multi_star, False, 0,
```

```
         "multigraph undirected, edge\n$m=3$, $n=9$: star at multiplicity $2$ ($16$ edges)"),
        ("g", bidir_k4, True, None,
         "simple directed, arc\n$m=4$, $n=4$: bidirected $K_4$ ($12$ arcs)"),
        ("g", bip_2b34, True, bip_pos,
         "multigraph directed, arc\n$m=3$, $n=7$: bipartite wall $2 \\cdot B_{3,4}$ ($24$ arcs)"),
        ("h", star_hypertree(10, 3), False, 0,
         "hypergraph, $r=3$\n$m=2$, $n=10$: star hypertree ($4$ hyperedges)"),
        ("h", star_hypertree(13, 4), False, 0,
         "hypergraph, $r=4$\n$m=2$, $n=13$: star hypertree ($4$ hyperedges)"),
        ("h", star_hypertree(16, 3), False, 0,
         "hypergraph, $r=3$\n$m=2$, $n=16$: star hypertree ($7$ hyperedges)"),
        ("hd", (dir_hypertree, 7), True, 0,
         "directed hypergraph, $r=3$\n$m=2$, $n=7$: tail-to-heads star"),
    ]
    fig, axes = plt.subplots(3, 3, figsize=(12, 12))
    for ax, (kind, obj, directed, hub, title) in zip(axes.flat, panels):
        if kind == "g":
            _draw_extremal_panel(ax, obj.mu.tolist(), directed=directed, hub=hub)
        elif kind == "h":
            _draw_hyper_panel(ax, list(obj.hyperedges), obj.num_vertices,
                              directed=False, hub=hub)
        else:  # "hd": (hyperedges, n)
            hes, nn = obj
            _draw_hyper_panel(ax, hes, nn, directed=True, hub=hub)
        ax.set_title(title, fontsize=9.5)
        ax.set_aspect("equal")
        ax.axis("off")
    fig.suptitle("Extremal graphs of the named families, drawn large", fontsize=14)
    _save(path)


_GUESS_STYLE = (0, (1, 1))   # dense dots: "we are very much guessing"


def plot_variant_grid(panels: list[dict], path: str | Path,
                      m: int | None = None) -> None:
    """All twelve variants on one grid, in the proved/conjectured/guessed language.

    ``panels`` is a row-major list of twelve dictionaries (rows = model simple,
    multi, hyper; columns = undirected edge, undirected vertex, directed arc,
    directed vertex).  Each panel may carry any of:

    ``proved`` ``(xs, ys)`` solid blue line, a theorem holding for all ``n``;
    ``conj``   ``(xs, ys)`` solid red line, a conjecture formula;
    ``guess``  ``(xs, ys)`` solid yellow line, a bare extrapolation of the
               machine points where no formula is known;
    ``band``   ``(xs, lo, hi)`` the certain interval, an easy construction below
               and the trivial maximum edge count above;
    ``exact``  ``(xs, ys)`` filled squares, sizes the machine proved;
    ``search`` ``(xs, ys)`` open circles, the search lower bounds.

    The point is one honest picture, distinguished by colour rather than dash
    pattern: a blue line is settled, a red line is conjectured, a yellow line is
    a guess, and the shaded band is the interval we are certain the truth lies in.
```

```
    """
    def draw_panel(ax, panel):
        band = panel.get("band")
        if band is not None:
            xs, lo, hi = band
            ax.fill_between(xs, lo, hi, color=_WARM, alpha=0.12,
                            label="certain interval")
            ax.plot(xs, lo, "-", color=_WARM, linewidth=0.8, alpha=0.6)
            ax.plot(xs, hi, "-", color=_WARM, linewidth=0.8, alpha=0.6)
        for branch in panel.get("branches", []):
            bxs, bys, blabel = branch
            ax.plot(bxs, bys, ":", color=_KUL_LIGHT, linewidth=1.3, label=blabel)
        if panel.get("proved") is not None:
            xs, ys = panel["proved"]
            ax.plot(xs, ys, "-", color=_KUL_BLUE, linewidth=2.3, label="proved")
        if panel.get("conj") is not None:
            xs, ys = panel["conj"]
            ax.plot(xs, ys, "-", color=_RED, linewidth=2.0, label="conjectured")
        if panel.get("guess") is not None:
            xs, ys = panel["guess"]
            ax.plot(xs, ys, "-", color=_GUESS, linewidth=2.0,
                    label="guess (interpolated)")
        if panel.get("exact") is not None:
            xs, ys = panel["exact"]
            if len(xs):
                ax.plot(xs, ys, "s", color=_GREEN, markersize=7,
                        label="machine-checked (exact)")
        if panel.get("search") is not None:
            xs, ys = panel["search"]
            if len(xs):
                ax.plot(xs, ys, "o", mfc="none", mec=_VIOLET, mew=1.8,
                        markersize=8, label="search (lower bound)")
        ax.set_title(panel["title"], fontsize=9.5)
        ax.set_xlabel("vertices $n$", fontsize=8.5)
        ax.set_ylabel(panel.get("ylabel", "edges"), fontsize=8.5)
        ax.tick_params(labelsize=8)
        ax.grid(True, alpha=0.3)
        ax.legend(fontsize=6.8, loc="upper left", framealpha=0.85)

    # The line styles (solid / dashed / dotted) are named in the per-panel
    # legends and spelled out in the caption, so the suptitle stays short and
    # just records the variant family and the threshold m.
    suptitle = "Erdős 915 across the twelve variants"
    if m is not None:
        suptitle += fr",  $m = {m}$"
    _variant_panel_grid(draw_panel, configs=panels, suptitle=suptitle, path=path,
                        suptitle_fontsize=13)
```

C.1.7 Walking the whole space

At small sizes it is possible to visit every graph there is and record what it looks like, which is what these routines do. The result is the distribution behind the landscape figures, and, more importantly, a way to confirm a claimed maximum by exhaustion rather than by search. The three-dimensional surfaces are built on the same idea, run across a grid of sizes and route limits and cached so that the sweep is paid for once.

```
# --- ENUMERATION LANDSCAPES: visit every labeled graph, collect (edges, lambda^max) ---

# Row-major table of all twelve variants, matching gather_variant_grid.
# ``enum_n`` is the vertex count used for full enumeration.
# ``max_mult`` caps per-cell multiplicity for multigraph variants.
_VARIANT_ENUM_CONFIGS: list[dict] = [
    # row 1 -- simple
    dict(key="simple_undirected_edge",   title="simple undirected edge",
         directed=False, simple=True,  hypergraph=False, r=3, separation="edge",
         enum_n=6, max_mult=1),
    dict(key="simple_undirected_vertex", title="simple undirected vertex",
         directed=False, simple=True,  hypergraph=False, r=3, separation="vertex",
         enum_n=6, max_mult=1),
    dict(key="simple_directed_edge",     title="simple directed arc",
         directed=True,  simple=True,  hypergraph=False, r=3, separation="edge",
         enum_n=4, max_mult=1),
    dict(key="simple_directed_vertex",   title="simple directed vertex",
         directed=True,  simple=True,  hypergraph=False, r=3, separation="vertex",
         enum_n=4, max_mult=1),
    # row 2 -- multigraph
    dict(key="multi_undirected_edge",    title="multigraph undirected edge",
         directed=False, simple=False, hypergraph=False, r=3, separation="edge",
         enum_n=5, max_mult=2),
    dict(key="multi_undirected_vertex",  title="multigraph undirected vertex",
         directed=False, simple=False, hypergraph=False, r=3, separation="vertex",
         enum_n=5, max_mult=2),
    dict(key="multi_directed_edge",      title="multigraph directed arc",
         directed=True,  simple=False, hypergraph=False, r=3, separation="edge",
         enum_n=3, max_mult=5),
    dict(key="multi_directed_vertex",    title="multigraph directed vertex",
         directed=True,  simple=False, hypergraph=False, r=3, separation="vertex",
         enum_n=3, max_mult=5),
    # row 3 -- hypergraph r=3
    dict(key="hyper_undirected_edge",    title="hypergraph undirected edge",
         directed=False, simple=True,  hypergraph=True,  r=3, separation="edge",
         enum_n=5, max_mult=1),
    dict(key="hyper_undirected_vertex",  title="hypergraph undirected vertex",
         directed=False, simple=True,  hypergraph=True,  r=3, separation="vertex",
         enum_n=5, max_mult=1),
    dict(key="hyper_directed_edge",      title="hypergraph directed arc",
         directed=True,  simple=True,  hypergraph=True,  r=3, separation="edge",
         enum_n=4, max_mult=1),
    dict(key="hyper_directed_vertex",    title="hypergraph directed vertex",
         directed=True,  simple=True,  hypergraph=True,  r=3, separation="vertex",
         enum_n=4, max_mult=1),
]


def enumerate_all_graphs(
    n: int, *,
    directed: bool,
    simple: bool,
    hypergraph: bool = False,
    r: int = 3,
    max_mult: int = 3,
    separation: str = "edge",
) -> list[tuple[int, int]]:
```

```
    """Return ``(edge_count, lambda_max)`` for every labeled graph of this type on ``n`` vertices.

    For simple graphs each cell is 0/1; for multigraphs each cell ranges over
    ``{0,...,max_mult}``.  For hypergraphs each candidate hyperedge is present or
    absent.  The list covers the full labeled graph space (isomorphic copies
    included), which is what we want for the distribution over the search space.
    """
    results: list[tuple[int, int]] = []

    if hypergraph:
        vertex_split = (separation == "vertex")
        candidates = _hyperedge_candidates(n, r, directed)
        for mask in product((0, 1), repeat=len(candidates)):
            chosen = [candidates[i] for i, flag in enumerate(mask) if flag]
            h = Hypergraph(n, chosen, directed=directed)
            conn = max_hyper_connectivity(h, vertex_split=vertex_split)
            results.append((h.edge_count(), conn))
        return results

    variant = _variant_for(directed, simple)
    measure = _connectivity_measure(separation)
    cells = _matrix_cells(n, directed)
    span = 2 if simple else (max_mult + 1)
    base = Graph(n, variant)
    for values in product(range(span), repeat=len(cells)):
        candidate = base.copy()
        for (u, v), value in zip(cells, values):
            if value:
                candidate.set_multiplicity(u, v, value)
        conn = measure(candidate)
        results.append((candidate.edge_count(), conn))
    return results


def _pair_connectivities(obj, *, separation: str, hypergraph: bool) -> list[int]:
    """Local connectivity of every vertex pair of one graph or hypergraph.

    The same pair sweep ``max_connectivity`` runs, but keeping each value rather
    than only the maximum.  For the edge case the capacity matrix is built once
    and reused across pairs (as ``max_connectivity`` does); vertex and hypergraph
    cases reuse their named per-pair routines.
    """
    vsplit = (separation == "vertex")
    if hypergraph:
        return [hyper_connectivity(obj, s, t, vertex_split=vsplit)
                for s, t in _pairs(obj)]
    if not vsplit:
        csr = _csr(obj.mu, dtype=int)
        return [int(_csgraph_maxflow(csr, s, t).flow_value) for s, t in _pairs(obj)]
    return [local_connectivity(obj, s, t, vertex_split=True) for s, t in _pairs(obj)]


def enumerate_pair_connectivities(
    n: int, *,
    directed: bool,
```

```python
    simple: bool,
    hypergraph: bool = False,
    r: int = 3,
    max_mult: int = 3,
    separation: str = "edge",
) -> dict[tuple[int, int], int]:
    """Pooled 2-D histogram ``{(lambda_max, pair_conn): count}`` over all graphs.

    For every labeled graph on ``n`` vertices and every vertex pair, record the
    pair's local connectivity tagged with the graph's own ``lambda^max``.  The
    result is a small threshold-independent table: at plot time a feasibility
    threshold ``T`` splits each pair-connectivity column into observations from
    graphs that stay at ``lambda^max <= T`` and those that exceed it.  Much
    smaller than the raw observation list (a few dozen entries per variant), so
    it pickles compactly even though the sweep itself enumerates every graph.
    """
    from collections import Counter
    table: Counter = Counter()

    def record(obj):
        pcs = _pair_connectivities(obj, separation=separation, hypergraph=hypergraph)
        lmax = max(pcs, default=0)
        for c in pcs:
            table[(lmax, c)] += 1

    if hypergraph:
        candidates = _hyperedge_candidates(n, r, directed)
        for mask in product((0, 1), repeat=len(candidates)):
            chosen = [candidates[i] for i, flag in enumerate(mask) if flag]
            record(Hypergraph(n, chosen, directed=directed))
        return dict(table)

    variant = _variant_for(directed, simple)
    cells = _matrix_cells(n, directed)
    span = 2 if simple else (max_mult + 1)
    base = Graph(n, variant)
    for values in product(range(span), repeat=len(cells)):
        candidate = base.copy()
        for (u, v), value in zip(cells, values):
            if value:
                candidate.set_multiplicity(u, v, value)
        record(candidate)
    return dict(table)


def compute_pair_enumeration_cache(
    cache_path: str | Path = "figures/pair_enumeration_cache.pkl",
    configs: list[dict] | None = None,
) -> dict[str, dict[tuple[int, int], int]]:
    """Per-variant pooled pair-connectivity tables, loading from cache if present.

    The pooled analogue of :func:`compute_enumeration_cache`: one
    ``{(lambda_max, pair_conn): count}`` table per variant.  Slow on first run
    (it re-sweeps the enumeration measuring every pair), cached thereafter.
    """
    cache_path = Path(cache_path)
    if configs is None:
```

```python
        configs = _VARIANT_ENUM_CONFIGS

    if cache_path.exists():
        with open(cache_path, "rb") as f:
            cached = pickle.load(f)
    else:
        cached = {}

    changed = False
    for cfg in configs:
        key = cfg["key"]
        if key in cached:
            continue
        print(f"  pooling pairs for {cfg['title']} (n={cfg['enum_n']})...", flush=True)
        cached[key] = enumerate_pair_connectivities(
            cfg["enum_n"],
            directed=cfg["directed"], simple=cfg["simple"],
            hypergraph=cfg["hypergraph"], r=cfg["r"],
            max_mult=cfg["max_mult"], separation=cfg["separation"])
        changed = True

    if changed:
        cache_path.parent.mkdir(parents=True, exist_ok=True)
        with open(cache_path, "wb") as f:
            pickle.dump(cached, f)

    return cached


def compute_enumeration_cache(
    cache_path: str | Path = "figures/enumeration_cache.pkl",
    configs: list[dict] | None = None,
) -> dict[str, list[tuple[int, int]]]:
    """Return the full enumeration for every variant, loading from cache if available.

    On first run this takes several minutes; subsequent runs load the pickle
    instantly.  The cache lives at ``cache_path`` relative to the caller's
    working directory.
    """
    cache_path = Path(cache_path)
    if configs is None:
        configs = _VARIANT_ENUM_CONFIGS

    if cache_path.exists():
        with open(cache_path, "rb") as f:
            cached = pickle.load(f)
    else:
        cached = {}

    changed = False
    for cfg in configs:
        key = cfg["key"]
        if key in cached:
            continue
        print(f"  enumerating {cfg['title']} (n={cfg['enum_n']})...", flush=True)
        data = enumerate_all_graphs(
```

```
            cfg["enum_n"],
            directed=cfg["directed"],
            simple=cfg["simple"],
            hypergraph=cfg["hypergraph"],
            r=cfg["r"],
            max_mult=cfg["max_mult"],
            separation=cfg["separation"],
        )
        cached[key] = data
        changed = True

    if changed:
        cache_path.parent.mkdir(parents=True, exist_ok=True)
        with open(cache_path, "wb") as f:
            pickle.dump(cached, f)

    return cached


def plot_scatter_lambda_edges(
    enum_data: dict[str, list[tuple[int, int]]],
    path: str | Path,
) -> None:
    """Extremal envelope of edge count against binding connectivity, all twelve variants.

    For each variant the full enumeration is drawn as a faint grey cloud (one
    point per labeled graph) with the binding connectivity ``lambda^max`` on the
    horizontal axis and the edge count on the vertical, matching the
    edges-against-parameter layout of the all-variant grid.  The extremal
    envelope -- the densest feasible graph available at each connectivity level
    -- is overlaid as an integer staircase with one dot per level, so every
    connectivity value carries an exact extremal edge count and the frontier is
    read off without any interpolation between levels.
    """
    def draw_panel(ax, cfg):
        key = cfg["key"]
        data = enum_data.get(key, [])
        if not data:
            ax.set_title(cfg["title"], fontsize=9.5)
            return

        edges_arr = [e for e, _ in data]
        conn_arr  = [c for _, c in data]

        # Faint cloud of every graph: connectivity (x) against edge count (y).
        ax.scatter(conn_arr, edges_arr, s=1.5, c="grey", alpha=0.25,
                   linewidths=0, rasterized=True)

        # Extremal envelope e(lambda) = densest feasible graph with lambda^max
        # at most this level: a non-decreasing integer staircase.  Reading it at
        # lambda = m-1 gives the extremal value of Problem 915 at this n.
        levels = sorted(set(conn_arr))
        running, env_edges = 0, []
        for lv in levels:
            running = max(running, max(e for e, c in data if c == lv))
            env_edges.append(running)
        ax.step(levels, env_edges, where="mid", color=_KUL_BLUE, linewidth=1.8,
```

```
                zorder=3)
        ax.plot(levels, env_edges, "o", color=_KUL_BLUE, markersize=6,
                zorder=4, label="extremal envelope")

        ax.set_title(cfg["title"], fontsize=9.5)
        ax.set_xlabel(r"$\lambda^{\max}$", fontsize=8.5)
        ax.set_ylabel("edge count", fontsize=8.5)
        ax.tick_params(labelsize=8)
        # Both axes count discrete objects (independent routes, edges).
        ax.xaxis.set_major_locator(MaxNLocator(integer=True))
        ax.yaxis.set_major_locator(MaxNLocator(integer=True))
        ax.grid(True, alpha=0.3)
        ax.text(0.02, 0.98, f"$n={cfg['enum_n']}$", transform=ax.transAxes,
                ha="left", va="top", fontsize=8, color="grey")

    _variant_panel_grid(
        draw_panel, path=path,
        suptitle=("Extremal envelope: edge count against binding connectivity "
                  "across all twelve variants (full enumeration)"),
        suptitle_fontsize=13)


def _variant_panel_grid(draw_panel, *, suptitle: str, path: str | Path,
                        configs=None, suptitle_fontsize: float = 12.0,
                        row_label_fontsize: float = 12.0) -> None:
    """Shared scaffold for every twelve-panel variant grid (three model rows by
    four columns): the distribution grids, the proved/conjectured bound grid, the
    sampled grid, and the extremal-envelope scatter.  Builds the axes, calls
    ``draw_panel(ax, cfg)`` on each panel config in turn, adds the per-row model
    labels and the suptitle, and saves.  Only the per-panel drawing and the panel
    configs differ between the grids, so the caller supplies both; everything else
    (layout, row labels, save) lives here once.  ``configs`` defaults to the
    twelve enumeration variants but may be any 12-item list (e.g. the sampled
    configs or a precomputed ``panels`` list).
    """
    if configs is None:
        configs = _VARIANT_ENUM_CONFIGS
    fig, axes = plt.subplots(3, 4, figsize=(16, 11))
    for cfg, ax in zip(configs, axes.flat):
        draw_panel(ax, cfg)
    for row, name in enumerate(("simple", "multigraph", "hypergraph $r=3$")):
        axes[row, 0].annotate(name, xy=(-0.32, 0.5), xycoords="axes fraction",
                              rotation=90, ha="center", va="center",
                              fontsize=row_label_fontsize, fontweight="bold",
                              color=_KUL_DARK)
    fig.suptitle(suptitle, fontsize=suptitle_fontsize)
    fig.tight_layout(rect=(0.02, 0.0, 1.0, 0.97))
    _save(path)


def plot_conn_dist_grid(
    enum_data: dict[str, list[tuple[int, int]]],
    m: int,
    path: str | Path,
    known_maxima: dict[str, int] | None = None,
) -> None:
    """12-panel histogram of lambda_max distribution, all variants, fixed m.

```

```
    Bars to the left of the feasibility boundary m-1 are coloured blue (the
    graphs we study); bars at or above m are coloured red (infeasible for this
    m).  A vertical dashed line marks the boundary, and a dotted line marks
    the known or conjectured maximum if supplied in ``known_maxima``.
    """
    threshold = m - 1  # feasible means lambda_max <= threshold

    def draw_panel(ax, cfg):
        key = cfg["key"]
        data = enum_data.get(key, [])
        if not data:
            ax.set_title(cfg["title"], fontsize=9)
            return

        conn_vals = [c for _, c in data]
        lo, hi = min(conn_vals), max(conn_vals)
        levels = list(range(lo, hi + 1))
        counts = [conn_vals.count(lv) for lv in levels]
        bar_colours = [_KUL_BLUE if lv <= threshold else _RED for lv in levels]
        ax.bar(levels, counts, color=bar_colours, edgecolor="white", linewidth=0.4)

        ax.axvline(threshold + 0.5, color=_WARM, linestyle="--", linewidth=1.8)
        if known_maxima and key in known_maxima:
            ax.axvline(known_maxima[key], color=_KUL_DARK, linestyle=":",
                       linewidth=1.6, label=f"known max = {known_maxima[key]}")

        ax.set_title(cfg["title"], fontsize=9.5)
        ax.set_xlabel(r"$\lambda^{\max}$", fontsize=8.5)
        ax.set_ylabel("graphs", fontsize=8.5)
        ax.tick_params(labelsize=8)
        # Connectivity is integer-valued: force integer ticks so the directed
        # panels stop showing spurious half-integer labels (0.5, 1.5, ...).
        ax.set_xticks(levels)
        ax.grid(True, axis="y", alpha=0.3)
        # The feasibility boundary is identical in all twelve panels, so it is
        # explained once in the title and caption rather than repeated as a
        # legend in every panel. A legend is drawn only when a per-panel known
        # maximum line is present (a value that genuinely differs by panel).
        if known_maxima and key in known_maxima:
            ax.legend(fontsize=6.8, loc="upper right", framealpha=0.85)
        ax.text(0.02, 0.98, f"$n={cfg['enum_n']}$", transform=ax.transAxes,
                ha="left", va="top", fontsize=8, color="grey")

    suptitle = (fr"Connectivity distribution across all twelve variants, $m = {m}$ "
                fr"(blue = feasible $\lambda^{{\max}} \leq {threshold}$, red = infeasible)")
    _variant_panel_grid(draw_panel, suptitle=suptitle, path=path,
                        row_label_fontsize=12)


def _midrange_lambda_threshold(hi: int) -> int:
    """A per-variant connectivity boundary at the midpoint of the achievable range.

    The distribution figures split graphs by ``lambda^max`` into a blue (low) and
```

```
    a red (high) population.  A single global boundary collapses some panels to one
    colour (every graph below it, or almost none), because the twelve variants
    reach very different connectivity ranges at the sizes enumeration allows.
    Splitting instead at ``round(hi/2)`` -- the middle of what each enumeration can
    reach -- keeps both populations visible in every panel, scaled to what is
    possible there.  Floored at one so the boundary is never zero, which would
    push every graph above it and leave the blue (low) side empty.
    """
    return max(1, round(hi / 2))


def plot_pair_conn_dist_grid(
    pair_data: dict[str, dict[tuple[int, int], int]],
    path: str | Path,
) -> None:
    """12-panel histogram of per-PAIR connectivity, pooled over the full enumeration.

    Where :func:`plot_conn_dist_grid` plots one ``lambda^max`` per graph, this
    pools every vertex pair of every graph: one observation per (graph, pair).
    Each bar is split and STACKED by the connectivity of the graph the pair came
    from -- blue (bottom) for pairs from graphs whose ``lambda^max`` stays at or
    below the per-panel boundary ``T``, red (top) for pairs from graphs above it
    -- so the bar's full height is the true number of observations at that level.

    ``T`` is set per variant by :func:`_midrange_lambda_threshold`, not by a single
    global ``m``: at a fixed boundary some panels are entirely one colour, while the
    mid-range split keeps both populations visible everywhere.  Pairs whose own
    connectivity exceeds ``T`` can only come from graphs above ``T``, so those bars
    are wholly red; below ``T`` the colours mix, since a high-connectivity graph
    still has many low-connectivity pairs.
    """
    def draw_panel(ax, cfg):
        table = pair_data.get(cfg["key"], {})
        if not table:
            ax.set_title(cfg["title"], fontsize=9)
            return

        hi = max(lmax for (lmax, _pc) in table)
        T = _midrange_lambda_threshold(hi)
        levels = sorted({pc for (_lmax, pc) in table})
        blue = [sum(c for (lmax, pc), c in table.items()
                    if pc == lv and lmax <= T) for lv in levels]
        red = [sum(c for (lmax, pc), c in table.items()
                   if pc == lv and lmax > T) for lv in levels]

        ax.bar(levels, blue, color=_KUL_BLUE, edgecolor="white", linewidth=0.4,
               label=fr"from $\lambda^{{\max}}\!\leq\!{T}$")
        ax.bar(levels, red, bottom=blue, color=_RED, edgecolor="white",
               linewidth=0.4, label=fr"from $\lambda^{{\max}}\!>\!{T}$")
        ax.axvline(T + 0.5, color=_WARM, linestyle="--", linewidth=1.8)

        ax.set_title(cfg["title"], fontsize=9.5)
```

```python
        ax.set_xlabel("pair connectivity", fontsize=8.5)
        ax.set_ylabel("pair observations", fontsize=8.5)
        ax.tick_params(labelsize=8)
        ax.set_xticks(levels)
        ax.grid(True, axis="y", alpha=0.3)
        ax.legend(fontsize=6.8, loc="upper right", framealpha=0.85)
        ax.text(0.02, 0.98, f"$n={cfg['enum_n']}$", transform=ax.transAxes,
                ha="left", va="top", fontsize=8, color="grey")

    suptitle = ("Pair-connectivity distribution across all twelve variants "
                "(every vertex pair of every graph pooled, split at each variant's "
                r"mid-range $\lambda^{\max}$: blue from low-connectivity graphs, red from high)")
    _variant_panel_grid(draw_panel, suptitle=suptitle, path=path,
                        row_label_fontsize=12)


def plot_edge_dist_grid(
    enum_data: dict[str, list[tuple[int, int]]],
    path: str | Path,
) -> None:
    """12-panel histogram of edge count distribution, all variants.

    Each bar is split and STACKED by graph connectivity: blue (bottom) for graphs
    whose ``lambda^max`` stays at or below the per-panel boundary ``T``, red (top)
    for graphs above it, so the full bar height is the true number of graphs at
    that edge count.  ``T`` is the midpoint of each variant's achievable
    connectivity range (:func:`_midrange_lambda_threshold`); a single global
    boundary leaves some panels all one colour, the mid-range split keeps both
    populations visible.  A dotted line marks the densest graph at or below ``T``.
    """
    def draw_panel(ax, cfg):
        key = cfg["key"]
        data = enum_data.get(key, [])
        if not data:
            ax.set_title(cfg["title"], fontsize=9)
            return

        hi = max(c for _e, c in data)
        T = _midrange_lambda_threshold(hi)
        max_e = max(e for e, _ in data)
        levels = list(range(0, max_e + 1))
        blue = [0] * (max_e + 1)
        red = [0] * (max_e + 1)
        for e, c in data:
            (blue if c <= T else red)[e] += 1

        # Stacked, not overlaid: low-connectivity graphs (blue) on the bottom,
        # high-connectivity (red) on top, so the full height is the true count.
        ax.bar(levels, blue, color=_KUL_BLUE, edgecolor="white", linewidth=0.3,
               label=fr"$\lambda^{{\max}}\!\leq\!{T}$")
        ax.bar(levels, red, bottom=blue, color=_RED, edgecolor="white",
               linewidth=0.3, label=fr"$\lambda^{{\max}}\!>\!{T}$")

        # The densest graph in the lower-connectivity population: read straight
        # off the data, so it lands on the right edge of the blue mass.
        if any(blue):
```

```python
            kmax = max(e for e in levels if blue[e])
            ax.axvline(kmax, color=_KUL_DARK, linestyle=":", linewidth=1.8,
                       label=fr"densest $\leq\!{T}$: {kmax}")

        ax.set_title(cfg["title"], fontsize=8.5)
        ax.set_xlabel("edge count", fontsize=7.5)
        ax.set_ylabel("graphs", fontsize=7.5)
        ax.tick_params(labelsize=7)
        # Edge counts are integers: keep the tick labels integer-valued.
        ax.xaxis.set_major_locator(MaxNLocator(integer=True))
        ax.grid(True, axis="y", alpha=0.25)
        ax.legend(fontsize=6.2, loc="upper right", framealpha=0.8)
        ax.text(0.98, 0.02, f"$n={cfg['enum_n']}$", transform=ax.transAxes,
                ha="right", va="bottom", fontsize=7, color="grey")

    suptitle = ("Edge-count distribution across all twelve variants "
                r"(stacked by connectivity, split at each variant's mid-range "
                r"$\lambda^{\max}$: blue low-connectivity graphs, red high)")
    _variant_panel_grid(draw_panel, suptitle=suptitle, path=path,
                        row_label_fontsize=11)


# --- 3-D BOUND SURFACE: cache solve() over (variant, n, m) grid; plot_variant_3d_surfaces draws it ---

# The two multigraph vertex problems reduce exactly to the simple ones: vertex
# mode is blind to parallel copies (see _split_capacity_matrix), so a multigraph
# and its underlying simple graph have the same vertex connectivity.  We mirror
# the simple-vertex surface into these keys rather than searching multigraphs,
# whose degenerate free-parallel fills would report a lower bound far above the
# true (simple) value and spike the plot.
_SURFACE_ALIASED_VERTEX = {
    "multi_undirected_vertex": "simple_undirected_vertex",
    "multi_directed_vertex":   "simple_directed_vertex",
}

# 12 variant configs for the surface computation.
_SURFACE_VARIANT_CONFIGS: list[dict] = [
    dict(key="simple_undirected_edge",   title="simple undirected edge",
         directed=False, simple=True,  hypergraph=False, r=3, separation="edge"),
    dict(key="simple_undirected_vertex", title="simple undirected vertex",
         directed=False, simple=True,  hypergraph=False, r=3, separation="vertex"),
    dict(key="simple_directed_edge",     title="simple directed arc",
         directed=True,  simple=True,  hypergraph=False, r=3, separation="edge"),
    dict(key="simple_directed_vertex",   title="simple directed vertex",
         directed=True,  simple=True,  hypergraph=False, r=3, separation="vertex"),
    dict(key="multi_undirected_edge",    title="multigraph undirected edge",
         directed=False, simple=False, hypergraph=False, r=3, separation="edge"),
    dict(key="multi_undirected_vertex",  title="multigraph undirected vertex",
         directed=False, simple=False, hypergraph=False, r=3, separation="vertex"),
    dict(key="multi_directed_edge",      title="multigraph directed arc",
         directed=True,  simple=False, hypergraph=False, r=3, separation="edge"),
    dict(key="multi_directed_vertex",    title="multigraph directed vertex",
         directed=True,  simple=False, hypergraph=False, r=3, separation="vertex"),
```

```
    dict(key="hyper_undirected_edge",    title="hypergraph undirected edge",
         directed=False, simple=True,  hypergraph=True,  r=3, separation="edge"),
    dict(key="hyper_undirected_vertex",  title="hypergraph undirected vertex",
         directed=False, simple=True,  hypergraph=True,  r=3, separation="vertex")
             ,
    dict(key="hyper_directed_edge",      title="hypergraph directed arc",
         directed=True,  simple=True,  hypergraph=True,  r=3, separation="edge"),
    dict(key="hyper_directed_vertex",    title="hypergraph directed vertex",
         directed=True,  simple=True,  hypergraph=True,  r=3, separation="vertex")
             ,
]


def _surface_known_value(vkey: str, n: int, m: int) -> int | None:
    """The proved exact value at ``(n, m)`` for ``vkey``, or ``None`` if open here
        .

    Returns the closed-form extremal value precisely in the regime where the
    thesis proves it, so the surface can colour those bars as exact (blue)
    rather than leaving every bar a search lower bound.  Outside the proved
    regime it returns ``None`` and the caller falls back to a discovery search
    (a purple lower bound).  Each value is capped at the trivial maximum for
    its model, since the closed form overshoots in the small-``n`` large-``m``
    corner where no graph that dense exists.
    """
    tri_simple = n * (n - 1) // 2
    tri_dir = n * (n - 1)
    tri_hyp = math.comb(n, 3)
    if vkey == "simple_undirected_edge":             # Mader, all m
        return min(simple_undirected_edge(n, m), tri_simple)
    if vkey == "simple_undirected_vertex":           # Leonard, m<=4
        return min(simple_undirected_edge(n, m), tri_simple) if m <= 4 else None
    if vkey == "simple_directed_edge":               # exact only at m=2
        return min(directed_arc_lower_bound(n, 2), tri_dir) if m == 2 else None
    if vkey == "simple_directed_vertex":             # exact only at m=2
        return min(directed_arc_lower_bound(n, 2), tri_dir) if m == 2 else None
    if vkey == "multi_undirected_edge":              # multi-tree bound, all m
        return min(multigraph_undirected_edge(n, m), (m - 1) * tri_simple)
    if vkey == "multi_undirected_vertex":            # = simple vertex, m<=4
        return min(simple_undirected_edge(n, m), tri_simple) if m <= 4 else None
    if vkey == "multi_directed_edge":                # cut-counting proves n<=6
        return min(directed_multigraph_arc(n, m), (m - 1) * tri_dir) if n <= 6 else None
    if vkey == "multi_directed_vertex":              # = simple digraph, exact m=2
        return min(directed_arc_lower_bound(n, 2), tri_dir) if m == 2 else None
    if vkey == "hyper_undirected_edge":              # incidence-rank, all m
        return min(hypergraph_edge(n, m, 3), tri_hyp)
    if vkey == "hyper_undirected_vertex":            # incidence-rank lemma, m<=3
        return min(hypergraph_edge(n, m, 3), tri_hyp) if m <= 3 else None
    # hyper_directed_edge, hyper_directed_vertex: open (new model), no formula.
    return None


def compute_surface_cache(
    n_range: tuple[int, int] = (3, 9),
    m_range: tuple[int, int] = (2, 6),
    max_seconds: float = 20.0,
    cache_path: str | Path = "figures/surface_cache.json",
```

```python
) -> dict:
    """Compute the optimal bound at every (variant, n, m) triple, save to a JSON cache.

    Returns the cache dict: ``{variant_key: {str(n): {str(m): {value, bound}}}}``.
    Where the value is proved (see :func:`_surface_known_value`) the bound is
    recorded as ``"exact"`` using the closed form; otherwise a discovery search
    supplies a ``"lower"`` bound.  Missing entries are skipped on later runs.
    """
    cache_path = Path(cache_path)
    if cache_path.exists():
        with open(cache_path) as f:
            cache = json.load(f)
    else:
        cache = {}

    changed = False
    ns = list(range(n_range[0], n_range[1] + 1))
    ms = list(range(m_range[0], m_range[1] + 1))

    for cfg in _SURFACE_VARIANT_CONFIGS:
        vkey = cfg["key"]
        if vkey in _SURFACE_ALIASED_VERTEX:
            continue  # filled by mirroring its simple counterpart, below
        if vkey not in cache:
            cache[vkey] = {}
        for n in ns:
            sn = str(n)
            if sn not in cache[vkey]:
                cache[vkey][sn] = {}
            for m in ms:
                sm = str(m)
                known = _surface_known_value(vkey, n, m)
                if known is not None:
                    # Proved regime: use the closed form, mark it exact (blue).
                    # Always (re)write, so an older cache whose every cell was
                    # tagged "lower" is corrected here without recomputation.
                    entry = {"value": known, "bound": "exact"}
                    if cache[vkey][sn].get(sm) != entry:
                        cache[vkey][sn][sm] = entry
                        changed = True
                    continue
                if sm in cache[vkey][sn]:
                    continue  # open cell already searched; keep the lower bound
                print(f"  surface: {cfg['title']}  n={n}  m={m}", flush=True)
                res = solve(
                    n, m,
                    directed=cfg["directed"],
                    simple=cfg["simple"],
                    hypergraph=cfg["hypergraph"],
                    r=cfg["r"],
                    exhaustive=False,
                    separation=cfg["separation"],
                    max_seconds=max_seconds,
                    seed=0,
                )
                cache[vkey][sn][sm] = {"value": res.value, "bound": res.bound}
                changed = True
```

```python

    # Mirror the simple-vertex surfaces into their multigraph aliases, so the two
    # provably-equal problems show identical bars instead of two independent
    # (and possibly degenerate) searches.
    for multi_key, simple_key in _SURFACE_ALIASED_VERTEX.items():
        if simple_key in cache and cache.get(multi_key) != cache[simple_key]:
            cache[multi_key] = copy.deepcopy(cache[simple_key])
            changed = True

    if changed:
        cache_path.parent.mkdir(parents=True, exist_ok=True)
        with open(cache_path, "w") as f:
            json.dump(cache, f)

    return cache


def plot_variant_3d_surfaces(
    cache_path: str | Path,
    path: str | Path,
) -> None:
    """3-D bar chart of the optimal bound over the (n, m) grid, all twelve variants.

    Each bar's height is the bound value; exact points are blue, lower-bound
    points are purple.  The 3x4 layout matches the flat grid figure.
    """
    with open(cache_path) as f:
        cache = json.load(f)

    fig = plt.figure(figsize=(20, 13))
    row_names = ("simple", "multigraph", "hypergraph $r=3$")

    for idx, cfg in enumerate(_SURFACE_VARIANT_CONFIGS):
        vkey = cfg["key"]
        ax = fig.add_subplot(3, 4, idx + 1, projection="3d")

        variant_data = cache.get(vkey, {})
        ns_exact, ms_exact, vs_exact = [], [], []
        ns_lower, ms_lower, vs_lower = [], [], []

        for sn, m_dict in sorted(variant_data.items(), key=lambda x: int(x[0])):
            n = int(sn)
            for sm, entry in sorted(m_dict.items(), key=lambda x: int(x[0])):
                m_val = int(sm)
                v = entry["value"]
                if entry["bound"] == "exact":
                    ns_exact.append(n)
                    ms_exact.append(m_val)
                    vs_exact.append(v)
                else:
                    ns_lower.append(n)
                    ms_lower.append(m_val)
                    vs_lower.append(v)

        dx = dy = 0.6
        if ns_lower:
            ax.bar3d(
```

```
                [x - dx / 2 for x in ns_lower],
                [y - dy / 2 for y in ms_lower],
                [0] * len(vs_lower),
                dx, dy, vs_lower,
                color=_VIOLET, alpha=0.7, shade=True,
                edgecolor="white", linewidth=0.25,
            )
        if ns_exact:
            ax.bar3d(
                [x - dx / 2 for x in ns_exact],
                [y - dy / 2 for y in ms_exact],
                [0] * len(vs_exact),
                dx, dy, vs_exact,
                color=_KUL_BLUE, alpha=0.9, shade=True,
                edgecolor="white", linewidth=0.25,
            )

        # Same camera for every panel so the twelve are read side by side, and a
        # z-axis anchored at 0 so bar heights are comparable within a panel.
        ax.view_init(elev=24, azim=-58)
        peak = max(vs_exact + vs_lower, default=1)
        ax.set_zlim(0, peak * 1.05)
        for pane in (ax.xaxis, ax.yaxis, ax.zaxis):
            pane.pane.set_alpha(0.04)

        ax.set_title(cfg["title"], fontsize=7.5, pad=2)
        ax.set_xlabel("$n$", fontsize=7, labelpad=2)
        ax.set_ylabel("$m$", fontsize=7, labelpad=2)
        ax.set_zlabel("bound", fontsize=7, labelpad=2)
        ax.tick_params(labelsize=6)

    # Row labels
    for row, name in enumerate(row_names):
        row_ax = fig.axes[row * 4]
        row_ax.text2D(-0.28, 0.5, name, transform=row_ax.transAxes,
                      rotation=90, ha="center", va="center",
                      fontsize=11, fontweight="bold", color=_KUL_DARK)

    # Manual legend patches
    from matplotlib.patches import Patch
    legend_elems = [
        Patch(facecolor=_KUL_BLUE, alpha=0.85, label="exact (proved)"),
        Patch(facecolor=_VIOLET,   alpha=0.65, label="lower bound (search)"),
    ]
    fig.legend(handles=legend_elems, loc="lower center", ncol=2,
               fontsize=10, framealpha=0.9)

    fig.suptitle("Erdős 915: optimal bound over $(n, m)$, "
                 "blue where proved, purple where only a search lower bound is known",
                 fontsize=13, y=0.99)
    _save(path, tight=False)


# --- 3-D THRESHOLD HISTOGRAM: lambda^max distribution vs p for three variants ---


def plot_conn_threshold_3d(
```

```
    path: str | Path,
    n: int = 30,
    m: int = 3,
    p_values: list[float] | None = None,
    samples: int = 400,
    seed: int = 7,
) -> None:
    """3-D bar chart of the lambda_max distribution over density, for three variants.

    x = density in units of that model's OWN threshold, y = lambda_max value,
    z = fraction of samples.  Each panel is swept in units of its own p*, and the
    threshold line sits at 1 in all three, because the models do NOT share a
    density scale.  The threshold is where a vertex's expected degree reaches m,
    which for a graph is p*(n-1) = m, giving p* = m/n, but for an r-uniform
    binomial hypergraph is p*C(n-1, r-1) = m, giving p* = m/C(n-1, r-1) =
    Theta(m/n^(r-1)).  Plotting the hypergraph panel against m/n would put the
    line in the wrong place by a factor of order n^(r-2).  Only the graph panels
    are covered by thm:gnp-threshold; the hypergraph panel is an observation.

    Three panels: undirected edge (uses ``n``), directed arc (uses ``n``),
    hypergraph edge (uses min(n, 8) to keep flow computation fast).
    """
    # Sweep in units of the model's own threshold, so all three panels are
    # directly comparable and each transition is visible where it actually is.
    ratios = ([i / 5.0 for i in range(1, 20)] if p_values is None else None)

    rng = random.Random(seed)
    # Hypergraph flow computation scales poorly: cap n at 8 for that panel.
    n_hyper = min(n, 8)

    variants_3d = [
        dict(label="undirected edge", directed=False, separation="edge",
             hypergraph=False, panel_n=n, p_star=m / n,
             star_text=f"$m/n = {m}/{n}$"),
        dict(label="directed arc",    directed=True,  separation="edge",
             hypergraph=False, panel_n=n, p_star=m / n,
             star_text=f"$m/n = {m}/{n}$"),
        dict(label=f"hypergraph edge ($r=3$, $n={n_hyper}$)", directed=False,
             separation="edge", hypergraph=True, panel_n=n_hyper,
             p_star=m / math.comb(n_hyper - 1, 2),
             star_text=(fr"$m/\binom{{{n_hyper}-1}}{{2}} = "
                        fr"{m}/{math.comb(n_hyper - 1, 2)}$")),
    ]

    fig = plt.figure(figsize=(18, 6))

    for panel_idx, vdesc in enumerate(variants_3d):
        ax = fig.add_subplot(1, 3, panel_idx + 1, projection="3d")
        panel_n = vdesc["panel_n"]
        p_star = vdesc["p_star"]
        panel_ps = (p_values if p_values is not None
                    else [min(1.0, r * p_star) for r in ratios])

        all_conn_vals: list[int] = []
        dist_by_p: list[tuple[float, list[int]]] = []

        for p in panel_ps:
```

```
            conn_list: list[int] = []
            for _ in range(samples):
                if vdesc["hypergraph"]:
                    cands = _hyperedge_candidates(panel_n, 3, vdesc["directed"])
                    chosen = [e for e in cands if rng.random() < p]
                    h = Hypergraph(panel_n, chosen, directed=vdesc["directed"])
                    c = max_hyper_connectivity(h, vertex_split=False)
                else:
                    g = sample_random_graph(panel_n, p, vdesc["directed"], rng)
                    if vdesc["separation"] == "edge":
                        c = max_edge_connectivity(g)
                    else:
                        c = max_vertex_connectivity(g)
                conn_list.append(c)
            dist_by_p.append((p, conn_list))
            all_conn_vals.extend(conn_list)

        if not all_conn_vals:
            continue

        lo, hi = 0, max(all_conn_vals)
        levels = list(range(lo, hi + 1))

        # x is the density in units of this panel's own threshold.
        xs = [p / p_star for p, _ in dist_by_p]
        width = 0.9 * min((b - a) for a, b in zip(xs, xs[1:])) if len(xs) > 1 else 0.1
        for x, (_, conn_list) in zip(xs, dist_by_p):
            total = len(conn_list)
            for lv in levels:
                frac = conn_list.count(lv) / total
                if frac > 0:
                    ax.bar3d(x - width / 2, lv - 0.4, 0, width, 0.8, frac,
                             color=_KUL_BLUE, alpha=0.7, shade=True)

        # The threshold sits at 1 in these units, by construction.
        for lv in levels:
            ax.plot([1.0, 1.0], [lv - 0.4, lv + 0.4], [0, 0],
                    color=_WARM, linewidth=0.5, alpha=0.4)
        zmax = max(conn_list.count(lv) / len(conn_list)
                   for _, conn_list in dist_by_p
                   for lv in levels
                   if conn_list.count(lv) > 0)
        ax.plot([1.0, 1.0], [lo, hi], [zmax * 0.9, zmax * 0.9],
                color=_WARM, linewidth=2.5, linestyle="--", label="$p / p^* = 1$")

        ax.set_title(f"{vdesc['label']}\n$p^* = $ {vdesc['star_text']}", fontsize=9)
        ax.set_xlabel("$p / p^*$", fontsize=9)
        ax.set_ylabel(r"$\lambda^{\max}$", fontsize=9)
        ax.set_zlabel("fraction", fontsize=9)
        ax.tick_params(labelsize=7)

    fig.suptitle(
        fr"Distribution of $\lambda^{{\max}}$ against density, $m = {m}$, each model "
        fr"in units of its own threshold $p^*$ (the density at which a vertex's "
        fr"expected degree reaches $m$)",
```

```
        fontsize=11,
    )
    _save(path, tight=False)
```

C.1.8 The open variants

These routines exist to attack the questions the thesis leaves open. They measure the hypergraph vertex problem, work with fractional weightings where the integer problem is out of reach, and provide the exhaustive maximisers used to test conjectures at the sizes where testing is still possible. Several results in Appendix A were found, or in two cases refuted, by running exactly these functions.

```
# --- OPEN-VARIANT EXPLORATION: hypergraph vertex connectivity, fractional search tools ---


def max_hyper_vertex_connectivity(hypergraph: Hypergraph) -> int:
    """``kappa^max`` for the hypergraph vertex problem (readability wrapper)."""
    return max_hyper_connectivity(hypergraph, vertex_split=True)


def _hyper_codegree_ok(edges, bound: int) -> bool:
    """Necessary feasibility filter: two vertices in > ``bound`` common
    hyperedges already carry that many one-step disjoint Berge routes."""
    from collections import Counter

    codegree: Counter = Counter()
    for edge in edges:
        for pair in combinations(sorted(edge), 2):
            codegree[pair] += 1
            if codegree[pair] > bound:
                return False
    return True


def hyper_vertex_feasible_exists(n: int, r: int, m: int, q: int,
                                 multi: bool = True) -> bool:
    """Exhaustively decide whether some ``r``-uniform (multi-)hypergraph on
    ``n`` vertices with ``q`` hyperedges has ``kappa^max <= m - 1``.

    Repeated hyperedges are allowed when ``multi`` is set (they are forced
    infeasible at m = 2 by the codegree filter, so m = 2 needs no repeats).
    Practical up to roughly C(C(n,r), q) ~ a few hundred thousand candidates.
    """
    all_edges = list(combinations(range(n), r))
    chooser = combinations_with_replacement if multi else combinations
    for sub in chooser(all_edges, q):
        if not _hyper_codegree_ok(sub, m - 1):
            continue
        candidate = Hypergraph(n, [frozenset(edge) for edge in sub])
        feasible = True
        for s in range(n):
            for t in range(s + 1, n):
                if hyper_connectivity(candidate, s, t, vertex_split=True) > m - 1:
                    feasible = False
```

```
                    break
            if not feasible:
                break
        if feasible:
            return True
    return False


def verify_hyper_vertex_value(n: int, r: int, m: int) -> bool:
    """Confirm exhaustively that the ``r``-uniform vertex value at ``(n, m)``
    equals ``floor((m-1)(n-1)/(r-1))``: the bound's witness is feasible
    (kappa^max <= m-1) and one more hyperedge never is."""
    target = ((m - 1) * (n - 1)) // (r - 1)
    if m == 2:
        witness = star_hypertree(n, r)
        attained = (len(witness.hyperedges) == target
                    and max_hyper_vertex_connectivity(witness) <= 1)
    else:
        attained = hyper_vertex_feasible_exists(n, r, m, target)
    return attained and not hyper_vertex_feasible_exists(n, r, m, target + 1)


def max_feasible_hyperedges(
    n: int, r: int, m: int,
    *, directed: bool = False, vertex_split: bool = False, kind: str = "forward",
    time_limit: float = 20.0, seed_lb: int = 0,
) -> tuple[int, bool]:
    """Largest number of ``r``-uniform hyperedges with Berge connectivity ``<= m -1``.

    Returns ``(value, exact)``.  ``exact`` is ``True`` when the branch and bound
    below proved optimality within ``time_limit`` (the value is then the true
    extremal number for that variant); otherwise ``value`` is the best feasible
    construction found, a lower bound.  ``kind`` selects the directed orientation
    model (forward / backward / general); for the undirected case it is ignored.

    The search is a depth-first include/exclude over the candidate hyperedges with
    two prunings.  Feasibility is monotone (adding a hyperedge never lowers any
    Berge connectivity), so once the active set is infeasible no superset can be
    feasible and the include branch is dropped; and a branch is cut once
    ``active + remaining`` cannot beat the best feasible count already seen.  A
    short randomised warm start raises that incumbent first, so the bound prune
    bites early.  ``seed_lb`` is an externally known feasible lower bound (e.g. the
    forward value when maximising over the general model, which contains every
    forward hypergraph) used to start the incumbent; it never changes the proven
    optimum, only the pruning.
    """
    deadline = time.time() + time_limit
    # Warm start: a short greedy randomised lower bound sharpens the bound prune;
    # kept brief so the branch and bound, which does the actual proving, keeps
    # the budget.
    warm_share = min(0.3 * time_limit, 1.5)
    lb, _ = _random_hypergraph_search(
        n, r, m, time.time() + warm_share, seed=0,
        directed=directed, vertex_split=vertex_split, kind=kind)
    lb = max(lb, seed_lb)

```

```python
    candidates = _hyperedge_candidates(n, r, directed, kind=kind)
    total = len(candidates)
    best = [lb]
    timed_out = [False]
    active = Hypergraph(n, directed=directed)

    def dfs(pos: int, count: int) -> None:
        if timed_out[0]:
            return
        if count + (total - pos) <= best[0]:
            return                                  # cannot beat the incumbent
        if time.time() > deadline:
            timed_out[0] = True
            return
        if pos == total:
            if count > best[0]:
                best[0] = count
            return
        # Include candidates[pos], but only if the set stays feasible.
        active.add_hyperedge(candidates[pos])
        if max_hyper_connectivity(active, vertex_split=vertex_split) <= m - 1:
            dfs(pos + 1, count + 1)
        active.hyperedges.pop()
        # Exclude candidates[pos].
        dfs(pos + 1, count)

    dfs(0, 0)
    return best[0], (not timed_out[0])


def _c_flat(mu: np.ndarray) -> tuple[np.ndarray, "_ct.POINTER[_ct.c_int]"]:
    """Return a contiguous int32 copy of ``mu`` and a ctypes c_int pointer into it."""
    flat = np.ascontiguousarray(mu, dtype=np.int32)
    ptr = flat.ctypes.data_as(_ct.POINTER(_ct.c_int))
    return flat, ptr   # caller must keep flat alive while ptr is in use


def _tiny_maxflow(mu: np.ndarray, n: int, s: int, t: int, cap: int) -> bool:
    """Return True iff the integer max-flow from s to t under capacities ``mu``
    exceeds ``cap``.  Plain DFS augmentation (Ford-Fulkerson with a stack, not
    BFS/Edmonds-Karp); built for n <= 7, where it beats both the networkx call
    and scipy's C ``maximum_flow`` by orders of magnitude inside the hot
    enumeration/search loops (measured ~16x faster than scipy at n=7: the
    library calls' per-call sparse-matrix construction and dispatch overhead
    dominates at this size, while the ``cap`` lets this routine stop after
    cap+1 augmenting paths).  Correct for integer capacities: each augmentation
    increases flow by at least 1, so the loop terminates in at most
    max-flow-value iterations."""
    if _C is not None and n <= 7:
        flat, ptr = _c_flat(mu)
        return bool(_C.tiny_maxflow(ptr, n, s, t, cap))
    residual = mu.astype(int)           # astype already returns a fresh, mutable copy
    flow = 0
    while flow <= cap:
        parent = [-1] * n
        parent[s] = s
```

```python
        queue = [s]
        while queue and parent[t] == -1:
            u = queue.pop()
            for v in range(n):
                if parent[v] == -1 and residual[u, v] > 0:
                    parent[v] = u
                    queue.append(v)
        if parent[t] == -1:
            return False  # flow is maximal and <= cap
        bottleneck = 10 ** 9
        v = t
        while v != s:
            bottleneck = min(bottleneck, residual[parent[v], v])
            v = parent[v]
        v = t
        while v != s:
            residual[parent[v], v] -= bottleneck
            residual[v, parent[v]] += bottleneck
            v = parent[v]
        flow += bottleneck
    return True


def _canonical_form(mu: np.ndarray) -> bytes:
    """A canonical key for the isomorphism class of a directed multigraph.

    The key is the lexicographically smallest flattened multiplicity matrix over
    all vertex relabellings: two multiplicity matrices have the same key iff one
    is a vertex permutation of the other.  This is the SOURCE OF TRUTH for
    isomorphism here, with no dependency on any external library, so the
    enumeration's correctness never rests on an outside binary.  For ``n=7`` it is
    5040 permutations per graph, which is cheap on CPU; storing one key per class
    is what bounds the enumeration's memory to the number of classes rather than
    the number of labelled copies.
    """
    n = mu.shape[0]
    if _C is not None and n <= 7:
        flat, ptr = _c_flat(mu)
        out = np.empty(n * n, dtype=np.int32)
        out_ptr = out.ctypes.data_as(_ct.POINTER(_ct.c_int))
        _C.canonical_form_min(ptr, n, out_ptr)
        return out.tobytes()
    best: bytes | None = None
    for perm in permutations(range(n)):
        p = list(perm)
        cand = mu[np.ix_(p, p)].tobytes()
        if best is None or cand < best:
            best = cand
    assert best is not None  # n >= 1, so at least the identity permutation exists
    return best


def enumerate_extremal_directed_multigraphs(
    n: int, m: int, target_arcs: int, max_degree: int | None = None,
    up_to_iso: bool = True,
) -> list[np.ndarray]:
    """Exhaustively list the directed multigraphs on ``n`` vertices with
    multiplicities in {0..m-1}, ``lambda^max <= m-1``, exactly ``target_arcs``
```

```
    arcs and (optionally) maximum total degree at most ``max_degree``.

    This is the finite-base tool for the m = 3 chain (see claude.md and the
    commented rem:odd-step-roadmap): the characterisations at n = 4, 5 feed
    the recursion behind statement (b) at n = 7.  DFS in vertex-block order
    with three prunings: multiplicity capacity, the induced-subgraph arc bound
    on every completed prefix, and exact flow checks on completed prefixes
    (induced flows only grow, so a violation is final).

    SOUNDNESS: the induced-subgraph bound is a PROVED upper bound for j <= 6
    (from the cut-counting MILP; see ``known`` dict below).  For j >= 7 it
    falls back to the CONJECTURED bipartite bound floor(j^2/4), which is not
    yet proved.  Therefore this function is a sound complete search only for
    n <= 6.  For n >= 7 it is a heuristic lower-bound search: it may miss
    extremal graphs whose j=7 prefix exceeds the bipartite bound (if the
    conjecture is false) or whose search branch was pruned by it.
    Returns multiplicity matrices, deduplicated up to vertex permutation when
    ``up_to_iso`` is set.  Deduplication is STREAMED through a canonical form
    (:func:`_canonical_form`) as graphs are found, so peak memory is bounded by
    the number of isomorphism classes rather than the number of labelled copies
    (the previous buffer-everything approach exhausted RAM on n=7).  The returned
    list, and its order, are unchanged by this.
    """
    # Ordered pairs grouped so that all pairs inside {0..j} precede vertex j+1.
    order: list[tuple[int, int]] = []
    for j in range(1, n):
        for i in range(j):
            order.append((i, j))
            order.append((j, i))
    block_end = {j: 2 * ((j + 1) * j // 2) for j in range(1, n)}  # prefix length
    # SOUNDNESS NOTE: for j <= 6 the prefix cap (m-1)*M*(j) is a PROVED upper
    # bound, so pruning at that cap is safe.  For j >= 7, _PROVEN_MSTAR.get(j, 0) == 0
    # and we fall back to (m-1)*floor(j^2/4), which equals the conjectured
    # bipartite bound and is NOT yet proved.  Any call with n >= 7 therefore
    # uses an unverified pruning at the j=7 boundary: the enumeration is sound
    # as a lower-bound search but cannot certify completeness for n >= 7.
    # To produce a proof for n=7 the j=7 pruning must be disabled or replaced
    # by a proved bound (e.g. from the MILP once M*(7) is confirmed).
    prefix_cap = {j: (m - 1) * max(_PROVEN_MSTAR.get(j, 0), (j * j) // 4)
                  for j in range(2, n + 1)}

    mu = np.zeros((n, n), dtype=int)
    total_pairs = len(order)
    # Stream results straight into the deduplicated output instead of buffering
    # every labelled feasible matrix first.  Memory is then bounded by the number
    # of ISOMORPHISM CLASSES (one canonical key in `seen`, one representative in
    # the list), not by the number of labelled copies, which is what previously
    # blew up to tens of gigabytes on n=7.  The set of matrices returned, and
    # their order, are identical to the old buffer-then-dedup code.
    seen: set[bytes] = set()
    representatives: list[np.ndarray] = []

    def _record(matrix: np.ndarray) -> None:
        # The DFS only checked prefixes {0..j} for j < n, so confirm the full
        # n-vertex graph is feasible before keeping it (cheap, and done first so a
        # rejected graph never pays for the canonical form).
        if any(_tiny_maxflow(matrix, n, s, t, m - 1)
```

```
               for s in range(n) for t in range(n) if s != t):
            return
        if up_to_iso:
            canon = _canonical_form(matrix)
            if canon in seen:
                return
            seen.add(canon)
        representatives.append(matrix.copy())

    def dfs(pos: int, arcs: int) -> None:
        if arcs + (m - 1) * (total_pairs - pos) < target_arcs or arcs > target_arcs:
            return
        if pos == total_pairs:
            if arcs == target_arcs:
                _record(mu)
            return
        completed = next((j for j in range(1, n) if block_end[j] == pos), None)
        if completed is not None:
            j = completed + 1  # prefix {0..completed} has j vertices
            if arcs > prefix_cap.get(j, 10 ** 9):
                return
            for s in range(j):
                for t in range(j):
                    if s != t and _tiny_maxflow(mu, n, s, t, m - 1):
                        return
        u, v = order[pos]
        for value in range(m):
            mu[u, v] = value
            if max_degree is None or (mu[u, :].sum() + mu[:, u].sum() <= max_degree
                                      and mu[v, :].sum() + mu[:, v].sum() <= max_degree):
                dfs(pos + 1, arcs + value)
        mu[u, v] = 0

    dfs(0, 0)
    return representatives


def _geng_support_graphs(
    n: int, min_edges: int, max_edges: int, geng_path: str = "geng",
) -> Iterator[list[tuple[int, int]]]:
    """Yield one edge list per isomorphism class of simple graph on ``n``
    vertices with ``min_edges`` to ``max_edges`` edges, via nauty's ``geng``.

    Each edge is an ordered tuple ``(u, v)`` with ``u < v``; isolated vertices
    are kept (``geng`` always emits all ``n`` vertices).  ``geng`` writes the
    non-isomorphic graphs in graph6 to stdout and we parse each with _graph6_edges.
    Raises ``RuntimeError`` if ``geng`` is not on PATH.
    """
    exe = shutil.which(geng_path)
    if exe is None:
        raise RuntimeError(
            f"nauty's '{geng_path}' was not found on PATH.  Install nauty "
            "(e.g. the 'nauty' package) or pass geng_path=...; the "
            "generation enumerator needs it."
```

```
        )
    if max_edges < min_edges:
        return
    proc = subprocess.run(
        [exe, str(n), f"{min_edges}:{max_edges}"],
        capture_output=True, check=True,
    )
    for line in proc.stdout.splitlines():
        line = line.strip()
        if not line:
            continue
        yield _graph6_edges(line)


def _graph6_edges(data: bytes) -> list[tuple[int, int]]:
    """Decode one graph6 line (geng's output format) into its ``(i, j)`` edges, ``i < j``.

    graph6 stores ``n`` in the first byte (offset 63), then the upper-triangle
    adjacency bits column by column (for ``j`` from 1, all ``i < j``), six bits per
    subsequent byte, high bit first.  We only ever generate ``n <= 12`` supports, so
    the single-byte header case is all that is needed and the format needs no
    networkx to read.
    """
    values = [byte - 63 for byte in data]
    n = values[0]
    bits = [(value >> shift) & 1 for value in values[1:] for shift in range(5, -1, -1)]
    index = 0
    edges: list[tuple[int, int]] = []
    for j in range(1, n):
        for i in range(j):
            if bits[index]:
                edges.append((i, j))
            index += 1
    return edges


def _decorate_support_worker(task: tuple) -> list[np.ndarray]:
    """Every feasible extremal decoration of ONE undirected support graph.

    Module-level and picklable, so :func:`enumerate_extremal_directed_multigraphs_via_generation`
    can fan the supports out across processes (each support is independent, and
    two non-isomorphic supports can never yield the same multigraph).  Returns one
    multiplicity matrix per decoration, deduplicated within this support when
    ``up_to_iso``.  The decoration logic and its pruning are identical to the
    sequential generator; only the ownership of the scratch arrays moves here.
    """
    n, m, target_arcs, max_degree, up_to_iso, edges = task
    per_edge_max = 2 * (m - 1)
    # Per-edge states (a, b) = (mu[u,v], mu[v,u]) excluding (0,0): a support edge
    # carries at least one arc in some direction.
    states = [(a, b) for a in range(m) for b in range(m) if (a, b) != (0, 0)]
    mu = np.zeros((n, n), dtype=int)
    deg = np.zeros(n, dtype=int)
```

```python
    out_deg = np.zeros(n, dtype=int)
    in_deg = np.zeros(n, dtype=int)
    seen: set[bytes] = set()
    reps: list[np.ndarray] = []

    def feasible_prefix(j: int) -> bool:
        cap = m - 1
        for s in range(j):
            if out_deg[s] <= cap:
                continue
            for t in range(j):
                if s != t and in_deg[t] > cap and _tiny_maxflow(mu, n, s, t, cap):
                    return False
        return True

    # Decorate in vertex-block order: pairs sorted by larger endpoint, then
    # smaller, so after the last pair whose larger endpoint is w the induced
    # subgraph on {0..w} is complete and the prefix bound for size w+1 applies.
    sps = sorted(edges, key=lambda e: (e[1], e[0]))
    bound: list[int | None] = [None] * len(sps)
    for i, (_, w) in enumerate(sps):
        if i + 1 == len(sps) or sps[i + 1][1] > w:
            bound[i] = w + 1

    def decorate(idx: int, arcs: int) -> None:
        remaining = len(sps) - idx
        if arcs + remaining > target_arcs:
            return
        if arcs + remaining * per_edge_max < target_arcs:
            return
        if idx == len(sps):                          # every support edge decorated
            if arcs == target_arcs and feasible_prefix(n):
                if up_to_iso:
                    canon = _canonical_form(mu)
                    if canon in seen:
                        return
                    seen.add(canon)
                reps.append(mu.copy())
            return
        u, v = sps[idx]
        j = bound[idx]              # prefix size to verify after this edge, or None
        for a, b in states:
            s = a + b
            if arcs + s > target_arcs:
                continue
            if max_degree is not None and (deg[u] + s > max_degree
                                           or deg[v] + s > max_degree):
                continue
            mu[u, v] = a
            mu[v, u] = b
            deg[u] += s
            deg[v] += s
            out_deg[u] += a; in_deg[v] += a
            out_deg[v] += b; in_deg[u] += b
            ok = True
            if j is not None:       # block boundary: prefix {0..j-1} is complete
                cap = _PROVEN_MSTAR.get(j)         # j == prefix size
                if cap is not None and arcs + s > (m - 1) * cap:
```

```python
                    ok = False                    # PROVED induced-arc bound
                elif not feasible_prefix(j):
                    ok = False
            if ok:
                decorate(idx + 1, arcs + s)
            deg[u] -= s
            deg[v] -= s
            out_deg[u] -= a; in_deg[v] -= a
            out_deg[v] -= b; in_deg[u] -= b
            mu[u, v] = 0
            mu[v, u] = 0

    decorate(0, 0)
    return reps


def enumerate_extremal_directed_multigraphs_via_generation(
    n: int, m: int, target_arcs: int, max_degree: int | None = None,
    up_to_iso: bool = True, geng_path: str = "geng", parallel: bool = True,
) -> list[np.ndarray]:
    """Sound, geng-seeded twin of :func:`enumerate_extremal_directed_multigraphs`.

    Lists every directed multigraph on ``n`` vertices with multiplicities in
    {0..m-1}, ``lambda^max <= m-1``, exactly ``target_arcs`` arcs and
    (optionally) maximum total degree ``<= max_degree``, deduplicated up to
    isomorphism.  Same object and same return format (multiplicity matrices) as
    the DFS enumerator.

    Following J. Goedgebeur's suggestion, the underlying undirected SUPPORT
    graph (the pairs carrying at least one arc) is generated once per
    isomorphism class by nauty's ``geng``; each support is then decorated, in
    vertex-block order, with a directed multiplicity pair ``(mu[u,v], mu[v,u])``
    in {0..m-1}^2 minus (0,0) per support edge.  Pruning is the running arc
    total, the degree cap, and -- at each completed prefix on ``j`` vertices --
    the PROVED induced-arc bound ``arcs(prefix) <= (m-1) M*(j)`` together with a
    prefix feasibility check (induced max-flows only grow, so a prefix that
    already exceeds the cap is dead).

    Soundness for every ``n``.  Two directed multigraphs with non-isomorphic
    supports are non-isomorphic, and every isomorphism class with a given
    support class has a labelling whose support equals geng's representative, so
    decorating every representative and canonical-deduplicating returns exactly
    one matrix per isomorphism class.  Crucially the induced-arc bound is used
    ONLY for ``j <= 6``, where ``M*(j)`` is proved; for ``j >= 7`` no arc bound
    is applied (only the global ``target_arcs`` and feasibility).  This is the
    difference from :func:`enumerate_extremal_directed_multigraphs`, which falls
    back to the CONJECTURED ``floor(j^2/4)`` at ``j >= 7`` and is therefore a
    complete search only for ``n <= 6``.  This generator is complete for all
    ``n`` (it never assumes an unproved value), so it can certify the finite
    ``n = 7`` classification once it finishes.

    Requires nauty's ``geng`` on PATH.  For ``n <= 6`` it returns the same set
    of isomorphism classes as :func:`enumerate_extremal_directed_multigraphs`;
    ``tests/test_solve`` checks that equality.

    With ``parallel`` (default) the supports are decorated concurrently across
    processor cores via a :class:`~concurrent.futures.ProcessPoolExecutor`, since
    each support is independent.  This is where the ``n = 7`` classification
```

```
    becomes practical on a multi-core machine: the work scales with the number of
    supports, which is exactly what fans out.  It falls back to a sequential run
    (with a warning) where multiprocessing is unavailable, and the result is
    identical either way.
    """
    if m < 1:
        raise ValueError("m must be >= 1")
    per_edge_max = 2 * (m - 1)             # most arcs one undirected pair can hold
    if per_edge_max == 0:                  # m == 1: only the empty graph is feasible
        return [np.zeros((n, n), dtype=int)] if target_arcs == 0 else []
    min_edges = (target_arcs + per_edge_max - 1) // per_edge_max
    max_edges = min(target_arcs, n * (n - 1) // 2)

    # geng emits each undirected support iso-class once, and decorating one
    # support is independent of the others (two non-isomorphic supports can never
    # yield the same multigraph), so the supports fan out across processes, each
    # with its own scratch arrays inside _decorate_support_worker.  The PROVED
    # M*(j) bound is applied there only for j <= 6; j >= 7 gets no arc bound, so
    # the search stays sound at every n.
    tasks = [(n, m, target_arcs, max_degree, up_to_iso, edges)
             for edges in _geng_support_graphs(n, min_edges, max_edges, geng_path)]

    representatives: list[np.ndarray] = []
    if parallel and len(tasks) > 1:
        try:
            with concurrent.futures.ProcessPoolExecutor() as executor:
                # chunksize=1: supports vary wildly in cost, so hand them out one
                # at a time to keep every core busy to the end.
                for reps in executor.map(_decorate_support_worker, tasks,
                                         chunksize=1):
                    representatives.extend(reps)
        except (OSError, concurrent.futures.BrokenExecutor) as exc:
            warnings.warn(
                f"parallel support enumeration unavailable ({exc!r}); running "
                "sequentially", RuntimeWarning, stacklevel=2,
            )
            for task in tasks:
                representatives.extend(_decorate_support_worker(task))
    else:
        for task in tasks:
            representatives.extend(_decorate_support_worker(task))

    if up_to_iso:
        # Distinct geng supports give non-isomorphic multigraphs, so this final
        # canonical pass is only a safety net over the process-local per-support
        # dedup; it keeps exactly one representative per isomorphism class.
        seen: set[bytes] = set()
        unique: list[np.ndarray] = []
        for matrix in representatives:
            key = _canonical_form(matrix)
            if key not in seen:
                seen.add(key)
                unique.append(matrix)
        representatives = unique
    return representatives
```

C.1.9 The gallery

Given a size and a route limit, these routines find every extremal graph there is, up to relabelling, and count the symmetries of each. Deciding when two graphs are the same graph drawn differently is done by trying all relabellings and keeping the smallest resulting table, which is slow in principle but perfectly quick at the sizes involved.

```
# --- GALLERY: all non-isomorphic extremal graphs for small (n, m); entry point gallery_extremal_graphs ---


def _graph_from_mu(mu: np.ndarray, variant: Variant) -> Graph:
    """Wrap a multiplicity matrix in a fresh Graph (zero-copy)."""
    g = Graph(mu.shape[0], variant)
    g.mu[:] = mu
    return g


def _aut_count_matrix(mu: np.ndarray) -> int:
    """Count vertex permutations that preserve ``mu`` (equals |Aut(G)|)."""
    n = mu.shape[0]
    canon = _canonical_form(mu)
    return sum(1 for perm in permutations(range(n))
               if mu[np.ix_(list(perm), list(perm))].tobytes() == canon)


def _dir_relabel_key(edge, p: list) -> tuple:
    """Permutation-relabelled key of one directed hyperedge.

    Keeps the legacy forward shape ``(int, tuple_of_heads)`` for forward edges
    (so their canonical keys and dedup counts are unchanged) and uses
    ``(tuple_of_tails, tuple_of_heads)`` for general edges.
    """
    first, heads = edge
    if isinstance(first, (set, frozenset)):
        return (tuple(sorted(p[t] for t in first)), tuple(sorted(p[h] for h in heads)))
    return (p[first], tuple(sorted(p[h] for h in heads)))


def _hyper_canonical(hyperedges: list, n: int, directed: bool) -> tuple:
    """Canonical key for a hyperedge collection under vertex permutation.

    Each edge is a frozenset (undirected) or a directed hyperedge in either the
    forward ``(tail, heads)`` or general ``(tails, heads)`` form.  Returns the
    lex-minimum over all n! relabellings.
    """
    best: tuple | None = None
    for perm in permutations(range(n)):
        p = list(perm)
        if directed:
            relabeled: tuple = tuple(sorted(_dir_relabel_key(edge, p) for edge in hyperedges))
        else:
            relabeled = tuple(sorted(
                tuple(sorted(p[v] for v in edge)) for edge in hyperedges
            ))
```

```python
        if best is None or relabeled < best:
            best = relabeled
    return best  # type: ignore[return-value]


def _aut_count_hyper(hyperedges: list, n: int, directed: bool) -> int:
    """Count vertex permutations that preserve a hyperedge collection."""
    canon = _hyper_canonical(hyperedges, n, directed)

    def _relabel(p: list) -> tuple:
        if directed:
            return tuple(sorted(_dir_relabel_key(edge, p) for edge in hyperedges))
        return tuple(sorted(
            tuple(sorted(p[v] for v in edge)) for edge in hyperedges
        ))

    return sum(1 for perm in permutations(range(n))
               if _relabel(list(perm)) == canon)


def _hyper_to_lists(hyperedges: list, directed: bool) -> list:
    """Convert hyperedges to JSON-serialisable sorted integer lists.

    Forward edges keep their ``[tail, [heads]]`` shape; general edges use
    ``[[tails], [heads]]``.
    """
    if directed:
        out = []
        for edge in hyperedges:
            first, heads = edge
            if isinstance(first, (set, frozenset)):
                out.append([sorted(first), sorted(heads)])
            else:
                out.append([first, sorted(heads)])
        return out
    return [sorted(edge) for edge in hyperedges]


def _enum_matrix_extremals(
    variant: Variant,
    n: int,
    m: int,
    separation: str,
    target: int,
    deadline: float,
) -> tuple[list[np.ndarray], bool]:
    """DFS over all (variant, n, m) multiplicity matrices achieving exactly ``target`` edges.

    Generalises :func:`enumerate_extremal_directed_multigraphs` to all four
    matrix variants (simple/multi × directed/undirected) and to vertex as well
    as edge separation.  Returns ``(reps, timed_out)`` where ``reps`` are
    deduplicated representatives (one per isomorphism class).

    Pruning strategy: arc-count reachability at every step; prefix edge-
    connectivity check at each completed vertex block (edge separation only;
    vertex separation checks only the final graph because the per-node cost is
    too high for DFS).
```

```
    """
    max_mult = 1 if variant.simple else (m - 1)
    is_directed = variant.directed

    # Vertex-block order: all pairs involving vertex j before j+1 appears.
    order: list[tuple[int, int]] = []
    for j in range(1, n):
        for i in range(j):
            order.append((i, j))
            if is_directed:
                order.append((j, i))
    total = len(order)

    # Position at which the subgraph on {0..j} is fully decided.
    # Directed: j*(j+1) positions; undirected: j*(j+1)//2 positions.
    step_per_vertex = 2 if is_directed else 1
    block_end: dict[int, int] = {
        j: step_per_vertex * j * (j + 1) // 2 for j in range(1, n)
    }

    mu = np.zeros((n, n), dtype=int)
    seen: set[bytes] = set()
    reps: list[np.ndarray] = []
    timed_out = [False]

    def _feasible_full() -> bool:
        g = _graph_from_mu(mu, variant)
        if separation == "edge":
            return max_edge_connectivity(g) <= m - 1
        return max_vertex_connectivity(g) <= m - 1

    def dfs(pos: int, edges: int) -> None:
        if timed_out[0]:
            return
        if time.time() > deadline:
            timed_out[0] = True
            return
        if edges + max_mult * (total - pos) < target or edges > target:
            return
        if pos == total:
            if edges == target and _feasible_full():
                canon = _canonical_form(mu)
                if canon not in seen:
                    seen.add(canon)
                    reps.append(mu.copy())
            return
        # At each block boundary: prune if the prefix subgraph is already
        # infeasible (edge connectivity only -- cheap with _tiny_maxflow).
        completed_j = next(
            (j for j in range(1, n) if block_end.get(j) == pos), None
        )
        if completed_j is not None and separation == "edge":
            sub_n = completed_j + 1
            if any(_tiny_maxflow(mu, n, s, t, m - 1)
                   for s in range(sub_n) for t in range(sub_n) if s != t):
                return
        u, v = order[pos]
        for val in range(max_mult + 1):
```

```python
            mu[u, v] = val
            if not is_directed:
                mu[v, u] = val
            dfs(pos + 1, edges + val)
        mu[u, v] = 0
        if not is_directed:
            mu[v, u] = 0

    dfs(0, 0)
    return reps, timed_out[0]


def _enum_hyper_extremals(
    directed: bool,
    vertex_split: bool,
    r: int,
    n: int,
    m: int,
    target: int,
    deadline: float,
    *,
    kind: str = "forward",
) -> tuple[list[list], bool]:
    """DFS over all r-uniform hypergraphs on n vertices with exactly ``target`` edges.

    Iterates candidate hyperedges in order (include / exclude) and prunes
    immediately when adding a hyperedge pushes the connectivity above m-1
    (monotonicity: adding edges never decreases connectivity, so no superset
    can be feasible at that branch either). ``kind`` selects the directed
    orientation model. Returns ``(reps, timed_out)`` where each representative
    in ``reps`` is a list of JSON-serialisable sorted vertex lists.
    """
    candidates = _hyperedge_candidates(n, r, directed, kind=kind)
    total = len(candidates)
    active: list = []
    seen: set[tuple] = set()
    reps: list[list] = []
    timed_out = [False]

    def dfs(pos: int, count: int) -> None:
        if timed_out[0]:
            return
        if time.time() > deadline:
            timed_out[0] = True
            return
        if count + (total - pos) < target or count > target:
            return
        if pos == total:
            if count == target:
                canon = _hyper_canonical(active, n, directed)
                if canon not in seen:
                    seen.add(canon)
                    reps.append(_hyper_to_lists(active, directed))
            return
        # Include candidates[pos] and recurse only if still feasible.
        active.append(candidates[pos])
        h = Hypergraph(n, active, directed=directed)
```

```
        if max_hyper_connectivity(h, vertex_split=vertex_split) <= m - 1:
            dfs(pos + 1, count + 1)
        active.pop()
        # Exclude candidates[pos].
        dfs(pos + 1, count)

    dfs(0, 0)
    return reps, timed_out[0]


def gallery_extremal_graphs(
    max_n: int = 7,
    max_m: int = 4,
    r: int = 3,
    time_per_case: float = 3.0,
) -> dict:
    """Collect every non-isomorphic extremal graph for all 12 variants at small n, m.

    For each (variant, n, m) triple where the enumeration finishes within
    ``time_per_case`` seconds, the function:

    1. runs a short repeated search to discover the extremal edge count;
    2. exhaustively enumerates every graph achieving that count;
    3. deduplicates by isomorphism class and records ``n! / |Aut(G)|`` (the
       number of labelled copies) per class.

    Returns a JSON-serialisable dict with structure::

        result[variant_key]["n={n}_m={m}"] = {
            "extremal_value": int,
            "classes": [{"repr": <matrix or edge list>, "labelled_count": int}],
            "total_labelled": int,   # sum of labelled_count over all classes
            "complete": bool,        # False when the deadline was hit
        }

    For matrix variants ``repr`` is the multiplicity matrix (list of lists of
    int).  For hypergraph variants it is a list of hyperedges: each edge is a
    sorted list of vertex indices (undirected) or ``[tail, [head, ...]]``
    (directed).

    The twelve variant keys are:
    ``simple_undirected_{edge,vertex}``,
    ``simple_directed_{edge,vertex}``,
    ``multi_undirected_{edge,vertex}``,
    ``multi_directed_{edge,vertex}``,
    ``hyper_undirected_r{r}_{edge,vertex}``,
    ``hyper_directed_r{r}_{edge,vertex}``.

    The two ``multi_*_vertex`` keys report the reduced SIMPLE problem, because
    the thesis's vertex measure is blind to parallel copies and the variant
    collapses onto the simple one (see the config comment below).
    """
    # The two multi-vertex rows run on the SIMPLE variant: the checker's vertex
    # mode gives every adjacency capacity one, so parallel copies never change
    # kappa and a raw multigraph search would just fill every cell to m-1 for
    # free (it once reported a "36-arc extremal K_4 at multiplicity 3").  The
    # thesis reduces these variants to the simple ones (tab:summary), and the
```

```python
    # gallery must report the reduced problem, exactly as the variant grids do.
    matrix_configs: list[tuple[str, Variant, str]] = [
        ("simple_undirected_edge",   SIMPLE_UNDIRECTED, "edge"),
        ("simple_undirected_vertex", SIMPLE_UNDIRECTED, "vertex"),
        ("simple_directed_edge",     SIMPLE_DIRECTED,   "edge"),
        ("simple_directed_vertex",   SIMPLE_DIRECTED,   "vertex"),
        ("multi_undirected_edge",    MULTI_UNDIRECTED,  "edge"),
        ("multi_undirected_vertex",  SIMPLE_UNDIRECTED, "vertex"),
        ("multi_directed_edge",      MULTI_DIRECTED,    "edge"),
        ("multi_directed_vertex",    SIMPLE_DIRECTED,   "vertex"),
    ]
    hyper_configs: list[tuple[str, bool, bool]] = [
        (f"hyper_undirected_r{r}_edge",   False, False),
        (f"hyper_undirected_r{r}_vertex", False, True),
        (f"hyper_directed_r{r}_edge",     True,  False),
        (f"hyper_directed_r{r}_vertex",   True,  True),
    ]

    result: dict = {}

    for key, variant, separation in matrix_configs:
        result[key] = {}
        for n in range(2, max_n + 1):
            for m in range(2, max_m + 1):
                case_deadline = time.time() + time_per_case
                # Step 1: discover the extremal value via repeated search.
                # 2000 steps/restart is enough to converge for n <= 6;
                # we run at least 3 seeds before the deadline to be robust
                # against unlucky single-seed runs.
                best_val = 0
                seed = 0
                while time.time() < case_deadline - 2.0 or seed < 3:
                    if time.time() > case_deadline - 0.5:
                        break
                    res = search_for_dense_graph(
                        variant, n, m, separation=separation,
                        steps=2000, seed=seed)
                    best_val = max(best_val, res.best_edge_count)
                    seed += 1
                # Step 2: enumerate all graphs at that value.
                enum_deadline = min(case_deadline, time.time() + 2.0)
                reps, timed_out = _enum_matrix_extremals(
                    variant, n, m, separation, best_val, enum_deadline)
                # Step 3: count automorphisms and labelled copies.
                fn = math.factorial(n)
                classes = [
                    {"repr": mu.tolist(),
                     "labelled_count": fn // _aut_count_matrix(mu)}
                    for mu in reps
                ]
                result[key][f"n={n}_m={m}"] = {
                    "extremal_value": best_val,
                    "classes": classes,
                    "total_labelled": sum(c["labelled_count"] for c in classes),
                    "complete": not timed_out,
                }

    for key, directed, vertex_split in hyper_configs:
```

```
        result[key] = {}
        for n in range(r, max_n + 1):
            for m in range(2, max_m + 1):
                case_deadline = time.time() + time_per_case
                # Brute-force search (small n): finds the extremal value.
                best_val, _, completed = _brute_force_hypergraph(
                    n, r, m, case_deadline,
                    directed=directed, vertex_split=vertex_split)
                if not completed:
                    result[key][f"n={n}_m={m}"] = {
                        "extremal_value": best_val, "classes": [],
                        "total_labelled": 0, "complete": False,
                    }
                    continue
                # Enumerate all achieving best_val.
                enum_deadline = min(case_deadline, time.time() + 2.0)
                hyper_reps, timed_out = _enum_hyper_extremals(
                    directed, vertex_split, r, n, m, best_val, enum_deadline)
                fn = math.factorial(n)
                classes = []
                for edge_lists in hyper_reps:
                    raw = ([(row[0], frozenset(row[1])) for row in edge_lists]
                           if directed else [frozenset(e) for e in edge_lists])
                    classes.append({
                        "repr": edge_lists,
                        "labelled_count": fn // _aut_count_hyper(raw, n, directed),
                    })
                result[key][f"n={n}_m={m}"] = {
                    "extremal_value": best_val,
                    "classes": classes,
                    "total_labelled": sum(c["labelled_count"] for c in classes),
                    "complete": not timed_out,
                }

    return result


def save_gallery_json(gallery: dict, path: str | Path) -> None:
    """Write the gallery dict to a JSON file (pretty-printed)."""
    with open(path, "w") as f:
        json.dump(gallery, f, indent=2)


def prove_integral_arc_bound(n: int, m: int, target: int, *,
                             time_limit: float = 3000.0,
                             use_gurobi: bool | None = None,
                             show_solver_log: bool = False) -> str:
    """Decide by MILP whether some directed multigraph on ``n`` vertices with
    multiplicities in {0..m-1} and ``lambda^max <= m-1`` has >= ``target`` arcs.

    Returns "INFEASIBLE" (proving L_m^dir(n) < target), "FEASIBLE", or
    "LIMIT".  This is the integral companion of the fractional prover: the
    m = 3 chain (rem:odd-step-roadmap) needs only L_3^dir(7) = 24, i.e.
    INFEASIBLE at target 25, which is a far friendlier MILP than M*(7) because the
    weights themselves are integers.  Same exact :func:`_cut_counting_model` with
    cap = m-1 and integer multiplicities, all its proved-valid families on, plus the
```

```
    one constraint that the multiplicities total at least ``target``.
    """
    _require_pulp()
    prob, w = _cut_counting_model(
        n, cap=float(m - 1), integer=True, two_hop=True,
        symmetry=True, deletion=True, degree_pair=True,
    )
    prob += pulp.lpSum(w.values()) >= target        # the arc target
    prob += 0                                        # feasibility, no objective

    prob.solve(_pick_solver(time_limit, show_solver_log, use_gurobi))
    status = pulp.LpStatus[prob.status]
    if status == "Infeasible":
        return "INFEASIBLE"
    if status == "Optimal":
        return "FEASIBLE"
    return "LIMIT"


def _float_maxflow_value(capacity: np.ndarray, source: int, target: int,
                         eps: float = 1e-12) -> float:
    """Max-flow value for FLOAT capacities (Edmonds-Karp, shortest augmenting path).

    scipy's csgraph max-flow is integer-only, so the fractional check below needs
    its own tiny routine.  BFS augmenting paths give a polynomial bound independent
    of the capacity magnitudes, so the loop terminates even on irrational-looking
    floats; ``eps`` treats a near-zero residual as saturated.
    """
    n = capacity.shape[0]
    residual = capacity.astype(float).copy()
    flow = 0.0
    while True:
        parent = [-1] * n
        parent[source] = source
        queue = deque([source])
        while queue and parent[target] == -1:
            u = queue.popleft()                      # FIFO: shortest augmenting path
            for v in range(n):
                if parent[v] == -1 and residual[u, v] > eps:
                    parent[v] = u
                    queue.append(v)
        if parent[target] == -1:
            return flow
        bottleneck = float("inf")
        v = target
        while v != source:
            bottleneck = min(bottleneck, residual[parent[v], v])
            v = parent[v]
        v = target
        while v != source:
            residual[parent[v], v] -= bottleneck
            residual[v, parent[v]] += bottleneck
            v = parent[v]
        flow += bottleneck

```

```python

def fractional_flows_feasible(weights: np.ndarray, tol: float = 1e-9) -> bool:
    """Check the scaled multigraph constraint: all pairwise max-flows <= 1.

    ``weights`` is an ``n x n`` matrix with entries in [0,1] (zero diagonal),
    the scaled multiplicity matrix mu/(m-1) of lem:scaling-reduction.
    """
    n = weights.shape[0]
    capacity = np.where(weights > 1e-12, weights, 0.0).astype(float)
    np.fill_diagonal(capacity, 0.0)
    for s in range(n):
        for t in range(n):
            if t != s and _float_maxflow_value(capacity, s, t) > 1 + tol:
                return False
    return True


def fractional_search(n: int, objective: str = "min_degree", steps: int = 6000,
                      seed: int = 0, start: str = "bipartite") -> tuple[np.ndarray, float]:
    """Hunt for a fractional counterexample to the odd-n directed multigraph
    statements (conj:min-degree and the total-weight bound).

    ``objective``: "min_degree" maximises the minimum weighted total degree
    (conjecture: cannot exceed k = (n-1)/2 for odd n); "total" maximises the
    total weight (conjecture: cannot exceed max(2(n-1), floor(n^2/4))).
    ``start``: "bipartite" (the conjectured extremiser), "zero", or "random".
    Returns the best weighting found and its objective value.  A value
    exceeding the conjectured ceiling would refute the conjecture for every
    m - 1 divisible by the weight denominators; none was found at n = 7, 9.
    """
    rng = random.Random(seed)
    weights = np.zeros((n, n))
    if start == "bipartite":
        for a in range(n // 2):
            for b in range(n // 2, n):
                weights[a, b] = 1.0
    elif start == "random":
        weights = np.array([[rng.random() if i != j else 0.0 for j in range(n)]
                            for i in range(n)])
        while not fractional_flows_feasible(weights):
            weights *= 0.85

    def score(w: np.ndarray) -> float:
        if objective == "total":
            return float(w.sum())
        degrees = w.sum(axis=0) + w.sum(axis=1)
        return float(degrees.min()) + 1e-3 * float(w.sum())

    current = score(weights)
    best, best_w = current, weights.copy()
    for it in range(steps):
        temperature = 0.08 * (1 - it / steps) + 1e-4
        u, v = rng.randrange(n), rng.randrange(n)
        if u == v:
            continue
        old = weights[u, v]
        weights[u, v] = min(1.0, max(0.0, old + (rng.random() * 2 - 1)
```

```
                                        * max(0.02, temperature))
        if weights[u, v] == old:
            continue
        value = score(weights)
        accept = value > current or rng.random() < math.exp((value - current)
                                                           / temperature)
        if accept and fractional_flows_feasible(weights):
            current = value
            if value > best:
                best, best_w = value, weights.copy()
        else:
            weights[u, v] = old
    return best_w, best


# --- SELF-CHECK: python erdos915_unified.py runs every invariant; exits 0 on all-PASS ---

_failures = 0


def check(label: str, condition: bool) -> None:
    """Print a PASS/FAIL line and tally failures."""
    global _failures
    print(f"  {'PASS' if condition else 'FAIL'}  {label}")
    if not condition:
        _failures += 1


def section(title: str) -> None:
    """Print a section header."""
    print(f"\n{title}")
```

C.1.10 The self-check

The file ends with a suite of checks that runs when the program is executed directly. It re-derives known values, tests each construction against its predicted size and connectivity, confirms the fast routines agree with the slow ones they stand in for, and verifies the bounds proved in Appendix A on sampled graphs. It prints a line per check and fails loudly rather than quietly if any of them stops holding, which is what makes it useful as a regression test after an edit.

```
def _run_checks() -> int:
    """Run every invariant check; return 1 on any failure, else 0.

    This is the program checking itself end to end, using the bare top-level
    names defined above in this single file.
    """
    section("Checker: the directed m=2 values ell_2^dir(n) = 2, 4, 6, 8")
    for n, expected in [(2, 2), (3, 4), (4, 6), (5, 8)]:
        graph = double_star(n, m=2, directed=True)
        check(f"double star n={n}: {graph.edge_count()} arcs, lambda^max={max_edge_connectivity(graph)}",
              graph.edge_count() == expected and max_edge_connectivity(graph) == 1)

```

```
    section("Checker: one-directional bipartite is quadratic and feasible")
    for n in range(2, 8):
        graph = one_directional_bipartite(n)
        check(f"bipartite n={n}: {graph.edge_count()} arcs (floor(n^2/4)={ (n*n)//4 })",
              graph.edge_count() == (n * n) // 4 and max_edge_connectivity(graph) == 1)

    section("Checker: the undirected cycle is the infeasible witness for m=2")
    cycle = Graph(6, SIMPLE_UNDIRECTED)
    for v in range(6):
        cycle.add_edge(v, (v + 1) % 6)
    check(f"C_6 has lambda^max={max_edge_connectivity(cycle)}", max_edge_connectivity(cycle) == 2)

    if NETWORKX_AVAILABLE:
        section("Gomory-Hu shortcut agrees with brute-force lambda^max")
        tree = clique_tree(3, 4)
        check(f"GH={max_edge_connectivity_via_tree(tree)} vs direct={max_edge_connectivity(tree)}",
              max_edge_connectivity_via_tree(tree) == max_edge_connectivity(tree))
    else:
        section("Gomory-Hu shortcut skipped (optional networkx not installed)")

    section("Vertex connectivity: K_4-trees are globally 3-connected")
    for blocks in range(0, 5):
        ktree = clique_tree(4, blocks)
        check(f"K_4-tree on {ktree.num_vertices} vertices: kappa={min_vertex_connectivity(ktree)}",
              min_vertex_connectivity(ktree) == 3)

    section("Constructions: the augmented bipartite conjecture values")
    counterexample = augmented_bipartite(10, 3)
    check(f"m=3,n=10: {counterexample.edge_count()} arcs, lambda^max={max_edge_connectivity(counterexample)}",
          counterexample.edge_count() == 30 and max_edge_connectivity(counterexample) == 2)
    for n, m in [(8, 3), (10, 4), (12, 4), (10, 5)]:
        graph = augmented_bipartite(n, m)
        expected = (n + m - 2) ** 2 // 4
        check(f"n={n},m={m}: {graph.edge_count()} arcs (expected {expected}), lambda^max={max_edge_connectivity(graph)}",
              graph.edge_count() == expected and max_edge_connectivity(graph) == m - 1)

    section("Hypergraphs: Berge connectivity, exact and generalising the graph case")
    k43 = complete_uniform_hypergraph(4, 3)
    check(f"complete 3-uniform K_4^(3): Berge lambda^max = {max_hyperedge_connectivity(k43)} (expect 3)",
          max_hyperedge_connectivity(k43) == 3)
    for n in [3, 4, 5]:
        complete = complete_uniform_hypergraph(n, 2)
        check(f"2-uniform K_{n} reduces to edge-connectivity {max_hyperedge_connectivity(complete)} (expect {n-1})",
              max_hyperedge_connectivity(complete) == n - 1)
    for n, r in [(7, 3), (10, 4), (13, 4)]:
        star = star_hypertree(n, r)
```

```
        check(f"star hypertree n={n}, r={r}: {star.edge_count()} edges (expect {(n-1)//(r-1)}), "
              f"lambda^max={max_hyperedge_connectivity(star)}",
              star.edge_count() == (n - 1) // (r - 1) and max_hyperedge_connectivity(star) == 1)

    section("Hypergraphs: directed orientation models (forward / backward / general)")
    # The general gate admits a mixed arc {0,1} -> {2,3}: a route enters at a tail
    # and leaves at a head, so 0->2 carries one route and 2->0 carries none.
    mixed = Hypergraph(4, [(frozenset({0, 1}), frozenset({2, 3}))], directed=True)
    check("mixed arc {0,1}->{2,3}: lambda(0,2)=1 and lambda(2,0)=0",
          hyper_connectivity(mixed, 0, 2) == 1 and hyper_connectivity(mixed, 2, 0) == 0)
    # Backward is the arc-reversal dual of forward, so their extremal numbers agree.
    fwd, _ = max_feasible_hyperedges(4, 3, 3, directed=True, kind="forward", time_limit=1.5)
    bwd, _ = max_feasible_hyperedges(4, 3, 3, directed=True, kind="backward", time_limit=1.5)
    check(f"forward == backward at n=4, r=3, m=3 ({fwd} == {bwd})", fwd == bwd)
    # General contains forward and strictly beats it where vertices are scarce
    # (n = r); both values here are proved exact by the branch and bound.
    g3, e3 = max_feasible_hyperedges(3, 3, 3, directed=True, kind="general", time_limit=1.5)
    f3, _  = max_feasible_hyperedges(3, 3, 3, directed=True, kind="forward", time_limit=1.5)
    check(f"general 4 > forward 3 at n=3, r=3, m=3 (got {g3} vs {f3}, exact={e3})",
          g3 == 4 and f3 == 3 and e3)
    g4, e4 = max_feasible_hyperedges(4, 4, 4, directed=True, kind="general", time_limit=10.0)
    f4, _  = max_feasible_hyperedges(4, 4, 4, directed=True, kind="forward", time_limit=3.0)
    check(f"general 8 doubles forward 4 at n = r = 4, m = 4 ({g4} vs {f4}, exact={e4})",
          g4 == 8 and f4 == 4 and e4)

    section("Monte Carlo: the appearance probability crosses the m/n scale")
    n, m = 30, 3
    below = estimate_appearance_probability(n, 0.3 * m / n, m, trials=120, seed=1)
    above = estimate_appearance_probability(n, 1.8 * m / n, m, trials=120, seed=1)
    check(f"P[appears] rises from {below:.2f} (well below m/n) to {above:.2f} (well above)",
          below < 0.5 < above)

    section("Monte Carlo: vertex-connectivity never exceeds edge-connectivity (Whitney)")
    edge_dist = connectivity_distribution(20, 0.25, trials=40, separation="edge", seed=3)
    vertex_dist = connectivity_distribution(20, 0.25, trials=40, separation="vertex", seed=3)
    check("kappa^max <= lambda^max on every one of the 40 identical samples",
          all(kappa <= lam for kappa, lam in zip(vertex_dist, edge_dist)))

    section("Edge vs vertex in G(n,p): they agree at small m, part company at large m")
    lam_small, kap_small = edge_vertex_distribution(5, 0.5, trials=120, seed=7)
```

```
    small_gap = sum(k < l for k, l in zip(kap_small, lam_small))
    check("n=5: kappa^max = lambda^max on every sample (no gap)",
          small_gap == 0 and all(k <= l for k, l in zip(kap_small, lam_small)))
    lam_large, kap_large = edge_vertex_distribution(16, 0.25, trials=200, seed=7)
    large_gap = sum(k < l for k, l in zip(kap_large, lam_large))
    check(f"n=16: Whitney holds and kappa^max < lambda^max on {large_gap} of 200 samples",
          large_gap > 0 and all(k <= l for k, l in zip(kap_large, lam_large)))

    if not PULP_AVAILABLE:
        section("Prover sections skipped (optional pulp not installed)")
    else:
        section("Prover: cut-counting proves M*(n) = 2(n-1) optimal for small n")
        # The thesis (ch2) claims L_m^dir(n) = 2(n-1)(m-1) is proved for ALL n <= 6
        # via the cut-counting MILP (thm:dir-multi-small).  n=3,4,5 all finish in
        # well under 60 s and are verified here.  n=6 takes ~1315 s (~22 min) on a
        # typical laptop and is NOT included in this fast loop -- it must be
        # verified separately:
        #
        #   result = prove_directed_multigraph(6, time_limit=2000.0)
        #   assert result.is_proof() and round(result.scaled_optimum) == 10
        #
        # The confirmed run (2026-06-13) is logged in program/logs/selftest_check.log
        # (search "n=6 OPTIMAL M*(6)=10 in 1315s").  Omitting n=6 from this loop is
        # not an error in the proof -- the proof ran -- but it means this self-test
        # alone does not reproduce the full n<=6 certification.
        for n in [3, 4, 5]:
            result = prove_directed_multigraph(n, time_limit=300.0)
            check(f"n={n}: status={result.status}, M*={result.scaled_optimum:.1f}, proof={result.is_proof()}",
                  result.is_proof() and round(result.scaled_optimum) == 2 * (n - 1))

        section("Prover: the valid inequalities sharpen but never move the optimum")
        # Turning the two-hop and symmetry-breaking rows off leaves the BARE exact
        # cut formulation, which must still prove the same M*(3) = 4.  If any row we
        # call a "valid inequality" were in fact invalid, it would cut a feasible
        # point and shift this optimum -- so this is the regression tripwire that
        # guards the prover's soundness, not just a re-run of the value.
        bare = prove_directed_multigraph(3, time_limit=120.0,
                                         use_two_hop=False, use_symmetry_breaking=False)
        check(f"M*(3) with the tighteners off = {bare.scaled_optimum:.1f} (expect 4), proof={bare.is_proof()}",
              bare.is_proof() and round(bare.scaled_optimum) == 4)

        section("Prover (integral): an INFEASIBLE verdict is a genuine upper-bound proof")
        # L_3^dir(4) = 12: a 12-arc multigraph exists, no 13-arc one does.  This
        # exercises the exact mechanism behind the thesis's L_3(7) = 24 result
        # (INFEASIBLE one above the optimum) on a value small enough to re-run here,
```

```
        # so a regression in the integral encoding cannot pass unnoticed.
        feasible_at_12 = prove_integral_arc_bound(4, 3, 12, time_limit=120.0)
        infeasible_at_13 = prove_integral_arc_bound(4, 3, 13, time_limit=120.0)
        check(f"target 12 -> {feasible_at_12} (expect FEASIBLE), "
              f"target 13 -> {infeasible_at_13} (expect INFEASIBLE)",
              feasible_at_12 == "FEASIBLE" and infeasible_at_13 == "INFEASIBLE")

    section("Search: rediscovering ell_2^dir(4) = 6 by random search")
    result = search_for_dense_graph(SIMPLE_DIRECTED, n=4, m=2, steps=6000, seed=0)
    check(f"best found = {result.best_edge_count} arcs, feasible lambda^max={max_edge_connectivity(result.best_graph)}",
          result.best_edge_count == 6 and max_edge_connectivity(result.best_graph) <= 1)

    section("Driver: one solve() call handles each kind of question")
    # Exhaustive directed: the cut-counting proves ell_2^dir(4) = 6 exactly.
    proved = solve(4, 2, directed=True, simple=True, exhaustive=True,
                   max_seconds=120.0)
    check(f"solve exhaustive simple-directed n=4,m=2: {proved.value} ({proved.bound})",
          proved.proven and proved.value == 6)
    # Discovery finds the same 6 arcs as a lower bound within a short budget.
    # The checks use sa (lighter); tabu is the default for actually solving.
    found = solve(4, 2, directed=True, simple=True, exhaustive=False,
                  max_seconds=3.0, method="sa")
    check(f"solve discovery simple-directed n=4,m=2: {found.value} ({found.bound})",
          found.bound == "lower" and found.value == 6)
    # The VERTEX separation is a separate question, not a corollary of the arc
    # one: Whitney's kappa <= lambda makes the vertex-feasible family the LARGER
    # of the two, so an arc upper bound does not restrict it.  thm:dir-vertex-m2
    # needs its own base cases, and these are the first three of them.
    for base_n, base_value in ((3, 4), (4, 6), (5, 8)):
        v_value, _, v_done = _exhaustive_directed(base_n, 2, "vertex",
                                                  time.time() + 120.0)
        check(f"exhaustive simple-directed VERTEX n={base_n},m=2: {v_value}",
              v_done and v_value == base_value)
    # The vertex flow helper must agree with the exact checker it stands in for.
    _vg = Graph(4, SIMPLE_DIRECTED)
    for _a, _b in ((0, 1), (0, 2), (1, 3), (2, 3)):
        _vg.mu[_a, _b] = 1
    _vout = [{_b for _b in range(4) if _vg.mu[_a, _b]} for _a in range(4)]
    check("vertex flow helper agrees with the exact checker on a theta digraph",
          _vertex_flow_at_least(_vout, 4, 0, 3, 2)
          and not _vertex_flow_at_least(_vout, 4, 0, 3, 3)
          and local_connectivity(_vg, 0, 3, vertex_split=True) == 2)
    # prop:dir-arc-stability, the unconditional quadratic bound.  Both the
    # load-bearing counting step (sum of d+ d- capped by m n(n-1)) and the bound
    # it yields are checked on sampled feasible digraphs.
    _rng = random.Random(3)
    _worst_slack = None
    _count_ok = True
    for _ in range(120):
        _sn, _sm = _rng.randint(3, 7), _rng.randint(2, 4)
        _sg = Graph(_sn, SIMPLE_DIRECTED)
        for _a in range(_sn):
            for _b in range(_sn):
                if _a != _b and _rng.random() < 0.6:
```

```
                    _sg.mu[_a, _b] = 1
        while max_edge_connectivity(_sg) > _sm - 1:   # prune down to feasible
            _present = [(a, b) for a in range(_sn) for b in range(_sn) if _sg.mu[a, b]]
            _a, _b = _rng.choice(_present)
            _sg.mu[_a, _b] = 0
        _dout = [int(_sg.mu[x].sum()) for x in range(_sn)]
        _din = [int(_sg.mu[:, x].sum()) for x in range(_sn)]
        # The sharp form of the counting step: (m-1) n(n-1), not m n(n-1).
        if sum(_dout[x] * _din[x] for x in range(_sn)) > (_sm - 1) * _sn * (_sn - 1):
            _count_ok = False
        _slack = ((_sn * _sn) // 4 + math.sqrt(_sm) * _sn ** 1.5
                  - int(_sg.mu.sum()))
        _worst_slack = _slack if _worst_slack is None else min(_worst_slack, _slack)
        # thm:dir-arc-linear-error, the O_m(n) bound.
        if int(_sg.mu.sum()) > (_sn * _sn) // 4 + 4 * (_sm - 1) * (_sn - 1):
            _count_ok = False
        # Case 2 of its proof: min total degree >= n/2 forces the linear bound
        # on the sum of the smaller half-degrees.
        if min(_dout[x] + _din[x] for x in range(_sn)) >= _sn / 2:
            if (sum(min(_dout[x], _din[x]) for x in range(_sn))
                    > 4 * (_sm - 1) * (_sn - 1)):
                _count_ok = False
    check("prop:dir-arc-stability and thm:dir-arc-linear-error hold on 120 "
          f"sampled feasible digraphs (tightest slack {_worst_slack:.1f})",
          _count_ok and _worst_slack >= 0)
    # sec:multi-vertex-standard: the OTHER counting convention is a different
    # problem, and the theta construction beats the thickened tree at m=5, n=4.
    _mv4, _, _mv4_done = max_multigraph_vertex_standard(4, 3)
    _mv5, _, _mv5_done = max_multigraph_vertex_standard(4, 5)
    check(f"multigraph vertex, other convention: K_3(4) = {_mv4} = (m-1)(n-1)",
          _mv4_done and _mv4 == 6)
    check(f"multigraph vertex, other convention: K_5(4) = {_mv5} > 12, the "
          "multigraph edge value, so the two problems differ",
          _mv5_done and _mv5 == 14)
    # Exhaustive undirected by brute force: ell_2(5) = n-1 = 4 (a spanning tree).
    tree = solve(5, 2, directed=False, simple=True, exhaustive=True,
                 max_seconds=30.0)
    check(f"solve exhaustive simple-undirected n=5,m=2: {tree.value} ({tree.bound})",
          tree.proven and tree.value == 4)
    # Hypergraph discovery returns a feasible lower bound for the r=3, m=2 case.
    hyper = solve(7, 2, hypergraph=True, r=3, exhaustive=False, max_seconds=3.0,
                  method="sa")
    check(f"solve discovery 3-uniform hypergraph n=7,m=2: {hyper.value} ({hyper.bound})",
          hyper.bound == "lower" and hyper.value == 3)

    # --- open-variant exploration coverage --------------------------------
    # These three functions are the program's only handle on the OPEN parts of
    # the problem (hypergraph vertex value, the fractional relaxation behind the
    # odd-n conjectures).  The first author-reviewed draft of this block ran
    # exhaustive n>=7 sweeps and 6000-step searches, which added minutes; the
    # checks below cover the same code paths at n<=5 so they finish in ~2s.
    section("Open variants: hypergraph vertex value (kappa^max) at small n")
    # m=2 uses the incidence-rank/forest witness, m=3 the brute-force
```

```
    # feasibility sweep; both confirm value = floor((m-1)(n-1)/(r-1)).
    for n, r, m in [(4, 3, 2), (5, 3, 2), (4, 3, 3), (5, 3, 3)]:
        target = ((m - 1) * (n - 1)) // (r - 1)
        check(f"hypergraph vertex value at n={n}, r={r}, m={m} = {target}",
              verify_hyper_vertex_value(n, r, m))

    section("Open variants: the fractional [0,1]-weight checker is exact")
    # A directed path keeps every max-flow at 1 (feasible); two internally
    # disjoint s->t routes push the s->t flow to 2 (infeasible).  This is the
    # relaxation that fractional_search optimises over.
    path_w = np.zeros((4, 4))
    path_w[0, 1] = path_w[1, 2] = path_w[2, 3] = 1.0
    two_route = np.zeros((4, 4))
    two_route[0, 1] = two_route[0, 2] = two_route[1, 3] = two_route[2, 3] = 1.0
    check("path weighting feasible, two-route weighting infeasible",
          fractional_flows_feasible(path_w)
          and not fractional_flows_feasible(two_route))

    section("Open variants: fractional search respects the odd-n ceilings")
    # Short n=5 runs (the conjectured bipartite point is rigid).  The
    # min_degree score carries a 1e-3 * total-weight tie-breaker, so its
    # comparison allows that slack; a genuine refutation would clear the
    # ceiling by a full unit, far above it.
    _, total_best = fractional_search(5, "total", steps=1500, seed=1)
    check(f"fractional total weight at n=5 stays <= 8: {total_best:.3f}",
          total_best <= 8 + 1e-6)
    _, deg_best = fractional_search(5, "min_degree", steps=1500, seed=1)
    check(f"fractional min degree at n=5 stays <= k=2 (+tie-break slack): {deg_best:.3f}",
          deg_best <= 2 + 1e-3 * 5 * 5)

    print(f"\n{'ALL CHECKS PASSED' if _failures == 0 else f'{_failures} CHECK(S) FAILED'}")
    return 1 if _failures else 0


if __name__ == "__main__":
    print(f"C extension: {'LOADED (_erdos_fast.so)' if C_EXTENSION_LOADED else 'not found (pure Python)'}")
    raise SystemExit(_run_checks())
```

C.2 The figure script

This is the only script that writes image files. It imports the main program and calls its plotting routines, so it holds no mathematics of its own. Keeping it separate means the figures can be regenerated in one command without running anything else, and every random choice it makes is seeded, so running it twice produces identical files.

```
"""
Regenerate every figure the thesis uses, from the one program next door.

This is the only figure script.  It imports the single program
``erdos915_unified.py`` and calls its plotting routines, so the figures can never
drift from the code that produces the numbers.  Run it from this ``program/``
directory:
```

```

    python make_figures.py

Each figure is written into ``../figures/``.  Every randomised search uses a
fixed seed, so the output is reproducible.
"""

from __future__ import annotations

import math
import sys
from pathlib import Path

import numpy as np

sys.path.insert(0, str(Path(__file__).resolve().parent))

from erdos915_unified import (  # noqa: E402
    MULTI_DIRECTED,
    MULTI_UNDIRECTED,
    SIMPLE_DIRECTED,
    SIMPLE_UNDIRECTED,
    Graph,
    _VARIANT_ENUM_CONFIGS,
    search_for_dense_graph,
    compute_enumeration_cache,
    compute_pair_enumeration_cache,
    compute_surface_cache,
    connectivity_distribution,
    gallery_extremal_graphs,
    directed_arc_lower_bound,
    directed_multigraph_arc,
    draw_graph_with_sensitivity,
    edge_vertex_distribution,
    hypergraph_edge,
    multigraph_undirected_edge,
    plot_complexity_growth,
    plot_extremal_gallery,
    plot_conn_dist_grid,
    plot_conn_threshold_3d,
    plot_connectivity_distribution,
    plot_pair_conn_dist_grid,
    plot_directed_crossover,
    plot_edge_vertex_divergence,
    plot_edge_vertex_histograms,
    plot_edge_dist_grid,
    plot_scatter_lambda_edges,
    plot_search_trace,
    plot_sa_vs_tabu_convergence,
    plot_variant_3d_surfaces,
    plot_variant_grid,
    save_gallery_json,
    simple_undirected_edge,
    solve,
)

FIGURES = Path(__file__).resolve().parent.parent / "figures"

```

```python

# ---------------------------------------------------------------------------
#  The all-variant grid: proved / conjectured / guessed, with machine points.
#  Every number below comes from one driver, ``solve``: exhaustive for an exact
#  point, discovery for a search lower bound.  The formula curves come from the
#  closed forms; the guess is a fit; the band is [construction LB, trivial max].
# ---------------------------------------------------------------------------

def _exact_points(ns, m, budget, **kw):
    """Machine-proved sizes: increasing n until the exhaustion no longer finishes."""
    xs, ys = [], []
    for n in ns:
        res = solve(n, m, exhaustive=True, max_seconds=budget, **kw)
        if res.bound == "exact":
            xs.append(n)
            ys.append(res.value)
        else:
            break           # once it cannot finish, larger n cannot either
    return xs, ys


def _search_points(ns, m, budget, **kw):
    """Discovery lower bounds: a concrete feasible graph at each n (the LB construction)."""
    xs, ys = [], []
    for n in ns:
        res = solve(n, m, exhaustive=False, max_seconds=budget, seed=0, **kw)
        xs.append(n)
        ys.append(res.value)
    return xs, ys


def _band(search, trivial_max, ns):
    """The certain interval: an easy construction below, the trivial maximum above.

    The lower edge is the densest feasible graph the search exhibited (a real
    lower bound); the upper edge is the most edges any graph on n vertices can
    hold at all (a loose but certain upper bound).  The truth lies in between.
    """
    found = dict(zip(search[0], search[1]))
    lower = [min(found.get(n, 0), trivial_max(n)) for n in ns]
    upper = [trivial_max(n) for n in ns]
    return list(ns), lower, upper


def _extend_lower_bounds(search, lb_fn, ns):
    """Raise the search lower bounds to a known verified construction where it wins.

    Past the size the quick search reaches, an explicit construction the checker
    confirms is a better (still honest) lower bound, so we report the best of the
    two at each n, exactly as the rediscovery section does by hand.
    """
    found = dict(zip(search[0], search[1]))
    for n in ns:
        found[n] = max(found.get(n, 0), lb_fn(n))
```

```python
    xs = sorted(found)
    return xs, [found[n] for n in xs]


def _reconcile_panel(panel: dict) -> dict:
    """Make a panel internally consistent before plotting.

    Three honest invariants, enforced here once for every panel rather than
    scattered through the construction code:

    1. **Nothing exceeds a proved optimum.**  A machine-checked exact value is
       the true maximum at that ``n``; no proved/conjectured curve, no
       named-construction lower bound, and no search circle may sit above it.
       (This is what produced the green square *below* its own curve: a closed
       form -- e.g. the hypergraph bound capped at the complete hypergraph -- can
       overshoot what is actually achievable at small ``n``.)
    2. **Search lower bounds rise with ``n``.**  A feasible graph on ``n``
       vertices stays feasible on ``n+1`` (add an isolated vertex), so the best
       feasible edge count can never *drop* as ``n`` grows.  We take a running
       maximum, which is exactly that extend-by-an-isolated-vertex bound.
    3. **No certain-interval band, and the guess interpolates rather than
       overshoots.**  The loose [construction, trivial-max] band dwarfed the
       data and is dropped (axes rescale to the data).  On an open panel the
       dotted guess is the piecewise-linear interpolation *through* the search
       points -- never a fitted curve riding above points it should pass through.
    """
    panel.pop("band", None)  # invariant 3a: no shaded certain interval

    exact = dict(zip(*panel["exact"])) if panel.get("exact") else {}

    def cap_to_exact(curve):
        if curve is None:
            return None
        xs, ys = curve
        return xs, [min(y, exact[x]) if x in exact else y for x, y in zip(xs, ys)]

    # invariant 1 applied to every formula line and to the named branches
    for key in ("proved", "conj"):
        if panel.get(key) is not None:
            panel[key] = cap_to_exact(panel[key])
    if panel.get("branches"):
        panel["branches"] = [(*cap_to_exact((bx, by)), lbl)
                             for bx, by, lbl in panel["branches"]]

    # invariants 1 then 2 applied to the search circles
    if panel.get("search") is not None:
        sx, sy = panel["search"]
        sy = [min(y, exact[x]) if x in exact else y for x, y in zip(sx, sy)]
        running, mono = -1, []
        for y in sy:
            running = max(running, y)
            mono.append(running)
        panel["search"] = (sx, mono)

    # invariant 3b: the open-panel guess is the interpolation through those
    # (monotone, exact-capped) points, so it can never ride above them.
    if panel.get("guess") == "search":
        panel["guess"] = panel.get("search")
```

```
    return panel


def gather_variant_grid(m=3, exact_budget=4.0, search_budget=0.4, open_search_budget=4.0):
    """Build the twelve panels of the all-variant grid (row-major, model x col).

    ``m`` is the forbidden connectivity value shown in every panel.

    Two uniformity rules make the twelve panels directly comparable:

    1. Every matrix panel plots its search lower bounds over the *same* vertex
       range ``matrix_ns`` and every hypergraph panel over ``hyper_ns``, so each
       panel carries the same number of data points -- no panel trails off early.
    2. For every panel where a construction is known (proved or conjectured) the
       search lower bound is raised to that named construction with
       :func:`_extend_lower_bounds`, exactly as the caption of the figure and
       the rediscovery section state: the reported circle at each ``n`` is the
       better of the cooled search and the verified construction.  This is the
       honest "lands on the curve" rule, applied to *all* panels rather than
       only the two directed ones it used to cover, so the proved curves pass
       through their circles instead of floating above a jagged search tail.

    Open panels also get a verified construction planted, not only the proved
    ones: the open undirected vertex panels get the proved EDGE value (Whitney,
    kappa <= lambda, so the edge extremiser is vertex-feasible) and the two
    directed-hypergraph panels get the proved bipartite family of
    prop:dir-hyper-first.  This matters because at ``search_budget=0.4`` the
    vertex-mode search on a large matrix graph barely completes one move, so
    the RAW result actively *degrades* with growing ``n`` (measured:
    n=6/10/14/16 gave 15/12/5/4 at m=6), and the monotone running-max in
    :func:`_reconcile_panel` then freezes the plotted curve at its early peak,
    a flat line that looks like a finding but is really budget starvation.
    ``open_search_budget`` (default 4s, ~10x ``search_budget``) still gives
    those panels a stronger walk, since the search can genuinely beat the
    planted construction where the open problem is richer (it does at m=6,
    n=9 on the undirected vertex panel).
    """
    panels = []
    # Uniform x-ranges for the search / construction curves.  The machine-PROVED
    # (exact) points are genuinely limited by enumeration cost and stop early; the
    # search lower bounds and the formula curves are cheap, so they run well past
    # that, to show each variant's trend and let the search discover dense
    # ("maybe extremal") graphs at sizes exhaustion cannot reach.
    matrix_ns = list(range(2, 17))   # all matrix panels, n = 2..16
    hyper_ns = list(range(2, 13))    # all hypergraph panels, n = 2..12

    def tri_undirected(n):
        return n * (n - 1) // 2

    def tri_hyper(n, r=3):
        return math.comb(n, r)

    def tri_dir_hyper(n, r=3):
        return n * math.comb(n - 1, r - 1)

    # Construction lower bounds (verified, proved-or-conjectured values), each
```

```
    # capped at the trivial maximum for its model.  The cap matters at small n:
    # a closed form like Mader's floor(m(n-1)/2) is the value only for n >= m;
    # for n < m the complete graph K_n already has lambda^max = n-1 <= m-1, so it
    # is feasible and denser, and the true value is comb(n,2).  Without the cap
    # the plotted curve and its lower-bound circles float ABOVE the exact squares,
    # which is impossible (a lower bound can never exceed the proved optimum).
    def lb_simple_edge(n):
        return min(simple_undirected_edge(n, m), n * (n - 1) // 2)

    def lb_multi_edge(n):
        return min(multigraph_undirected_edge(n, m), (m - 1) * (n * (n - 1) // 2))

    def lb_dir(n):
        return min(directed_arc_lower_bound(n, m), n * (n - 1))

    def lb_multi_dir(n):
        return min(directed_multigraph_arc(n, m), (m - 1) * n * (n - 1))

    def lb_hyper_edge(n):
        return min(hypergraph_edge(n, m, 3), math.comb(n, 3))

    def lb_dir_hyper(n):
        # The PROVED bipartite family of prop:dir-hyper-first at r = 3:
        # alpha tails, and on the other n - alpha vertices a shared near-regular
        # head graph of max degree m - 1 (realisable iff m - 1 <= n - alpha - 1),
        # giving alpha * floor((m-1)(n-alpha)/2) single-step hyperarcs with
        # lambda^max = kappa^max = m - 1.  Sound for BOTH separations, since
        # every route in it is a single tail -> head step.
        best = 0
        for alpha in range(1, n - m + 1):
            best = max(best, alpha * ((m - 1) * (n - alpha) // 2))
        return min(best, n * math.comb(n - 1, 2))

    # Undirected vertex is proved for m<=4 (Leonard/Whitney), open for m>=5.
    vert_proved = (m <= 4)
    # Hypergraph vertex is proved for m<=3 (incidence-rank lemma, thm:hyper-vertex-m3).
    hyper_vert_proved = (m <= 3)

    def searched(ns, lb_fn, **solve_kw):
        """Search over the uniform range, then raise to the known construction."""
        se = _search_points(ns, m, search_budget, **solve_kw)
        if lb_fn is not None:
            se = _extend_lower_bounds(se, lb_fn, ns)
        return se

    # ----- row 1: simple -----------------------------------------------
    # (1) simple undirected edge -- proved (Mader) for all m.
    ex = _exact_points(range(2, 9), m, exact_budget,
                       directed=False, simple=True, separation="edge")
    se = searched(matrix_ns, lb_simple_edge,
                  directed=False, simple=True, separation="edge")
    panels.append(dict(
        title=f"undirected edge, $m={m}$  (proved)", ylabel="edges",
        proved=(matrix_ns, [lb_simple_edge(n) for n in matrix_ns]),
        exact=ex, search=se))

    # (2) simple undirected vertex -- proved for m<=4, open for m>=5.
```

```
    ex2 = _exact_points(range(2, 8), m, exact_budget,
                        directed=False, simple=True, separation="vertex")
    if vert_proved:
        se2 = searched(matrix_ns, lb_simple_edge,
                       directed=False, simple=True, separation="vertex")
        panels.append(dict(
            title=f"undirected vertex, $m={m}$  (proved)", ylabel="edges",
            proved=(matrix_ns, [lb_simple_edge(n) for n in matrix_ns]),
            exact=ex2, search=se2))
    else:
        se2 = _search_points(matrix_ns, m, open_search_budget,
                             directed=False, simple=True, separation="vertex")
        # Whitney: kappa <= lambda, so Mader's edge extremiser is vertex-feasible
        # and the proved edge value is an honest lower bound on the open vertex
        # panel too.  Without it the starved vertex-mode search freezes into a
        # flat plateau at large n (the "cut off" look); the search may still beat
        # the edge value where the vertex problem is genuinely richer.
        se2 = _extend_lower_bounds(se2, lb_simple_edge, matrix_ns)
        band2 = _band(se2, tri_undirected, matrix_ns)
        panels.append(dict(
            title=f"undirected vertex, $m={m}$  (open)", ylabel="edges",
            guess="search",
            band=band2, exact=ex2, search=se2))

    # (3) simple directed arc -- conjectured.
    ex3 = _exact_points(range(2, 8), m, exact_budget,
                        directed=True, simple=True, separation="edge")
    se3 = searched(matrix_ns, lb_dir,
                   directed=True, simple=True, separation="edge")
    panels.append(dict(
        title=f"directed arc, $m={m}$  (conjectured)", ylabel="arcs",
        conj=(matrix_ns, [lb_dir(n) for n in matrix_ns]),
        branches=[(matrix_ns, [min(m * (n - 1), n * (n - 1)) for n in matrix_ns],
                   "hub $m(n{-}1)$"),
                  # Shifted-partition augmented bipartite floor((n+m-2)^2/4)
                  # (the corrected conj:dir-arc branch; the old balanced count
                  # floor(n^2/4)+(m-2)ceil(n/2) agrees at m<=3 but is smaller
                  # at m>=4 and would sit below the conjecture curve at m=6).
                  (matrix_ns,
                   [min((n + m - 2) ** 2 // 4, n * (n - 1))
                    for n in matrix_ns], "bipartite")],
        exact=ex3, search=se3))

    # (4) simple directed vertex -- conjectured (= arc).
    ex4 = _exact_points(range(2, 8), m, exact_budget,
                        directed=True, simple=True, separation="vertex")
    se4 = searched(matrix_ns, lb_dir,
                   directed=True, simple=True, separation="vertex")
    panels.append(dict(
        title=f"directed vertex, $m={m}$  (conjectured)", ylabel="arcs",
        conj=(matrix_ns, [lb_dir(n) for n in matrix_ns]),
        exact=ex4, search=se4))

    # ----- row 2: multigraph ------------------------------------------
    # (5) multigraph undirected edge -- proved for all m.
    ex5 = _exact_points(range(2, 7), m, exact_budget,
                        directed=False, simple=False, separation="edge")
    se5 = searched(matrix_ns, lb_multi_edge,
```

```
                   directed=False, simple=False, separation="edge")
    panels.append(dict(
        title=f"undirected edge, $m={m}$  (proved)", ylabel="edges",
        proved=(matrix_ns, [lb_multi_edge(n) for n in matrix_ns]),
        exact=ex5, search=se5))

    # (6) multigraph undirected vertex -- equals simple (parallel edges irrelevant for vertex cuts).
    multi_vert_label = ("proved, $=$ simple" if vert_proved else "open, $=$ simple")
    if vert_proved:
        panels.append(dict(
            title=f"undirected vertex, $m={m}$  ({multi_vert_label})", ylabel="edges",
            proved=(matrix_ns, [lb_simple_edge(n) for n in matrix_ns]),
            exact=ex2, search=se2))
    else:
        panels.append(dict(
            title=f"undirected vertex, $m={m}$  ({multi_vert_label})", ylabel="edges",
            guess="search",
            band=_band(se2, tri_undirected, matrix_ns), exact=ex2, search=se2))

    # (7) multigraph directed arc -- certified (cut-counting) then conjectured.
    ex7 = _exact_points(range(3, 6), m, 10.0,
                        directed=True, simple=False, separation="edge")
    se7 = searched(matrix_ns, lb_multi_dir,
                   directed=True, simple=False, separation="edge")
    panels.append(dict(
        title=f"directed arc, $m={m}$  (multigraph, certified)", ylabel="arcs",
        conj=(matrix_ns, [lb_multi_dir(n) for n in matrix_ns]),
        exact=ex7, search=se7))

    # (8) multigraph directed vertex -- conjectured (reduces to simple digraph).
    panels.append(dict(
        title=f"directed vertex, $m={m}$  (conjectured, $=$ simple)", ylabel="arcs",
        conj=(matrix_ns, [lb_dir(n) for n in matrix_ns]),
        exact=ex4, search=se4))

    # ----- row 3: hypergraph (r=3) -------------------------------------
    # (9) hypergraph undirected edge -- proved for all m.
    ex9 = _exact_points(range(2, 8), m, exact_budget,
                        hypergraph=True, r=3, directed=False, separation="edge")
    se9 = searched(hyper_ns, lb_hyper_edge,
                   hypergraph=True, r=3, directed=False, separation="edge")
    panels.append(dict(
        title=f"undirected edge, $m={m}$  (proved)", ylabel="hyperedges",
        proved=(hyper_ns, [lb_hyper_edge(n) for n in hyper_ns]),
        exact=ex9, search=se9))

    # (10) hypergraph undirected vertex -- PROVED for m<=3 (incidence-rank lemma),
    #       open for m>=4.
    ex10 = _exact_points(range(2, 7), m, exact_budget,
                         hypergraph=True, r=3, directed=False, separation="vertex")
    if hyper_vert_proved:
        se10 = searched(hyper_ns, lb_hyper_edge,
```

```
                        hypergraph=True, r=3, directed=False, separation="vertex")
        panels.append(dict(
            title=f"undirected vertex, $m={m}$ (proved)", ylabel="hyperedges",
            proved=(hyper_ns, [lb_hyper_edge(n) for n in hyper_ns]),
            exact=ex10, search=se10))
    else:
        se10 = _search_points(hyper_ns, m, open_search_budget,
                              hypergraph=True, r=3, directed=False, separation="vertex")
        # Whitney again: the proved hyperedge value is a lower bound for the
        # vertex separation (its extremiser is vertex-feasible).  The exact cap
        # in _reconcile_panel corrects the small-n corner where the simple
        # attainment condition m-1 <= n-2 has not kicked in yet.
        se10 = _extend_lower_bounds(se10, lb_hyper_edge, hyper_ns)
        panels.append(dict(
            title=f"undirected vertex, $m={m}$ (open)", ylabel="hyperedges",
            guess="search",
            band=_band(se10, tri_hyper, hyper_ns), exact=ex10, search=se10))

    # (11) hypergraph directed arc -- OPEN (new directed Berge model).
    ex11 = _exact_points(range(2, 6), m, exact_budget,
                         hypergraph=True, r=3, directed=True, separation="edge")
    se11 = _search_points(hyper_ns, m, open_search_budget,
                          hypergraph=True, r=3, directed=True, separation="edge")
    # The proved bipartite construction of prop:dir-hyper-first is the named
    # lower bound here (the search alone slips below the quadratic at larger n).
    se11 = _extend_lower_bounds(se11, lb_dir_hyper, hyper_ns)
    panels.append(dict(
        title=f"directed arc, $m={m}$ (open, new model)", ylabel="hyperarcs",
        guess="search",
        band=_band(se11, tri_dir_hyper, hyper_ns), exact=ex11, search=se11))

    # (12) hypergraph directed vertex -- OPEN (new directed Berge model).
    ex12 = _exact_points(range(2, 6), m, exact_budget,
                         hypergraph=True, r=3, directed=True, separation="vertex")
    se12 = _search_points(hyper_ns, m, open_search_budget,
                          hypergraph=True, r=3, directed=True, separation="vertex"
                          )
    # Same construction: all its routes are single tail -> head steps, so it is
    # feasible for the vertex separation at the same value.
    se12 = _extend_lower_bounds(se12, lb_dir_hyper, hyper_ns)
    panels.append(dict(
        title=f"directed vertex, $m={m}$ (open, new model)", ylabel="hyperarcs",
        guess="search",
        band=_band(se12, tri_dir_hyper, hyper_ns), exact=ex12, search=se12))

    return [_reconcile_panel(p) for p in panels]


def main() -> None:
    FIGURES.mkdir(parents=True, exist_ok=True)

    plot_directed_crossover(m=3, max_n=24, path=FIGURES / "directed_crossover.png"
        )
    print("wrote directed_crossover.png")

    plot_edge_vertex_divergence(max_n=35, path=FIGURES / "edge_vertex_divergence.
        png")
```

```
    print("wrote edge_vertex_divergence.png")

    # Figure 3.2: cooling trace for the directed multigraph n=4, m=3 case.
    # record_exact_connectivity logs the true lambda^max per step (more expensive
    # but needed for an exact scatter; the search itself uses the cheaper upper bound).
    run = search_for_dense_graph(MULTI_DIRECTED, n=4, m=3, steps=3000, seed=1,
                                 record_exact_connectivity=True)
    plot_search_trace(run, path=FIGURES / "temperature_trace.png",
                      optimum=12, ceiling=2, show_histogram=False)
    print(f"wrote temperature_trace.png (search reached {run.best_edge_count} arcs of 12)")

    # Additional scatter-only traces for other variants (no cooling panel needed
    # since the cooling schedule is the same across all cases).
    trace_cases = [
        # (variant, n, m, filename_stem, ceiling, connectivity_label, edge_label, title)
        (SIMPLE_UNDIRECTED, 7, 3, "trace_simple_undirected_n7_m3", 2,
         r"$\lambda^{\max}$", "edges",
         r"Search trace: simple undirected, $n=7$, $m=3$"),
        (SIMPLE_DIRECTED, 5, 3, "trace_simple_directed_n5_m3", 2,
         r"$\lambda^{\max}$", "arcs",
         r"Search trace: simple directed, $n=5$, $m=3$"),
        (MULTI_UNDIRECTED, 5, 3, "trace_multi_undirected_n5_m3", 2,
         r"$\lambda^{\max}$", "edges",
         r"Search trace: multi undirected, $n=5$, $m=3$"),
        (MULTI_DIRECTED, 5, 3, "trace_multi_directed_n5_m3", 2,
         r"$\lambda^{\max}$", "arcs",
         r"Search trace: multi directed, $n=5$, $m=3$"),
        (MULTI_DIRECTED, 4, 3, "trace_multi_directed_n4_m3_vertex", 2,
         r"$\kappa^{\max}$", "arcs",
         r"Search trace: multi directed vertex-sep, $n=4$, $m=3$"),
    ]
    for variant, n, m, stem, ceil_val, clabel, elabel, ttitle in trace_cases:
        sep = "vertex" if "vertex" in stem else "edge"
        tr = search_for_dense_graph(variant, n=n, m=m, separation=sep,
                                    steps=3000, seed=1,
                                    record_exact_connectivity=True)
        plot_search_trace(tr, path=FIGURES / f"{stem}.png",
                          ceiling=ceil_val, show_cooling=False,
                          show_histogram=False,
                          connectivity_label=clabel, edge_label=elabel,
                          title=ttitle)
        print(f"wrote {stem}.png  (best={tr.best_edge_count})")

    # A larger directed multigraph: four s->t routes of differing capacity plus a
    # dead end.  Parallel arcs are drawn explicitly, so multiplicity is read by
    # counting.  The s->a route is over-provisioned (mu=3 into a but a single arc
    # a->t out), so it appears as three arcs in a COOL colour (sigma=1): the
    # visual point that multiplicity and load-bearing role are not the same.
    sens = Graph(8, MULTI_DIRECTED)   # 0=s 1=t 2=a 3=b 4=c 5=d 6=e 7=f
    sens.set_multiplicity(0, 2, 3); sens.set_multiplicity(2, 1, 1)  # s->a->t, over-provisioned in
    sens.set_multiplicity(0, 3, 3); sens.set_multiplicity(3, 1, 3)  # s->b->t, cap 3 (load-bearing)
    sens.set_multiplicity(0, 4, 2); sens.set_multiplicity(4, 1, 2)  # s->c->t, cap 2
```

```
    sens.set_multiplicity(0, 6, 1); sens.set_multiplicity(6, 1, 1)  # s->e->t, cap 1
    sens.set_multiplicity(0, 7, 2); sens.set_multiplicity(7, 1, 2)  # s->f->t, cap 2
    sens.set_multiplicity(0, 5, 2)                                  # s->d, dead end
    sens_layout = {0: (-3.2, 0.0), 1: (3.2, 0.0),
                   2: (0.0, 2.6), 3: (0.0, 1.3), 4: (0.0, 0.0),
                   6: (0.0, -1.3), 7: (0.0, -2.6), 5: (-3.2, -2.4)}
    sens_labels = {0: "s", 1: "t", 2: "a", 3: "b", 4: "c", 5: "d", 6: "e", 7: "f"}
    draw_graph_with_sensitivity(
        sens, path=FIGURES / "sensitivity_mixed.png",
        layout=sens_layout, node_labels=sens_labels)
    print("wrote sensitivity_mixed.png")

    # Annealing vs tabu convergence (Ch.3): wall-clock timed, so a representative
    # run rather than a seed-exact one (see the figure caption).
    plot_sa_vs_tabu_convergence(FIGURES / "sa_vs_tabu_convergence.pdf",
                                cases=((5, 3), (7, 3)), budget=8.0, seed=0)
    print("wrote sa_vs_tabu_convergence.pdf")

    plot_complexity_growth(path=FIGURES / "complexity_growth.png")
    print("wrote complexity_growth.png")

    # Gallery of machine-found extremal graphs (reads extremal_gallery.json).
    plot_extremal_gallery(FIGURES / "extremal_graphs_gallery.png")
    print("wrote extremal_graphs_gallery.png")

    # Bounds grids: one for m=3 and one for m=6 (more purple dots).
    for m_val in (3, 6):
        print(f"gathering all-variant grid m={m_val} (runs solve many times)...")
        panels = gather_variant_grid(m=m_val)
        out = FIGURES / f"variant_bounds_m{m_val}.png"
        plot_variant_grid(panels, path=out, m=m_val)
        print(f"wrote {out.name}")

    # ---------------------------------------------------------------
    # Full-enumeration scatter and distribution figures.
    # The enumeration cache is written once to figures/enumeration_cache.pkl.
    # ---------------------------------------------------------------
    print("building enumeration cache (slow on first run, cached thereafter)...")
    enum_cache_path = FIGURES / "enumeration_cache.pkl"
    enum_data = compute_enumeration_cache(cache_path=enum_cache_path)
    print("enumeration cache ready")

    plot_scatter_lambda_edges(enum_data, path=FIGURES / "scatter_lambda_edges.png")
    print("wrote scatter_lambda_edges.png")

    # Pooled per-pair connectivity: every vertex pair of every enumerated graph,
    # tagged with its graph's lambda^max.  Slow first run, cached thereafter.
    print("building pair-connectivity cache (slow on first run, cached thereafter)...")
    pair_cache_path = FIGURES / "pair_enumeration_cache.pkl"
    pair_data = compute_pair_enumeration_cache(cache_path=pair_cache_path)
    print("pair-connectivity cache ready")

    # Pooled per-pair connectivity histograms (Ch.4): each panel stacks blue/red
```

```python
    # at its own mid-range connectivity boundary, so the split is meaningful in
    # every variant rather than collapsing to one colour at a fixed m.
    plot_pair_conn_dist_grid(pair_data, path=FIGURES / "pair_conn_dist.png")
    print("wrote pair_conn_dist.png")

    # Edge-count histograms (Ch.4): same mid-range stacked colouring.
    plot_edge_dist_grid(enum_data, path=FIGURES / "edges_dist.png")
    print("wrote edges_dist.png")

    # Per-graph connectivity distribution at the fixed forbidden threshold m=6
    # (App. B), showing the m=6 feasibility split; m=3 body version removed 2026-06-20.
    plot_conn_dist_grid(enum_data, 6, path=FIGURES / "conn_dist_m6.png")
    print("wrote conn_dist_m6.png")

    # ---------------------------------------------------------------
    # 3-D bound surface over the (n, m) grid.
    # Uses a JSON cache (slow first run, instant thereafter).
    # ---------------------------------------------------------------
    surface_cache_path = FIGURES / "surface_cache.json"
    print("building surface cache (slow on first run, cached thereafter)...")
    compute_surface_cache(
        n_range=(3, 9), m_range=(2, 6),
        max_seconds=20, cache_path=surface_cache_path)
    plot_variant_3d_surfaces(surface_cache_path, path=FIGURES / "variant_surface_3d.png")
    print("wrote variant_surface_3d.png")

    # ---------------------------------------------------------------
    # 3-D threshold histogram (appendix): how the lambda_max distribution shifts
    # with density p, for three representative variants.  Kept as a standalone
    # appendix figure (fig:threshold-3d) after the Figure 4.2 threshold story was
    # removed from the body on 2026-06-20.
    # ---------------------------------------------------------------
    # No p_values: each panel sweeps in units of its own threshold, since the
    # graph and hypergraph models do not share a density scale.
    plot_conn_threshold_3d(
        path=FIGURES / "threshold_3d.png",
        n=12, m=3, samples=150, seed=7)
    print("wrote threshold_3d.png")

    # NOTE: the 2-D random-sampling threshold figures (degree_threshold,
    # sampled_variant_grid) were removed from the thesis on 2026-06-20.  Their
    # generators are kept in erdos915_unified.py and the restore recipe is in
    # research_notes/removed_threshold_phenomenon.md.

    # Edge vs vertex disjointness in G(16, p) at three densities (Chapter 1).
    p_values = [0.25, 0.5, 0.75]
    data = {pp: edge_vertex_distribution(16, pp, trials=400, seed=7)
            for pp in p_values}
    plot_edge_vertex_histograms(data, 16, p_values,
                                FIGURES / "edge_vertex_sampling.png")
    print("wrote edge_vertex_sampling.png")

    # Gallery of extremal graphs: all 12 variants × small (n, m), ~3s per case.
    print("building extremal graph gallery (this takes a few minutes)...")
    gallery_path = FIGURES / "extremal_gallery.json"
    gal = gallery_extremal_graphs(max_n=7, max_m=4, r=3, time_per_case=3.0)
```

```python
    save_gallery_json(gal, gallery_path)
    total_classes = sum(
        len(cases.get("classes", []))
        for variant_data in gal.values()
        for cases in variant_data.values()
    )
    print(f"wrote {gallery_path.name} ({total_classes} iso-classes across all variants)")


if __name__ == "__main__":
    main()
```

C.3 The C accelerator

Two operations dominate the running time at the sizes the searches reach: deciding whether a pair of vertices already has too many independent routes, and reducing a graph to a standard form so that two drawings of the same graph can be recognised as equal. Both are rewritten here in C, which is compiled separately and loaded if it is present.

Nothing depends on it. Every function below has a pure Python counterpart in the main program, the two are tested against each other, and if the compiled library is missing the program simply uses the Python version. The size limits in the code are deliberate: the fixed-size buffers are safe only up to the stated number of vertices, and above it the program falls back rather than risking a buffer it cannot hold.

```c
/*
 * _erdos_fast.c  -  hot-path C helpers for erdos915_unified.py
 *
 * Compile:  gcc -O3 -march=native -shared -fPIC -o _erdos_fast.so _erdos_fast.c
 *
 * All functions take a flat row-major int32 array `mu` of size n*n.
 * Access:  mu[u*n + v]  is the capacity (multiplicity) of arc u->v.
 */

#include <stdint.h>
#include <string.h>
#include <stdlib.h>

/* ---------------------------------------------------------------- */
/* tiny_maxflow                                                        */
/* DFS Ford-Fulkerson; returns 1 iff max-flow(s,t) > cap.           */
/* Stops as soon as it finds cap+1 augmenting paths (early exit).   */
/* n <= 7 in practice; residual lives on the stack.                 */
/* ---------------------------------------------------------------- */
int tiny_maxflow(const int *mu, int n, int s, int t, int cap)
{
    /* local mutable residual; sized for n up to 16 (vertex-split at n=8) */
    int res[256];
    memcpy(res, mu, (size_t)(n * n) * sizeof(int));

    int flow = 0;
    while (flow <= cap) {
```

```
        /* DFS to find an augmenting path */
        int parent[16];
        for (int i = 0; i < n; i++) parent[i] = -1;
        parent[s] = s;

        /* stack-based DFS (max depth = n vertices) */
        int stack[16];
        int sp = 0;
        stack[sp++] = s;

        while (sp > 0 && parent[t] == -1) {
            int u = stack[--sp];
            for (int v = 0; v < n; v++) {
                if (parent[v] == -1 && res[u * n + v] > 0) {
                    parent[v] = u;
                    stack[sp++] = v;
                }
            }
        }

        if (parent[t] == -1)
            return 0;   /* max-flow <= cap */

        /* find bottleneck */
        int bottleneck = 1 << 30;
        for (int v = t; v != s; ) {
            int u = parent[v];
            int cap_uv = res[u * n + v];
            if (cap_uv < bottleneck) bottleneck = cap_uv;
            v = u;
        }

        /* augment */
        for (int v = t; v != s; ) {
            int u = parent[v];
            res[u * n + v] -= bottleneck;
            res[v * n + u] += bottleneck;
            v = u;
        }

        flow += bottleneck;
    }
    return 1;   /* max-flow > cap */
}

/* ------------------------------------------------------------------ */
/* max_connectivity_exceeds                                             */
/* Sweeps all pairs; returns 1 iff any pair has flow > k.             */
/* directed=1: all ordered pairs; directed=0: unordered pairs.        */
/* ------------------------------------------------------------------ */
int max_connectivity_exceeds(const int *mu, int n, int k, int directed)
{
    for (int s = 0; s < n; s++) {
        int start = directed ? 0 : s + 1;
        for (int t = start; t < n; t++) {
            if (s == t) continue;
            if (tiny_maxflow(mu, n, s, t, k))
                return 1;
```

```
        }
    }
    return 0;
}

/* ------------------------------------------------------------------ */
/* canonical_form_min                                                   */
/* Writes the lex-min row-major permutation of the n×n matrix into     */
/* out (n*n int32 values).  Uses Heap's algorithm to enumerate n!      */
/* permutations.  At n=7: 5040 iterations × 49 ints each.             */
/* ------------------------------------------------------------------ */
void canonical_form_min(const int *mu, int n, int *out)
{
    int perm[7], best[49], cand[49];
    int c[7];    /* Heap's algorithm counters */
    int nn = n * n;

    for (int i = 0; i < n; i++) { perm[i] = i; c[i] = 0; }

    /* identity permutation is the first candidate */
    for (int r = 0; r < n; r++)
        for (int col = 0; col < n; col++)
            best[r * n + col] = mu[perm[r] * n + perm[col]];

    /* Heap's algorithm */
    int i = 1;
    while (i < n) {
        if (c[i] < i) {
            /* swap */
            if (i % 2 == 0) {
                int tmp = perm[0]; perm[0] = perm[i]; perm[i] = tmp;
            } else {
                int tmp = perm[c[i]]; perm[c[i]] = perm[i]; perm[i] = tmp;
            }

            /* build candidate permuted matrix */
            for (int r = 0; r < n; r++)
                for (int col = 0; col < n; col++)
                    cand[r * n + col] = mu[perm[r] * n + perm[col]];

            /* update best if cand is lex-smaller */
            if (memcmp(cand, best, (size_t)nn * sizeof(int)) < 0)
                memcpy(best, cand, (size_t)nn * sizeof(int));

            c[i]++;
            i = 1;
        } else {
            c[i] = 0;
            i++;
        }
    }

    memcpy(out, best, (size_t)nn * sizeof(int));
}
```

C.4 The test suite

The suite checks the program against facts established independently of it: closed forms proved by hand, constructions whose size and connectivity are known in advance, and small cases where an exhaustive walk gives the answer outright. Several tests compare two routines that are supposed to agree, such as a fast path against the slow one it replaces, which is what catches an optimisation that changes an answer. Tests needing an optional library skip themselves when it is absent, so the suite passes on a minimal installation rather than reporting failures for tools it was never given.

C.4.1 Shared setup

```
"""Test suite for the unified Erdos-915 program.

The tests import ``erdos915_unified`` from the parent directory, so we put that
directory on the path here, once, for every test module. The whole suite runs
with the standard library alone:

    cd program
    python -m unittest discover -s tests

(no pytest required, though pytest will also discover and run these if present).
"""

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
```

C.4.2 The graph model

```
"""The graph model: the multiplicity matrix and every operation on it."""

import unittest

from erdos915_unified import (
    Graph,
    MULTI_DIRECTED,
    MULTI_UNDIRECTED,
    SIMPLE_DIRECTED,
    SIMPLE_UNDIRECTED,
)


class VariantBooleans(unittest.TestCase):
    def test_the_two_booleans(self):
        self.assertEqual((SIMPLE_UNDIRECTED.directed, SIMPLE_UNDIRECTED.simple), (False, True))
        self.assertEqual((MULTI_UNDIRECTED.directed, MULTI_UNDIRECTED.simple), (False, False))
        self.assertEqual((SIMPLE_DIRECTED.directed, SIMPLE_DIRECTED.simple), (True, True))
        self.assertEqual((MULTI_DIRECTED.directed, MULTI_DIRECTED.simple), (True,
```

```python
            False))

    def test_describe_is_nonempty(self):
        for variant in (SIMPLE_UNDIRECTED, MULTI_UNDIRECTED, SIMPLE_DIRECTED, MULTI_DIRECTED):
            self.assertTrue(variant.describe())


class EmptyGraph(unittest.TestCase):
    def test_fresh_graph_is_empty(self):
        g = Graph(5, SIMPLE_UNDIRECTED)
        self.assertEqual(g.num_vertices, 5)
        self.assertEqual(list(g.vertices()), list(range(5)))
        self.assertEqual(g.edge_count(), 0)
        self.assertEqual(list(g.edges()), [])
        self.assertFalse(g.has_edge(0, 1))


class UndirectedModel(unittest.TestCase):
    def test_add_edge_is_symmetric(self):
        g = Graph(3, SIMPLE_UNDIRECTED)
        g.add_edge(0, 1)
        self.assertTrue(g.has_edge(0, 1))
        self.assertTrue(g.has_edge(1, 0))
        self.assertEqual(g.multiplicity(0, 1), 1)
        self.assertEqual(g.multiplicity(1, 0), 1)
        self.assertEqual(g.edge_count(), 1)
        self.assertEqual(list(g.edges()), [(0, 1, 1)])

    def test_simple_graph_saturates_at_one(self):
        g = Graph(3, SIMPLE_UNDIRECTED)
        g.add_edge(0, 1)
        g.add_edge(0, 1)
        self.assertEqual(g.multiplicity(0, 1), 1)
        self.assertEqual(g.edge_count(), 1)

    def test_multigraph_keeps_parallel_edges(self):
        g = Graph(3, MULTI_UNDIRECTED)
        g.add_edge(0, 1, 3)
        self.assertEqual(g.multiplicity(0, 1), 3)
        self.assertEqual(g.multiplicity(1, 0), 3)
        self.assertEqual(g.edge_count(), 3)
        self.assertEqual(g.degree(0), 3)
        self.assertEqual(g.degree(1), 3)

    def test_undirected_degree_counts_incident_multiplicity(self):
        g = Graph(3, MULTI_UNDIRECTED)
        g.add_edge(0, 1, 2)
        g.add_edge(0, 2, 1)
        self.assertEqual(g.degree(0), 3)
        self.assertEqual(g.edge_count(), 3)


class DirectedModel(unittest.TestCase):
    def test_arc_is_one_directional(self):
        g = Graph(3, SIMPLE_DIRECTED)
        g.add_edge(0, 1)
        self.assertTrue(g.has_edge(0, 1))
```

```
        self.assertFalse(g.has_edge(1, 0))
        self.assertEqual(g.edge_count(), 1)
        self.assertEqual(g.out_degree(0), 1)
        self.assertEqual(g.in_degree(1), 1)
        self.assertEqual(g.in_degree(0), 0)
        self.assertEqual(g.degree(0), 1)
        self.assertEqual(g.degree(1), 1)

    def test_directed_multiplicities_are_independent(self):
        g = Graph(3, MULTI_DIRECTED)
        g.add_edge(0, 1, 2)
        g.add_edge(1, 0, 1)
        self.assertEqual(g.multiplicity(0, 1), 2)
        self.assertEqual(g.multiplicity(1, 0), 1)
        self.assertEqual(g.edge_count(), 3)
        self.assertEqual(g.out_degree(0), 2)
        self.assertEqual(g.in_degree(0), 1)
        self.assertEqual(g.degree(0), 3)

    def test_edges_yields_both_arc_directions(self):
        g = Graph(3, SIMPLE_DIRECTED)
        g.add_edge(0, 1)
        g.add_edge(1, 0)
        self.assertEqual(sorted(g.edges()), [(0, 1, 1), (1, 0, 1)])


class Mutation(unittest.TestCase):
    def test_remove_never_goes_below_zero(self):
        g = Graph(3, MULTI_UNDIRECTED)
        g.add_edge(0, 1, 3)
        g.remove_edge(0, 1, 5)
        self.assertEqual(g.multiplicity(0, 1), 0)
        self.assertFalse(g.has_edge(0, 1))

    def test_set_multiplicity_directly(self):
        g = Graph(3, MULTI_DIRECTED)
        g.set_multiplicity(0, 1, 4)
        self.assertEqual(g.multiplicity(0, 1), 4)
        self.assertEqual(g.multiplicity(1, 0), 0)

    def test_simple_rejects_multiplicity_above_one(self):
        g = Graph(3, SIMPLE_UNDIRECTED)
        with self.assertRaises(ValueError):
            g.set_multiplicity(0, 1, 2)

    def test_negative_multiplicity_rejected(self):
        g = Graph(3, MULTI_UNDIRECTED)
        with self.assertRaises(ValueError):
            g.set_multiplicity(0, 1, -1)

    def test_self_loops_rejected(self):
        g = Graph(3, MULTI_UNDIRECTED)
        with self.assertRaises(ValueError):
            g.add_edge(1, 1)

    def test_copy_is_independent(self):
        g = Graph(3, MULTI_UNDIRECTED)
        g.add_edge(0, 1, 2)
```

```python
        clone = g.copy()
        clone.add_edge(0, 2, 1)
        self.assertEqual(g.edge_count(), 2)
        self.assertEqual(clone.edge_count(), 3)


if __name__ == "__main__":
    unittest.main()
```

C.4.3 The closed-form bounds

```python
"""The closed-form bounds library: every formula, on a range of inputs."""

import unittest

from erdos915_unified import (
    directed_arc_lower_bound,
    directed_arc_m2,
    directed_multigraph_arc,
    hypergraph_edge,
    multigraph_undirected_edge,
    simple_undirected_edge,
    simple_undirected_vertex_le4,
    simple_undirected_vertex_m5,
)


class Bounds(unittest.TestCase):
    def test_simple_undirected_edge(self):
        self.assertEqual(simple_undirected_edge(5, 3), 6)
        for n in range(2, 12):
            for m in range(2, 6):
                self.assertEqual(simple_undirected_edge(n, m), (m * (n - 1)) // 2)

    def test_multigraph_undirected_edge(self):
        for n in range(2, 12):
            for m in range(2, 6):
                self.assertEqual(multigraph_undirected_edge(n, m), (m - 1) * (n - 1))

    def test_vertex_le4_matches_edge_and_caps_at_four(self):
        for m in range(2, 5):
            self.assertEqual(simple_undirected_vertex_le4(10, m), simple_undirected_edge(10, m))
        with self.assertRaises(ValueError):
            simple_undirected_vertex_le4(10, 5)

    def test_vertex_m5(self):
        for n in range(6, 20):
            self.assertEqual(simple_undirected_vertex_m5(n), (8 * n) // 3 - 3)

    def test_directed_arc_m2_branches_and_crossover(self):
        self.assertEqual(directed_arc_m2(4), 6)    # linear hub branch wins
        self.assertEqual(directed_arc_m2(7), 12)  # the two branches meet
        self.assertEqual(directed_arc_m2(8), 16)  # quadratic branch wins
        for n in range(2, 20):
            self.assertEqual(directed_arc_m2(n), max(2 * (n - 1), (n * n) // 4))
```

```python

    def test_directed_arc_lower_bound(self):
        self.assertEqual(directed_arc_lower_bound(10, 3), 30)  # the 30-arc graph
        self.assertEqual(directed_arc_lower_bound(4, 2), 6)
        for n in range(3, 14):
            for m in range(2, 5):
                expected = max(m * (n - 1), (n + m - 2) ** 2 // 4)
                self.assertEqual(directed_arc_lower_bound(n, m), expected)

    def test_hypergraph_edge(self):
        self.assertEqual(hypergraph_edge(7, 2, 3), 3)
        self.assertEqual(hypergraph_edge(13, 2, 4), 4)
        for n in range(3, 14):
            for m in range(2, 5):
                for r in range(2, 5):
                    self.assertEqual(hypergraph_edge(n, m, r), ((m - 1) * (n - 1)) // (r - 1))

    def test_directed_multigraph_arc(self):
        self.assertEqual(directed_multigraph_arc(4, 3), 12)
        self.assertEqual(directed_multigraph_arc(6, 2), 10)
        for n in range(2, 12):
            for m in range(2, 6):
                expected = max(2 * (n - 1) * (m - 1), (m - 1) * ((n * n) // 4))
                self.assertEqual(directed_multigraph_arc(n, m), expected)


if __name__ == "__main__":
    unittest.main()
```

C.4.4 The named constructions

```python
"""The named extremal constructions: their exact sizes and connectivities."""

import unittest

from erdos915_unified import (
    augmented_bipartite,
    clique_tree,
    double_star,
    max_edge_connectivity,
    min_vertex_connectivity,
    one_directional_bipartite,
)


class Constructions(unittest.TestCase):
    def test_double_star_directed(self):
        for n in range(2, 7):
            for m in range(2, 5):
                g = double_star(n, m, directed=True)
                self.assertEqual(g.edge_count(), 2 * (n - 1) * (m - 1))
                self.assertEqual(max_edge_connectivity(g), m - 1)

    def test_double_star_undirected(self):
        for n in range(2, 7):
            for m in range(2, 5):
```

```
                g = double_star(n, m, directed=False)
                self.assertEqual(g.edge_count(), (m - 1) * (n - 1))
                self.assertEqual(max_edge_connectivity(g), m - 1)

    def test_one_directional_bipartite_is_quadratic_and_feasible(self):
        for n in range(2, 8):
            g = one_directional_bipartite(n)
            self.assertEqual(g.edge_count(), (n * n) // 4)
            self.assertEqual(max_edge_connectivity(g), 1)

    def test_augmented_bipartite_conjecture_values(self):
        self.assertEqual(augmented_bipartite(10, 3).edge_count(), 30)
        for n, m in [(8, 3), (10, 4), (12, 4), (10, 5)]:
            g = augmented_bipartite(n, m)
            self.assertEqual(g.edge_count(), (n + m - 2) ** 2 // 4)
            self.assertEqual(max_edge_connectivity(g), m - 1)

    def test_augmented_bipartite_guards_its_precondition(self):
        with self.assertRaises(ValueError):
            augmented_bipartite(3, 5)  # needs n >= m

    def test_clique_tree_is_globally_connected(self):
        for blocks in range(0, 5):
            self.assertEqual(min_vertex_connectivity(clique_tree(4, blocks)), 3)


if __name__ == "__main__":
    unittest.main()
```

C.4.5 The connectivity checker

```
"""The connectivity checker (edge and vertex) and the Gomory-Hu shortcut."""

import random
import unittest

from erdos915_unified import (
    Graph,
    MULTI_DIRECTED,
    MULTI_UNDIRECTED,
    NETWORKX_AVAILABLE,
    SIMPLE_DIRECTED,
    SIMPLE_UNDIRECTED,
    _vertex_flow_at_least,
    exceeds_bound,
    local_edge_connectivity,
    local_vertex_connectivity,
    max_connectivity,
    max_edge_connectivity,
    max_edge_connectivity_via_tree,
    max_vertex_connectivity,
    sample_random_graph,
)


def random_multigraph(variant, n, rng, max_mult=2):
    """A random graph of ``variant`` on ``n`` vertices for predicate testing.
```

```

    Simple variants take multiplicities in ``{0, 1}``; multigraph variants in
    ``{0, ..., max_mult}``.  Directed variants fill every ordered pair, undirected
    variants the upper triangle (``set_multiplicity`` mirrors the lower half).
    """
    g = Graph(n, variant)
    top = 1 if variant.simple else max_mult
    for u in range(n):
        start = 0 if variant.directed else u + 1
        for v in range(start, n):
            if u != v:
                g.set_multiplicity(u, v, rng.randint(0, top))
    return g


def complete_graph(n, variant=SIMPLE_UNDIRECTED):
    g = Graph(n, variant)
    for u in range(n):
        for v in range(u + 1, n):
            g.add_edge(u, v)
    return g


def star(n):
    g = Graph(n, SIMPLE_UNDIRECTED)
    for leaf in range(1, n):
        g.add_edge(0, leaf)
    return g


def cycle(n):
    g = Graph(n, SIMPLE_UNDIRECTED)
    for v in range(n):
        g.add_edge(v, (v + 1) % n)
    return g


class EdgeConnectivity(unittest.TestCase):
    def test_tree_is_one(self):
        self.assertEqual(max_edge_connectivity(star(6)), 1)

    def test_cycle_is_two(self):
        self.assertEqual(max_edge_connectivity(cycle(6)), 2)

    def test_complete_graph(self):
        for n in (3, 4, 5):
            self.assertEqual(max_edge_connectivity(complete_graph(n)), n - 1)
            self.assertEqual(local_edge_connectivity(complete_graph(n), 0, 1), n - 1)

    def test_parallel_edges_count_as_routes(self):
        g = Graph(3, MULTI_UNDIRECTED)
        g.add_edge(0, 1, 4)
        self.assertEqual(local_edge_connectivity(g, 0, 1), 4)
        self.assertEqual(max_edge_connectivity(g), 4)

    def test_directed_arc_disjoint_routes(self):
        g = Graph(3, SIMPLE_DIRECTED)
```

```
        g.add_edge(0, 1)  # direct route
        g.add_edge(0, 2)  # detour 0 -> 2 -> 1
        g.add_edge(2, 1)
        self.assertEqual(local_edge_connectivity(g, 0, 1), 2)

    def test_removing_an_edge_never_raises_lambda(self):
        rng = random.Random(11)
        for _ in range(20):
            g = sample_random_graph(8, 0.4, False, rng)
            before = max_edge_connectivity(g)
            for u, v, _count in list(g.edges()):
                reduced = g.copy()
                reduced.set_multiplicity(u, v, 0)
                self.assertLessEqual(max_edge_connectivity(reduced), before)


class VertexConnectivity(unittest.TestCase):
    def test_complete_graph(self):
        for n in (3, 4, 5):
            self.assertEqual(max_vertex_connectivity(complete_graph(n)), n - 1)
            self.assertEqual(local_vertex_connectivity(complete_graph(n), 0, 1), n - 1)

    def test_whitney_inequality_on_random_graphs(self):
        rng = random.Random(0)
        for _ in range(40):
            g = sample_random_graph(12, 0.3, False, rng)
            self.assertLessEqual(max_vertex_connectivity(g), max_edge_connectivity(g))


class VertexFlowPredicate(unittest.TestCase):
    """``_vertex_flow_at_least`` is the exhaustive search's inner vertex test.

    It must agree with the exact checker it replaces on every (pair, k), or the
    base cases of the directed vertex theorem would rest on a faster wrong test.
    """

    def test_matches_the_exact_checker(self):
        rng = random.Random(17)
        for _ in range(150):
            n = rng.randint(2, 6)
            g = Graph(n, SIMPLE_DIRECTED)
            out = [set() for _ in range(n)]
            for a in range(n):
                for b in range(n):
                    if a != b and rng.random() < 0.35:
                        g.mu[a, b] = 1
                        out[a].add(b)
            for s in range(n):
                for t in range(n):
                    if s == t:
                        continue
                    exact = local_vertex_connectivity(g, s, t)
                    for k in range(1, 5):
                        self.assertEqual(_vertex_flow_at_least(out, n, s, t, k),
                                         exact >= k)

```

```python
    def test_a_direct_arc_is_one_route_with_no_interior(self):
        # s -> t plus the detour s -> x -> t: two internally disjoint routes.
        out = [{1, 2}, set(), {1}, set()]
        self.assertTrue(_vertex_flow_at_least(out, 4, 0, 1, 2))
        self.assertFalse(_vertex_flow_at_least(out, 4, 0, 1, 3))


@unittest.skipUnless(NETWORKX_AVAILABLE, "Gomory-Hu trees need the optional networkx")
class GomoryHu(unittest.TestCase):
    def test_tree_matches_direct_sweep(self):
        for g in (cycle(6), complete_graph(5), star(7)):
            self.assertEqual(max_edge_connectivity_via_tree(g), max_edge_connectivity(g))

    def test_multigraph_matches(self):
        g = Graph(4, MULTI_UNDIRECTED)
        g.add_edge(0, 1, 3)
        g.add_edge(1, 2, 2)
        g.add_edge(2, 3, 1)
        g.add_edge(0, 3, 2)
        self.assertEqual(max_edge_connectivity_via_tree(g), max_edge_connectivity(g))


class CappedPredicate(unittest.TestCase):
    """``exceeds_bound`` must agree with the exact checker, pair set for pair set.

    This pins the cheap capped predicate the search relies on to the trusted
    exact ``max_connectivity`` over every variant, separation and threshold.
    """

    def test_predicate_matches_exact_value(self):
        variants = (SIMPLE_UNDIRECTED, MULTI_UNDIRECTED,
                    SIMPLE_DIRECTED, MULTI_DIRECTED)
        rng = random.Random(2024)
        for variant in variants:
            for n in (3, 4, 5):
                for _ in range(8):
                    g = random_multigraph(variant, n, rng)
                    for sep in ("edge", "vertex"):
                        split = sep == "vertex"
                        exact = max_connectivity(g, vertex_split=split)
                        for k in range(-1, exact + 2):
                            self.assertEqual(
                                exceeds_bound(g, k, separation=sep),
                                exact > k,
                                msg=f"{variant.name} n={n} sep={sep} k={k} "
                                    f"exact={exact}",
                            )

    def test_predicate_does_not_mutate_the_graph(self):
        rng = random.Random(7)
        g = random_multigraph(MULTI_DIRECTED, 5, rng)
        before = g.mu.copy()
        exceeds_bound(g, 1, separation="vertex")
        exceeds_bound(g, 1, separation="edge")
        self.assertTrue((g.mu == before).all())
```

```python


if __name__ == "__main__":
    unittest.main()
```

C.4.6 Hypergraphs

```python
"""The hypergraph model and Berge connectivity by the helper-point network."""

import unittest

from erdos915_unified import (
    Hypergraph,
    complete_uniform_hypergraph,
    hyper_connectivity,
    hyperedge_connectivity,
    max_feasible_hyperedges,
    max_hyperedge_connectivity,
    star_hypertree,
    _hyperedge_candidates,
)


class BergeConnectivity(unittest.TestCase):
    def test_single_hyperedge_carries_one_route(self):
        h = Hypergraph(3, [[0, 1, 2]])
        self.assertEqual(h.edge_count(), 1)
        self.assertEqual(hyperedge_connectivity(h, 0, 1), 1)
        self.assertEqual(hyperedge_connectivity(h, 0, 2), 1)
        self.assertEqual(hyperedge_connectivity(h, 1, 2), 1)

    def test_two_hyperedges_give_two_routes(self):
        h = Hypergraph(4, [[0, 1, 2], [0, 1, 3]])
        self.assertEqual(h.edge_count(), 2)
        self.assertEqual(hyperedge_connectivity(h, 0, 1), 2)
        self.assertEqual(len(h.incident_hyperedges(0)), 2)
        self.assertEqual(len(h.incident_hyperedges(2)), 1)

    def test_complete_three_uniform_k4(self):
        self.assertEqual(max_hyperedge_connectivity(complete_uniform_hypergraph(4,
            3)), 3)

    def test_two_uniform_reduces_to_edge_connectivity(self):
        for n in (3, 4, 5):
            self.assertEqual(max_hyperedge_connectivity(
                complete_uniform_hypergraph(n, 2)), n - 1)

    def test_star_hypertree(self):
        for n, r in [(7, 3), (10, 4), (13, 4)]:
            h = star_hypertree(n, r)
            self.assertEqual(h.edge_count(), (n - 1) // (r - 1))
            self.assertEqual(max_hyperedge_connectivity(h), 1)

    def test_bad_hyperedge_size_rejected(self):
        with self.assertRaises(ValueError):
            star_hypertree(7, 1)

```

```python

class DirectedOrientationModels(unittest.TestCase):
    """Forward / backward / general directed hyperedge models."""

    def test_general_gate_is_one_way(self):
        # A mixed arc {0,1} -> {2,3}: a route enters at a tail, leaves at a head.
        h = Hypergraph(4, [(frozenset({0, 1}), frozenset({2, 3}))], directed=True)
        self.assertEqual(sorted(h.members(h.hyperedges[0])), [0, 1, 2, 3])
        self.assertEqual(hyper_connectivity(h, 0, 2), 1)   # tail -> head
        self.assertEqual(hyper_connectivity(h, 2, 0), 0)   # head -> tail: none

    def test_forward_legacy_equals_general_form(self):
        legacy = Hypergraph(3, [(0, frozenset({1, 2}))], directed=True)
        general = Hypergraph(3, [(frozenset({0}), frozenset({1, 2}))], directed=True)
        for s, t in [(0, 1), (0, 2), (1, 2)]:
            self.assertEqual(hyper_connectivity(legacy, s, t),
                             hyper_connectivity(general, s, t))

    def test_candidate_counts_split_at_r3(self):
        # At r = 3 every split has a singleton side, so general = forward + backward.
        fwd = _hyperedge_candidates(4, 3, True, kind="forward")
        bwd = _hyperedge_candidates(4, 3, True, kind="backward")
        gen = _hyperedge_candidates(4, 3, True, kind="general")
        self.assertEqual(len(fwd), 12)
        self.assertEqual(len(bwd), 12)
        self.assertEqual(len(gen), 24)

    def test_forward_backward_duality(self):
        # Arc reversal makes backward the dual of forward: same extremal numbers.
        for n, m in [(3, 3), (4, 3), (4, 2)]:
            f, _ = max_feasible_hyperedges(n, 3, m, directed=True, kind="forward", time_limit=2.0)
            b, _ = max_feasible_hyperedges(n, 3, m, directed=True, kind="backward", time_limit=2.0)
            self.assertEqual(f, b)

    def test_general_beats_forward_when_vertices_scarce(self):
        # n = r is the regime where the richer orientation set packs more arcs.
        g3, e3 = max_feasible_hyperedges(3, 3, 3, directed=True, kind="general", time_limit=2.0)
        f3, _ = max_feasible_hyperedges(3, 3, 3, directed=True, kind="forward", time_limit=2.0)
        self.assertTrue(e3)
        self.assertEqual((g3, f3), (4, 3))

    def test_general_contains_forward(self):
        # General can never be worse than forward (forward hypergraphs are general).
        for n, m in [(4, 3), (5, 2)]:
            f, _ = max_feasible_hyperedges(n, 3, m, directed=True, kind="forward", time_limit=2.0)
            g, _ = max_feasible_hyperedges(n, 3, m, directed=True, kind="general",
                                           time_limit=3.0, seed_lb=f)
            self.assertGreaterEqual(g, f)

```

```python
if __name__ == "__main__":
    unittest.main()
```

C.4.7 Edge sensitivity

```python
"""Edge sensitivity: the load-bearing measure that guides the search."""

import random
import unittest

from erdos915_unified import (
    Graph,
    MULTI_DIRECTED,
    MULTI_UNDIRECTED,
    SIMPLE_DIRECTED,
    SIMPLE_UNDIRECTED,
    double_star,
    edge_sensitivity,
    max_edge_connectivity,
    max_vertex_connectivity,
    one_directional_bipartite,
    sensitivity_map,
)


def _random_graph(variant, n, rng, top):
    g = Graph(n, variant)
    cap = 1 if variant.simple else top
    for u in range(n):
        start = 0 if variant.directed else u + 1
        for v in range(start, n):
            if u != v:
                g.set_multiplicity(u, v, rng.randint(0, cap))
    return g


class Sensitivity(unittest.TestCase):
    def test_absent_edge_has_zero_sensitivity(self):
        g = Graph(3, SIMPLE_UNDIRECTED)
        self.assertEqual(edge_sensitivity(g, 0, 1), 0)

    def test_sensitivity_matches_its_definition(self):
        g = one_directional_bipartite(6)
        before = max_edge_connectivity(g)
        for (u, v), sigma in sensitivity_map(g).items():
            reduced = g.copy()
            reduced.set_multiplicity(u, v, 0)
            self.assertEqual(sigma, before - max_edge_connectivity(reduced))
            self.assertGreaterEqual(sigma, 0)  # removing an edge never raises
                ↪ lambda

    def test_map_covers_exactly_the_present_edges(self):
        g = double_star(5, 3, directed=True)
        self.assertEqual(len(sensitivity_map(g)), sum(1 for _ in g.edges()))

    def test_map_equals_per_edge_sensitivity(self):
        # The before-once refactor must return the IDENTICAL dict as the old
```

```python
        # per-edge definition, across variants and both connectivity measures.
        variants = (SIMPLE_UNDIRECTED, MULTI_UNDIRECTED,
                    SIMPLE_DIRECTED, MULTI_DIRECTED)
        rng = random.Random(99)
        for variant in variants:
            for measure in (max_edge_connectivity, max_vertex_connectivity):
                for _ in range(6):
                    g = _random_graph(variant, 5, rng, top=2)
                    expected = {(u, v): edge_sensitivity(g, u, v, measure)
                                for u, v, _ in g.edges()}
                    self.assertEqual(sensitivity_map(g, measure), expected)


if __name__ == "__main__":
    unittest.main()
```

C.4.8 The search

```python
"""The temperature search: it rediscovers the known small extremal values.

The search is seeded, so these outcomes are deterministic and reproducible.
"""

import unittest

from erdos915_unified import (
    MULTI_DIRECTED,
    MULTI_UNDIRECTED,
    SIMPLE_DIRECTED,
    SIMPLE_UNDIRECTED,
    best_of_searches,
    max_connectivity,
    max_edge_connectivity,
    search_for_dense_graph,
    tabu_search_for_dense_graph,
)


class Search(unittest.TestCase):
    def test_search_rediscovers_directed_m2(self):
        result = search_for_dense_graph(SIMPLE_DIRECTED, n=4, m=2, steps=6000, seed=0)
        self.assertEqual(result.best_edge_count, 6)  # ell_2^dir(4) = 6
        self.assertTrue(result.feasible_found)
        self.assertLessEqual(max_edge_connectivity(result.best_graph), 1)

    def test_acceptance_rate_is_a_fraction(self):
        result = search_for_dense_graph(SIMPLE_DIRECTED, n=4, m=2, steps=6000, seed=0)
        self.assertTrue(0.0 <= result.acceptance_rate() <= 1.0)
        self.assertEqual(len(result.history), 6000)

    def test_search_rediscovers_undirected_m3(self):
        # ell_3(n) = floor(3(n-1)/2): n=4 -> 4, n=5 -> 6.
        for n, expected in [(4, 4), (5, 6)]:
            result = best_of_searches(SIMPLE_UNDIRECTED, n, 3, restarts=6, steps=3000, seed=0)
```

```python
            self.assertEqual(result.best_edge_count, expected)
            self.assertLessEqual(max_edge_connectivity(result.best_graph), 2)


class TabuSearch(unittest.TestCase):
    """Tabu search is the deterministic twin of the annealer: same energy and
    neighbourhood, so it rediscovers the same small extremal values.
    """

    def test_tabu_rediscovers_directed_m2(self):
        result = tabu_search_for_dense_graph(SIMPLE_DIRECTED, n=4, m=2,
                                             steps=150, seed=0)
        self.assertEqual(result.best_edge_count, 6)  # ell_2^dir(4) = 6
        self.assertTrue(result.feasible_found)
        self.assertLessEqual(max_edge_connectivity(result.best_graph), 1)

    def test_tabu_rediscovers_undirected_m3(self):
        result = tabu_search_for_dense_graph(SIMPLE_UNDIRECTED, n=4, m=3,
                                             steps=150, seed=0)
        self.assertEqual(result.best_edge_count, 4)  # ell_3(4) = 4
        self.assertLessEqual(max_edge_connectivity(result.best_graph), 2)

    def test_best_of_searches_accepts_tabu_method(self):
        result = best_of_searches(SIMPLE_DIRECTED, 4, 2, restarts=2,
                                  method="tabu", parallel=False, steps=120)
        self.assertEqual(result.best_edge_count, 6)

    def test_unknown_method_rejected(self):
        with self.assertRaises(ValueError):
            best_of_searches(SIMPLE_UNDIRECTED, 4, 3, method="genetic")


class FastPathEquivalence(unittest.TestCase):
    """The fast (capped + monotone) search must reproduce the slow exact search
    step for step.  This is the "nothing under the rug" guarantee: the speedup
    changes no reported value because it changes no single step of the walk.
    """

    def test_fast_path_matches_exact(self):
        variants = (SIMPLE_UNDIRECTED, MULTI_UNDIRECTED,
                    SIMPLE_DIRECTED, MULTI_DIRECTED)
        for variant in variants:
            for n in (3, 4, 5):
                for m in (2, 3):
                    for sep in ("edge", "vertex"):
                        for seed in range(2):
                            opts = dict(separation=sep, steps=300, seed=seed)
                            ref = search_for_dense_graph(
                                variant, n, m, reference_mode=True, **opts)
                            fast = search_for_dense_graph(variant, n, m, **opts)
                            tag = (f"{variant.name} n={n} m={m} sep={sep} "
                                   f"seed={seed}")
                            # Bit-for-bit: same energies, same accept/reject
                            # decisions, same density at every step.
                            self.assertEqual(
                                [s.energy for s in fast.history],
                                [s.energy for s in ref.history], tag)
                            self.assertEqual(
```

```
                                [s.accepted for s in fast.history],
                                [s.accepted for s in ref.history], tag)
                            self.assertEqual(
                                [s.edge_count for s in fast.history],
                                [s.edge_count for s in ref.history], tag)
                            self.assertEqual(
                                fast.best_edge_count, ref.best_edge_count, tag)
                            self.assertTrue(
                                (fast.best_graph.mu == ref.best_graph.mu).all(),
                                tag)
                            # The returned witness is genuinely feasible by the
                            # exact checker (certified, not just flagged).
                            split = sep == "vertex"
                            self.assertLessEqual(
                                max_connectivity(fast.best_graph,
                                                 vertex_split=split),
                                m - 1, tag)


if __name__ == "__main__":
    unittest.main()
```

C.4.9 The prover

```
"""The cut-counting prover: a zero-gap solve is a genuine upper-bound proof."""

import os
import unittest

import numpy as np

from erdos915_unified import (
    MULTI_DIRECTED,
    PULP_AVAILABLE,
    Graph,
    _canonical_form,
    enumerate_extremal_directed_multigraphs,
    max_edge_connectivity,
    prove_directed_multigraph,
)


@unittest.skipUnless(PULP_AVAILABLE, "the MILP certifier needs the optional pulp")
class Prover(unittest.TestCase):
    def test_small_optima_are_proved(self):
        # M*(n) = 2(n-1), optimal with zero gap, for the reachable sizes.
        for n in (3, 4, 5):
            result = prove_directed_multigraph(n, time_limit=120.0)
            self.assertTrue(result.is_proof())
            self.assertEqual(result.status, "OPTIMAL")
            self.assertEqual(round(result.scaled_optimum), 2 * (n - 1))

    def test_one_solve_settles_every_m(self):
        result = prove_directed_multigraph(4, time_limit=120.0)
        self.assertTrue(result.is_proof())
        for m in (2, 3, 4, 5):
            # L_m^dir(4) = (m-1) * M*(4) = (m-1) * 6.
```

```
            self.assertEqual(result.value_for(m), (m - 1) * 2 * (4 - 1))


class EnumerationDedup(unittest.TestCase):
    """The streamed canonical-form dedup keeps one representative per iso-class
    and returns exactly the known small extremal sets.

    The active path stays at n in {3, 4} (sub-second).  The n=5 enumeration is
    minutes long (the DFS, not the dedup), so its known count is checked only when
    ``RUN_SLOW_ENUM`` is set; the dedup logic it exercises is identical to n=4.
    """

    # m=3 directed multigraph, target = 4(n-1) arcs: the doubled bidirected
    # spanning trees.  Known counts (claude.md): n=3 -> 1 (path), n=4 -> 2
    # (star, path), n=5 -> 3 (star, broom, path).
    FAST_COUNTS = {3: 1, 4: 2}

    def test_known_iso_class_counts(self):
        for n, expected in self.FAST_COUNTS.items():
            reps = enumerate_extremal_directed_multigraphs(n, 3, 4 * (n - 1))
            self.assertEqual(len(reps), expected, f"n={n}")

    def test_representatives_are_distinct_and_feasible(self):
        for n in self.FAST_COUNTS:
            reps = enumerate_extremal_directed_multigraphs(n, 3, 4 * (n - 1))
            keys = [_canonical_form(mu) for mu in reps]
            # One representative per class: all canonical keys distinct.
            self.assertEqual(len(set(keys)), len(keys), f"n={n}")
            for mu in reps:
                g = Graph(n, MULTI_DIRECTED)
                g.mu = mu.copy()
                self.assertEqual(g.edge_count(), 4 * (n - 1))
                self.assertLessEqual(max_edge_connectivity(g), 2, f"n={n}")

    def test_dedup_preserves_the_iso_classes(self):
        # The deduped run must cover exactly the iso-classes of the full labelled
        # run: no class invented, none dropped.
        for n in self.FAST_COUNTS:
            labelled = enumerate_extremal_directed_multigraphs(
                n, 3, 4 * (n - 1), up_to_iso=False)
            deduped = enumerate_extremal_directed_multigraphs(n, 3, 4 * (n - 1))
            self.assertEqual(
                {_canonical_form(mu) for mu in labelled},
                {_canonical_form(mu) for mu in deduped},
                f"n={n}")
            # Every labelled matrix is itself one of the kept iso-classes.
            kept = {_canonical_form(mu) for mu in deduped}
            for mu in labelled:
                self.assertIn(_canonical_form(mu), kept, f"n={n}")

    def test_canonical_form_is_permutation_invariant(self):
        rng = np.random.default_rng(0)
        for n in (3, 4, 5, 6):
            mu = rng.integers(0, 3, size=(n, n))
            np.fill_diagonal(mu, 0)
            key = _canonical_form(mu)
            for _ in range(5):
                p = rng.permutation(n)
```

```
                self.assertEqual(_canonical_form(mu[np.ix_(p, p)]), key, f"n={n}")

    @unittest.skipUnless(os.environ.get("RUN_SLOW_ENUM"),
                         "n=5 enumeration is minutes long; set RUN_SLOW_ENUM=1")
    def test_n5_known_count(self):
        reps = enumerate_extremal_directed_multigraphs(5, 3, 16)
        self.assertEqual(len(reps), 3)  # star, broom, path


if __name__ == "__main__":
    unittest.main()
```

C.4.10 The random model

```
"""The random model: the appearance threshold and the Whitney relation.

All sampling is seeded, so these outcomes are deterministic.
"""

import unittest

from erdos915_unified import (
    connectivity_distribution,
    edge_vertex_distribution,
    estimate_appearance_probability,
    threshold_curve,
)


class MonteCarlo(unittest.TestCase):
    def test_appearance_probability_crosses_m_over_n(self):
        n, m = 30, 3
        below = estimate_appearance_probability(n, 0.3 * m / n, m, trials=120, seed=1)
        above = estimate_appearance_probability(n, 1.8 * m / n, m, trials=120, seed=1)
        self.assertLess(below, 0.5)
        self.assertGreater(above, 0.5)

    def test_whitney_holds_on_every_sample(self):
        edge = connectivity_distribution(20, 0.25, trials=40, separation="edge", seed=3)
        vertex = connectivity_distribution(20, 0.25, trials=40, separation="vertex", seed=3)
        self.assertEqual(len(edge), 40)
        for kappa, lam in zip(vertex, edge):
            self.assertLessEqual(kappa, lam)

    def test_edge_and_vertex_agree_at_small_n(self):
        lam, kap = edge_vertex_distribution(5, 0.5, trials=120, seed=7)
        self.assertEqual(sum(k < l for k, l in zip(kap, lam)), 0)  # no gap at n = 5
        self.assertTrue(all(k <= l for k, l in zip(kap, lam)))  # Whitney

    def test_edge_and_vertex_part_company_at_larger_n(self):
        lam, kap = edge_vertex_distribution(16, 0.25, trials=200, seed=7)
        gap = sum(k < l for k, l in zip(kap, lam))
```

```
        self.assertGreater(gap, 0)
        self.assertTrue(all(k <= l for k, l in zip(kap, lam)))

    def test_threshold_curve_reports_the_predicted_threshold(self):
        curve = threshold_curve(20, 2, [0.05, 0.2], trials=20, seed=0)
        self.assertAlmostEqual(curve.predicted_threshold, 2 / 20)
        self.assertEqual(len(curve.probabilities), 2)


if __name__ == "__main__":
    unittest.main()
```

C.4.11 The solver

```
"""The single solver: one solve() call handles every kind of question."""

import shutil
import unittest

import itertools

import numpy as np

from erdos915_unified import (
    solve,
    enumerate_extremal_directed_multigraphs as _dfs_enum,
    enumerate_extremal_directed_multigraphs_via_generation as _gen_enum,
    _decorate_support_worker,
    _canonical_form,
    PULP_AVAILABLE,
)


class Solve(unittest.TestCase):
    def test_exhaustive_simple_directed_is_exact(self):
        r = solve(4, 2, directed=True, simple=True, exhaustive=True, max_seconds
            ↪ =120.0)
        self.assertTrue(r.proven)
        self.assertEqual(r.bound, "exact")
        self.assertEqual(r.value, 6)

    def test_discovery_returns_a_lower_bound(self):
        # verification stays on sa; tabu is the default for solving the problems
        r = solve(4, 2, directed=True, simple=True, exhaustive=False,
                  max_seconds=3.0, method="sa")
        self.assertEqual(r.bound, "lower")
        self.assertEqual(r.value, 6)

    def test_exhaustive_undirected_finds_the_spanning_tree(self):
        r = solve(5, 2, directed=False, simple=True, exhaustive=True, max_seconds
            ↪ =30.0)
        self.assertTrue(r.proven)
        self.assertEqual(r.value, 4)  # a spanning tree on 5 vertices

    @unittest.skipUnless(PULP_AVAILABLE,
                         "solve() proves this one through the MILP certifier, "
                         "which needs the optional pulp")
```

```python
    def test_directed_multigraph_is_proved(self):
        r = solve(4, 3, directed=True, simple=False, exhaustive=True, max_seconds=120.0)
        self.assertTrue(r.proven)
        self.assertEqual(r.value, 12)  # L_3^dir(4) = 2(n-1)(m-1) = 12

    def test_vertex_separation_undirected(self):
        # k_2(n) = n - 1: no cycle is allowed, so a spanning tree is extremal.
        r = solve(5, 2, directed=False, simple=True, separation="vertex",
                  exhaustive=True, max_seconds=30.0)
        self.assertTrue(r.proven)
        self.assertEqual(r.value, 4)

    def test_hypergraph_discovery(self):
        r = solve(7, 2, hypergraph=True, r=3, exhaustive=False, max_seconds=3.0,
                  method="sa")
        self.assertEqual(r.bound, "lower")
        self.assertEqual(r.value, 3)

    def test_result_describe_is_readable(self):
        r = solve(4, 2, directed=True, simple=True, exhaustive=True, max_seconds=120.0)
        self.assertIn("value", r.describe())


@unittest.skipIf(shutil.which("geng") is None, "nauty's geng is not installed")
class GengGeneration(unittest.TestCase):
    """The geng-seeded enumerator must return exactly the same isomorphism
    classes as the DFS enumerator wherever the DFS one is a sound complete
    search (n <= 6).  This pins down the new generation pipeline against the
    method whose values are already in the thesis."""

    @staticmethod
    def _classes(reps):
        return {_canonical_form(mu) for mu in reps}

    def test_matches_dfs_at_n4(self):
        # n = 4, m = 3: full target range and a degree cap.  Fast, and the DFS
        # enumerator is provably complete here, so equality is the gold standard.
        for target, cap in [(12, None), (10, None), (8, None), (8, 6), (6, None)]:
            with self.subTest(target=target, cap=cap):
                a = self._classes(_dfs_enum(4, 3, target, max_degree=cap))
                b = self._classes(_gen_enum(4, 3, target, max_degree=cap))
                self.assertEqual(a, b)

    def test_extremal_is_the_doubled_spanning_trees(self):
        # At n <= 6 the extremal directed multigraphs are exactly the doubled
        # bidirected spanning trees, one per unlabelled tree: 2 on 4 vertices.
        reps = _gen_enum(4, 3, 12)            # L_3^dir(4) = 2(n-1)(m-1) = 12
        self.assertEqual(len(self._classes(reps)), 2)


class SupportWorker(unittest.TestCase):
    """The per-support decorator is the unit the generation enumerator fans out
    across processes.  Over every support it must reproduce the DFS classes, so
    the parallel refactor is covered here even without geng on PATH."""

    @staticmethod
```

```python
    def _all_supports(n, min_e, max_e):
        # Every non-isomorphic simple graph on n vertices in the edge range, the
        # output geng would feed the enumerator, by brute force + canonical dedup.
        pairs = [(i, j) for i in range(n) for j in range(i + 1, n)]
        seen, out = set(), []
        for mask in itertools.product((0, 1), repeat=len(pairs)):
            if not (min_e <= sum(mask) <= max_e):
                continue
            mu = np.zeros((n, n), dtype=int)
            edges = []
            for (i, j), bit in zip(pairs, mask):
                if bit:
                    mu[i, j] = mu[j, i] = 1
                    edges.append((i, j))
            key = _canonical_form(mu)
            if key not in seen:
                seen.add(key)
                out.append(edges)
        return out

    def test_worker_union_matches_dfs_at_n4(self):
        n, m, target = 4, 3, 12
        supports = self._all_supports(n, (target + 3) // 4, min(target, n * (n - 1) // 2))
        reps = []
        for edges in supports:
            reps.extend(_decorate_support_worker((n, m, target, None, True, edges)))
        worker = {_canonical_form(mu) for mu in reps}
        dfs = {_canonical_form(mu) for mu in _dfs_enum(n, m, target)}
        self.assertEqual(worker, dfs)


if __name__ == "__main__":
    unittest.main()
```

Bibliography

[1] Béla Bollobás and Pál Erdős. Extremal problems in graph theory. *Matematikai Lapok*, 13:143–152, 1962. Original statement of Problem 915.

[2] Karl Menger. Zur allgemeinen Kurventheorie. *Fundamenta Mathematicae*, 10:96–115, 1927. The original statement of Menger's theorem.

[3] Pál Erdős. Erdős problems. `https://www.erdosproblems.com/915`. Problem 915.

[4] Wolfgang Mader. Ein Extremalproblem des Zusammenhangs von Graphen. *Mathematische Zeitschrift*, 131:223–231, 1973. Solves Erdős Problem 915 for edge-disjoint paths.

[5] Hassler Whitney. Congruent graphs and the connectivity of graphs. *American Journal of Mathematics*, 54(1):150–168, 1932.

[6] John L. Leonard. On graphs with at most four line-disjoint paths connecting any two vertices. *Journal of Combinatorial Theory, Series B*, 13:242–250, 1972.

[7] B. A. Sørensen and Carsten Thomassen. On k-rails in graphs. *Journal of Combinatorial Theory, Series B*, 17(2):143–159, 1974. Determines $k_5(n) = \lfloor 8n/3 \rfloor - 3$.

[8] John L. Leonard. On a conjecture of Bollobás and Erdős. *Periodica Mathematica Hungarica*, 3(3–4):281–284, 1973. Proves $\ell_m(n) = k_m(n)$ for $m \leq 4$.

[9] Kristóf Bérczi, Karthekeyan Chandrasekaran, Tamás Király, and Shubhang Kulkarni. Splitting-off in hypergraphs. In *Proceedings of the 51st International Colloquium on Automata, Languages, and Programming (ICALP 2024)*, Leibniz International Proceedings in Informatics (LIPIcs), 2024. arXiv:2307.08555. A journal version has since appeared, which should replace this entry once the volume and pages are to hand.

[10] Brendan D. McKay and Adolfo Piperno. Practical graph isomorphism, II. *Journal of Symbolic Computation*, 60:94–112, 2014. Describes the nauty/Traces canonical-labelling tools underlying geng.

[11] Brendan D. McKay and Stanisław P. Radziszowski. $R(4,5) = 25$. *Journal of Graph Theory*, 19(3):309–322, 1995. Exhaustive-generation determination of a Ramsey number.

[12] Marijn J. H. Heule, Oliver Kullmann, and Victor W. Marek. Solving and verifying the Boolean Pythagorean triples problem via cube-and-conquer. In *Theory and Applications of Satisfiability Testing (SAT 2016)*, volume 9710 of *Lecture Notes in Computer Science*, pages 228–245. Springer, 2016. The certified SAT proof is roughly 200 terabytes in DRAT format.

[13] Marijn J. H. Heule. Schur number five. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI-18)*, pages 6598–6606. AAAI Press, 2018.

[14] Alexander A. Razborov. Flag algebras. *The Journal of Symbolic Logic*, 72(4):1239–1282, 2007.

[15] Ralph E. Gomory and Tien Chung Hu. Multi-terminal network flows. *Journal of the Society for Industrial and Applied Mathematics*, 9(4):551–570, 1961. Introduces the Gomory-Hu tree structure.

[16] Zsolt Baranyai. On the factorization of the complete uniform hypergraph. In *Infinite and Finite Sets (Colloq., Keszthely, 1973), Vol. I*, volume 10 of *Colloq. Math. Soc. Janos Bolyai*, pages 91–108. North-Holland, 1975.

[17] W. T. Tutte. *Connectivity in Graphs*. University of Toronto Press, Toronto, 1966.

[18] John E. Hopcroft and Robert E. Tarjan. Dividing a graph into triconnected components. *SIAM Journal on Computing*, 2(3):135–158, 1973.

[19] Wolfgang Mulzer. Five proofs of Chernoff's bound with applications. *Bulletin of the EATCS*, 124, 2018. Accessible survey of Chernoff bound variants.

[20] Béla Bollobás. *Random Graphs*, volume 73 of *Cambridge Studies in Advanced Mathematics*. Cambridge University Press, 2 edition, 2001. Chapter 7 gives $\kappa(G(n,p)) = \delta(G(n,p))$ with high probability in this regime.

[21] Béla Bollobás and Andrew Thomason. Random graphs of small order. In *Random Graphs '83*, volume 28 of *Annals of Discrete Mathematics*, pages 47–97. North-Holland, 1985. Hitting-time form of the connectivity result.

[22] Alan Frieze and Michał Karoński. *Introduction to Random Graphs*. Cambridge University Press, 2016. Chapter 7 covers connectivity in $G(n,p)$ and $D(n,p)$; the directed analogue of Bollobás's $\kappa = \delta$ result appears in Section 7.3.

DEPARTMENT OF MATHEMATICS
Celestijnenlaan 200B
3001 Leuven, Belgium
wis.kuleuven.be